



Фёдор Меньшиков

ОЛИМПИАДНЫЕ ЗАДАЧИ ПО ПРОГРАММИРОВАНИЮ

 ПИТЕР®

Фёдор Меньшиков

ОЛИМПИАДНЫЕ ЗАДАЧИ ПО ПРОГРАММИРОВАНИЮ



Москва · Санкт-Петербург · Нижний Новгород · Воронеж
Ростов-на-Дону · Екатеринбург · Самара · Новосибирск
Киев · Харьков · Минск

2006

ББК 32.973.2-018

УДК 004.42

М51

Меньшиков Ф. В.

М51 Олимпиадные задачи по программированию (+CD). — СПб.: Питер, 2006. — 315 с.: ил.

ISBN 5-469-00765-0

Цель этой книги — познакомить читателей с некоторыми часто встречающимися типами задач, предлагаемых на олимпиадах по программированию. В разных источниках можно найти немало олимпиадных задач, но примеры решений публикуются далеко не всегда, а без них начинающему программисту-олимпийцу подготовиться к реальным соревнованиям бывает тяжело. В этой книге опубликован детальный разбор задач, взятых из ранее проводившихся олимпиад. Кроме примеров решения, автор обращает внимание читателей на особенности языков программирования, которые часто упускают из виду, но которые очень важны для успешного решения задачи. К книге прилагается компакт-диск с набором тестовых данных и тестирующей системой, позволяющими проверить правильность решения каждой из 90 задач.

ББК 32.973.2-018

УДК 004.42

Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 5-469-00765-0

© ЗАО Издательский дом «Питер», 2006

Краткое содержание

Предисловие	17
ЧАСТЬ 1. ЗАДАЧИ	26
Тренировка 1	26
Тренировка 2	29
Тренировка 3	32
Тренировка 4	35
Тренировка 5	38
Тренировка 6	42
Тренировка 7	46
Тренировка 8	50
Тренировка 9	56
Тренировка 10	62
Тренировка 11	67
Тренировка 12	71
Тренировка 13	77
Тренировка 14	82
Тренировка 15	87
ЧАСТЬ 2. РЕШЕНИЯ	92
Тренировка 1	92
Тренировка 2	107
Тренировка 3	123

Тренировка 4	137
Тренировка 5	146
Тренировка 6	160
Тренировка 7	171
Тренировка 8	185
Тренировка 9	199
Тренировка 10	217
Тренировка 11	230
Тренировка 12	239
Тренировка 13	252
Тренировка 14	266
Тренировка 15	277

Приложение А. Характеристики встроенных типов	
Турбо Паскаля	294
Приложение Б. Процедуры длинной арифметики	295
Дополнительная информация о задачах	300

Содержание

Предисловие	17
Структура книги	17
Как работать с книгой	18
Турбо Паскаль	19
Технический момент	20
Рекомендуемые настройки среды	21
Компакт-диск	21
Установка проверяющей системы	22
Работа с проверяющей системой	23
История создания книги	24
Благодарности	25
От издательства	25
 ЧАСТЬ 1. ЗАДАЧИ	 26
Тренировка 1	26
Задача 1 А. Простые числа	26
Задача 1 В. Выражение	26
Задача 1 С. Возрастающая подпоследовательность	27
Задача 1 D. Треугольник и точка	27
Задача 1 Е. Степень	28
Задача 1 F. Покер	28
 Тренировка 2	 29
Задача 2 А. Простые числа (2)	29
Задача 2 В. Перестановки	29
Задача 2 С. Маршрут	30
Задача 2 D. Пересечение отрезков	30
Задача 2 Е. Длинная сумма	31
Задача 2 F. Спираль	31

Тренировка 3	32
Задача 3 А. Разложение на простые множители	32
Задача 3 В. Перестановки (2)	32
Задача 3 С. Копилка	33
Задача 3 D. Открытка и конверт	33
Задача 3 Е. Длинное произведение	34
Задача 3 F. Змейка	34
 Тренировка 4	 35
Задача 4 А. Совершенные числа	35
Задача 4 В. Разложение на слагаемые	35
Задача 4 С. Гангстеры	36
Задача 4 D. Площадь многоугольника ..	36
Задача 4 Е. Деление длинного числа на короткое	37
Задача 4 F. Скобки	37
 Тренировка 5	 38
Задача 5 А. Дружественные числа	38
Задача 5 В. Скобки (2)	38
Задача 5 С. Маршрут (2)	39
Задача 5 D. Выпуклая оболочка	39
Задача 5 Е. Системы счисления	40
Задача 5 F. День рождения	40
 Тренировка 6	 42
Задача 6 А. Закраска прямой	42
Задача 6 В. Суммы	42
Задача 6 С. Игра «Даты»	43
Задача 6 D. Площадь прямоугольников	43
Задача 6 Е. Lines	43
Задача 6 F. Покраска лабиринта	44
 Тренировка 7	 46
Задача 7 А. Упорядоченные дроби	46
Задача 7 В. Сообщение	46
Задача 7 С. Игра умножения	47
Задача 7 D. Прямая и квадраты	47
Задача 7 Е. Lines (2)	47
Задача 7 F. Удаление клеток	48

Тренировка 8	50
Задача 8 А. Несоставляемое число	50
Задача 8 В. SMS	50
Задача 8 С. Дерево игры	52
Задача 8 D. Дуга на сфере	53
Задача 8 Е. Путь коня	54
Задача 8 F. Грядки	54
Тренировка 9	56
Задача 9 А. Диалог компьютеров	56
Задача 9 В. Бросание кубика	57
Задача 9 С. Игра с монетами	58
Задача 9 D. Бассейн реки	58
Задача 9 Е. Ближайшее число	60
Задача 9 F. Прямоугольное деление	61
Тренировка 10	62
Задача 10 А. Анти-QuickSort	62
Задача 10 В. Строки Фибоначчи	63
Задача 10 С. Игра в зачеркивание	64
Задача 10 D. Граница многоугольника	64
Задача 10 Е. Путь спелеолога	65
Задача 10 F. Дырявая ткань	65
Тренировка 11	67
Задача 11 А. Последовательность	67
Задача 11 В. Провода	67
Задача 11 С. Палиндромы	68
Задача 11 D. Круговая площадь	68
Задача 11 Е. Гомер Симпсон	69
Задача 11 F. Дробная арифметика	69
Тренировка 12	71
Задача 12 А. Последовательность (2)	71
Задача 12 В. Гирлянда	71
Задача 12 С. Головоломка умножения	72
Задача 12 D. Точки в многоугольнике	73
Задача 12 Е. Водопровод	73
Задача 12 F. Химические реакции	74

Тренировка 13	77
Задача 13 А. Двойная решетка	77
Задача 13 В. Последовательность Фибоначчи	77
Задача 13 С. Скобки (3)	78
Задача 13 D. Центр тяжести	78
Задача 13 Е. Сумма произведений	79
Задача 13 F. Статическая сложность	79
Тренировка 14	82
Задача 14 А. Марковский цикл	82
Задача 14 В. Д-44	83
Задача 14 С. Упаковка символов	83
Задача 14 D. Пирамиды	84
Задача 14 Е. Поле для крикета	84
Задача 14 F. Электронная таблица	86
Тренировка 15	87
Задача 15 А. Игра с калькулятором	87
Задача 15 В. Площадь треугольника	87
Задача 15 С. Формирование поезда	88
Задача 15 D. Стена	89
Задача 15 Е. Семечки	90
Задача 15 F. Умножение многочленов	91
ЧАСТЬ 2. РЕШЕНИЯ	92
Тренировка 1	92
Решение задачи 1 А. Простые числа	92
Решение задачи 1 В. Выражение	94
Вариант 1	94
Вариант 2	95
Решение задачи 1 С. Возрастающая подпоследовательность	97
Решение задачи 1 D. Треугольник и точка	99
Вариант 1	99
Вариант 2	100
Вариант 3	101
Вариант 4	103
Вариант 5	104
Решение задачи 1 Е. Степень	105

Решение задачи 1 F. Покер	105
Вариант 1	105
Вариант 2	106
Вариант 3	106
Сравнение алгоритмов	106
Тренировка 2	107
Решение задачи 2 A. Простые числа (2)	107
Решение задачи 2 B. Перестановки	108
Вариант 1	108
Вариант 2	110
Вариант 3	110
Вариант 4	111
Решение задачи 2 C. Маршрут	112
Решение задачи 2 D. Пересечение отрезков	114
Вариант 1	114
Вариант 2	116
Решение задачи 2 E. Длинная сумма	119
Решение задачи 2 F. Спираль	121
Тренировка 3	123
Решение задачи 3 A. Разложение на простые множители	123
Решение задачи 3 B. Перестановки (2)	123
Вариант 1	124
Вариант 2	125
Вариант 3	126
Решение задачи 3 C. Копилка	126
Вариант 1	126
Вариант 2	127
Решение задачи 3 D. Открытка и конверт	128
Вариант 1	128
Вариант 2	130
Решение задачи 3 E. Длинное произведение	132
Вариант 1	132
Вариант 2	134
Вариант 3	135
Сравнение алгоритмов	135
Решение задачи 3 F. Змейка	135

Тренировка 4	137
Решение задачи 4 А. Совершенные числа	137
Решение задачи 4 В. Разложение на слагаемые	137
Решение задачи 4 С. Гангстеры	139
Вариант 1	139
Вариант 2	140
Решение задачи 4 D. Площадь многоугольника	141
Вариант 1	141
Вариант 2	142
Решение задачи 4 Е. Деление длинного числа на короткое	143
Решение задачи 4 F. Скобки	144
Тренировка 5	146
Решение задачи 5 А. Дружественные числа	146
Решение задачи 5 В. Скобки (2)	147
Решение задачи 5 С. Маршрут (2)	149
Решение задачи 5 D. Выпуклая оболочка	152
Решение задачи 5 Е. Системы счисления	155
Вариант 1	156
Вариант 2	157
Вариант 3	158
Сравнение алгоритмов	158
Решение задачи 5 F. День рождения	158
Тренировка 6	160
Решение задачи 6 А. Закраска прямой	160
Вариант 1	160
Вариант 2	161
Решение задачи 6 В. Суммы	161
Решение задачи 6 С. Игра «Даты»	162
Решение задачи 6 D. Площадь прямоугольников	163
Решение задачи 6 Е. Lines	166
Решение задачи 6 F. Покраска лабиринта	169
Тренировка 7	171
Решение задачи 7 А. Упорядоченные дроби	171
Вариант 1	171
Вариант 2	172
Решение задачи 7 В. Сообщение	173

Решение задачи 7 С. Игра умножения	173
Вариант 1	173
Вариант 2	176
Решение задачи 7 D. Прямая и квадраты	177
Решение задачи 7 E. Lines (2)	177
Решение задачи 7 F. Удаление клеток	181

Тренировка 8 185

Решение задачи 8 A. Несоставляемое число	185
Решение задачи 8 B. SMS	186
Решение задачи 8 C. Дерево игры	188
Вариант 1	188
Вариант 2	190
Решение задачи 8 D. Дуга на сфере	191
Вариант 1	191
Вариант 2	192
Вариант 3	193
Решение задачи 8 E. Путь коня	193
Решение задачи 8 F. Грядки	197

Тренировка 9 199

Решение задачи 9 A. Диалог компьютеров	199
Решение задачи 9 B. Бросание кубика	202
Решение задачи 9 C. Игра с монетами	204
Решение задачи 9 D. Бассейн реки	204
Решение задачи 9 E. Ближайшее число	207
Вариант 1	207
Вариант 2	209
Решение задачи 9 F. Прямоугольное деление	211
Вариант 1	211
Вариант 2	212
Вариант 3	213

Тренировка 10 217

Решение задачи 10 A. Анти-QuickSort	217
Решение задачи 10 B. Строки Фибоначчи	219
Решение задачи 10 C. Игра в зачеркивание	221
Решение задачи 10 D. Граница многоугольника	224
Решение задачи 10 E. Путь спелеолога	225
Решение задачи 10 F. Дырявая ткань	226

Тренировка 11	230
Решение задачи 11 А. Последовательность	230
Вариант 1	230
Вариант 2	231
Решение задачи 11 В. Провода	232
Решение задачи 11 С. Палиндромы	232
Решение задачи 11 D. Круговая площадь	234
Вариант 1	234
Вариант 2	235
Решение задачи 11 Е. Гомер Симпсон	236
Решение задачи 11 F. Дробная арифметика	236
Тренировка 12	239
Решение задачи 12 А. Последовательность (2)	239
Вариант 1	239
Вариант 2	240
Решение задачи 12 В. Гирлянда	240
Вариант 1	241
Вариант 2	242
Вариант 3	242
Решение задачи 12 С. Головоломка умножения	243
Решение задачи 12 D. Точки в многоугольнике	244
Решение задачи 12 Е. Водопровод	244
Вариант 1	244
Вариант 2	245
Решение задачи 12 F. Химические реакции	245
Тренировка 13	252
Решение задачи 13 А. Двойная решетка	252
Решение задачи 13 В. Последовательность Фибоначчи	253
Вариант 1	253
Вариант 2	255
Решение задачи 13 С. Скобки (3)	256
Вариант 1	256
Вариант 2	258
Решение задачи 13 D. Центр тяжести	261
Решение задачи 13 Е. Сумма произведений	262
Решение задачи 13 F. Статическая сложность	262

Тренировка 14	266
Решение задачи 14 А. Марковский цикл	266
Вариант 1	266
Вариант 2	269
Решение задачи 14 В. Д-44	270
Решение задачи 14 С. Упаковка символов	272
Решение задачи 14 D. Пирамиды	274
Решение задачи 14 Е. Поле для крикета	275
Решение задачи 14 F. Электронная таблица	275
Тренировка 15	277
Решение задачи 15 А. Игра с калькулятором	277
Вариант 1	277
Вариант 2	278
Решение задачи 15 В. Площадь треугольника	278
Вариант 1	278
Вариант 2	280
Сравнение алгоритмов	281
Решение задачи 15 С. Формирование поезда	281
Вариант 1	281
Вариант 2	287
Решение задачи 15 D. Стена	289
Решение задачи 15 Е. Семечки	289
Решение задачи 15 F. Умножение многочленов	291
Приложение А. Характеристики встроенных типов	
Турбо Паскаля	294
Целые типы	294
Вещественные типы	294
Приложение Б. Процедуры длинной арифметики	295
Дополнительная информация о задачах	300
Тренировка 1	300
Тренировка 2	301
Тренировка 3	301
Тренировка 4	302
Тренировка 5	303
Тренировка 6	303

Тренировка 7	304
Тренировка 8	304
Тренировка 9	305
Тренировка 10	306
Тренировка 11	306
Тренировка 12	307
Тренировка 13	308
Тренировка 14	308
Тренировка 15	309
Использованные сокращения	309
Авторы задач и решений	313

Предисловие

Уровень сложности задач, предлагаемых на олимпиадах, с течением времени возрастает. Сейчас трудно рассчитывать на успешное выступление, если участник специально не готовился к олимпиаде. Основным методом подготовки является решение задач, предлагавшихся на олимпиадах прошлых лет.

Цель этой книги — познакомить учащихся с наиболее часто встречающимися типами олимпиадных задач. Книга идеальна для людей, знающих основы языка программирования, умеющих программировать, но еще не добивавшихся значительных успехов на олимпиадах. В настоящее время можно найти значительное количество олимпиадных задач, в том числе и на русском языке (например, <http://neerc.ifmo.ru/school>), но разборы задач публикуются далеко не всегда, а без них начинающему трудно прийти к идеям решения задач, предлагаемых на реальных соревнованиях. В нашей книге публикуются детальные разборы задач. В разборах кроме идей решения обсуждаются также некоторые особенности языков программирования, на которые обычно не обращают внимания, но которые очень важны при решении олимпиадных задач. На компакт-диске, прилагаемом к книге, есть тестирующая система, которая позволяет проверить правильность решения каждой из 90 задач.

Структура книги

Книга состоит из 15 тренировок, по 6 задач в каждой. Основной смысл разбиения задач на тренировки — поставить читателя книги в условия реальной олимпиады: есть набор задач, которые нужно решить за ограниченное время.

Тренировки можно условно объединить в циклы по пять. Номера задач сопровождаются буквами, каждой из которых соответствует определенная тема. Таким образом, на протяжении последовательных пяти тренировок даются задачи на одни и те же темы. Это позволяет глубже изучить их, кроме того, даже если задачи первой тренировки цикла были решены с помощью разборов, в дальнейшем человек вникает в тему и имеет больше шансов самостоятельно решить подобные задачи. Пользу же самостоятельного решения трудно переоценить.

Темы задач:

1–5 А — теория чисел;

1–5 В — перебор;

- 1–5 С — динамическое программирование;
- 1–5 D — геометрия;
- 1–5 E — длинная арифметика;
- 1–5 F — свободная тема;
- 6–10 A — быстрая сортировка;
- 6–10 B — динамическое программирование;
- 6–10 C — антагонистические игры;
- 6–10 D — геометрия;
- 6–10 E — поиск в ширину;
- 6–10 F — поиск в глубину;
- 11–15 A — последовательности;
- 11–15 B — метод половинного деления;
- 11–15 C — динамическое программирование;
- 11–15 D — геометрия;
- 11–15 E — сокращение перебора;
- 11–15 F — синтаксический анализ.

Некоторые комментарии к названиям тем:

- Длинная арифметика — это работа с числами, которые не могут быть представлены встроеными типами данных компьютера.
- Антагонистические игры — это игры, в которых выигрыш одного из участников равносильно проигрышу другого. То есть участники не могут получить больше выгоды при совместной игре, каждый играет за себя.
- Поиск в ширину — это способ обхода графа. Применительно к задачам на клетчатой бумаге существуют эквивалентные неофициальные олимпиадные термины — «метод лесного пожара» и «волновой метод». Название «метод лесного пожара» вызвано следующей аналогией: в каком-то квадрате леса произошло возгорание, и за единицу времени загораются квадраты, которые до этого были не тронуты огнем, но соседствовали с горящими.
- Поиск в глубину — другой способ обхода графа. Применительно к задачам на клетчатой бумаге употребляется эквивалентный термин «заливка». Инструмент «заливка» можно встретить в любом растровом графическом редакторе, например Windows Paint. Этот инструмент служит для изменения цвета связной одноцветной области.

Как работать с книгой

Рекомендуется следующий порядок работы с материалом книги: начать с первой тренировки и не переходить к следующей, не решив (возможно, с помощью чтения разборов) *все* задачи. В тренировке содержится несколько задач — это обеспечива-

ет возможность маневра, позволяет не заикливаться на одной задаче, а решать одновременно все. В цикле несколько тренировок (а не все задачи вперемишку), чтобы, прочитав разбор одной задачи, человек мог решить подобные задачи самостоятельно. Кроме того, материал задачи более поздней тренировки может предполагать знание материала задачи (умение ее решать) более ранней тренировки.

Работу над задачей рекомендуется организовать следующим образом:

1. Прочитать условие задачи (желательно прочитать условия всех задач тренировки сразу).
2. Попытаться решить задачу. Оценить, подойдет ли придуманный алгоритм под ограничения задачи.
3. Если совсем нет идей, то прочитать разбор (не раньше чем через три дня после прочтения условия задачи!).
4. Написать решение на языке программирования.
5. Протестировать решение. Если все тесты пройдены — замечательно, вы можете считать, что решили задачу. Прочитайте разбор (вдруг там есть интересные идеи) и приступайте к другой задаче. Если разбор сильно отличается от вашего решения, возможно, вы захотите решить задачу и другим способом. Если ваш способ проще описанного в книге, возможно, вы захотите поделиться им с друзьями. Тогда пришлите его автору книги.
6. Если решение не прошло тесты, нужно, не заглядывая в тесты жюри, самому попытаться написать тест, на котором программа «завалится». Если это удалось (повод для гордости! — вы теперь почти жюри), изменить решение соответствующим образом и перейти к шагу 5.
7. Если на всех составленных вами тестах ваше решение работает так, как, по вашему мнению, оно должно работать, посмотрите тест жюри. Измените решение так, чтобы пройти этот тест, и переходите к шагу 5.
8. Может случиться, что ваше решение нельзя изменить так, чтобы оно прошло все тесты. Тогда читайте решение жюри и переходите к шагу 4.

Изначально задачи предназначались для подготовки к олимпиадам по формуле Международного (студенческого) командного чемпионата мира по программированию ассоциации АСМ. Задача считается решенной, если она прошла *все* тесты жюри. Решения, не прошедшие хотя бы один тест, не засчитываются. Тестирование происходит в реальном времени, то есть решение, не прошедшее все тесты, может быть исправлено и снова отправлено на проверку. На проверку сдается файл исходного текста, то есть нестандартные опции компилятора нужно в нем написать явно.

Турбо Паскаль

В книге используется язык программирования Турбо Паскаль 7.0. Основная причина этого — почти на всех олимпиадах, задачи которых использованы в книге, другие компиляторы Паскаля не предоставлялись. Кроме того, ограничения на

память в Турбо Паскале более жесткие, чем в современных компиляторах, так что это несколько повышает уровень сложности задач и тем самым стимулирует изобретательность.

Технический момент

Ограничение на время работы программы указывается для компьютера Intel Celeron 400.

Наиболее простой способ организации работы с файлами, указанными в условии задачи, следующий: сразу после `begin` в главной программе нужно написать:

```
assign(input, 'filename.in');  
reset(input);  
assign(output, 'filename.out');  
rewrite(output);
```

где `filename` — название файлов задачи. После этого можно вводить данные из файла так, как вы вводили бы их с клавиатуры, и писать в файлы так, как вы писали бы на экран (не указывая в качестве первого параметра процедур `read` и `write` файловую переменную). Заметьте, переменные `input` и `output` — предопределенные, в своей программе их объявлять не следует.

Во всех разборах предполагается, что работа с файлами происходит именно описанным способом. Чтение из входного файла значения переменной `x` — просто `read(x)`, запись значения переменной `y` в выходной файл — просто `write(y)`, проверка конца строки входного файла — просто `eoln`.

Создание входного файла. Нажать F3, в открывшемся окне диалога Открыть файл написать имя входного файла (например, `filename.in`). Если файл не существовал, среда Турбо Паскаля откроет пустое окошко с нужным именем. Ввести данные. *Не забыть нажать F2 — сохранить изменения.* Если этого не сделать, программа не сможет прочитать эти данные.

Открытие входного и выходного файлов. Переместить курсор на строку в одиночных кавычках, содержащую имя файла, например `'primes.in'` в операторе `assign`. Нажать Ctrl+Enter. Этот способ не работает, если имя файла содержит знак «минус». Тогда нужно открывать файл через окно диалога, вызванное нажатием F3. Также можно открыть недавно использовавшийся файл, выбрав его имя в списке недавно открытых файлов в меню File. После изменения входных данных *не забыть сохранить файл* — в TP¹ автосохранение не сохраняет ничего, кроме исходного текста программы.

Переход между окнами входных, выходных данных и текстом программы осуществляется нажатием клавиши F6.

Закрывается окно нажатием клавиш Alt+F3.

¹ TP = Turbo Pascal.

Рекомендуемые настройки среды

В режиме отладки по умолчанию все проверки включить:

Options/Compiler...

Runtime errors

☒ Range checking

☒ Stack checking

☒ I/O checking

☒ Overflow checking

Запретить использовать сопроцессор без явного указания директивы `{ $N+.E- }` в программе:

Numeric processing

☐ 8087/80287

☒ Emulation

Если бы возле строки 8087/80287 стоял крестик, у вас программы, использующие, например, тип `extended`, без этой директивы компилировались бы, а в тестирующей системе — нет.

Автосохранение очень помогает, когда в ходе работы программы происходит аварийная ситуация:

Options/Environment/Preferences...

Auto save

☒ Editor files

☒ Environment

☒ Desktop

Стиль отступов, использовавшийся в примерах:

Options/Environment/Editor...

Editor options

☒ Use tab characters

☒ Optimal fill

Tab size 3

После изменения настроек нужно не забыть сохранить параметры — Options/Save TURBO.TP.

Компакт-диск

На прилагаемом к книге компакт-диске содержатся тесты и проверяющая система, а также условия задач книги в формате HTML и материалы реальных олимпиад.

Диск содержит:

- Материалы сайта <http://neerc.ifmo.ru/school/> — «Олимпиады по информатике. Сайт в Санкт-Петербурге». Сайт содержит множество задач, предлагавшихся на реальных олимпиадах. Сложность большинства задач высокая. Почти для всех задач есть тесты. Материал помещен на компакт-диск с согласия автора сайта — Андрея Станкевича.
- Материалы проекта «Задача в неделю» (<http://subscribe.ru/catalog/job.education.zvn>, <http://zvn.uriit.ru>, <http://attend.to/zvn>) за учебные 2002–2003 и 2003–2004 годы. Есть задачи различной сложности. Для большинства задач есть тесты, для значительной части есть разборы. Материал помещен на компакт-диск с согласия ведущего проекта — Александра Владимировича Алексева.
- Материалы сайта Северо-Восточного европейского полуфинала¹ чемпионата мира по программированию ACM (<http://neerc.ifmo.ru/>) — задачи прошлых лет и тесты к ним.
- Материалы Центрального четвертьфинала NEERC² (г. Рыбинск). Некоторые материалы были предоставлены членом жюри соревнования — Владимиром Николаевичем Пинаевым и публикуются с его согласия.
- Материалы сайта Дальневосточного четвертьфинала NEERC (<http://imcs.dvgu.ru/acm/>).
- Материалы сайта Чемпионата Урала 1999 года (<http://contest.ur.ru/ural99/>), организатором соревнований был Уральский государственный университет.
- Материалы сайта Чемпионата Урала 2000 года (<http://contest.psu.ru/>).
- Материалы Северного четвертьфинала NEERC 2002 года (<http://neerc.ifmo.ru/subregions/northern.html>).
- Материалы сайта Вологодских олимпиад школьников по информатике (<http://www.uni-vologda.ac.ru/olympiads/school/informatics/>).
- Материалы сайта Вологодской межвузовской олимпиады по программированию (<http://www.uni-vologda.ac.ru/olympiads/interuni/>).
- А также задачи и тесты некоторых других олимпиад, как на русском, так и на английском языках.

Установка проверяющей системы

Работа проверяющей системы тестировалась на компьютере Intel Celeron 400 с операционной системой Windows 95 и на компьютере Intel Celeron 2400 с операционной системой Windows XP (SP2).

Сначала проверяющую систему нужно установить — с компакт-диска она работать не будет. Скопируйте каталог `check` с компакт-диска в корень диска C:. Снимите атрибут «только для чтения» с файлов этого каталога. Это можно сделать следующим образом. Запустите сеанс MS-DOS (Пуск/Выполнить/command для Windows 95/98, Пуск/

¹ Россия относится к этому региону.

² NEERC — Northeastern European Regional Contest, Северо-Восточный европейский полуфинал.

Выполнить/cmd для Windows NT/2000/XP). Перейдите в каталог C:\check, сначала выполнив команду C:, затем cd \check. Выполните команду снятия атрибута «только для чтения» с файлов каталога: attrib -R /S. После этого можно завершить сеанс MS-DOS с помощью команды exit.

Если у вас операционная система Windows 95/98, скопируйте в папку Автозагрузка меню Пуск файл C:\check\95.bat. Если у вас операционная система Windows NT/2000/XP, скопируйте в папку Автозагрузка меню Пуск файл C:\check\NT.bat.

Скопируйте файлы tpc.exe и turbo.tpl из каталога исполнимых файлов Турбо Паскаля (обычно C:\TP\BIN или C:\BP\BIN) в каталог C:\check\lang\TP.

Перезагрузитесь. Проверяющая система установлена, диск T: стал отображать содержимое каталога C:\check.

При желании удалить систему нужно удалить bat-файл из папки Автозагрузка меню Пуск, после чего перезагрузиться и удалить каталог C:\check.

Работа с проверяющей системой

На тестирование принимается файл с исходным текстом программы. Файл должен иметь расширение pas и имя, соответствующее названию входного файла задачи. Например, в задаче «Простые числа» первой тренировки имя входного файла primes.in. Решение этой задачи, отправляемое на тестирование, должно иметь имя primes.pas.

Чтобы отправить решение на тестирование, нужно перейти в каталог, где лежит файл с исходным текстом программы и в командной строке выполнить команду

```
T:\checker <имя файла>
```

например

```
T:\checker primes.pas
```

Если тестирующая программа установлена правильно, система выдаст один из следующих ответов:

- **Compilation error** — ошибка компиляции. Если у вас решение в среде TP компилируется, попробуйте откомпилировать его компилятором командной строки (команда t:\lang\tp\tpc primes.pas).
- **Runtime error** — ошибка времени выполнения. Программа «сломалась» на тесте жюри.
- **Time limit exceeded** — превышение предела времени. Программа не уложилась в ограничение по времени. Ограничение по времени для вашего компьютера может не совпадать с ограничением, указанным в условии задачи для компьютера Intel Celeron 400. Проверяющая система устанавливает ограничение по времени индивидуально для каждого компьютера исходя из времени работы эталонного решения (решения жюри).
- **Output file absent** — отсутствует выходной файл. Программа успешно завершила работу, но выходной файл не был создан. Как правило, это связано с ошибкой написания его имени в программе.

- Presentation error — ошибка представления. Программа выдала ответ, но так ответ для этой задачи выглядеть не может. Например, требуется вывести число, а программа выдала букву.
- Wrong answer — неверный ответ.
- Accepted — принято. Программа прошла все тесты, на каждом уложилась в отведенное время и на каждом выдала правильный ответ.

История создания книги

Осенью 2002 года команда Вологодского государственного педагогического университета, капитаном которой был автор, заняла 9 место из 116 в полуфинале чемпионата мира по программированию АСМ (участвовали команды стран, входивших в СССР, за исключением Украины и Молдавии) и попала в финал. Таким образом, Вологодский государственный педагогический университет стал одним из 66 университетов, представленных в финале, и одним из 16, представляющих Европу.

Этому событию предшествовали полтора года тренировок, большинство из которых проводил автор. Основа материала книги (40 из 90 задач) появилась именно тогда.

С осени 2001 автор ведет факультатив по решению олимпиадных задач в Вологодском государственном естественно-математическом лицее (позднее Вологодский многопрофильный лицей). С осени 2002 автор занимается олимпиадными задачами со школьниками, посещающими занятия Центра дополнительного образования.

По окончании университета автору было предложено место преподавателя кафедры прикладной математики ВГПУ. Таким образом, осенью 2003 автор стал тренером по олимпиадному программированию в трех местах: в ВГПУ, в ВМЛ и в ЦДО.

Эта книга отражает курс, предлагаемый школьникам и студентам, только начинающим заниматься олимпиадами.

Результатом преподавательской деятельности автора можно считать следующие достижения учеников:

- Осень 2003 года, полуфинал (центральный регион) Всероссийской командной олимпиады школьников (по формуле АСМ): ВМЛ — 1 место, школьники Вологды (ЦДО) — 2 место.
- Осень 2004 года, полуфинал (центральный регион) Всероссийской командной олимпиады школьников: ВМЛ — 1 место, школьники Вологды — 2 место.
- Осень 2004 года, финал Всероссийской командной олимпиады школьников: ВМЛ — 12 место из 110 и диплом II степени.
- Весна 2003, Всероссийская олимпиада по информатике, у Вологодской области один диплом II степени и один диплом III степени¹.

¹ До 2003 года по крайней мере в течение двух лет у Вологодской области на Всероссийской олимпиаде школьников не было дипломов даже III степени.

- Весна 2004, Всероссийская олимпиада по информатике, у Вологодской области два диплома III степени.

Благодарности

Автор хотел бы выразить благодарность студентам и школьникам, решавшим приводимые в книге задачи, слушавшим их разборы, находившим ошибки в решениях и тестах жюри, а особенно тем, кто придумывал свои решения задач. Он благодарен также Сергею Залмановичу Свердлову, скрупулезно вычитавшему рукопись, и Александру Израилевичу Зейфману, предоставившему автору место работы, которое позволило написать эту книгу.

За помощь в создании тренировок 6–10 автор благодарит Никиту Рыбака, постоянно требовавшего все новые задачи для решения, и Александра Коротаева, который подвиг автора на написание разборов и проверил их реализацией.

За помощь в создании тренировок 11–15 автор благодарит всех, благодаря которым состоялся слет «Интеллект-2004», Александра Коротаева и Алексея Сергушичева, которые проверили разборы реализацией, а также Виктора Сергеевича Губу, сделавшего ценные комментарии относительно некоторых задач.

Автор благодарит Андрея Станкевича, Александра Владимировича Алексеева и Владимира Николаевича Пинаева, предоставивших дополнительные материалы для компакт-диска.

От издательства

Ваши замечания, предложения и вопросы отправляйте по адресу электронной почты comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

Подробную информацию о наших книгах вы найдете на веб-сайте издательства: <http://www.piter.com>.

ЧАСТЬ 1. ЗАДАЧИ

Тренировка 1

Задача 1 А. Простые числа

Вывести все простые числа от M до N включительно.

Ограничения: $2 \leq M \leq N \leq 300\,000$, время 6 с¹.

Ввод из файла `primes.in`². В первой строке находятся разделенные пробелом M и N .

Вывод в файл `primes.out`. Вывести числа в порядке возрастания, по одному в строке. Если между M и N включительно нет простых — вывести «Absent».

Примеры

Ввод 1 **Ввод 2**

2 5 4 4

Вывод 1 **Вывод 2**

2 Absent

3

5

Задача 1 В. Выражение

Даны N целых чисел $X_1, X_2, \dots, X_N$. Расставить между ними знаки «+» и «-» так, чтобы значение получившегося выражения было равно заданному целому S .

Ограничения: $2 \leq N \leq 24$, $0 \leq X_i \leq 50\,000\,000$, $-1\,000\,000\,000 \leq S \leq 1\,000\,000\,000$, время 3 с.

Ввод из файла `expr.in`. В первой строке находятся числа N и S . В следующей строке — N чисел через пробел.

Вывод в файл `expr.out`. Если получить требуемый результат невозможно, вывести «No solution», если можно — то вывести равенство. Если решение не единственное, вывести любое.

¹ Для какого компьютера указывается время, можно узнать в предисловии («Технический момент»).

² О простом способе работы с файлами можно узнать в предисловии («Технический момент»).

Примеры

Ввод 1	Ввод 2
3 10	2 100
15 25 30	10 10
Вывод 1	Вывод 2
15+25-30=10	No solution

Задача 1 C. Возрастающая подпоследовательность

Даны N целых чисел $X_1, X_2, \dots, X_N$. Требуется вычеркнуть из них минимальное количество чисел так, чтобы оставшиеся шли в порядке возрастания.

Ограничения: $1 \leq N \leq 10\,000$, $1 \leq X_i \leq 60\,000$, время 4 с.

Ввод из файла incseq.in. В первой строке находится число N . В следующей строке — N чисел через пробел.

Вывод в файл incseq.out. В первой строке выводится количество невычеркнутых чисел, во второй — сами невычеркнутые числа через пробел в исходном порядке. Если вариантов несколько, вывести любой.

Пример

Ввод
6
2 5 3 4 6 1

Вывод
4
2 3 4 6

Задача 1 D. Треугольник и точка

В декартовой системе координат на плоскости заданы координаты вершин треугольника и еще одной точки. Определить, принадлежит ли эта точка треугольнику.

Ограничения: координаты вершин — целые числа, для любой точки выполняются следующие условия: $-10\,000 \leq x, y \leq 10\,000$, время 1 с.

Ввод из файла tria-pt.in. В четырех строках находятся пары чисел — координаты точек. Числа в первых трех строках — это координаты вершин треугольника, в четвертой строке — координаты тестируемой точки.

Вывод в файл tria-pt.out. Вывести слово In, если точка находится внутри треугольника, или Out — если снаружи.

Примеры

Ввод 1	Ввод 2	Ввод 3	Ввод 4
0 0	0 0	0 0	0 0
100 0	100 0	100 0	100 0

0 100	0 100	0 100	0 100
100 100	10 10	50 50	0 0
Вывод 1	Вывод 2	Вывод 3	Вывод 4
Out	In	In	In

Задача 1 Е. Степень

Для натуральных чисел a и n вычислить a^n .

Ограничения: $1 \leq a \leq 9$, $1 \leq n \leq 7000$, время 5 с.

Ввод из файла power.in. В первой строке находятся разделенные пробелом a и n .

Вывод в файл power.out. Выводится одно число — результат без стоящих впереди нулей, стоящих впереди и позади пробелов.

Примеры

Ввод 1	Ввод 2
3 20	5 50
Вывод 1	Вывод 2
3486784401	88817841970012523233890533447265625

Задача 1 F. Покер

Даны пять целых чисел. Среди них:

- если одинаковы 5, то вывести «Impossible», иначе;
- если одинаковы 4, то вывести «Four of a Kind», иначе;
- если одинаковы 3 и 2, то вывести «Full House», иначе;
- если есть 5 последовательных, то вывести «Straight», иначе;
- если одинаковы 3, то вывести «Three of a Kind», иначе;
- если одинаковы 2 и 2, то вывести «Two Pairs», иначе;
- если одинаковы 2, то вывести One «Pair», иначе;
- вывести «Nothing».

Ограничения: все числа от 1 до 13 включительно, время 1 с.

Ввод из файла poker.in. В первой строке находятся пять чисел через пробел.

Вывод в файл poker.out. Выводится одна строка — результат анализа.

Примеры

Ввод 1	Ввод 2	Ввод 3	Ввод 4
1 3 9 3 2	1 5 5 4 4	1 5 2 4 3	10 11 12 13 1
Вывод 1	Вывод 2	Вывод 3	Вывод 4
One Pair	Two Pairs	Straight	Nothing

Тренировка 2

Задача 2 А. Простые числа (2)

Вывести все простые числа от M до N включительно.

Ограничения: $2 \leq M \leq N \leq 1\,000\,000$, время 6 с.

Ввод из файла `primes2.in`. В первой строке находятся разделенные пробелом M и N .

Вывод в файл `primes2.out`. Вывести числа в порядке возрастания, по одному в строке. Если между M и N включительно нет простых — вывести «Absent».

Примеры

Ввод 1 **Ввод 2**

2 5 4 4

Вывод 1 **Вывод 2**

2 Absent

3

5

Задача 2 В. Перестановки

Дана строка, состоящая из M попарно различных символов. Вывести все перестановки символов данной строки.

Ограничения: $2 \leq M \leq 8$, символы — буквы латинского алфавита и цифры, время 1 с.

Ввод из файла `permut.in`. В первой строке файла находится исходная строка.

Вывод в файл `permut.out`. Вывести в каждой строке файла по одной перестановке. Перестановки можно выводить в любом порядке. Повторений и строк, не являющихся перестановками исходной, быть не должно.

Примеры

Ввод 1 **Ввод 2**

AB IOX

Вывод 1 **Вывод 2**

AB XOI

BA OIX

IXO

XIO

OXI

IOX

Задача 2 С. Маршрут

В таблице из N строк и N столбцов клетки заполнены цифрами от 0 до 9. Требуется найти такой путь из клетки $(1, 1)$ в клетку (N, N) , чтобы сумма цифр в клетках, через которые он пролегает, была минимальной; из любой клетки ходить можно только вниз или вправо.

Ограничения: $2 \leq N \leq 250$, время 1 с.

Ввод из файла `route.in`. В первой строке находится число N . В следующих N строках содержатся по N цифр без пробелов.

Вывод в файл `route.out`. Выводятся N строк по N символов. Символ «решетка» (#) показывает, что маршрут проходит через эту клетку, а минус — что не проходит. Если путей с минимальной суммой цифр несколько, вывести любой.

Пример

Ввод

```
3
943
216
091
```

Вывод

```
#--
###
--#
```

Задача 2 D. Пересечение отрезков

Два отрезка на плоскости заданы целочисленными координатами своих концов в декартовой системе координат. Требуется определить, существует ли у них общая точка.

Ограничения: координаты целые и по модулю не превосходят 10 000, время 1 с.

Ввод из файла `segments.in`. В первой строке содержатся координаты первого конца первого отрезка, во второй — второго конца первого отрезка, в третьей и четвертой — координаты концов второго отрезка.

Вывод в файл `segments.out`. Выводится слово «Yes», если общая точка есть, или слово «No» — в противном случае.

Примеры

Ввод 1 **Ввод 2**

```
0 0      0 0
1 0      1 0
1 0      2 0
1 1      3 0
```

Вывод 1 **Вывод 2**

```
Yes      No
```

Задача 2 E. Длинная сумма

Даны два целых неотрицательных числа, M и N . Найти их сумму.

Ограничения: $0 \leq M, N < 10^{30\,000}$, время 1 с.

Ввод из файла `longsum.in`. В первой строке содержится M , во второй — N .

Вывод в файл `longsum.out`. В первой строке вывести сумму без пробелов и ведущих нулей.

Пример

Ввод

```
12345678901234567890123456789
1111111111111111111111111111
```

Вывод

```
13456790012345679001234567900
```

Задача 2 F. Спираль

Вывести квадрат, состоящий из $N \times N$ клеток, заполненных числами от 1 до N^2 по спирали (см. примеры).

Ограничения: $2 \leq N \leq 100$, время 1 с.

Ввод из файла `spiral.in`. В первой строке находится единственное число N .

Вывод в файл `spiral.out`. Выводится N строк по N чисел, разделенных пробелами. Не допускается начинать спираль в ином, кроме верхнего левого, углу, закручивать спираль против часовой стрелки или изнутри наружу.

Примеры

Ввод 1	Ввод 2	Ввод 3
3	4	5
Вывод 1	Вывод 2	Вывод 3
1 2 3	1 2 3 4	1 2 3 4 5
8 9 4	12 13 14 5	16 17 18 19 6
7 6 5	11 16 15 6	15 24 25 20 7
	10 9 8 7	14 23 22 21 8
		13 12 11 10 9

Тренировка 3

Задача 3 А. Разложение на простые множители

Вывести представление целого числа N в виде произведения простых чисел.

Ограничения: $2 \leq N \leq 2^{31} - 1$, время 2 с.

Ввод из файла pfactor.in. В первой строке находится единственное число N .

Вывод в файл pfactor.out. Выводится список чисел в порядке неубывания, разделенных знаком «*».

Примеры

Ввод 1	Ввод 2
--------	--------

5	30
---	----

Вывод 1	Вывод 2
---------	---------

5	2*3*5
---	-------

Задача 3 В. Перестановки (2)

Дана строка, состоящая из M символов. Вывести все перестановки символов данной строки.

Ограничения: $2 \leq M \leq 8$, символы — буквы латинского алфавита и цифры, время 1 с.

Ввод из файла permut2.in. В первой строке файла находится исходная строка.

Вывод в файл permut2.out. Вывести в каждой строке файла по одной перестановке. Перестановки можно выводить в любом порядке. Повторений и строк, не являющихся перестановками исходной, быть не должно.

Примеры

Ввод 1	Ввод 2
--------	--------

AB	122
----	-----

Вывод 1	Вывод 2
---------	---------

AB	122
----	-----

BA	212
----	-----

	221
--	-----

Задача 3 C. Копилка

Заданы вес E пустой копилки и вес F копилки с монетами. В копилке могут находиться монеты N видов; известны ценность P_i каждого вида монет и вес W_i одной монеты. Найти минимальную и максимальную суммы денег, которые могут находиться в копилке.

Ограничения: $1 \leq E \leq F \leq 10\,000$, $1 \leq N \leq 500$, $1 \leq P_i \leq 50\,000$, $1 \leq W_i \leq 10\,000$, все числа целые, время 2 с.

Ввод из файла `riggy.in`. В первой строке находятся числа E и F , во второй — число N , в следующих N строках — по два числа, P_i и W_i .

Вывод в файл `riggy.out`. Выводятся два числа через пробел — минимальная и максимальная суммы. Если копилка не может иметь точно заданный вес при условии, что она наполнена монетами заданных видов, — вывести `This is impossible..`

Примеры

Ввод 1	Ввод 2	Ввод 3
1000 1100	1000 1010	1000 2000
2	2	1
1 1	6 3	10 3
5 2	2 2	
Вывод 1	Вывод 2	Вывод 3
100 250	10 16	This is impossible.

Задача 3 D. Открытка и конверт

Даны размеры прямоугольных открытки и конверта. Требуется определить, поместится ли открытка в конверт.

Ограничения: размеры открытки и конверта — целые положительные числа, не превосходящие 100, время 1 с.

Ввод из файла `postcard.in`. В первой строке находятся размеры открытки, во второй — размеры конверта.

Вывод в файл `postcard.out`. Если открытку можно вложить в конверт, вывести «Possible», если нет — вывести «Impossible».

Пример

Ввод

```
1 10
9 9
```

Вывод

```
Possible
```

Задача 3 Е. Длинное произведение

Даны целые неотрицательные числа M и N . Найти $M \cdot N$.

Ограничения: $0 \leq M, N \leq 10^{2500}$, время 5 с.

Ввод из файла `longprod.in`. В первой строке находится число M , во второй — N .

Вывод в файл `longprod.out`. Вывести одно число — результат умножения.

Пример

Ввод

9876543210

1023456789

Вывод

10108215200126352690

Задача 3 F. Змейка

Вывести квадрат, состоящий из $N \times N$ ячеек, заполненных числами от 1 до N^2 «змейкой» (см. примеры).

Ограничения: $2 \leq N \leq 100$, время 1 с.

Ввод из файла `serpent.in`. В первой строке находится единственное число N .

Вывод в файл `serpent.out`. Выводится N строк по N чисел, разделенных пробелами. Не допускается начинать змейку в ином, кроме верхнего левого, углу или задавать ей другое направление.

Примеры

Ввод 1

3

Ввод 2

4

Ввод 3

5

Вывод 1

1 2 6

Вывод 2

1 2 6 7

Вывод 3

1 2 6 7 15

3 5 7

3 5 8 13

3 5 8 14 16

4 8 9

4 9 12 14

4 9 13 17 22

10 11 15 16

10 12 18 21 23

11 19 20 24 25

Тренировка 4

Задача 4 А. Совершенные числа

Число называется совершенным, если оно равно сумме всех своих делителей, меньших его самого. Требуется найти все совершенные числа от M до N .

Ограничения: M и N целые, $1 \leq M \leq N \leq 10^9$, $(N - M)\sqrt{N} \leq 10^7$, время 5 с.

Ввод из файла `perfect.in`. В первой строке находятся разделенные пробелом числа M и N .

Вывод в файл `perfect.out`. В каждой строке вывести по одному числу в порядке возрастания. Если совершенных чисел в промежутке нет, вывести `Absent`.

Примеры

Ввод 1 **Ввод 2**

6 6 4 5

Вывод 1 **Вывод 2**

6 Absent

Задача 4 В. Разложение на слагаемые

Вывести все представления натурального числа N суммой натуральных чисел. Перестановка слагаемых нового способа представления не дает.

Ограничения: $2 \leq N \leq 40$, время 2 с.

Ввод из файла `decomp.in`. В первой строке находится единственное число N .

Вывод в файл `decomp.out`. В каждой строке выводится одно из представлений. В сумме слагаемые разделяются знаком «+».

Пример

Ввод

4

Вывод

1+1+1+1

1+2+1

1+3

2+2

Задача 4 С. Гангстеры

N гангстеров собираются в ресторан. i -й гангстер приходит в момент времени T_i , богатство этого гангстера — P_i . Дверь ресторана имеет $K + 1$ степень открытости, они обозначаются целыми числами из интервала $[0, K]$. Степень открытости двери может изменяться на единицу в единицу времени, то есть дверь может открыться на единицу, закрыться на единицу или остаться в том же положении, в котором была. В начальный момент времени дверь закрыта (положение 0). i -й гангстер заходит в ресторан, только если дверь открыта специально для него, то есть когда степень открытости двери соответствует его полноте S_i . Если в момент, когда гангстер подходит к ресторану, степень открытости двери не соответствует его полноте, он уходит и больше не возвращается. Ресторан работает в интервале времени $[0, T]$. Требуется собрать гангстеров с максимальным суммарным богатством в ресторане, открывая и закрывая дверь соответствующим образом.

Ограничения: $1 \leq N \leq 100$, $1 \leq K \leq 100$, $1 \leq T \leq 30\,000$, $0 \leq T_i \leq T$, $1 \leq P_i \leq 300$, $1 \leq S_i \leq K$, все числа целые, время 2 с.

Ввод из файла `gangster.in`. В первой строке находятся числа N, K, T , во второй — $T_1, T_2, \dots, T_N$, в третьей — $P_1, P_2, \dots, P_N$, в четвертой — $S_1, S_2, \dots, S_N$. Числа в строках разделены пробелами.

Вывод в файл `gangster.out`. Вывести одно число — максимальное суммарное богатство гангстеров, попавших в ресторан. Если зайти не удалось никому, вывести 0.

Примеры

Ввод 1	Ввод 2
4 10 20	2 17 100
10 16 8 16	5 0
10 11 15 1	50 33
10 7 1 8	6 1
Вывод 1	Вывод 2
26	0

Задача 4 D. Площадь многоугольника

Многоугольник на плоскости задан целочисленными координатами своих N вершин в декартовой системе координат. Требуется найти площадь многоугольника. Стороны многоугольника не соприкасаются (за исключением соседних — в вершинах) и не пересекаются.

Ограничения: $3 \leq N \leq 50\,000$, координаты вершин целые и по модулю не превосходят 20 000, время 1 с.

Ввод из файла `area.in`. В первой строке находится число N . В следующих N строках находятся пары чисел — координаты точек. Если соединить точки в данном порядке, а также первую и последнюю точки, получится заданный многоугольник.

Вывод в файл `area.out`. Вывести одно число — площадь многоугольника. Его следует округлить до ближайшего числа с одной цифрой после запятой.

Примеры

Ввод 1	Ввод 2
4	4
5 0	0 4
0 5	0 0
-5 0	3 0
0 -5	1 1
Вывод 1	Вывод 2
50.0	3.5

Задача 4 Е. Деление длинного числа на короткое

Даны целое неотрицательное число M и целое положительное число N . Найти $M \div N$ и $M \bmod N$.

Ограничения: $0 \leq M \leq 10^{60\,000}$, $1 \leq N \leq 1\,000\,000$, время 1 с.

Ввод из файла `divshort.in`. В первой строке находится число M , во второй — N .

Вывод в файл `divshort.out`. В первой строке вывести значение выражения $M \div N$, во второй — выражения $M \bmod N$.

Пример

Ввод
12345678901234567890
1000
Вывод
12345678901234567
890

Задача 4 F. Скобки

Дана последовательность из N круглых, квадратных и фигурных скобок. Выяснить, можно ли добавить в нее цифры и знаки арифметических действий так, чтобы получилось правильное арифметическое выражение.

Ограничения: $1 \leq N \leq 100\,000$, время 1 с.

Ввод из файла `bracket.in`. В первой строке находится число скобок N , во второй — N символов из набора `(, ), [, ], {, }`.

Вывод в файл `bracket.out`. Выводится слово `Yes`, если получить правильное арифметическое выражение можно, или `No`, если нельзя.

Примеры

Ввод 1	Ввод 2
6	24
([()])	{[(())({})][{}]{}}[[]]
Вывод 1	Вывод 2
No	Yes

Тренировка 5

Задача 5 А. Дружественные числа

Два различных натуральных числа называются дружественными, если первое из них равно сумме делителей второго числа, за исключением самого второго числа, а второе равно сумме делителей первого числа, за исключением самого первого числа. Требуется найти все пары дружественных чисел, оба из которых принадлежат промежутку от M до N .

Ограничения: $1 \leq M \leq N \leq 1\,000\,000$, все числа целые, время 1 с.

Ввод из файла `friendly.in`. В первой строке находятся числа M и N .

Вывод в файл `friendly.out`. В каждой строке вывести по паре чисел через пробел. Первое число пары должно быть меньше второго. Строки должны быть отсортированы в порядке возрастания первого числа пары. Если пар дружественных чисел в промежутке нет, вывести «Absent».

Примеры

Ввод 1	Ввод 2	Ввод 3
200 300	200 250	185000 205000
Вывод 1	Вывод 2	Вывод 3
220 284	Absent	185368 203432
		196724 202444

Комментарий к примеру 1

$220 = 1 + 2 + 4 + 71 + 142$ (все делители числа 284);

$284 = 1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110$ (все делители числа 220).

Задача 5 В. Скобки (2)

Вывести все правильные скобочные выражения длиной N , состоящие из круглых и квадратных скобок.

Ограничения: $1 \leq N \leq 14$, N — четное, время 2 с.

Ввод из файла `bracket2.in`. В первой строке находится единственное число N .

Вывод в файл `bracket2.out`. Каждое выражение выводится в отдельной строке.

Пример

Ввод

4

Вывод

```
((  
[[  
[(  
()[  
[](  
()(  
([  
[[[
```

Задача 5 C. Маршрут (2)

Дана матрица $N \times N$, заполненная положительными числами. Путь по матрице начинается в левом верхнем углу. За один ход можно пройти в соседнюю по вертикали или горизонтали клетку (если она существует). Нельзя ходить по диагонали, нельзя оставаться на месте. Требуется найти максимальную сумму чисел, стоящих в клетках по пути длиной K (клетку можно посещать несколько раз).

Ограничения: $2 \leq N \leq 100$, элементы матрицы имеют значения от 1 до 9999, $1 \leq K \leq 2000$, все числа целые, время 5 с.

Ввод из файла route2.in. В первой строке находятся разделенные пробелом числа N и K . Затем идут N строк по N чисел в каждой.

Вывод в файл route2.out. Вывести одно число — максимальную сумму.

Пример

Ввод

```
5 7  
1 1 1 1 1  
1 1 3 1 9  
1 1 6 1 1  
1 1 3 1 1  
1 1 1 1 1
```

Вывод

```
21
```

Задача 5 D. Выпуклая оболочка

На плоскости заданы N точек своими декартовыми координатами. Найти минимальный периметр многоугольника, содержащего все эти точки. Гарантируется, что искомый многоугольник имеет ненулевую площадь.

Ограничения: $3 \leq N \leq 1000$, $-10\,000 \leq x_i, y_i \leq 10\,000$, все числа целые, все точки различны, время 2 с.

Ввод из файла conhull.in. В первой строке находится число N , далее — N строк с парами координат.

Вывод в файл `conhull.out`. Вывести одно число — длину периметра с одним знаком после запятой.

Пример

Ввод

```
5
1 0
0 1
-1 0
0 -1
0 0
```

Вывод

```
5.7
```

Задача 5 Е. Системы счисления

Дано целое неотрицательное число в I -ричной системе счисления. Вывести это число в J -ричной системе счисления.

Ограничения: $2 \leq I, J \leq 36$, для представления цифр 10...35 используются прописные латинские буквы A...Z соответственно, число разрядов исходного числа не превышает 1000, время 5 с.

Ввод из файла `scale.in`. В первой строке находятся числа I и J (в десятичной системе счисления), во второй строке — число для перевода.

Вывод в файл `scale.out`. Вывести искомое число. Если число начинается с буквы, перед ней не должно быть нуля.

Пример

Ввод

```
10 36
29234652
```

Вывод

```
HELLO
```

Задача 5 F. День рождения

Заданы день и месяц рождения, а также текущие день, месяц и год. Определить, сколько дней осталось до дня рождения.

Примечание. Високосные годы — это те, номер которых делится на 400, а также те, номер которых делится на 4, но не делится на 100.

Ограничения: год от 1920 до 3000, месяц — от 1 до 12, день — от 1 до числа дней в месяце, время 1 с.

Ввод из файла `birthday.in`. В первой строке находятся разделенные пробелами день и месяц рождения, во второй — разделенные пробелами текущие день, месяц и год.

Вывод в файл birthday.out. Вывести число дней, оставшихся до дня рождения.

Примеры

Ввод 1

19 04

19 04 2002

Вывод 1

0

Ввод 2

05 05

19 04 2002

Вывод 2

16

Ввод 3

29 02

28 02 2001

Вывод 3

1096

Тренировка 6

Задача 6 А. Закраска прямой

На числовой прямой окрасили N отрезков. Известны координаты левого и правого концов каждого отрезка (L_i и R_i). Найти длину окрашенной части числовой прямой.

Ограничения: L_i и R_i — целые, $-1\,000\,000\,000 \leq L_i \leq R_i \leq 1\,000\,000\,000$, $1 \leq N \leq 15\,000$, время 1 с.

Ввод из файла `cover.in`. В первой строке находится число N , в следующих N строках — пары L_i и R_i .

Вывод в файл `cover.out`. Вывести одно число — длину окрашенной части прямой.

Примеры

Ввод 1 Ввод 2

2 1

1 3 10 10

2 4

Вывод 1 Вывод 2

3 0

Задача 6 В. Суммы

Дано N целых чисел $A_1, A_2, \dots, A_N$. Требуется найти количество различных значений сумм вида $k_1A_1 + k_2A_2 + \dots + k_NA_N$.

Ограничения: $1 \leq N \leq 500$, $0 \leq A_i \leq 100$, $0 \leq k_i \leq 1$, все числа целые, время 2 с.

Ввод из файла `sums.in`. В первой строке находится число N , во второй — $A_1, A_2, \dots, A_N$ через пробел.

Вывод в файл `sums.out`. Вывести одно число — количество различных значений сумм.

Примеры

Ввод 1 Ввод 2 Ввод 3

3 3 5

1 1 2 1 3 2 49 100 98 49 0

Вывод 1 Вывод 2 Вывод 3

5 7 10

Задача 6 С. Игра «Даты»

Играют двое. Задается какая-то дата 2004 года. Каждый игрок на своем ходе называет более позднюю дату, увеличивая на 1 или 2 либо день в месяце, либо месяц, но не то и другое сразу. При этом сочетание дня и месяца должно оставаться датой. Игрок, назвавший 31 декабря, проигрывает. Оба играют наилучшим образом. Исходя из заданной даты, вывести, кто выиграет.

Ограничения: месяц от 1 до 12, день от 1 до числа дней в месяце, даты «31 декабря» во входных данных нет, время 1 с.

Ввод из файла `dategame.in`. В первой строке находятся разделенные пробелом числа, обозначающие день и месяц.

Вывод в файл `dategame.out`. Вывести 1, если выигрывает первый (начинающий) игрок, или 2 — в противном случае.

Примеры

Ввод 1	Ввод 2	Ввод 3
30 12	29 12	29 11
Вывод 1	Вывод 2	Вывод 3
2	1	2

Задача 6 D. Площадь прямоугольников

Дано N прямоугольников со сторонами, параллельными осям координат. Требуется определить площадь фигуры, образованной объединением данных прямоугольников.

Ввод из файла `rectarea.in`. В первой строке находится число прямоугольников — N . Затем идут N строк, содержащих по 4 числа: x_1, y_1, x_2, y_2 — координаты двух противоположных углов прямоугольника.

Ограничения: $1 \leq N \leq 100$, координаты целые и по абсолютному значению не превосходят 10 000, время 3 с.

Вывод в файл `rectarea.out`. Вывести одно число — площадь фигуры.

Пример

```
Ввод
2
1 1 3 3
2 2 4 4
Вывод
7
```

Задача 6 E. Lines

В таблице из N строк и N столбцов некоторые клетки заняты шариками, другие свободны. Выбран шарик, который нужно переместить, и место, куда его нужно

переместить. Выбранный шарик за один шаг перемещается в соседнюю по горизонтали или вертикали свободную клетку. Требуется выяснить, возможно ли переместить шарик из начальной клетки в заданную, и если возможно, то найти путь из наименьшего количества шагов.

Ограничения: $2 \leq N \leq 40$, время 1 с.

Ввод из файла `lines.in`. В первой строке находится число N , в следующих N строках — по N символов. Символом точки обозначена свободная клетка, латинской заглавной O — шарик, $@$ — исходное положение шарика, который должен двигаться, латинской заглавной X — конечное положение шарика.

Вывод в файл `lines.out`. В первой строке выводится Y , если движение возможно, или N , если нет. Если движение возможно, далее следует N строк по N символов — как и на вводе, но буква X , а также все точки по пути заменяются плюсами.

Примеры

Ввод 1	Ввод 2	Ввод 3
5	5	5
....X	..X..	...X.
.0000		
.....	00000	0.000
0000.		
@....	..@..	@
Вывод 1	Вывод 2	Вывод 3
Y	N	Y
+++++		..++.
+0000		..+..
+++++		0+000
0000+		..++++
@++++		@

Задача 6 F. Покраска лабиринта

Лабиринт представляет собой квадрат, состоящий из $N \times N$ сегментов. Каждый из сегментов может быть либо пустым, либо заполненным камнем. Гарантируется, что левый верхний и правый нижний сегменты пусты. Лабиринт обнесен сверху, снизу, слева и справа стенами, оставляющими свободными только левый верхний и правый нижний углы. Директор лабиринта решил покрасить стены лабиринта, видимые изнутри (рис. 6.1). Помогите ему рассчитать количество краски, необходимой для этого.

Ограничения: $3 \leq N \leq 33$, размер сегмента 3×3 м, высота стен 3 м, время 1 с.

Ввод из файла `paintlab.in`. В первой строке находится число N , затем идут N строк по N символов: точка обозначает пустой сегмент, решетка — сегмент со стеной.

Вывод в файл `paintlab.out`. Вывести одно число — площадь видимой части внутренних стен лабиринта в квадратных метрах.

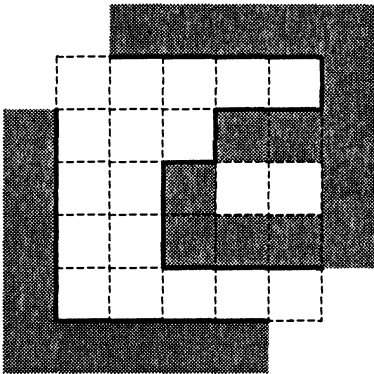


Рис. 6.1. Вид на лабиринт сверху

Пример

Ввод

5
.....
...##
..#..
...###
.....

Вывод

198

Тренировка 7

Задача 7 А. Упорядоченные дроби

Вывести в порядке возрастания все несократимые дроби, заключенные между 0 и 1, знаменатели которых не превышают N .

Ограничения: $2 \leq N \leq 255$, время 1 с.

Ввод из файла `ordfrac.in`. В первой строке находится единственное число N .

Вывод в файл `ordfrac.out`. В каждой строке выводится дробь.

Пример

Ввод

5

Вывод

1/5

1/4

1/3

2/5

1/2

3/5

2/3

3/4

4/5

Задача 7 В. Сообщение

В сообщении, состоящем из одних русских букв и пробелов, каждую букву заменили ее порядковым номером в русском алфавите (А — 1, Б — 2, ..., Я — 33), а пробел — нулем. Требуется по заданной последовательности цифр найти количество исходных сообщений, из которых она могла получиться.

Ограничения: цифр не более 100, время 1 с.

Ввод из файла `message.in`. В первой строке содержится последовательность цифр.

Вывод в файл `message.out`. Вывести одно число.

Пример

Ввод

1025

Вывод

4

Задача 7 C. Игра умножения

Слава и Оля играют в игру умножения — умножают целое число P на одно из чисел от 2 до 9. Слава всегда начинает с $P = 1$, делает умножение, затем число умножает Оля, затем Слава и т. д. Перед началом игры им задают случайное число N , и победителем считается тот, кто первым получит $P \geq N$. Определить, кто выиграет при заданном N , если оба играют наилучшим образом.

Ограничения: $2 \leq N \leq 4\,294\,967\,295$, время 1 с.

Ввод из файла `multgame.in`. В первой строке находится единственное число N .

Вывод в файл `multgame.out`. Выводится одна строка — `Stan wins.`, если победит Слава, или `Ollie wins.`, если победит Оля.

Примеры

Ввод 1	Ввод 2	Ввод 3
162	17	34012226
Вывод 1	Вывод 2	Вывод 3
Stan wins.	Ollie wins.	Stan wins.

Задача 7 D. Прямая и квадраты

В прямоугольной декартовой системе координат прямая задана двумя принадлежащими ей точками $(0, W)$ и $(100N, E)$. Также заданы N^2 квадратов со сторонами, параллельными осям координат. Квадрат $S_{i,j}$ имеет координаты углов $(100i, 100j)$ и $(100i + 100, 100j + 100)$, $i, j = 1, 2, \dots, N$. Требуется найти количество квадратов, имеющих общую точку с прямой.

Ограничения: $1 \leq N \leq 100$, $0 \leq W, E \leq 100N$, все числа целые, время 1 с.

Ввод из файла `sqline.in`. В первой строке находятся три целых числа, N , W и E , разделенных пробелами.

Вывод в файл `sqline.out`. Вывести одно число — количество квадратов.

Пример

Ввод
3 150 50
Вывод
4

Задача 7 E. Lines (2)

В таблице из N строк и N столбцов некоторые клетки заняты шариками, другие свободны. Выбран шарик, который нужно переместить, и место, куда его нужно

переместить. Выбранный шарик за один шаг перемещается в соседнюю по горизонтали или вертикали свободную клетку. Требуется выяснить, возможно ли переместить шарик из начальной клетки в заданную, и если возможно, то найти путь из наименьшего количества шагов.

Ограничения: $2 \leq N \leq 250$, время 1 с.

Ввод из файла `lines2.in`. В первой строке находится число N , в следующих N строках — по N символов. Символом точки обозначена свободная клетка, латинской заглавной O — шарик, $@$ — исходное положение шарика, который должен двигаться, латинской заглавной X — конечное положение шарика.

Вывод в файл `lines2.out`. В первой строке выводится Y , если движение возможно, или N , если нет. Если движение возможно, далее следует N строк по N символов — как и на вводе, но X , а также все точки по пути заменяются плюсами.

Примеры

Ввод 1	Ввод 2	Ввод 3
5	5	5
...X	..X..	...X.
.0000		
.....	00000	0.000
0000.		
@....	..@..	@
Вывод 1	Вывод 2	Вывод 3
Y	N	Y
+++++		..++.
+0000		..++..
+++++		0+000
0000+		..++++
@++++		@

Задача 7 F. Удаление клеток

Из прямоугольного листа клетчатой бумаги (M строк, N столбцов) удалили некоторые клетки. На сколько кусков распадется оставшаяся часть листа? Две клетки не распадаются, если они имеют общую сторону.

Ограничения: $1 \leq M, N \leq 100$, время 1 с.

Ввод из файла `remsquar.in`. В первой строке находятся числа M и N , в следующих M строках — по N символов. Если клетка не была вырезана, этому соответствует знак $\#$, если вырезана — точка.

Вывод в файл `remsquar.out`. Вывести одно число.

Пример**Ввод**

4 8

#.#.#.#

.....##

#.###.##

##.##.##

Вывод

6

Тренировка 8

Задача 8 А. Несоставляемое число

Даны N натуральных чисел. Найти минимальное натуральное число, не представимое суммой никаких из этих чисел, если в эту сумму каждое исходное число может входить не более одного раза.

Ограничения: $1 \leq N \leq 10\,000$, значения исходных чисел от 1 до 1 000 000 000, время 1 с.

Ввод из файла posum.in. В первой строке находится число N , в следующих N строках — по одному натуральному числу.

Вывод в файл posum.out. Вывести одно число.

Примеры

Ввод 1	Ввод 2
4	4
1	1
1	2
1	4
5	8
Вывод 1	Вывод 2
4	16

Задача 8 В. SMS

Сообщения SMS сотового телефона MOBILA (рис. 8.1) составлены из прописных латинских букв. Если буква первая на кнопке, нужно нажать эту кнопку один раз, чтобы добавить букву в сообщение. Если буква вторая — нужно нажать кнопку дважды и т. д. Так, чтобы набрать слово «SMS», нужно нажать

(PQRS)(PQRS)(PQRS)(PQRS)(MNO)(PQRS)(PQRS)(PQRS)(PQRS)

Чтобы ввести две буквы, находящиеся на одной кнопке, нужно между нажатиями клавиши сделать паузу. Например, чтобы ввести сообщение «AA», нужно нажать (ABC)(пауза)(ABC)

Если на кнопке три буквы, то, как только такая кнопка нажата три раза, последняя буква добавляется в сообщение немедленно, а следующие нажатия той же кнопки относятся к следующей букве сообщения. Аналогично, если на кнопке четыре бук-

вы, то после четырех нажатий в сообщение будет добавлена последняя буква. То есть последовательность нажатий

(ABC)(ABC)(ABC)(ABC)(пауза)(ABC)

соответствует сообщению «СAA». К сожалению, сотовые телефоны этой модели давно не производятся, и остался только один такой телефон. Он может произвольно вставлять и игнорировать паузы во время ввода сообщения, что может привести к некоторым изменениям в сообщениях. Например, введя MOSCOWQUARTER-FINAL, можно получить вместо этого OMSCMNWQTTARTERPDEINAL. Вы получили SMS-сообщение и знаете, что оригинальное сообщение содержало N букв. Чтобы определить вероятность угадывания оригинального сообщения, найдите число возможных сообщений, которые могли превратиться в то, которое вы получили.

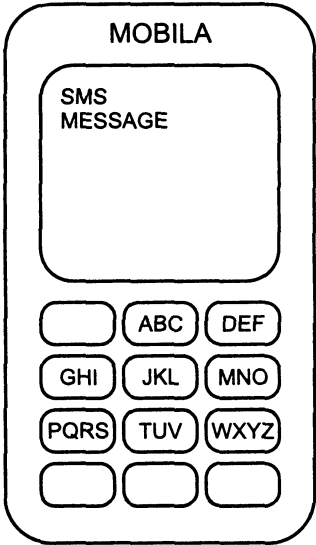


Рис. 8.1. Раскладка клавиатуры мобильного телефона

Ограничения: $1 \leq N \leq 80$, полученное сообщение состоит только из прописных латинских букв, длина полученного сообщения — от 1 до 80 букв, время 2 с.

Ввод из файла `sms.in`. В первой строке задана длина оригинального сообщения N . Вторая строка содержит полученное SMS-сообщение.

Вывод в файл `sms.out`. Вывести число сообщений из N букв, которые, будучи набранными на этом телефоне, могут превратиться в данное сообщение.

Примеры

Ввод 1	Ввод 2	Ввод 3
4	2	80
МАМА	WWW	QUARTERFINAL
Вывод 1	Вывод 2	Вывод 3
1	2	0

Задача 8 С. Дерево игры

Игра для двух игроков определяется ее деревом. Соперники делают ходы по очереди. Первый игрок начинает игру. Игра кончается или вничью, или победой одного из игроков. Листья дерева этой игры могут иметь значения, равные одному из трех чисел: +1 — победа первого игрока, -1 — победа второго игрока, 0 — ничья. Ваша задача — определить, кто выиграет, если оба противника следуют правильной стратегии.

Ввод из файла `gametree.in`. Узлы дерева пронумерованы последовательными целыми числами. Корень дерева всегда имеет номер 1. Первая строка входного файла содержит целое N — число узлов в дереве игры. Следующая $N - 1$ строка описывает узлы — одна строка для каждого узла (за исключением первого). Вторая строка содержит описание второго узла дерева, третья — третьего узла и т. д. Если узел является листом, первый символ строки — `L`, затем идет пробел, затем номер родительского узла, еще пробел и результат игры (+1 — победа первого игрока, -1 — победа второго, 0 — ничья). Если узел внутренний, то строка содержит N — первый символ, затем пробел и номер родительского узла.

Вывод в файл `gametree.out`. Выводится +1, если выигрывает первый игрок, -1, если второй, и 0 — в случае ничейного исхода.

Ограничения: $2 \leq N \leq 1000$, время 1 с.

Примеры

Ввод 1	Ввод 2
7	7
N 1	N 1
N 1	N 1
L 2 -1	L 2 -1
L 2 +1	L 2 +1
L 3 +1	L 3 +1
L 3 +1	L 3 0
Вывод 1	Вывод 2
+1	0

Ввод 3 (рис. 8.2)

```

18
N 1
N 1
N 2
L 2 +1
N 3
L 3 +1
L 3 +1
L 4 -1
L 4 +1

```

N 4
N 6
L 6 -1
L 6 -1
L 11 -1
L 11 +1
L 12 +1
L 12 -1
Вывод 3
+1

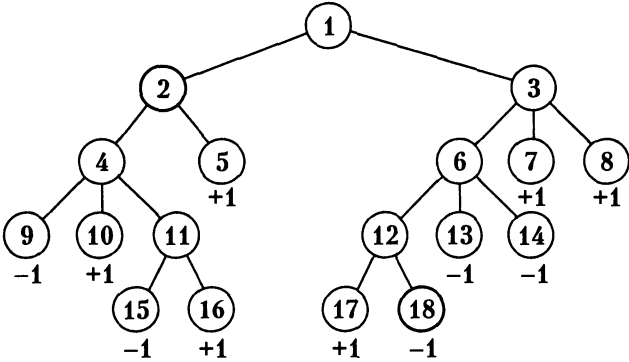


Рис. 8.2. Пример дерева игры

Задача 8 D. Дуга на сфере

На поверхности планеты, являющейся шаром с радиусом R , заданы две точки своими широтой и долготой. Найти минимальную длину пути по поверхности этой планеты из одной точки в другую.

Ограничения: широта — в градусах от -90 до 90 , долгота — в градусах от -180 до 180 , $100 \leq R \leq 10\,000$, все числа вещественные, время 1 с.

Ввод из файла `spherarc.in`. В первой строке находится число R , во второй строке заданы широта и долгота первой точки, в третьей строке — широта и долгота второй точки.

Вывод в файл `spherarc.out`. Вывести длину пути с двумя знаками после запятой.

Пример

Ввод
4000
45 120
0 120
Вывод
3141.59

Задача 8 Е. Путь коня

Дана шахматная доска, состоящая из $N \times N$ клеток; несколько из них вырезано. Провести ходом коня через невырезанные клетки путь минимальной длины из одной заданной клетки в другую.

Ограничения: $2 \leq N \leq 50$, время 1 с.

Ввод из файла knightw.in. В первой строке задано число N . В следующих N строках содержится по N символов. Символом # обозначена вырезанная клетка, точкой — невырезанная клетка, @ — заданные клетки (таких символов два).

Вывод в файл knightw.out. Если путь построить невозможно, вывести «Impossible», в противном случае вывести такую же карту, как и на входе, но пометить все промежуточные положения коня символом @.

Примеры

Ввод 1	Ввод 2	Ввод 3
5	5	5
.....	@...@.	@.....
..@@..	..##..	..#...
.....		..#...
.....		
.....		@
Вывод 1	Вывод 2	Вывод 3
...@.	@...@.	Impossible
..@@..	..##..	
....@	..@...@	
.....	..@...	
.....	@.....	

Задача 8 F. Грядки

Прямоугольный садовый участок шириной N и длиной M метров разбит на квадраты со стороной 1 м. На этом участке вскопаны грядки. Грядкой называется совокупность квадратов, удовлетворяющая таким условиям:

- из любого квадрата этой грядки можно попасть в любой другой квадрат этой же грядки, последовательно переходя по грядке из квадрата в квадрат через их общую сторону;
- никакие две грядки не пересекаются и не касаются друг друга ни по вертикальной, ни по горизонтальной сторонам квадратов (касание грядок углами квадратов допускается).

Подсчитайте количество грядок на садовом участке.

Ограничения: $1 \leq N, M \leq 200$, время 1 с.

Ввод из файла `beds.in`. В первой строке находятся числа N и M через пробел, далее идут N строк по M символов. Символ `#` обозначает территорию грядки, точка соответствует незанятой территории. Других символов в исходном файле нет.

Вывод в файл `beds.out`. Вывести одно число — количество грядок на садовом участке.

Пример

Ввод

5 10

##.....#.

.#.#...#.

.###...#.

..##...#.

.....#.

Вывод

3

Тренировка 9

Задача 9 А. Диалог компьютеров

Три компьютера соединены сетью. Один из них — сервер, два других — клиенты. На сервере есть несколько файлов. Полные имена файлов, состоящие из двух частей (имя и расширение), различны. Оба клиента знают полные имена всех файлов, находящихся на сервере. Сервер выбирает один из своих файлов и посылает его имя одному из клиентов, а расширение — второму.

Затем клиенты начинают общаться друг с другом, пытаясь определить, какой файл был выбран сервером (они хотят узнать полное имя файла). Однако клиенты вынуждены общаться очень ограниченным способом. Они по очереди посылают сообщения друг другу, но могут сказать только, что не знают полного имени файла. Если клиент не знает полного имени выбранного файла, он может послать другому клиенту сообщение, говорящее: «Я не знаю полного имени файла». Клиенты чередуются, посылая только это сообщение туда и обратно. Так продолжается до тех пор, пока один из клиентов не узнает полное имя файла или они не решат закончить диалог. Клиент, получивший первую часть полного имени файла, всегда ждет, что второй клиент пошлет первое сообщение.

Пусть вы знаете все полные имена файлов, находящихся на сервере, и слушаете разговор клиентов. Основываясь на этой беседе, вы должны определить набор файлов, которые могли быть выбраны сервером. Файлы в этом наборе называются файлами-кандидатами.

Ввод из файла `dialogue.in`. В первой строке находятся два целых числа, N и M , разделенные пробелом: N — число файлов на сервере, M — число сообщений, посланных клиентами, пытающимися определить полное имя файла.

Каждая из следующих N строк содержит одно полное имя файла. Полное имя файла дано в стиле, аналогичном формату 8.3 MS-DOS. Каждое полное имя представлено в форме `имя.расширение`, где `имя`, и `расширение` состоят только из заглавных латинских букв и цифр. Имя всегда имеет от одного до восьми символов. Расширение имеет до трех символов и может быть пусто. Если расширение пусто, разделяющая точка может быть опущена.

Каждое полное имя файла появляется во входном файле не более одного раза.

Ограничения: $1 \leq N \leq 1000$, $1 \leq M \leq 100$, время 3 с.

Вывод в файл `dialogue.out`. В первой строке выводится число файлов-кандидатов для данных набора файлов и числа сообщений между клиентами. Выводится 0, если файлы-кандидаты отсутствуют.

В следующих строках находятся полные имена файлов-кандидатов, каждое в отдельной строке. Они должны идти в том же порядке и в том же написании, что и во входном файле. Это означает, что если разделяющая точка в названии конкретного файла была опущена во входном файле, то она должна быть опущена и в выводе, и наоборот. Файл нельзя упоминать более одного раза.

Пример

Ввод

```
19 2
LICENCE.TMP
WIN32.LOG
FILEID.
PSTOTEXT.TXT
GSVIEW32.EXE
GSVIEW32.ICO
GSVIEWDE.HLP
LICENCE
GSVIEWEN.HLP
GSVW32DE.DLL
FILEID.TMP
GSVW32EN.DLL
PSTOTXT3.DLL
PSTOTXT3.EXE
GSV16SPL.EXE
GVWGS32.EXE
ZLIB32.DLL
PRINTER.INI
README.TXT
```

Вывод

```
6
LICENCE.TMP
FILEID.
LICENCE
FILEID.TMP
PSTOTXT3.DLL
PSTOTXT3.EXE
```

Задача 9 В. Бросание кубика

Кубик, грани которого помечены цифрами от 1 до 6, бросают N раз. Найти вероятность того, что сумма выпавших чисел будет равна Q .

Ограничения: $1 \leq N \leq 500$, $1 \leq Q \leq 3000$, время 1 с.

Ввод из файла `die.in`. В первой строке находятся числа N и Q через пробел.

Вывод в файл `die.out`. Выводится единственное вещественное число, которое должно отличаться от истинного значения не более чем на 0,01 истинного значения.

Примеры

Ввод 1	Ввод 2	Ввод 3	Ввод 4
1 6	1 7	4 14	100 100
Вывод 1	Вывод 2	Вывод 3	Вывод 4
0.16666	0	0.11265	1.53e-78

Задача 9 C. Игра с монетами

Однорублевые монетки разложены в стопки (в стопках может быть различное количество монет), а стопки поставлены на столе в ряд слева направо. Двое противников по очереди делают ходы. Ход состоит в том, что один из игроков берет слева несколько стопок подряд, не меньше одной, но и не больше, чем перед этим взял его соперник. Первый игрок своим первым ходом берет не более K стопок. Игра заканчивается, когда стопок не остается. Требуется найти максимальное число монет, которое может получить первый участник после окончания игры, если второй игрок тоже старается ходить так, чтобы получить как можно больше монет.

Ввод из файла `coingame.in`. В первой строке находится сначала число стопок N , за ним идут N чисел, задающих количество монет в стопках слева направо, а затем число K . Все числа в строке разделены пробелами.

Ограничения: $1 \leq N \leq 180$, $1 \leq K \leq 80$, количество монет в стопке — не менее 1 и не более 20 000, время 1 с.

Вывод в файл `coingame.out`. Вывести одно число — максимальное количество монет, которое заведомо может получить первый игрок, как бы ни играл второй.

Примеры

Ввод 1	Ввод 2	Ввод 3
3 4 9 1 3	4 1 2 2 7 3	5 3 4 8 1 7 2
Вывод 1	Вывод 2	Вывод 3
14	5	18

Задача 9 D. Бассейн реки

Дана карта рек некоторого континента (рис. 9.1). Каждая река показана как ломаная линия, которая начинается у истока реки и заканчивается или в точке, где река впадает в другую, или устьем реки. Вершины ломаной — или точки поворота реки, или точки впадения притоков.

Будем рассматривать бассейн реки как выпуклый многоугольник минимальной площади, который содержит реку и все ее притоки.

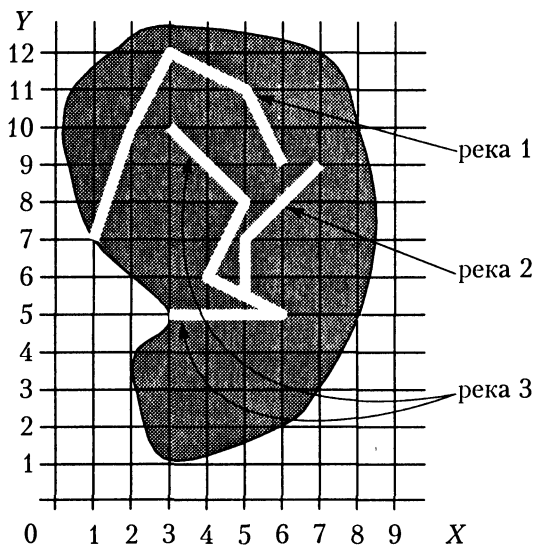


Рис. 9.1. Пример карты континента

Примечание. Согласно этому определению бассейна реки одна и та же территория может принадлежать бассейнам различных рек.

Пример. Показан континент с тремя реками. Координаты рек и площади бассейнов даны в таблице.

Название реки	x	y	Площадь бассейна реки без притоков
Река 1	6	9	12,5
	5	11	
	3	12	
	2	10	
	1	7	
Река 2	7	9	1,5
	5	7	
	5	5,5	
Река 3	3	10	9,5
	5	8	
	4	6	
	5	5,5	
	6	5	
	3	5	

Требуется найти максимальную площадь бассейна реки, расположенной на данном континенте.

Ввод из файла `basin.in`. Первая строка содержит число рек N . В следующих строках файла содержится N блоков, описывающих реки.

Каждый блок номер i состоит:

- из одной строки с k_i — числом вершин ломаной, описывающей реку;
- k_i строк, содержащих пары вещественных чисел x_j и y_j ($1 \leq j \leq k_i$), разделенных пробелом, — координаты точек, описывающих реку.

Ограничения: $1 \leq N \leq 10$, сумма $k_i \leq 1000$, $-1000 \leq x_j, y_j \leq 1000$, время 3 с.

Вывод в файл `basin.out`. Вывести одно число — площадь наибольшего бассейна реки с двумя знаками после запятой.

Примеры

Ввод 1	Ввод 2
3	2
5	5
6 9	6 9
5 11	5 11
3 12	3 12
2 10	2 10
1 7	1 7
3	6
7 9	3 10
5 7	5 8
5 5.5	4 6
6	5 5.5
3 10	6 5
5 8	3 5
4 6	
5 5.5	
6 5	
3 5	
Вывод 1	Вывод 2
16.00	12.50

Задача 9 Е. Ближайшее число

Дана матрица A размером $N \times N$, заполненная неотрицательными целыми числами. Расстояние между двумя элементами A_{ij} и A_{pq} определено как $|i - p| + |j - q|$. Требуется заменить каждый нулевой элемент матрицы ближайшим ненулевым. Если есть две или больше ближайших ненулевых ячейки, нуль должен быть оставлен.

Ограничения: $1 \leq N \leq 200$, $0 \leq A_{ij} \leq 1\,000\,000$, время 3 с.

Ввод из файла `pearnum.in`. В первой строке содержится число N . Затем идут N строк по N чисел, разделенных пробелами и представляющих собой матрицу.

Вывод в файл `pearnum.out`. Выводится N строк по N чисел, разделенных пробелами, — модифицированная матрица.

Пример

Ввод

```
3
0 0 0
1 0 2
0 3 0
```

Вывод

```
1 0 2
1 0 2
0 3 0
```

Задача 9 F. Прямоугольное деление

Дано N прямоугольников со сторонами, параллельными осям координат. Требуется определить, на сколько частей эти прямоугольники разбивают плоскость (внутри частей не должно быть границ прямоугольников).

Ввод из файла `rectpart.in`. В первой строке содержится число прямоугольников N . Далее идут N строк, содержащих по четыре числа, x_1, y_1, x_2, y_2 , — координаты двух противоположных углов прямоугольника.

Ограничения: $1 \leq N \leq 100$, координаты представляют собой целые числа и по абсолютной величине не превосходят 10 000, время 3 с.

Вывод в файл `rectpart.out`. Вывести одно число — количество частей, на которые разбивается плоскость.

Пример

Ввод

```
3
10 20 50 30
40 10 50 25
40 25 80 30
```

Вывод

```
6
```


Тренировка 10

Задача 10 А. Анти-QuickSort

Для сортировки последовательности чисел широко используется быстрая сортировка — QuickSort. Далее приведена программа, которая сортирует массив *a*, используя этот алгоритм.

```
var
  a : array [1..N] of integer;

procedure QSort(left, right : integer);
var
  i, j : integer;
  key : integer;
  buf : integer;
begin
  key := a[(left + right) div 2];
  i := left;
  j := right;
  repeat
    while a[i] < key do {первый while}
      inc(i);
    while key < a[j] do {второй while}
      dec(j);
    if i <= j then begin
      buf := a[i];
      a[i] := a[j];
      a[j] := buf;
      inc(i);
      dec(j);
    end;
  until i > j;

  if left < j then
    QSort(left, j);
  if i < right then
```

```

        QSort(i, right);
    end;

begin
    ...
    QSort(1, N);
end.

```

Хотя QuickSort является самой быстрой сортировкой в среднем, существуют тесты, на которых она работает очень долго. Оценивать время работы алгоритма будем количеством сравнений с элементами массива (то есть суммарным количеством сравнений в первом и втором while). Требуется написать программу, генерирующую тест, на котором быстрая сортировка сделает наибольшее число таких сравнений.

Ввод из файла antiqs.in. В первой строке находится единственное число N .

Ограничения: $1 \leq N \leq 70\,000$, время 1 с.

Вывод в файл antiqs.out. Вывести перестановку чисел от 1 до N , на которой быстрая сортировка выполнит максимальное число сравнений. Если таких перестановок несколько, вывести любую из них.

Пример

Ввод
3
Вывод
1 3 2

Задача 10 В. Строки Фибоначчи

Строку Фибоначчи $F(K)$ для натуральных чисел K определим так: $F(1) = 'A'$, $F(2) = 'B'$, $F(K) = F(K-1) + F(K-2)$ при $K > 2$, где «+» означает конкатенацию строк. Требуется найти количество вхождений строки S , состоящей из символов A и B, в строку Фибоначчи $F(N)$.

Ограничения: Длина строки S составляет от 1 до 25 символов, $1 \leq N \leq 45$, время 1 с.

Примечание. Длина $F(45)$ равна 1 134 903 170.

Ввод из файла fibostr.in. В первой строке содержится число N , во второй — строка S .

Вывод в файл fibostr.out. Вывести одно число — количество вхождений строки S в строку Фибоначчи $F(N)$.

Примеры

Ввод 1	Ввод 2	Ввод 3	Ввод 4
1	2	8	35
A	ABA	BBABAB	BBABAB
Вывод 1	Вывод 2	Вывод 3	Вывод 4
1	0	3	1346268

Задача 10 C. Игра в зачеркивание

Бумажная полоска разделена на N клеток. Двое играющих по очереди выбирают и зачеркивают ровно K пустых смежных клеток. Выигрывает сделавший последний ход. Оба игрока придерживаются правильной стратегии. Дана ситуация игры. Требуется определить, кто выигрывает.

Ограничения: $1 \leq K \leq N \leq 40$, время 10 с.

Ввод из файла `crossgam.in`. В первой строке содержатся числа N и K , во второй строке — N символов: латинская заглавная O — пустая клетка, латинская заглавная X — зачеркнутая клетка.

Вывод в файл `crossgam.out`. Вывести одно число: 1, если выиграет первый сделавший ход; 2, если выиграет второй; 0, если ход сделать нельзя.

Примеры

Ввод 1	Ввод 2	Ввод 3
4 2	5 2	7 2
0000	00000	0XX0XX0
Вывод 1	Вывод 2	Вывод 3
1	2	0

Задача 10 D. Граница многоугольника

Многоугольник на плоскости задан целочисленными координатами своих N вершин в декартовой системе координат. Требуется найти количество точек с целочисленными координатами, лежащих на границе многоугольника. Стороны многоугольника друг с другом не соприкасаются (за исключением соседних — в вершинах) и не пересекаются.

Ограничения: $3 \leq N \leq 100\,000$, координаты вершин целые и по модулю не превосходят $1\,000\,000\,000$, время 2 с.

Ввод из файла `border.in`. В первой строке содержится число N , в следующих N строках — пары чисел — координаты точек. Если соединить точки в данном порядке, а также соединить первую и последнюю точки, получится заданный многоугольник.

Вывод в файл `border.out`. Вывести одно число — количество точек с целочисленными координатами на границе многоугольника.

Пример

Ввод

```
3
10 0
0 10
0 0
```

Вывод

```
30
```

Задача 10 E. Путь спелеолога

Пещера представлена кубом, разбитым на N частей по каждому измерению (то есть на N^3 кубических клеток). Каждая клетка может быть или пустой, или полностью заполненной камнем. Исходя из положения спелеолога в пещере, требуется найти, какое минимальное количество перемещений по клеткам ему требуется, чтобы выбраться на поверхность. Переходить из клетки в клетку можно, только если они обе свободны и имеют общую грань.

Ограничения: $1 \leq N \leq 30$, время 1 с.

Ввод из файла `speleo.in`. В первой строке содержится число N . Далее следует N блоков. Блок состоит из пустой строки и N строк по N символов: `#` обозначает клетку, заполненную камнями, точка — свободную клетку. Начальное положение спелеолога обозначено заглавной буквой `S`. Первый блок представляет верхний уровень пещеры, достижение любой свободной его клетки означает выход на поверхность. Выход на поверхность всегда возможен.

Вывод в файл `speleo.out`. Вывести одно число — длину пути до поверхности.

Пример

Ввод

3

```
###
###
.##
```

```
.#
.#S
.#
```

```
###
...
###
```

Вывод

6

Комментарий

Нужно спуститься на уровень вниз, сделать два движения на запад, подняться на уровень вверх, сделать движение на юг, подняться на уровень вверх.

Задача 10 F. Дырявая ткань

На столе лежат несколько кусков ткани, не перекрывая друг друга. Эти куски могут иметь дыры, в том числе и настолько большие, что в них может поместиться целый кусок ткани. Был получен черно-белый образ поверхности стола, на котором

области, покрытые тканью, представлены символами *, а свободные площади — точками. Один кусок ткани, таким образом, представлен 4-связной областью символов *, то есть группой *, соседствующих друг с другом горизонтально или вертикально, но не по диагонали.

```

.***.***
.*.***.*
.***.***
*...**.*

```

На схеме три куска — один без дыр, а два — с одной дырой каждый: первый — площадью 8, второй — площадью 12.

Ваша цель — найти кусок с наибольшим количеством дыр в нем. Дыра — это 4-связная область точек, полностью окруженных символами *. Если несколько кусков имеют одинаковое количество дыр, нужно выбрать кусок минимальной площади.

Ввод из файла `holey.in`. В первой строке содержатся два числа W и H , разделенные пробелами. Следующие H строк содержат по W символов каждая. Символы в этих строках — или * (ASCII 42), или точка (ASCII 46).

Ограничения: $1 \leq W, H \leq 100$, время 1 с.

Вывод в файл `holey.out`. Вывести одно целое число — площадь минимального из наиболее дырявых кусков. Если нет кусков с дырами, выходной файл должен содержать ноль.

Пример

Ввод

```

9 5
.*****
.*.....*
.*...**.*
.*.....*
.*****

```

Вывод

```

22

```

Тренировка 11

Задача 11 А. Последовательность

В последовательности чисел $a_1, a_2, a_3, \dots$ задан первый член, а остальные вычисляются по формуле $a_i = (a_{i-1})^2 \bmod 10\,000$. Найти N -й член последовательности.

Ограничения: $0 \leq a_1 < 10\,000$, $1 \leq N \leq 2\,000\,000\,000$, время 1 с.

Ввод из файла seq.in. В первой строке находятся числа a_1 и N , разделенные пробелом.

Вывод в файл seq.out. Вывести одно число — a_N .

Примеры

Ввод 1	Ввод 2
4 3	0 2000000000
Вывод 1	Вывод 2
256	0

Задача 11 В. Провода

Дано N отрезков провода длиной $L_1, L_2, \dots, L_N$ сантиметров. Требуется разрезанием получить из них K равных отрезков как можно большей длины, выражающейся целым числом сантиметров. Если нельзя получить K отрезков длиной даже 1 см, вывести 0.

Ограничения: $1 \leq N \leq 10\,000$, $1 \leq K \leq 10\,000$, $100 \leq L_i \leq 10\,000\,000$, все числа целые, время 1 с.

Ввод из файла cable.in. В первой строке находятся числа N и K . В следующих N строках — $L_1, L_2, \dots, L_N$, по одному числу в строке.

Вывод в файл cable.out. Вывести одно число — полученную длину отрезков.

Пример

Ввод
4 11
802
743
457
539
Вывод
200

Задача 11 С. Палиндромы

Непустая строка, содержащая некоторое слово, называется палиндромом, если это слово одинаково читается как слева направо, так и справа налево. Пусть дана строка, в которой записано слово S , состоящее из N прописных букв латинского алфавита. Вычеркиванием из этого слова некоторого набора символов можно получить строку, которая будет палиндромом. Требуется найти количество способов вычеркивания из данного слова некоторого (возможно, пустого) набора таких символов, что полученная в результате строка является палиндромом. Способы, различающиеся только порядком вычеркивания символов, считаются одинаковыми.

Ограничения: $1 \leq N \leq 60$, время 1 с.

Ввод из файла `palindr.in`. В первой строке записано слово S .

Вывод в файл `palindr.out`. Вывести одно целое число — количество способов вычеркивания.

Пример

Ввод

BAOBAB

Вывод

22

Задача 11 D. Круговая площадь

Два круга заданы координатами центров в прямоугольной декартовой системе координат и радиусами. Найти площадь их пересечения.

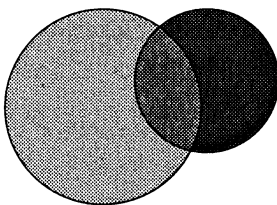


Рис. 11.1. Пересечение кругов

Ограничения: во входных данных числа вещественные и по модулю не превосходят 1000, время 1 с.

Ввод из файла `circarea.in`. В первой строке находятся шесть вещественных чисел через пробел — координаты центров и радиусы двух кругов: $x_1, y_1, r_1, x_2, y_2, r_2$.

Вывод в файл `circarea.out`. Вывести одно вещественное число с двумя знаками после запятой — площадь пересечения кругов.

Пример

Ввод

20.0 30.0 15.0 40.0 30.0 30.0

Вывод

608.37

Задача 11 Е. Гомер Симпсон

Обеденный перерыв Гомера Симпсона составляет T мс. Один гамбургер Гомер съедает за N мс, один чизбургер — за M . Требуется найти максимальное суммарное число гамбургеров и чизбургеров, которые Гомер может съесть в течение обеденного перерыва.

Ограничения: $1 \leq M, N, T \leq 1\,000\,000$, все числа целые, время 2 с.

Ввод из файла `homer.in`. В первой строке находятся три числа — M, N и T , разделенные пробелами.

Вывод в файл `homer.out`. Вывести максимальное суммарное число гамбургеров и чизбургеров. Если остается какое-то время, требуется указать его через пробел. Предпочтителен вариант, когда дополнительного времени остается как можно меньше.

Примеры

Ввод 1	Ввод 2	Ввод 3
3 5 54	3 5 55	4 4 6
Вывод 1	Вывод 2	Вывод 3
18	17	1 2

Задача 11 F. Дробная арифметика

Напишите программу, реализующую сложение, вычитание, умножение и деление дробей. Формат дробей во входных и выходных данных:

- знак числа (пишется только в случае, когда его отсутствие изменяет число);
- целая часть числа (нулевая целая часть не пишется, если есть числитель и знаменатель);
- пробел (не пишется, если отсутствует целая или дробная часть);
- числитель (если он не равен нулю);
- знак / (если есть числитель);
- знаменатель (если есть числитель).

Примеры представления дробных чисел: $-7\frac{3}{4}$, $8\frac{1}{2}$, $-7/11$, 0, 11.

Ограничения (как на входные, так и на выходные данные): целая часть может принимать значения из диапазона $0 \dots 30\,000$, числитель и знаменатель могут принимать значения от 1 до 30 000, при делении второй операнд не равен нулю, время 1 с.

Ввод из файла `frac-ar.in`. В первой строке вводится дробь (первый операнд), во второй — знак операции («+» — сложение, «-» — вычитание, «*» — умножение, «/» — деление), в третьей строке — дробь (второй операнд). Обе дроби могут быть сократимы.

Вывод в файл `frac-ag.out`. В единственной строке выводится *несократимая правильная* дробь (результат) в описанном формате.

Пример

Ввод

-3 1/6

+

2/4

Вывод

-2 2/3

Тренировка 12

Задача 12 А. Последовательность (2)

Каждый член последовательности десятичных цифр $d_1, d_2, d_3, \dots$, начиная с четвертого, равен последней цифре суммы трех предыдущих. По заданным d_1, d_2, d_3 найти N -й член последовательности.

Ограничения: $1 \leq N \leq 10^{15}$, время 1 с.

Ввод из файла `seq2.in`. В первой строке находятся цифры d_1, d_2, d_3 , разделенные пробелами, во второй — число N .

Вывод в файл `seq2.out`. Вывести одну цифру — d_N .

Примеры

Ввод 1	Ввод 2
1 4 8	5 5 5
4	1000000000000000
Вывод 1	Вывод 2
3	5

Задача 12 В. Гирлянда

Гирлянда состоит из N лампочек на общем проводе (рис. 12.1). Один ее конец закреплен на заданной высоте A мм ($H_1 = A$). Благодаря силе тяжести гирлянда прогибается: высота каждой неконцевой лампы на 1 мм меньше, чем средняя высота ближайших соседей ($H_i = (H_{i-1} + H_{i+1})/2 - 1$ для $1 < i < N$). Требуется найти минимальную высоту второго конца B ($B = H_N$) при условии, что ни одна из лампочек не должна лежать на земле ($H_i > 0$ для $1 \leq i \leq N$).

Ограничения: $3 \leq N \leq 1000$ — целое, $10 \leq A \leq 1000$ — вещественное, время 1 с.

Ввод из файла `garland.in`. В первой строке находятся два числа, N и A .

Вывод в файл `garland.out`. Вывести одно вещественное число B с двумя знаками после запятой.

Примеры

Ввод 1	Ввод 2
8 15	692 532.81
Вывод 1	Вывод 2
9.75	446113.34

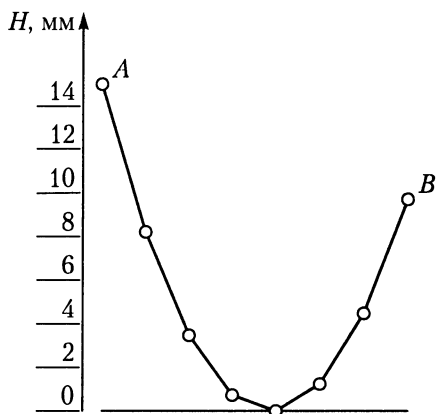


Рис. 12.1. Пример расположения гирлянды

Задача 12 С. Головоломка умножения

В головоломку умножения играют с рядом карт, каждая из которых содержит одно положительное целое число. Во время хода игрок убирает одну карту из ряда и получает число очков, равное произведению числа на убранной карте и чисел на картах, лежащих непосредственно слева и справа от нее. Не разрешено убирать первую и последнюю карты ряда. После последнего хода в ряду остается только две карты.

Цель игры — убрать карты в таком порядке, чтобы минимизировать общее количество набранных очков.

Например, если карты содержат числа 10, 1, 50, 20 и 5, игрок может взять карту с числом 1, затем 20 и 50, и подсчитать очки по следующей формуле:

$$10 \cdot 1 \cdot 50 + 50 \cdot 20 \cdot 5 + 10 \cdot 50 \cdot 5 = 500 + 5000 + 2500 = 8000.$$

Если бы он взял карты в обратном порядке, то есть 50, затем 20, затем 1, количество очков было бы таким:

$$1 \cdot 50 \cdot 20 + 1 \cdot 20 \cdot 5 + 10 \cdot 1 \cdot 5 = 1000 + 100 + 50 = 1150.$$

Ввод из файла `mpuzzle.in`. В первой строке находится число карт N , во второй — разделенные пробелами N чисел на картах.

Ограничения: $3 \leq N \leq 100$, числа на картах целые от 1 до 100, время 1 с.

Вывод в файл `mpuzzle.out`. Вывести одно целое число — минимально возможное число очков.

Пример

Ввод

6

10 1 50 50 20 5

Вывод

3650

Задача 12 D. Точки в многоугольнике

Многоугольник на плоскости задан целочисленными координатами своих N вершин в декартовой системе координат. Требуется найти число точек с целочисленными координатами, лежащих внутри многоугольника (не на границе). Стороны многоугольника друг с другом не соприкасаются (за исключением соседних — в вершинах) и не пересекаются.

Ограничения: $3 \leq N \leq 10\,000$, координаты вершин целые и по модулю не превосходят $1\,000\,000$, время 1 с.

Ввод из файла `polygonr.in`. В первой строке находится число N , в следующих N строках — пары чисел — координаты точек. Если соединить точки в данном порядке, а также соединить первую и последнюю точки, получится заданный многоугольник.

Вывод в файл `polygonr.out`. Вывести одно число — искомое количество точек.

Пример

Ввод

4

-10 -10

-10 10

10 10

10 -10

Вывод

361

Задача 12 Е. Водопровод

Город Восточный постоянно страдает от недостатка воды. Для устранения этой проблемы была построена новая водопроводная труба. Строительство трубы началось с обоих концов одновременно, и спустя некоторое время половины соединились. Ну, почти. Первая половина трубы заканчивалась в точке (x_1, y_1) , а вторая — в точке (x_2, y_2) .

К сожалению, осталось лишь несколько отрезков трубы различной длины. Более того, из-за специфики местной технологии трубы могут быть проложены только в направлении с севера на юг или с востока на запад и соединяются, образуя или прямую, или угол 90° . Требуется, зная длины отрезков труб $L_1, L_2, \dots, L_K$ и количество отрезков каждой длины $C_1, C_2, \dots, C_K$, сконструировать трубу, соединяющую две заданные точки, или определить, что это невозможно.

Ограничения: $1 \leq K \leq 4$, $1 \leq x_1, y_1, x_2, y_2$, $L_i \leq 1000$, $1 \leq C_i \leq 10$, все числа целые, время 3 с.

Ввод из файла `wpipe.in`. В первой строке находятся числа x_1, y_1, x_2, y_2, K , затем $2K$ чисел: $L_1, L_2, \dots, L_K, C_1, C_2, \dots, C_K$.

Вывод в файл `wpipe.out`. Вывести одно число — минимальное количество нужных отрезков труб или -1 , если соединение невозможно.

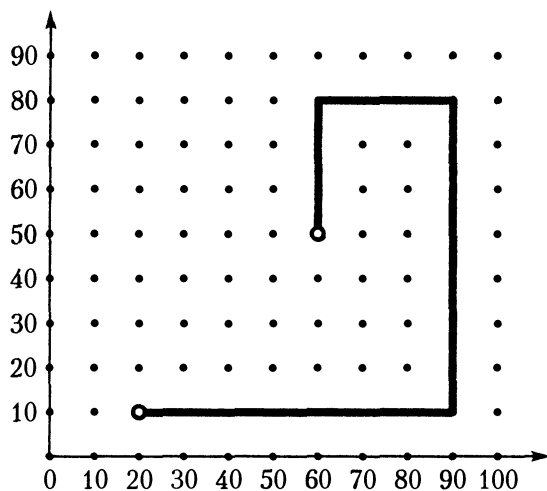


Рис. 12.2. Карта водопровода

Примеры

Ввод 1

5 5 5 6 1 2 10

Вывод 1

-1

Ввод 2

20 10 60 50 2 70 30 2 2

Вывод 2

4

Задача 12 F. Химические реакции

Билл преподает химию в школе, он подготовил несколько тестов для учеников. Каждый тест состоит из химической формулы и нескольких возможных результатов реакции. Среди этих результатов ученики должны выбрать правильный. Билл хочет убедиться в том, что, вводя свои тесты в компьютер, он не допустил опечаток, благодаря которым ученики могли бы отбросить неверные ответы, просто подсчитав число химических элементов в левой и правой частях уравнения (в правильном уравнении химической реакции должно соблюдаться равенство).

Ваша задача — написать программу, которая поможет Биллу. Программа должна прочитать описание теста, состоящее из заданной левой части уравнения и нескольких возможных правых частей, и определить, равно ли количество химических элементов в каждой предложенной правой части уравнения количеству химических элементов в заданной левой части.

Билл формализовал задачу. И левая, и правая части уравнения представлены строкой символов без пробелов, состоящей из одной или более химических последовательностей, разделенных знаком плюс. Каждая последовательность имеет необязательный предшествующий целый множитель, относящийся ко всей последовательности, и несколько элементов. Каждый элемент может сопровождаться необязательным целым множителем, относящимся к нему. Элемент в этом урав-

нении может быть или отдельным химическим элементом, или целой последовательностью в круглых скобках. Каждый отдельный химический элемент представлен или одной прописной буквой, или прописной буквой, сопровождаемой строчной.

Еще более формально, используя нотацию, аналогичную форме Бэкуса–Наура, можно написать:

```
<формула> ::= [<число>] <последовательность> {"+" [<число>] <последовательность>}  
<последовательность> ::= <элемент> [<число>] {<элемент> [<число>]}  
<элемент> ::= <химический элемент> | "(" <последовательность> ")"  
<химический элемент> ::= <прописная буква> [<строчная буква>]  
<прописная буква> ::= "A".."Z"  
<строчная буква> ::= "a".."z"  
<число> ::= "1".."9" {"0".."9"}
```

Будем говорить, что каждый отдельный химический элемент встречается в формуле всего X раз, если X — сумма всех различных вхождений этого химического элемента, умноженных на все числа, относящиеся к ним. Например, в формуле $C_2H_{50}N+3O_2+3(SiO_2)$

- C встречается всего 2 раза;
- H встречается всего 6 раз ($5 + 1$);
- O встречается всего 13 раз ($1 + 3 \cdot 2 + 3 \cdot 2$);
- Si встречается всего 3 раза.

Все множители в формулах — целые числа не меньше 2, если заданы явно, или равны 1 — по умолчанию.

Ввод из файла `chem.in`. В первой строке находится формула — левая часть уравнения, во второй — одно число N — количество рассматриваемых правых частей, в каждой из следующих N строк — одна формула — предлагаемая правая часть уравнения.

Ограничения: $1 \leq N \leq 10$, длина формулы не превосходит 100 символов, каждый отдельный химический элемент встречается всего не более 10 000 раз в каждой формуле, время 1 с.

Вывод в файл `chem.out`. Для каждой из N заданных правых частей выведите одну строку вида

<формула левой части>==<формула правой части>

если общее количество вхождений каждого отдельного химического элемента в левую часть равно общему числу вхождений этого химического элемента в правую часть. В противном случае выведите:

<формула левой части>!=<формула правой части>

Здесь <формула левой части> должна быть замещена посимвольной копией формулы левой части, как она дана в первой строке входного файла, а <формула правой части> —

замещена точной копией формулы правой части, как она дана во входном файле. В строках не должно быть пробелов.

Пример

Ввод

C2H5OH+3O2+3(SiO2)

7

2CO2+3H2O+3SiO2

2C+6H+13O+3Si

99C2H5OH+3SiO2

3SiO4+C2H5OH

C2H5OH+3O2+3(SiO2)+Ge

3(Si(O)2)+2CO+3H2O+O2

2CO+3H2O+3O2+3Si

Вывод

C2H5OH+3O2+3(SiO2)==2CO2+3H2O+3SiO2

C2H5OH+3O2+3(SiO2)==2C+6H+13O+3Si

C2H5OH+3O2+3(SiO2)!=99C2H5OH+3SiO2

C2H5OH+3O2+3(SiO2)==3SiO4+C2H5OH

C2H5OH+3O2+3(SiO2)!=C2H5OH+3O2+3(SiO2)+Ge

C2H5OH+3O2+3(SiO2)==3(Si(O)2)+2CO+3H2O+O2

C2H5OH+3O2+3(SiO2)!=2CO+3H2O+3O2+3Si

ПРИМЕЧАНИЕ

Пример не содержит цифры ноль, потому что она выглядит практически так же, как символ химического элемента кислорода. Настоящие тесты могут содержать любые допустимые символы.

Тренировка 13

Задача 13 А. Двойная решетка

Две бесконечные равномерные прямоугольные решетки заданы размерами ячеек $x_1 \times y_1$ и $x_2 \times y_2$. Решетки расположены на плоскости параллельно друг другу и координатным осям так, что смещение одного из узлов второй решетки относительно узла первой составляет D_x по горизонтали и D_y по вертикали. В результате наложения образуется новая, «составная» решетка с более мелкими ячейками различного размера. Требуется вывести в порядке возрастания все различные площади ячеек составной решетки.

Ограничения: $1 \leq x_1, y_1, x_2, y_2 \leq 100, 0 \leq D_x < x_1, 0 \leq D_y < y_1$, все числа целые, время 1 с.

Ввод из файла `dlattice.in`. В первой строке находятся числа $x_1, y_1, x_2, y_2, D_x, D_y$, разделенные пробелами.

Вывод в файл `dlattice.out`. В первой строке вывести N — количество получившихся площадей, в следующих N строках — сами площади.

Пример

Ввод
20 20 12 10 2 0
Вывод
4
20
60
100
120

Задача 13 В. Последовательность Фибоначчи

$\{F_k\}_{k=-\infty}^{\infty}$ — бесконечная последовательность целых чисел, которая удовлетворяет условию Фибоначчи $F_k = F_{k-1} + F_{k-2}$ (для любого целого k). Даны также i, F_i, j, F_j, n ($i \neq j$). Найти F_n . Пример части последовательности:

k	-2	-1	0	1	2	3	4	5	6
F_k	-5	4	-1	3	2	5	7	12	19

Ограничения: $-1000 \leq i, j, n \leq 1000, -2\,000\,000\,000 \leq F_k \leq 2\,000\,000\,000$ ($k = \min(i, j, n) \dots \max(i, j, n)$), время 1 с.

Ввод из файла fiboseq.in. В первой строке находятся числа i, F_i, j, F_j, n .

Вывод в файл fiboseq.out. Вывести одно число F_n .

Пример

Ввод

3 5 -1 4 5

Вывод

12

Задача 13 С. Скобки (3)

Определим правильные скобочные выражения так:

1. Пустое выражение — правильное.
2. Если выражение S правильное, то (S) и $[S]$ также правильные.
3. Если выражения A и B правильные, то и выражение AB — правильное.

Дана последовательность скобок $(,), [\text{ и }]$. Требуется найти самое короткое правильное выражение, в котором данная последовательность является подпоследовательностью, то есть такое, из которого можно вычеркнуть некоторые символы (возможно, ноль) и получить исходную последовательность, не меняя порядок оставшихся.

Ограничения: исходная последовательность содержит не более 100 скобок, время 1 с.

Ввод из файла bracket3.in. В первой строке находятся символы $(,), [\text{ и }]$ без пробелов.

Вывод в файл bracket3.out. Выводится искомая последовательность скобок без пробелов.

Примеры

Ввод 1	Ввод 2	Ввод 3	Ввод 4
<code>(([</code>	<code>(([[]])</code>	<code>(([]])</code>	<code>(([[[]])])</code>
Вывод 1	Вывод 2	Вывод 3	Вывод 4
<code>()([])</code>	<code>(([[()]])</code>	<code>(([]]) []</code>	<code>() () [[(() ()]]</code>

Задача 13 D. Центр тяжести

По координатам вершин многоугольника требуется найти координаты его центра тяжести. Стороны многоугольника друг с другом не соприкасаются (за исключением соседних — в вершинах) и не пересекаются. Площадь многоугольника не равна нулю.

Ограничения: число вершин $3 \leq N \leq 100\,000$, координаты вершин в декартовой системе координат целые и по модулю не превосходят 20 000, время 2 с.

Ввод из файла centgrav.in. В первой строке находится число N , в следующих N строках — пары чисел — координаты точек. Если соединить точки в данном порядке,

а также соединить первую и последнюю точки, получится заданный многоугольник.

Вывод в файл `centgrav.out`. Вывести два числа с двумя знаками после запятой — координаты центра тяжести.

Примеры

Ввод 1	Ввод 2
4	4
5 0	1 1
0 5	11 1
-5 0	11 11
0 -5	1 11
Вывод 1	Вывод 2
0.00 0.00	6.00 6.00

Задача 13 E. Сумма произведений

Дан набор переменных $x_1, x_2, \dots, x_N$. Каждая переменная x_i может принимать значение только $-1, 0$ или $+1$. Для данного целого числа S требуется определить количество способов присвоить переменным x_i значения так, чтобы сумма всех возможных произведений $x_i x_j$ была равна S , где $i < j$ и $i, j = 1, 2, \dots, N$. Два способа считаются различными, если они содержат различное число $x_i = 0$.

Ограничения: $2 \leq N \leq 10\,000$, $-10\,000 < S < 10\,000$, время 1 с.

Ввод из файла `prodsum.in`. В первой строке находятся числа N и S , разделенные пробелом.

Вывод в файл `prodsum.out`. Вывести одно целое число — количество способов представить S как сумму произведений.

Примеры

Ввод 1	Ввод 2
5 0	3 -2
Вывод 1	Вывод 2
3	0

Задача 13 F. Статическая сложность

Анализ временной сложности алгоритмов — важный инструмент создания эффективных программ. Алгоритмы, выполняемые за линейное время, как правило, значительно быстрее алгоритмов, требующих для выполнения той же задачи квадратичного времени, так что предпочтение должно быть отдано первым.

Обычно определяют время выполнения алгоритма по отношению к n — «размеру» входных данных. Это может быть число объектов, которые нужно отсортировать, число точек многоугольника и т. п. Поскольку определение формулы зависимости временной сложности алгоритма от n — непростая задача, было бы замечательно,

если бы ее можно было автоматизировать. К сожалению, в общем случае это невозможно. Но в этой задаче мы будем рассматривать программы очень простой природы, над которыми это можно проделать. Рассматриваемые программы записаны согласно следующим правилам БНФ, где `<число>` может быть любым неотрицательным целым числом:

```

<Программа> ::= "BEGIN" <Список операторов> "END"
<Список операторов> ::= <Оператор> | <Оператор> <Список операторов>
<Оператор> ::= <Оператор LOOP> | <Оператор OP>
<Оператор LOOP> ::= <Заголовок LOOP> <Список операторов> "END"
<Заголовок LOOP> ::= "LOOP" <число> | "LOOP n"
<Оператор OP> ::= "OP" <число>

```

Время выполнения такой программы может быть вычислено следующим образом: выполнение оператора `OP` требует столько единиц времени, сколько указано в его параметре. Список операторов, заключенный в оператор `LOOP`, выполняется столько раз, сколько указано в параметре оператора, то есть или заданное константное число раз, если задано число, или n раз, если параметром является n . Время выполнения списка операторов равно сумме времени выполнения его частей. Таким образом, время выполнения программы в целом зависит от n .

Ввод из файла `icomplex.in`. В первой строке находится целое число k — число программ во входном файле. Затем идут k программ, удовлетворяющих приведенной грамматике. Пробелы и переводы строк могут встречаться везде в программе, но не в ключевых словах `BEGIN`, `END`, `LOOP` и `OP`, нет их и в целых числах.

Вывод в файл `icomplex.out`. Для каждой программы сначала идет строка с номером программы. В следующей строке записывается время работы программы в терминах n — многочлен степени не более 10. Многочлен должен быть записан обычным способом, то есть подобные слагаемые должны быть приведены, слагаемое большей степени должно предшествовать слагаемому меньшей степени, слагаемые с коэффициентом 0 не записываются, множители 1 не записываются. Общий вид второй строки «`Runtime = a*n^10+b*n^9+...+i*n^2+j*n+k`». Если время выполнения нулевое, нужно вывести «`Runtime = 0`». За строкой с многочленом должна следовать пустая строка.

Ограничения: вложенность операторов `LOOP` не превышает 10, размер входного файла не более 2 Кбайт, коэффициенты многочлена в ответе не превышают 50 000, время 1 с.

Пример

Ввод

2

BEGIN

 LOOP n

 OP 4

 LOOP 3

 LOOP n

```
      OP 1
    END
  OP 2
END
OP 1
END
OP 17
END

BEGIN
  OP 1997 LOOP n LOOP n OP 1 END END
END
```

Вывод

Program #1

Runtime = $3*n^2+11*n+17$

Program #2

Runtime = n^2+1997

Тренировка 14

Задача 14 А. Марковский цикл

Ограниченный алгоритм Маркова состоит из последовательности предложений

$$s_1 s_2 \dots s_N \rightarrow d_1 d_2 \dots d_N,$$

где s_i и d_i — символы из алфавита A, B, C. Подстрока $s_1 s_2 \dots s_N$ называется левой, а $d_1 d_2 \dots d_N$ — правой частью предложения.

Алгоритм выполняется над исходной текстовой строкой, состоящей из прописных латинских букв A, B, C, следующим образом: перебираются все предложения начиная с первого. Если левая часть предложения входит в текстовую строку, то самое левое вхождение заменяется правой частью этого предложения, и поиск вновь начинается с первого предложения. Если ни одно предложение не может быть применено, алгоритм останавливается.

При выполнении алгоритма возможны два результата: либо остановка, либо бесконечный цикл с определенным периодом. По данной строке и набору предложений алгоритма Маркова определить количество «ациклических» (выполненных до начала цикла) шагов и длину самого цикла. Если алгоритм останавливается, то длина цикла считается нулевой, а все выполненные шаги — ациклическими.

Ввод из файла `markovc.in`. В первой строке находится исходная текстовая строка, а в следующих строках — предложения, по одному в строке. Строки могут содержать произвольное количество незначащих пробелов.

Ограничения: длина исходной текстовой строки и левых частей предложений — от 1 до 12 букв, количество предложений — от 1 до 50, время 3 с.

Вывод в файл `markovc.out`. Вывести два целых числа, разделенных пробелом, — количество ациклических шагов и длину цикла.

Пример

Ввод

ABABC

C ->A

AB ->BA

BAA->ABC

Вывод

3 3

Задача 14 В. Д-44

При решении задач баллистики следует учитывать сопротивление воздуха. Пусть сила сопротивления воздуха пропорциональна квадрату модуля скорости движения снаряда и действует противоположно скорости. Считая коэффициент пропорциональности силы сопротивления воздуха квадрату скорости постоянной величиной, рассчитать дальность полета снаряда (с точностью до 10 м) при стрельбе из 85-миллиметровой пушки Д-44 под заданным углом к горизонту, если масса снаряда равна 9,6 кг, а начальная скорость — 800 м/с. Известно также, что при стрельбе под углом 45° дальность стрельбы составляет 15 650 м. Ускорение свободного падения считать равным $9,8 \text{ м/с}^2$.

Ограничения: угол стрельбы к горизонту — от 5 до 85° , время 2 с.

Ввод из файла d44.in. В первой строке находится единственное вещественное число — угол в градусах.

Вывод в файл d44.out. Вывести одно целое число — дальность в метрах.

Пример

Ввод

45

Вывод

15649

Задача 14 С. Упаковка символов

Билл пытается компактно представить последовательности прописных символов от А до Z с помощью упаковки повторяющихся подпоследовательностей внутри них. Например, один из способов представить последовательность AAAAAAAAAABABABCCD — это 10(A)2(BA)B2(C)D. Он формально определяет сжатые последовательности символов и правила перевода их в несжатый вид следующим образом:

- Последовательность, содержащая один символ от А до Z, является упакованной. Распаковка этой последовательности дает ту же последовательность из одного символа.
- Если S и Q — упакованные последовательности, то SQ — также упакованная последовательность. Если S распаковывается в S', а Q распаковывается в Q', то SQ распаковывается в S'Q'.
- Если S — упакованная последовательность, то X(S) — также упакованная последовательность, где X — десятичное представление целого числа, большего 1. Если S распаковывается в S', то X(S) распаковывается в S', повторенную X раз.

Следуя этим правилам, легко распаковать любую заданную упакованную последовательность. Однако Биллу более интересен обратный переход. Он хочет упаковать заданную последовательность так, чтобы результирующая сжатая последовательность содержала наименьшее возможное число символов.

Ограничения: длина исходной последовательности от 1 до 100, время 2 с.

Ввод из файла `folding.in`. В первой строке находится последовательность символов от A до Z.

Вывод в файл `folding.out`. В единственной строке выводится упакованная последовательность наименьшей длины, которая распаковывается в заданную последовательность. Если таких последовательностей несколько, можно выводить любую.

Примеры

Ввод 1

AAAAAAAAABABABCCD

Вывод 1

9(A)3(AB)CCD

Ввод 2

NEERCYESYESNEERCYESYESYES

Вывод 2

2(NEERC3(YES))

Задача 14 D. Пирамиды

Рассматриваемые пирамиды имеют треугольник в основании, то есть являются тетраэдрами. Требуется по заданным длинам ребер пирамиды найти ее объем.

Ограничения: длины ребер — целые положительные числа, не превосходящие 1000, время 1 с.

Ввод из файла `pyramids.in`. В первой строке находятся шесть чисел через пробел — длины ребер пирамиды $ABCD$. Порядок ребер: AB, AC, AD, BC, BD, CD .

Вывод в файл `pyramids.out`. Вывести одно вещественное число с четырьмя знаками после запятой — объем пирамиды.

Примеры

Ввод 1

1 1 1 1 1 1

Вывод 1

0.1179

Ввод 2

1000 1000 1000 3 4 5

Вывод 2

1999.9937

Задача 14 E. Поле для крикета

Жил-был жадный Король. Он приказал своему главному Архитектору построить поле для королевского крикета в парке. Король был таким жадным, что не послушал предложения своего Архитектора построить поле прямо в центре парка и окружить его живописным бордюром деревьев, специально посаженных вокруг. Вместо этого он приказал не срубать деревья и не сажать новых, но построить самое большое поле для крикета, какое только можно. Если Король обнаружит, что Архитектор посмел тронуть даже единственное дерево в парке или спроектировал меньшее поле, чем было возможно, Архитектор лишится головы. Более того, он потребовал от Архитектора представить план поля, где указаны его точное положение и размер.

Ваша задача — помочь бедному Архитектору сохранить голову, написав программу, которая найдет максимальный размер поля для крикета и его положение внутри парка, удовлетворяющие требованиям Короля.

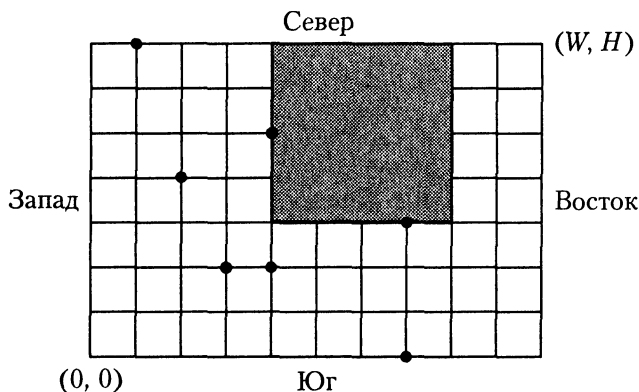


Рис. 14.1. Пример карты парка

Задача слегка упрощена тем, что парк Короля имеет прямоугольную форму и расположен на плоской поверхности. Более того, границы парка параллельны направлениям север — юг и восток — запад. В то же время игра в королевский крикет всегда происходит на квадратном поле, границы которого также параллельны направлениям север — юг и восток — запад. Архитектор уже сопоставил парку прямоугольную декартову систему координат и точно определил координаты каждого дерева. Оси этой системы координат, конечно, параллельны направлениям север — юг и восток — запад. Юго-западный угол парка имеет координаты $(0, 0)$, а северо-восточный — координаты (W, H) , где W и H — длина и ширина парка соответственно (рис. 14.1).

В этой задаче вы можете пренебречь диаметром деревьев. Деревья не могут находиться внутри поля для крикета, но могут располагаться на его сторонах. Поле для крикета может также касаться границы парка, но не должно лежать вне парка.

Ввод из файла `cricket.in`. Первая строка содержит три целых числа, N , W и H , разделенных пробелами: N — число деревьев в парке, W и H — длина и ширина парка соответственно.

Следующие N строк описывают координаты деревьев в парке. Каждая строка содержит два целых числа x_i и y_i , разделенных пробелом и представляющих собой координаты i -го дерева. Все деревья имеют различные координаты.

Ограничения: $1 \leq N \leq 100$, $1 \leq W, H \leq 10\,000$, $0 \leq x_i \leq W$, $0 \leq y_i \leq H$, время 1 с.

Вывод в файл `cricket.out`. Вывести через пробел три целых числа, P , Q и L , где (P, Q) — координаты юго-западного угла поля для крикета, L — длина его сторон. Если существует несколько возможных положений поля максимального размера, вывести любое.

Пример

Ввод

7 10 7

3 2

4 2

7 0

7 3

4 5

2 4

1 7

Вывод

4 3 4

Задача 14 F. Электронная таблица

Напишите программу, выполняющую функции очень простой электронной таблицы. Она работает с таблицей из 9 строк от 1 до 9 и 26 столбцов от A до Z. Клетки таблицы обозначаются именами, составленными из кодов столбца и строки, например B1, S8.

Каждая клетка содержит выражение. Выражения используют целые константы, ссылки на клетки, скобки, бинарные операторы +, -, * и / (целочисленное деление). Так, 567, E8/2, (3+B3)*(C4-1) являются правильными выражениями. Все операторы целочисленные. Деление на ноль дает в результате ноль.

Если значение ячейки, на которую ссылается некоторое выражение, не определено, оно считается равным нулю. Ситуация, когда две или более ячейки зависят друг от друга, является отдельным случаем — циклической ссылкой.

Ограничения: длина выражения в одной ячейке до 255 символов, все аргументы и результаты меньше 1 000 000, время 1 с.

Ввод из файла sprsheet.in. Первая строка содержит число выражений N . Следующие N строк имеют формат <Имя клетки>=<выражение>. Все выражения корректные, и каждая ячейка определена не более чем одним выражением.

Вывод в файл sprsheet.out. В единственной строке выводится или значение клетки A1, или число 1000000 (один миллион), если значение клетки A1 не может быть найдено из-за циклической ссылки.

Пример

Ввод

4

A1=B1+C5

B1=20

C5 =B1 /D7-E1*E1

E1=(3+1)*2

Вывод

-44

Тренировка 15

Задача 15 А. Игра с калькулятором

В калькулятор вводится натуральное число K и нажимается клавиша «+». Калькулятор все еще показывает K . Цель игры: получить на экране число, состоящее из одинаковых цифр. Для ее достижения можно производить только одно действие — нажимать на клавишу «=» (возможно, 0 раз). После первого нажатия получается результат $K + K$, после очередного нажатия результат увеличивается на K . Требуется определить, удастся ли достичь цели, а если удастся, то какое число, состоящее из одинаковых цифр, будет получено первым. Количество отображаемых калькулятором цифр считать неограниченным, время работы батареек — тоже.

Ограничения: $1 \leq K \leq 999$, время 1 с.

Ввод из файла `calcgamе.in`. В первой строке находится одно число — K .

Вывод в файл `calcgamе.out`. Если цели достичь невозможно, вывести «Impossible», если возможно, вывести два числа через пробел: цифру, из которой состоит искомое число, и количество цифр в числе.

Примеры

Ввод 1 **Ввод 2**

37 25

Вывод 1 **Вывод 2**

1 3 Impossible

Задача 15 В. Площадь треугольника

Даны длины трех отрезков. Если возможно, требуется построить треугольник, в котором один из этих отрезков был бы высотой, один — биссектрисой и один — медианой; все отрезки должны исходить из одной вершины.

Ограничения: длина каждого из трех отрезков от 0,01 до 100, точность результата должна быть 0,001, время 1 с.

Ввод из файла `tria-abm.in`. Вводятся три положительных числа, разделенных пробелами, — длины отрезков.

Вывод в файл `tria-abm.out`. Выводится одно число — площадь треугольника. Если треугольник нельзя построить, вывести -1. Если может быть построено несколько треугольников с разными площадями, вывести 0.

Примеры

Ввод 1	Ввод 2
4.349 4.0 5.0	3.1 3.1 3.1
Вывод 1	Вывод 2
16.0	0

Задача 15 С. Формирование поезда

Компания, занимающаяся железнодорожными перевозками, получила заказ сформировать поезд, состоящий из определенного числа вагонов. Проблема в том, что у компании есть вагоны, выпущенные в разное время, так что каждый из вагонов может иметь один из двух видов сцепления на каждом конце. У компании также есть один локомотив.

Сцепления и для локомотива, и для вагонов обозначены буквой А или В. Повернуть вагон или локомотив противоположной стороной невозможно.

Дана информация о вагонах и локомотиве. Требуется найти число способов сформировать разные поезда заданной длины из имеющихся видов вагонов. Дополнительным требованием является то, что тип сцеплений на каждом конце состава должен соответствовать типу сцепления локомотива.

Поезда считаются различными, если при их сравнении от одного конца к другому выявляется хотя бы одно отличие.

Пример 1. Пусть у компании есть вагоны АА, АА, АВ, ВА, ВА и локомотив АВ. В поезде должно быть 4 вагона. Из данных вагонов можно сформировать только два различных поезда: ВАААВВА и ВААВВААА. Локомотив можно присоединить к поезду как с левого (используя сцепление В), так и с правого конца (используя сцепление А).

Пример 2. Пусть у компании есть только по одному вагону каждого типа (АА, АВ, ВА, ВВ) и локомотив АА, а поезд должен состоять из трех вагонов. Существует три способа сформировать поезд: АААВВА, АВВААА и АВВВВА.

Ввод из файла `train-ab.in`. В первой строке находятся N — число вагонов, находящихся в распоряжении компании, и K — требуемая длина поезда в вагонах. Вторая строка описывает тип сцеплений локомотива. Следующие N строк описывают типы сцепления вагонов. Описания даны как АВ, АА, ВВ или ВА.

Ограничения: $1 \leq K \leq N \leq 40$, время 1 с.

Вывод в файл `train-ab.out`. В первой строке выводится слово «YES», если можно сформировать хотя бы один поезд, или «NO» — в противном случае.

Если поезд сформировать возможно, во второй строке должно указываться количество способов это сделать.

Примеры

Ввод 1	Ввод 2
4 4	4 4
АВ	ВА
АА	АА

AB	AB
BA	BA
BA	BA
Вывод 1	Вывод 2
YES	NO
2	

Задача 15 D. Стена

Жил-был жадный Король. Он приказал своему главному Архитектору построить стену вокруг его замка. Король был таким жадным, что не послушал предложение Архитектора построить красивую кирпичную стену совершенной формы с изящными высокими башнями. Вместо этого он приказал построить стену вокруг всего замка, используя минимальное количество камня, но потребовал, чтобы стена не подходила к замку ближе некоторого расстояния. Если Король узнает, что Архитектор использовал больше ресурсов для постройки стены, чем было абсолютно необходимо для удовлетворения требований, Архитектор лишится головы. Более того, Архитектор должен представить проект стены, где указано точное количество ресурсов (рис. 15.1).

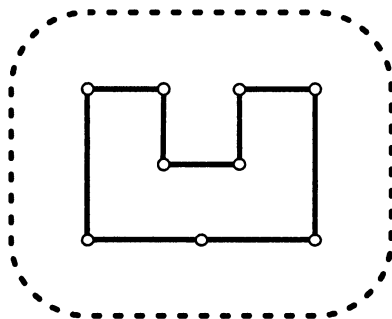


Рис. 15.1. Замок, окруженный стеной

Ваша задача — помочь бедному Архитектору сохранить голову, написав программу, определяющую минимальную длину стены, которую можно построить вокруг замка, удовлетворив требования Короля.

Задача слегка упрощается тем, что замок Короля представляет собой многоугольник и расположен на плоской поверхности. Архитектор уже сопоставил замку прямоугольную декартову систему координат и точно определил координаты каждого угла замка в футах.

Ввод из файла `wall.in`. Первая строка содержит два целых числа N и L , разделенных пробелом: N — число углов в замке Короля, а L — минимальное число футов, на которое Король разрешил приблизить стену к замку.

Следующие N строк описывают координаты углов замка в порядке обхода по часовой стрелке. Каждая строка содержит два целых числа x_i и y_i , разделенных пробелом

и представляющих собой координаты i -го угла в футах. Все углы имеют различные координаты, и стены замка не пересекаются иначе как в углах.

Ограничения: $3 \leq N \leq 1000$, $1 \leq L \leq 1000$, $-10\,000 \leq x_i, y_i \leq 10\,000$, время 2 с.

Вывод в файл wall.out. Выводится единственное число — минимальная длина стены в футах, которая может быть построена вокруг замка согласно требованиям Короля. Представить Королю длину стены нужно в виде целого числа футов, потому что вещественные числа еще не изобретены. Однако результат нужно округлить так, чтобы он отличался не более чем на 8 дюймов от правильного (1 фут = 12 дюймов), потому что большей неточности Король не потерпит.

Пример

Ввод

```
9 100
200 400
300 400
300 300
400 300
400 400
500 400
500 200
350 200
200 200
```

Вывод

1628

Задача 15 Е. Семечки

На базаре есть ряд из N мест, где продаются семечки подсолнечника. Потенциальные покупатели идут вдоль ряда, затем в некоторый момент останавливаются и покупают семечки. Качество семечек от места к месту различается незначительно, так что разница только в цене семечек и положении места.

Перед тем как выйти на рынок в качестве еще одного продавца семечек, вы провели исследование рынка, чтобы найти зависимость числа покупателей от двух названных факторов. Исследование показывает, что большинство покупателей следуют одному и тому же шаблону. Они проходят мимо нескольких мест, замечая и запоминая цены, а затем после обхода K мест возвращаются к месту с наименьшей замеченной ценой, совершают там покупку, затем покидают базар. Если есть несколько мест с одинаковой ценой, покупатель выбирает ближайшее.

Предположим, что есть пять мест с ценами 37, 34, 34, 35, 33. Если покупатель с $K = 4$ идет слева направо, он видит семечки по ценам 37, 34, 34, 35. В этот момент он решает, что видел достаточно, возвращается к третьему месту и покупает семечки там. Хотя на втором месте цена та же, что и на третьем, покупателю до него идти дальше. Если бы тот же покупатель зашел справа, он бы увидел цены 33, 35, 34, 34, затем остановился и вернулся бы к пятому месту.

Число мест, пройденных до принятия решения (K), является функцией жадности и терпеливости покупателя, и, очевидно, различается у разных покупателей. Исследование выявило средний процент B_K покупателей для всех значений K

$$(1 \leq K \leq N, 0 \leq B_K \leq 99, \sum_{k=1}^N B_k = 100).$$

Вам следует определить оптимальную стратегию на этом рынке (то есть цену и положение нового места, которое максимизирует ожидаемый средний доход) в предположении, что половина клиентов идет в направлении от первого места к N -му, а другая половина — от N -го места к первому, и они следуют описанному шаблону.

Ввод из файла `seeds.in`. В первой строке находится число существующих мест N , во второй строке — N целых чисел — цены на каждом месте, в третьей строке — N целых чисел в диапазоне от 0 до 99 — значения B_K для каждого K . Все числа в строках разделены пробелами.

Ограничения: $2 \leq N \leq 100$, исходные цены — целые числа от 1 до 9999, время 2 с.

Вывод в файл `seeds.out`. Выводятся два целых числа — L и P . L ($0 < L < N$) — это число существующих мест, после которых должно быть размещено новое (вам не разрешается устанавливать свое место первым или последним). Число P — оптимальная цена. Если существует более чем одно оптимальное решение, вы должны выбрать решение с минимальным L , а среди них — с минимальным P .

Пример

Ввод

```
5
37 34 34 35 33
10 20 30 30 10
```

Вывод

```
2 33
```

Задача 15 F. Умножение многочленов

Ввести в символьной форме два многочлена от x с целыми коэффициентами и вывести их произведение в порядке убывания степеней — также в символьной форме.

Ограничения: степень исходных многочленов не более 10, коэффициенты исходных многочленов по модулю не более 10^4 , время 1 с.

Ввод из файла `polymul.in`. В двух строках находятся многочлены.

Вывод в файл `polymul.out`. В единственной строке выводится многочлен.

Примеры

Ввод 1	Ввод 2	Ввод 3
0	$x+1$	-5
0	$x-1$	$x^2+x+x-2x^3$
Вывод 1	Вывод 2	Вывод 3
0	x^2-1	$10x^3-5x^2-10x$

ЧАСТЬ 2. РЕШЕНИЯ

Тренировка 1

Решение задачи 1 А. Простые числа

Тема: теория чисел

Под ограничения задачи (по времени и интервалу простых чисел) подходит следующее простое решение. Проходим от M до N и определяем, является ли каждое из чисел простым. Простоту определяем следующим образом: смотрим, делится ли это число на какое-нибудь из чисел от 2 до квадратного корня из этого числа включительно. Если оно на что-то делится, то число не простое, если ни на что не делится — простое.

Докажем, что достаточно проверить делимость только на числа, не превосходящие корня из числа, простота которого определяется. Пусть у положительного числа X есть положительный делитель $Y > \sqrt{X}$. Значит, $\frac{X}{Y}$ — натуральное число, обозначим его Z . Докажем, что $Z < \sqrt{X}$. Предположим противное: $Z \geq \sqrt{X}$. Тогда из того, что $Y > \sqrt{X}$ и $Z \geq \sqrt{X}$ следует, что $YZ > \sqrt{X}\sqrt{X} = X$, то есть $YZ > X$. Но $YZ = X$, по определению Z . Пришли к противоречию. Отсюда следует, что $Z < \sqrt{X}$. Итак, доказано, что если у числа есть делитель, больший корня из него, то найдется и меньший корня. А значит, если число не делится ни на одно число между 2 и корнем из него включительно, то оно простое. Следует отметить, что делимость на сам корень нужно проверять: например, $25 = 5 \cdot 5$ не является простым именно из-за этого делителя.

Детали реализации:

- Находить, делится ли X на Y , нужно с помощью целочисленной операции `mod`. Вещественные операции (деление «/», `frac`, `int`) выполняются намного дольше.
- Нужно прекращать проверять число на простоту, как только оно на что-то поделилось. Например, программа
`isPrime := true;`

```
for i := 2 to trunc(sqrt(x)) do
  if x mod i = 0 then
    isPrime := false;
```

работает намного дольше отведенного времени потому, что она делит на все числа до корня. И это при том, что каждое второе число четное, то есть то, что оно не простое, выясняется после первой же проверки.

- Вычисление корня числа — довольно долгая операция, намного дольше, чем сравнение, сложение или `mod`. Поэтому для каждого числа от M до N корень нужно находить только один раз. Например, фрагмент программы

```
isPrime := true;
divisor := 2;
while (divisor <= trunc(sqrt(x))) and isPrime do begin
  if x mod divisor = 0 then
    isPrime := false;
  divisor := divisor + 1;
end;
```

работает намного больше отведенного времени потому, что корень из числа вычисляется столько раз, сколько проверок условия цикла проводится. Это особенность циклов `while` и `repeat` — условие вычисляется каждый раз заново. Чтобы достичь нормального быстродействия, используя эти циклы, нужно вычислить корень заранее и занести в отдельную переменную, которую затем и использовать в условии цикла. В цикле же `for` границы вычисляются один раз — при входе в него. Поэтому для цикла `for` выражение `trunc(sqrt(x))` не требуется хранить в отдельной переменной.

- Если в исходном тексте программы написать директиву `{ $N+, E- }`, то все вещественные операции (извлечение корня, округление) будут выполняться значительно быстрее. Например, программа

```
var
  i, t : longint;
begin
  for i := 1 to 2000000 do
    t := trunc(sqrt(i));
end.
```

на Intel Celeron 400 с ключами по умолчанию работает 24,5 с, а с директивой `{ $N+, E- }` — 1,3 с.

ВНИМАНИЕ

Обратите внимание на то, что именно нужно выводить в случае отсутствия простых чисел в заданном промежутке.

В олимпиадных задачах нужно выводить именно то, что указано в условии. С заглавной ли буквы должно быть сообщение? Нужна ли точка после числа? Все это очень важно.

Решение задачи 1 В. Выражение

Вариант 1

Темы: перебор 2^n , рекурсия.

Глобальные объявления:

```
const
    maxN = 24;
var
    x : array [1..maxN] of longint;
    sign : array [2..maxN] of char;
    n : integer;
    s : longint;
```

В массиве x с 1-го по N -й элемент находятся исходные значения. Массив $sign$ хранит знаки — $sign[i]$ хранит знак перед i -м числом ('+' или '-'). Обратите внимание на то, что, по условию задачи, перед первым слагаемым знака быть не может. Теперь наша задача — заполнить массив $sign$ всеми возможными способами и подсчитать суммы для этих способов. Если какая-нибудь из этих сумм будет равна S , то нужно вывести соответствующее выражение и завершить работу программы, а если ни одна не будет равна S , вывести сообщение об этом (обратите внимание на регистр, количество пробелов между словами и наличие точки).

Особенность задачи заключается в том, что если просто сгенерировать все варианты массива $sign$ и для каждого отдельно найти сумму, то такое решение не пройдет по времени при $N = 24$. В этой задаче требуется по мере изменения массива $sign$ получать сумму для новой комбинации небольшим изменением суммы предыдущей комбинации. Рекурсивная процедура, реализующая перебор, такова:

```
procedure p(k : integer; sum : longint);
```

Эта процедура названа ничего не значащим именем p . Все дело в том, что название процедуры, отражающее ее смысл, выглядело бы так: «Перебрать все варианты заполнения в массиве $sign$ элементов со 2-го по k -й символами '+' и '-'». Очевидно, нет никакого смысла переводить эту фразу на английский и пытаться создать из нее имя процедуры. Вот почему обычно подобные процедуры имеют ничего особого не значащие имена: $perebor$, $recurse$, rec и т. п.

При вызове этой процедуры уже определены элементы массива $sign$ от $(k+1)$ -го до N -го включительно (k — первый параметр процедуры), причем сумма соответствующих $x[i]$ с учетом знака перед ними уже подсчитана в sum (второй параметр).

Процедура работает так. Если $k = 1$, то ей перебирать ничего не надо — все знаки уже определены. Нужно только сравнить накопленную сумму (sum) с заданной. А если $k > 1$, то процедура перебирает знак перед k -м слагаемым ('+' или '-') и для каждого из этих двух случаев вызывает себя рекурсивно, чтобы перебрать знаки перед элементами со 2-го по $(k-1)$ -й.

Рассмотрим работу процедуры более подробно.

Конечный случай. Если $k = 1$, то все знаки уже расставлены и сумма $x[i]$ для $i = 2 \dots N$ уже подсчитана. Ничего не стоит определить, соответствует ли эта сумма S (не забудьте $x[1]$). Если сумма соответствует, нужно ее вывести и сделать одно из двух: или сразу же завершить выполнение программы с помощью `halt`, или запомнить, что сумма уже выводилась и больше выводить ничего не надо.

Продолжение рекурсии ($k > 1$). При этом нужно перебрать все возможные значения $\text{sign}[k]$ — в данном случае это только '-' и '+'. Сначала в $\text{sign}[k]$ заносится значение '-', и процедура вызывает себя рекурсивно с параметрами $k-1$ и суммой, в которую $x[k]$ вошел с минусом. Потом в $\text{sign}[k]$ заносится значение '+', и процедура вызывает себя рекурсивно с параметром $k-1$ (опять) и суммой, в которую $x[k]$ вошел с плюсом.

Вызов рекурсии:

`p(n, 0):`

В первый раз вызванная процедура будет ставить N -й знак, а сумма уже расставленных элементов равна нулю.

ПРИМЕЧАНИЕ

Рекурсивные процедуры интенсивно используют стек. После того как программа заработает (но не раньше!), можно убрать проверку глубины стека, которая по умолчанию включена. Для этого нужно в начале программы приписать `{ $S- }`.

Вариант 2

Тема: перебор 2^n .

Получить все возможные последовательности из плюсов и минусов можно и без использования рекурсии, например, используя двоичную систему счисления: минус сопоставить нулю, плюс — единице. Двоичные представления чисел от 0 до $2^N - 1$, дополненные слева нулями до N двоичных разрядов, дают все возможные последовательности длиной N из нулей и единиц (на самом деле нужны последовательности длиной $N - 1$ — именно столько имеется мест для постановки знаков *между* N числами).

Будем получать комбинации в естественном порядке: сначала соответствующую 0, затем 1, затем 2 и так далее до $2^{N-1} - 1$. Пусть известно двоичное представление числа X . Как по этому представлению найти двоичную запись числа $X + 1$?

Посмотрим, как эта задача решается в десятичной системе счисления. Если последняя цифра числа не 9, то она увеличивается на единицу. Если последняя цифра числа 9, а предпоследняя — не 9, то последняя цифра заменяется нулем, а предпоследняя увеличивается на 1. Аналогично, если две последние цифры числа — девятки, а третья с конца — нет, то опять концевые девятки заменяются нулями, а первая с конца не-девятка увеличивается на 1. Если число состоит только из девяток, будем считать, что слева у него есть еще ноль. Тогда такой случай не будет особым.

В двоичной системе счисления все происходит точно так же. Только роль самой большой цифры вместо 9 играет 1. Чтобы получить следующее двоичное число,

нужно все концевые единицы заменить нулями, а первый с конца ноль заменить единицей, например:

1010 -> 1011

100011 -> 100100

111 = 0111 -> 1000

Глобальные объявления лишь немного отличаются от объявлений решения 1:

```
const
    maxN = 24;
var
    x : array [1..maxN] of longint;
    sign : array [1..maxN] of char;
    n : integer;
    s : longint;
    cursum : longint;
```

Сначала массив `sign` инициализируется минусами. Это соответствует двоичному нулю. Затем получаем комбинацию, соответствующую двоичной единице, — `sign[n] = '+'`, в остальных ячейках находятся минусы, то есть `sign[n]` соответствует младшему двоичному разряду. Когда нужно прекращать перебор? Когда очередная полученная двоичная комбинация имеет больше $N - 1$ разряда ($N - 1$, а не N потому, что нужно расставить знаки *между* N числами, то есть $N - 1$ знак). Признаком этого является условие `sign[1] = '+'`. Фиктивный N -й разряд `sign[1]` (в решении 1 его нет) существует специально для того, чтобы узнать, когда заканчивать перебор.

Не пройдет по времени такое решение: получать все возможные комбинации плюсов и минусов и для каждой проверять равенство получившегося выражения заданному числу. Нужно хранить (в переменной `cursum`), значение выражения с текущей расстановкой знаков. При изменении знаков нужно соответствующим образом корректировать `cursum`, а не вычислять заново. Если `sign[i]` изменяется с минуса на плюс, то к `cursum` нужно добавить два `x[i]` — раньше X_i входил в сумму с минусом, а теперь входит с плюсом. Наоборот, при изменении `sign[i]` с плюса на минус из `cursum` нужно вычесть два `x[i]`.

Корректировать `cursum` быстрее, чем вычислять значения выражения каждый раз заново. Ведь в половине случаев для получения следующего двоичного числа нужно поменять только один младший разряд с 0 на 1 (или, в наших терминах, с '-' на '+'). В половине оставшихся случаев нужно поменять два разряда. В половине оставшихся — три и т. д. Получается, что в среднем меняются $1\frac{1}{2} + 2\frac{1}{4} + 3\frac{1}{8} + 4\frac{1}{16} + 5\frac{1}{32} + 6\frac{1}{64} + \dots$ разрядов. Сумма этого ряда равна 2. Для $N = 24$ отличие в шестом знаке после запятой. Фактически для больших N (а для маленьких оптимизация количества действий не требуется) выполняется около двух сложений с `cursum` на каждую комбинацию плюсов и минусов. Если бы сумма вычислялась каждый раз заново, то для $N = 24$ требовалось бы 23 операции, то есть на порядок больше.

Можно возразить, что в описанном ранее алгоритме добавляются или вычитаются удвоенные значения элементов массива x , так что на выполнение операции умножения требуется дополнительное время. На самом деле можно объявить массив $x2$ и занести в него удвоенные значения из массива x . Но по времени проходит даже алгоритм с умножением на 2.

Так как изначально `sign` заполняется минусами, начальным значением `cursum` должно быть совсем не 0, а $X_1 - X_2 - X_3 - \dots - X_N$.

Убедитесь в том, что реализованный вами алгоритм находит расстановку знаков и тогда, когда все знаки '-', и когда все знаки '+'.

Решение задачи 1 С. Возрастающая подпоследовательность

Тема: динамическое программирование.

Для каждого члена исходной последовательности нужно вычислить максимальную длину возрастающей подпоследовательности, оканчивающейся этим элементом.

Для примера

2 5 3 4 6 1

эта характеристика будет выглядеть так:

1 2 2 3 4 1

- Левее двойки ничего нет, поэтому для нее допустима последовательность только из нее самой, то есть длины 1.
- Для пятерки есть две подпоследовательности — 2 5 и просто 5. Максимальная длина 2.
- Для тройки возможны подпоследовательности 2 3 и просто 3. Последовательность 2 5 3 не рассматривается — она не возрастающая.
- Для четверки возможны подпоследовательности 2 3 4, 2 4, 3 4 и просто 4. Максимальная длина у подпоследовательности 2 3 4.
- Для шестерки возможны подпоследовательности 2 3 4 6, 3 4 6, 2 3 6, 2 4 6, 2 5 6, 2 6, 5 6, 3 6, 4 6 и просто 6.
- У единицы единственная подпоследовательность — из нее самой.

Будем вычислять характеристики для всех членов последовательности от 1-го до N -го по порядку и заносить полученные результаты в массив.

Пусть известны характеристики для всех членов последовательности от 1-го до $(i - 1)$ -го и нужно узнать ее для i -го. Последовательность длиной 1 из одного (i) -го элемента всегда можно построить. Пусть можно построить последовательность длиной не меньше двух. Тогда какой-то из элементов от 1-го до $(i - 1)$ -го будет предпоследним. Очевидно, что предпоследним может быть любой элемент, меньший i -го. А наилучшая характеристика у i -го элемента получится, если взять предыдущий элемент с максимальной характеристикой.

Итак, чтобы получить максимальную длину подпоследовательности, кончающейся i -м элементом, нужно выбрать максимум из длин подпоследовательностей элементов от 1-го до $(i - 1)$ -го, меньших i -го, и добавить единицу. Если меньших элементов не существует, получится длина 1.

Очевидно, длина искомой подпоследовательности равна максимуму из найденных длин каждого элемента. А соответствующий максимум элемент является последним в последовательности, которую нужно вывести. Предыдущие элементы восстанавливаются следующим образом. Идем по массивам от индекса максимума к 1. Находим элемент, меньший максимума, с длиной последовательности, кончающейся им, на 1 меньше максимальной длины. Продолжаем идти к меньшим индексам, находим еще меньший элемент с длиной последовательности, меньшей еще на 1, и так до тех пор, пока не закончится массив.

Детали реализации:

- Для хранения исходных значений нужен массив из элементов типа `word`. В переменные типа `integer` (возможные значения от $-32\,768$ до $32\,767$) исходные данные не поместятся. Диапазон чисел, представимых типом `word`, — от 0 до 65 535, то есть для этой задачи подходит. Почему не рекомендуется использовать тип `longint`? Диапазон значений, представимых типом `longint`, — от $-2\,147\,483\,648$ до $2\,147\,483\,647$ — подходит. Но переменные типа `longint` занимают 4 байта (в отличие от 2 байтов для `integer` и `word`), поэтому работа с ними в Турбо Паскале происходит медленнее, чем с переменными типа `integer`. А работа с данными типа `word` происходит со скоростью работы с данными типа `integer`.
- Согласно приведенному алгоритму, элементы искомой подпоследовательности будут найдены в порядке, обратном тому, в котором их нужно вывести. Чтобы не объявлять дополнительный массив, при последнем проходе можно числа, не входящие в последовательность, заменять нулями (во входных данных 0 встретиться не может, там числа только от 1 до 60 000). А потом пройти по массиву от 1-го до N -го элемента и вывести все ненулевые числа.

Модификация решения

Чтобы избежать дополнительного прохода по массиву, можно находить другую величину — максимальную длину возрастающей подпоследовательности, *начинающейся* этим элементом. Тогда эта характеристика находится для i -го элемента на основании характеристик элементов с $(i + 1)$ -го по N -й.

Дополнение решения

Нашли Антон Малафеев и Федор Меньшиков.

Описанный способ вывода самой последовательности после расчета характеристики можно упростить — проходя по массивам, находить не меньший элемент с длиной подпоследовательности на 1 меньше длины подпоследовательности предыдущего выделенного, а просто первый попавшийся элемент с длиной подпоследовательности на 1 меньше длины подпоследовательности предыдущего.

Действительно, пусть подходит элемент X_i . Покажем, что также подходит любой элемент X_j ($j > i$) с такой же характеристикой. Из равенства характеристик следует, что $X_j \leq X_i$. Действительно, если бы выполнялось $X_j > X_i$, то характеристика у X_j была бы как минимум на 1 больше — X_i можно было бы включить в качестве предпоследнего члена в возрастающую подпоследовательность, оканчивающуюся X_j .

Получается, что при равных характеристиках элемент с большим индексом имеет меньшее значение. Значит, при проходе от больших индексов к меньшим первый попавшийся элемент с нужной характеристикой подойдет — если он не будет меньше предыдущего выделенного элемента, то другие (с меньшими индексами) элементы — и подавно.

Решение задачи 1 D. Треугольник и точка

Тема: геометрия.

Вариант 1

Уравнение прямой вида $Ax + By + C = 0$ обладает следующим свойством. Если в него подставить координаты точки, принадлежащей этой прямой, то оно превратится в тождество. Если точку взять из одной из полуплоскостей, на которые прямая делит плоскость, то будет выполняться неравенство $Ax + By + C < 0$, а если из другой — то $Ax + By + C > 0$. То есть если есть такое уравнение прямой и координаты двух точек, то очень легко узнать, находятся ли эти точки в одной полуплоскости относительно этой прямой или в разных.

Рассмотрим уравнения трех прямых, проходящих через две вершины треугольника. Если для всех трех прямых заданная точка лежит в той же полуплоскости, что и третья вершина треугольника (через которую не проходит прямая), то она принадлежит треугольнику.

Детали реализации:

- Обратите внимание на то, что следует отвечать, если точка лежит на границе треугольника. В условии в явном виде ответа нет, но следует обратить внимание на примеры. В одном из них такой случай рассматривается.
- Лучше всего запоминающееся уравнение прямой, проходящей через две точки, такое: $\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1}$. Преобразовать его к виду $Ax + By + C = 0$ не составляет труда.
- Наиболее часто встречающейся при решении этой задачи ошибкой является использование неподходящих типов данных. Увидели, что в исходных данных ограничение до 10 000, — и решили использовать `integer`. В описанном алгоритме при подстановке точки с такими координатами в уравнение прямой получаются значения, измеряемые сотнями миллионов. И неважно, переменной какого типа они присваиваются и присваиваются ли вообще — тип промежуточного результата — наибольший из типов операндов. Так что если вы умно-

жили одну переменную типа `integer`, содержащую 10 000, на вторую такую же, то правильного ответа не получите, потому что точное значение такого произведения не является допустимым значением типа `integer`. В лучшем случае вы узнаете об ошибке, получив ответ «Runtime error» — ошибка времени выполнения. Но это только если примените директиву `{ $R+.Q+ }`. В худшем же случае вы будете получать ответы «Wrong answer» — неверный ответ и думать, что идея вашего решения неправильная, тогда как неправильная только реализация. Один из лучших способов решения этой проблемы — в начале программы приписать

`type`

`integer = longint;`

Это позволит использовать в программе слово `integer`, подразумевая тип `longint`. Иногда, правда, требуется именно тип `integer` — например, для третьего параметра процедуры `val`. В таком контексте нужно будет писать `system.integer` вместо `integer`.

- Только в математике можно свободно писать $A \cdot B < 0$ для обозначения того, что одно число положительное, а второе — отрицательное. Написав такое в программе на Паскале, нужно хорошо подумать, может ли быть представлено значение $A \cdot B$ максимальным из типов A и B .

Пусть, например, $A_{1,2}x + B_{1,2}y + C_{1,2} = 0$ — уравнение прямой, проходящей через первую и вторую вершины треугольника, (x_3, y_3) — координаты третьей вершины треугольника, а (x_0, y_0) — координаты точки, принадлежность которой треугольнику нужно определить. Рассмотрим, можно ли записывать, что точки лежат не по разные стороны от прямой, следующим образом: $(A_{1,2}x_3 + B_{1,2}y_3 + C_{1,2})(A_{1,2}x_0 + B_{1,2}y_0 + C_{1,2}) \geq 0$.

При ограничениях задачи (значения координат по модулю не превосходят 10 000) значение каждого из множителей $(A_{1,2}x_3 + B_{1,2}y_3 + C_{1,2})$ и $(A_{1,2}x_0 + B_{1,2}y_0 + C_{1,2})$ может достигать нескольких сотен миллионов. Очевидно, их произведение не может быть представлено числом типа `longint`. Поэтому нужно или использовать вещественный тип для хранения координат точек и коэффициентов, или использовать тип `longint`, но отказаться от использования умножения для проверки того, что точки лежат не по разные стороны от прямой.

- В описанном решении выполняется мало вычислений. Поэтому лучше поставить проверки: `{ $R+.Q+ }`. Работу они не замедлят, а в случае переполнения вы узнаете о нем.
- Обратите внимание на то, что именно нужно выводить. В олимпиадных задачах нужно выводить именно то, что упомянуто в условии. С заглавной буквы? Есть ли точка в конце? Все это очень важно.

Вариант 2

Если точка D находится внутри треугольника ABC или на его границе, то сумма площадей треугольников ABD , BCD и CAD равна площади треугольника ABC , а если вне — то сумма площадей больше.

В общем случае такой подход нельзя назвать хорошим, так как площадь треугольника, вершины которого имеют вещественные координаты, получается приближенной, и поэтому, даже если точка находится внутри треугольника, сумма площадей треугольников ABD , BCD и CAD может чуть-чуть не совпасть с площадью треугольника ABC .

Для целых координат точек все обстоит значительно лучше. Площадь треугольника с вершинами в точках с целочисленными координатами может быть найдена и представлена в компьютере точно. Дело в том, что она — полуцелое число, то есть или оно само целое, или целым является его удвоенное значение.

Докажем полуцелость площади. Впишем треугольник в минимальный прямоугольник со сторонами, параллельными осям координат. Площадь прямоугольника будет равна сумме площади исходного треугольника и нескольких прямоугольных треугольников. Очевидно, площадь прямоугольника с вершинами в точках с целочисленными координатами — целое число. А площадь каждого прямоугольного треугольника — полуцелое число (произведение целой высоты на целое основание, деленное пополам).

Так что можно любым способом находить площадь четырех треугольников, умножать каждую из них на 2, округлять до ближайшего целого числа и для этих целых чисел проверять равенство.

Детали реализации:

- Самая простая формула для поиска удвоенной площади треугольника по координатам вершин, — $\text{abs}((x_2 - x_1) * (y_3 - y_1) - (x_3 - x_1) * (y_2 - y_1))$. Получается она из модуля векторного произведения

$$\begin{vmatrix} i & x_2 - x_1 & x_3 - x_1 \\ j & y_2 - y_1 & y_3 - y_1 \\ k & 0 & 0 \end{vmatrix}.$$

- См. комментарии к варианту 1 относительно типов данных и проверок.

Вариант 3

Решение нашли Екатерина Грозина и Федор Меньшиков.

Если точка принадлежит всем трем углам треугольника, то она принадлежит треугольнику. Чтобы точка D принадлежала треугольнику ABC , ей достаточно лежать внутри углов BAC , ABC и ACB (рис. P1.1).

Принадлежность углу эквивалентна принадлежности двум полуплоскостям, то есть точка D лежит внутри угла ABC тогда и только тогда, когда она лежит в одной полуплоскости с точкой C относительно прямой AB , а также в одной полуплоскости с точкой A относительно прямой BC .

Если это свойство использовать напрямую, получится удвоенный вариант 1. Оно требуется, чтобы доказать следующее утверждение: для принадлежности точки треугольнику достаточно принадлежности ее любым *двум* углам треугольника.

Действительно, если взять два угла, то, согласно этому свойству, принадлежность двум углам эквивалентна принадлежности всем трем полуплоскостям из варианта 1.

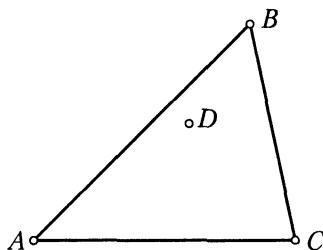


Рис. Р1.1. Точка в треугольнике

Сократить количество углов с трех до двух пришлось потому, что в описываемом далее решении очень неудобен угол треугольника $\geq 120^\circ$. Так как сумма углов треугольника равна 180° , то двух углов $\geq 120^\circ$ в нем быть не может. Значит, в треугольнике есть два угла $< 120^\circ$.

Принадлежность точки D углу $ABC < 120^\circ$ можно определить следующим образом: величина угла ABD не превосходит величины угла ABC , и величина угла CBD не превосходит величины угла ABC . Обратите внимание на то, что для угла $\geq 120^\circ$ этот критерий не выполняется (рис. Р1.2).

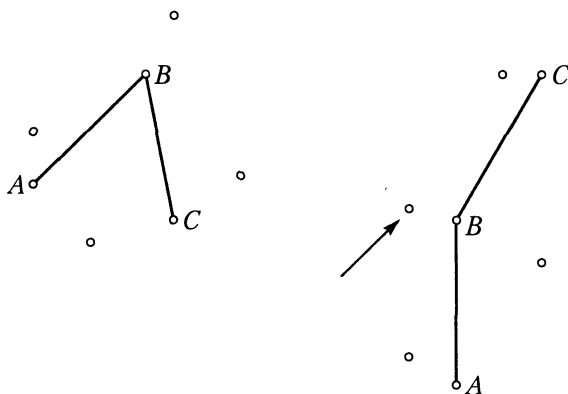


Рис. Р1.2. Точки вокруг углов, меньших и больших 120°

Слева изображен угол $< 120^\circ$, для всех четырех точек, расположенных около него, критерий выполняется. Справа изображен угол $\geq 120^\circ$, и стрелка указывает на точку D вне угла, для которой углы ABD и CBD все-таки не превосходят угла ABC .

Детали реализации:

- Прежде всего нужно определить, не совпадает ли точка D с какой-то из точек A , B или C . Если совпадает — можно выводить ответ. Дело в том, что если есть совпадение, то при получении косинуса через скалярное произведение векто-

ров (см. далее) произойдет деление на ноль, поэтому случай совпадения лучше исключить заранее.

- Косинус угла между двумя векторами можно найти следующим образом:

```
len1 := sqrt(sqr(dx1) + sqr(dy1));
```

```
len2 := sqrt(sqr(dx2) + sqr(dy2));
```

```
cosa := (dx1 * dx2 + dy1 * dy2) / (len1 * len2);
```

- Сравнение углов величиной от 0 до 180° , $\alpha < \beta$, эквивалентно сравнению косинусов этих углов со сменой порядка: $\cos \beta < \cos \alpha$.
- Можно не выяснять, какой из углов треугольника $\geq 120^\circ$. Если угол $\geq 120^\circ$, то некоторые точки, не принадлежащие углу, описанный метод будет считать принадлежащими ему, но ни одну точку, принадлежащую углу, он не будет считать не принадлежащей. Поэтому точки, случайно прошедшие по критерию для этого угла, будут отсеяны формулами для двух других углов $< 120^\circ$.

Вариант 4

Решение предложил Евгений Белов.

Если точка находится строго внутри треугольника, то сумма углов, под которыми из нее видны стороны, равна 2π . Если точка находится вне треугольника, то эта сумма меньше 2π .

Детали реализации:

- Как и в варианте 3, следует сразу же отсеять случай, когда проверяемая точка совпадает с одной из вершин треугольника, в противном случае в приведенном далее решении произойдет деление на ноль. Когда точка лежит на границе треугольника, приведенные ниже формулы дадут правильный ответ.
- Нахождение косинуса угла между векторами — см. вариант 3. Здесь требуется по косинусу угла найти еще и сам угол. В Турбо Паскале нет функции $\arccos$, есть только $\arctan$, то есть можно получить угол, если известен его тангенс. Чтобы воспользоваться $\arctan$, найдем значение тангенса через значение косинуса: $\operatorname{tg} \alpha = \frac{\sin \alpha}{\cos \alpha}$, $\sin \alpha = \sqrt{1 - \cos^2 \alpha}$. То есть если косинус угла равен x , то угол можно получить по формуле $\arctan(\sqrt{1 - \cos^2(x)} / x)$.
- Используя функцию $\arctan$, всегда следует иметь в виду, что она возвращает значение из диапазона $\left(-\frac{\pi}{2}; \frac{\pi}{2}\right)$. В случае нахождения угла от 0 до π такое значение нужно скорректировать. Положительный угол менять не надо, а к отрицательному нужно добавить π .
- Если косинус равен нулю, то подставлять это значение в приведенную формулу нельзя — произойдет деление на ноль. Этот случай нужно рассмотреть отдельно — если косинус равен нулю, угол в нашем случае равен $\pi/2$.
- Найденное значение косинуса может немного отличаться от точного значения. Для приведенной формулы критическим является случай, когда на самом деле

косинус должен быть равен 1 (угол 0) или -1 (угол π), а он получается по модулю чуть больше 1. При подстановке такого значения в формулу произойдет ошибка вычисления корня из отрицательного числа. Поэтому случаи $x > 1$ и $x < -1$ нужно рассмотреть отдельно.

- Для вещественных вычислений лучше всего использовать вещественный тип с максимальной точностью — для Турбо Паскаля это `extended`. Для минимального изменения программы, использующей тип `real`, нужно в ее начале написать:

```
{ $N+, E- }
type
  real = extended;
```

Если не записать директиву, будет выдана ошибка компиляции.

- Как уже было отмечено, вещественные значения получаются неточными, поэтому нельзя сравнивать сумму углов с $2 * \pi$ с помощью оператора `=`. Нужно записать:

```
if abs(a1 + a2 + a3 - 2 * pi) < epsilon then
  writeln('In')
```

Значение `epsilon` выбирается таким, что погрешность вычисления суммы углов не превосходит этого значения. Для типа `real` возможная погрешность $1e-6$, для типа `extended` погрешность $1e-18$ (оба значения получены экспериментально).

Вариант 5

Решение предложил Ярослав Музыкантов.

Z -координата (аппликата) векторного произведения векторов, находящихся на плоскости xOy , равная $(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)$, обладает следующим свойством: если угол поворота от первого вектора до второго составляет от 0 до 180° *по* часовой стрелке, она имеет один знак, а если, наоборот, от 0 до 180° *против* часовой стрелки, ее знак противоположен. Когда векторы коллинеарны, Z -координата равна 0.

Идея решения такова. Если поворот *всех* векторов $\vec{DA} \rightarrow \vec{DB}$, $\vec{DB} \rightarrow \vec{DC}$, $\vec{DC} \rightarrow \vec{DA}$ (см. рис. P1.1) или *по* часовой стрелке, или *против* часовой стрелки, то точка лежит внутри треугольника, а если есть два перехода: один *по* часовой стрелке, а второй — *против*, то точка вне треугольника.

Таким образом, нужно найти три числа: Z -координаты векторных произведений $[\vec{DA}, \vec{DB}]$, $[\vec{DB}, \vec{DC}]$, $[\vec{DC}, \vec{DA}]$. Если среди них есть и положительное, и отрицательное, точка лежит вне треугольника, в противном случае — внутри.

Случай, когда точка лежит на стороне треугольника или совпадает с вершиной, особым не является. Если точка совпадает с вершиной, два числа из трех будут равны нулю. Если точка лежит на прямой, проходящей через сторону, одно из трех чисел будет равно нулю.

Детали реализации: см. комментарии к варианту 1 относительно типов данных и проверок.

Решение задачи 1 E. Степень

Как видно из примеров, число нужно выводить абсолютно точно, а представить абсолютно точно такие большие числа с помощью стандартных типов Турбо Паскаля нельзя.

Будем хранить цифры числа в массиве:

var

```
digit : array [1..7000] of integer;
```

Выбор длины массива происходит следующим образом. Максимальный результат равен 9^{7000} , он меньше 10^{7000} , а в десятичной записи числа 10^{7000} — 7001 цифра.

digit[1] будет представлять самый младший разряд (единицы), digit[2] — десятки, digit[3] — сотни и т. д.

Сначала занесем в digit[1] единицу, а затем умножим число, представленное этим массивом, n раз на a .

Будем хранить текущую длину числа в переменной len.

Тогда новое число получается из предыдущего следующим образом (carry — перенос в следующий разряд):

```
carry := 0;
for i := 1 to len do begin
    carry := carry + digit[i] * a;
    digit[i] := carry mod 10;
    carry := carry div 10;
end;
if carry <> 0 then begin
    inc(len);
    digit[len] := carry;
end;
```

После окончания основного цикла в carry лежит число, не превосходящее 10. Поэтому в случае carry <> 0 нужно увеличить длину числа на 1 и присвоить старшей цифре это значение.

Решение задачи 1 F. Покер

Вариант 1

Пусть в массиве A лежат исходные числа. Объявим еще два массива, B и C:

var

```
A : array [1..5] of integer;
```

B : array [1..13] of integer;

C : array [0..5] of integer;

$B[i]$ должно указывать, сколько раз значение i содержится в массиве A. Например, если в массиве A содержатся три двойки, то $B[2] = 3$. Алгоритм заполнения массива B очевиден — сначала обнуляем его, затем проходим по массиву A и увеличиваем элемент массива B, соответствующий значению текущего элемента массива A.

В массив C занесем аналогичную информацию о массиве B. Таким образом, массив A будет содержать значения карт, массив B будет содержать, сколько есть карт соответствующего достоинства, а массив C будет содержать информацию, сколько есть пар, троек, четверок карт. Очевидно, что, имея такую информацию, вывести результат можно с помощью небольшого количества сравнений элементов массива C с константами.

Особым случаем является Straight — массив C для него аналогичен случаю Nothing. Поэтому разделить эти случаи можно по массиву B: для Straight он должен содержать 5 единиц в ячейках с последовательными индексами.

Не забудьте побуквенно проверить, что все, что выдает ваша программа, соответствует тому, что написано в условии задачи. Проверьте регистр, наличие точки, кавычек и число пробелов. Для этой задачи есть проверяющая программа, так что в случае неправильного написания будет выдано не «Wrong answer» — неверный ответ, а «Presentation error» — ошибка представления. Но так бывает не во всех задачах — и будете вы искать ошибку в алгоритме, когда ошиблись всего в одной букве ответа.

Вариант 2

Нужно сосчитать количество пар одинаковых чисел. Например, в наборе

1 1 1 2 2

количество пар 4 (первое число равно второму, первое — третьему, второе — третьему, четвертое — пятому). Удивительно, но для всех названных в задаче комбинаций эта характеристика различна (кроме Nothing и Straight).

Если известно, что все карты различны, то, чтобы отделить Nothing от Straight, нужно найти максимальное и минимальное значения из исходных. Если они различаются ровно на 4, то ответом является Straight.

Вариант 3

Нужно отсортировать массив исходных данных, после этого проверка каждого отдельного случая сводится не более чем к 12 сравнениям, а в целом задачу можно решить не более чем за 30 сравнений.

Сравнение алгоритмов

В вариантах 1 и 2 значительно меньше шансов допустить ошибку, чем в варианте 3. Если для всех типов ответа вы проверили работу программы, реализующей варианты 1 или 2, то, вероятнее всего, у вас получилась абсолютно правильная программа, чего нельзя сказать о программе, реализующей вариант 3.

Тренировка 2

Решение задачи 2 А. Простые числа (2)

Тема: теория чисел

Эта задача отличается от задачи 1 А только одним — вдвое большими ограничениями на входные данные. Значит, простейший алгоритм для решения этой задачи не подходит.

Здесь подходит следующий алгоритм: перебирать все числа по порядку от M до N и проверять делимость каждого числа на все *простые* числа до корня из него.

Почему достаточно перебирать до корня, доказано в разборе задачи 1 А, а почему достаточно перебирать только простые — сейчас докажем.

Пусть число X разделилось на какое-то составное число Y . Так как Y составное, то у него есть простой делитель. Обозначим этот простой делитель Z . Так как Z — делитель Y , то $\frac{Y}{Z}$ — целое число. Так как Y — делитель X , то $\frac{X}{Y}$ — целое число. Значит,

и $\frac{X}{Z} = \frac{X}{Y} \cdot \frac{Y}{Z}$ — целое число, так как оно является произведением двух целых. Итак, если X делится на какое-то составное число Y , то оно делится и на простое число Z , являющееся простым делителем Y . Значит, если X не делится ни на какое простое от 2 до $\sqrt{X}$ включительно, то оно само является простым.

Детали реализации:

- Как делить только на простые числа? Очевидно, нельзя перебирать все числа до корня, каждое проверять на простоту, и если оно простое — проверять делимость. Такой алгоритм будет работать еще медленнее, чем просто деление на все числа до корня. Нужно вычислить все необходимые простые числа заранее и поместить их в массив. Какие числа необходимы? Максимальное число во входных данных — 1 000 000. Корень из него — 1000. Соответственно, нужны все простые числа от 2 до 1000. Для упрощения алгоритма рекомендуется вычислить все простые числа от 2 до первого простого числа после 1000 включительно.

Вычислить простые числа до 1000 можно с помощью алгоритма, описанного в решении задачи 1 А.

- Вычисление корня числа — довольно длительная операция. Поэтому нельзя вычислять корень даже один раз для каждого числа ($N - M + 1$ раз). Нужно

в отдельной переменной целого типа хранить целую часть корня из числа и изменять эту переменную при последовательном изменении числа. Пусть в переменной i находится текущее число, простота которого проверяется, а в переменной $\text{sqrt } i$ — целая часть корня из предыдущего числа, то есть справедливо равенство $\text{sqrt } i = \text{trunc}(\text{sqrt}(i - 1))$. Требуется обновить $\text{sqrt } i$ — вычислить $\text{trunc}(\text{sqrt}(i))$ без использования вещественных операций. Очевидно, корень поменяется только в том случае, если i является полным квадратом. Также очевидно, что при переходе от $i - 1$ к i корень или останется прежним, или увеличится на 1. Поэтому изменение корня можно отследить с помощью следующего условия: $\text{sqr}(\text{sqrt } i + 1) = i$. Таким образом, целую часть корня нужно вычислить только один раз — для M , а для остальных чисел нужно получать его из предыдущего

- Как и корень из текущего числа, можно не находить каждый раз заново, а получать из предыдущего индекс массива, соответствующий простому числу, до которого нужно проверять делимость.
- Не нужно забывать, что проверку простоты нужно прекращать сразу же, как только выяснилось, что число не простое.

Решение задачи 2 В. Перестановки

Тема: перебор $n!$

Вариант 1

Общая идея решения задачи такова: ставим на первое место все возможные символы, ставим на второе место все оставшиеся после заполнения первого места символы, на третье — все оставшиеся после заполнения первого и второго мест и т. д.

Глобальные объявления:

```
const
    maxN = 8;
var
    s : string[maxN]; {исходная строка}
    rez : string[maxN]; {строка-результат}
    used : array [1..maxN] of boolean;
    n : integer; {количество символов в строке}
```

$\text{Used}[i]$ хранится, использовался ли уже символ $s[i]$ в строящейся строке. В начале программы все значения false — на первое место может быть поставлен любой символ. После определения первого символа соответствующий элемент массива used устанавливается в true . При изменении первого символа старый элемент восстанавливается в false , а в true устанавливается новый элемент.

Рекурсивная процедура перебора:

```
procedure p(current : integer);
```

По традиции (см. вариант 1 разбора задачи 1 В) процедура названа именем *p*, хотя полное ее название может выглядеть так: «Перебрать все варианты расстановки символов *s[i]*, *i* = 1...*n*, таких, что *used[i]* = false, на места с *current*-го по *n*-е строки *rez*».

При вызове этой процедуры уже определены символы строки *rez* с 1-го до (*current*-1)-го, и в массиве *used* на соответствующих местах стоят true, а на местах, соответствующих еще не использованным символам, стоят false.

Процедура работает так. Если *current* = *n* + 1, то перебирать ничего не надо, и процедура выводит построенную комбинацию. Если же *current* ≤ *n*, то процедура ставит на место *current* все допустимые символы по очереди и для каждого варианта вызывает себя рекурсивно, чтобы расставить оставшиеся символы на места с (*current*+1)-го по *n*-е.

Рассмотрим работу процедуры более подробно.

Конечный случай. Если *current* = *n* + 1, то все символы уже использованы, и можно выводить результат.

Продолжение рекурсии (*current* ≤ *n*). Процедура перебирает все символы исходной строки от 1-го до *n*-го и, если этот символ еще не использовался (определяется по *used*), помещает его на место *current* строки *rez*, отмечает в *used*, что символ уже использовался, и рекурсивно себя вызывает с параметром *current* + 1. После вызова нужно восстановить *used*.

Обратите внимание на то, что переменная цикла, используемого в процедуре, должна быть *локальной*. При использовании глобальной переменной ее значение будет испорчено циклом в рекурсивно вызванной процедуре.

Вызов рекурсии:

```
p(1);
```

В первый раз вызванная процедура будет устанавливать первый символ строки *rez*.

Детали реализации:

- В этом решении переменные типа *string* используются двумя способами — как массив символов (когда обращаемся к *rez[current]*) и как строка (при выводе *rez*). Когда используются оба эти способа, нужно учитывать внутреннее строение строк Турбо Паскаля.

Нулевой символ строки содержит информацию о ее длине. Можно получить длину строки *str* как *ord(str[0])* — это эквивалент *length(str)*. Можно установить длину строки *str* равной целому числу *num* так: *str[0] := chr(num)*.

Нулевой символ изменяется автоматически при присваивании строке нового значения (например, *str := 'hello'; str := str + 'q'*) и при выполнении стандартных процедур (например, *insert(str, str, 1)*, *delete(str, 1, 2)*).

Но нулевой символ *не* меняется при работе со строкой как с массивом символов, то есть при присваивании определенному символу строки какого-то значения, например:


```
var
  str : string[10];
begin
  str := 'hello';    {str[0] = #5}
  writeln(str);      {будет выведено 'hello'}
  str[6] := '?';
  writeln(str);      {будет выведено 'hello'}
  str[0] := #6;
  writeln(str);      {будет выведено 'hello?'}
  str[6] := '!';
  writeln(str);      {будет выведено 'hello!'}
end.
```

Какие выводы можно сделать из этого примера? При использовании `writeln` и строковых операций символы строки учитываются только до места, указанного в ее нулевом символе. Поэтому при присваивании `str[i]` какого-нибудь значения нужно убедиться в том, что перед выполнением строковых операций длина строки будет установлена $\geq i$.

Что можно посоветовать применительно к нашей задаче? После ввода строки `s` выполнить присваивание `rez := s`. При этом длина `rez` будет установлена равной длине `s`. Так как все выводимые строки имеют одинаковую длину, равную длине исходной строки, то о `rez[0]` теперь можно забыть и спокойно присваивать значения элементам строки `rez`.

Вариант 2

Предложил Александр Ржеуцкий.

Общая идея: ставим первый символ исходной строки на все возможные места, второй — на все оставшиеся от первого, третий — на все оставшиеся от первого и второго и т. д.

Этот вариант отличается от варианта 1 только тем, что `used[i]` хранит не то, использовался ли `s[i]`, а то, установлен ли `rez[i]`. Соответственно, и параметр рекурсивной процедуры `current` относится не к тому, на какое место поставить символ, а к тому, с какого места его взять.

Вариант 3

Решение найдено Александром Сорокиным на сайте <http://algotlist.manual.ru>.

Общая идея: работаем только с исходной строкой. Сначала обмениваем первый символ со всеми остальными, этим получаем все варианты первого символа. При фиксированном первом символе обмениваем второй со всеми, кроме первого, — получаем все варианты второго символа. При фиксированных первых двух символах обмениваем третий символ со всеми оставшимися и т. д.

Глобальные объявления:

```
const
    maxN = 8;
var
    s : string[maxN]:    {исходная строка}
    n : integer:         {количество символов в строке}
```

Дополнительная процедура обмена двух элементов строки:

```
procedure swap(i, j : integer);
```

Рекурсивная процедура перебора:

```
procedure p(current : integer);
```

При вызове процедуры уже определены символы строки от 1-го до (current-1)-го.

Конечный случай. Если $\text{current} = n + 1$, то все символы уже расставлены, и можно выводить результат.

Продолжение рекурсии ($\text{current} \leq n$). Перебираются индексы от current включительно (чтобы символ остался) до n. Символ с индексом current обменивается с выбранным. Процедура вызывает себя рекурсивно с параметром $\text{current} + 1$. После вызова для восстановления исходного состояния нужен обратный обмен.

Вызов рекурсии:

```
p(1);
```

В первый раз вызванная процедура будет устанавливать первый символ строки.

Вариант 4

Тема: лексикографический порядок.

Будем получать перестановки в лексикографическом порядке. Для получения первой перестановки отсортируем символы строки по возрастанию.

Получение лексикографически следующей строки. Нужно поменять местами настолько мало конечных символов, насколько это возможно. Перестановкой X конечных символов нельзя получить лексикографически следующую строку только в одном случае — если они идут в убывающем порядке. Двигаясь с конца строки, нужно найти первый символ, нарушающий порядок убывания. Затем нужно заменить самый левый символ этого участка на самый меньший из лексикографически больших символов, содержащихся в участке, а остальные символы в порядке возрастания поместить в конец строки. Перебор заканчивается, когда все символы строки окажутся отсортированными по убыванию.

Пример перехода к следующей перестановке. Возьмем строку 2361540. Оставить неизменными 6 первых символов этой строки не получится — тогда седьмой определяется автоматически. Попробуем оставить 5 первых символов и получить новую комбинацию перестановкой только цифр 4 и 0 в конце. Они уже отсортированы по убыванию, поэтому получить большее число их перестановкой не получится. Аналогично, фиксация первых четырех символов и попытка переставить уже отсортированные по убыванию 540 к успеху не приведет.

А вот если зафиксировать начало (236) и попытаться переставить 1540, то этим способом можно получить следующую перестановку. Следующая за 1540 перестановка не может начинаться на 0 — она должна быть больше 1540. Она не может начинаться и на 1, ведь 1540 — самое большое из чисел, которые начинаются на единицу. А на 4 она начинаться может. Нужно выбрать самую лексикографически меньшую перестановку цифр 0145, начинающуюся на 4. Очевидно, это 4015 — оставшиеся цифры 0, 1, 5 отсортированы по возрастанию. Итак, за 2361540 лексикографически следует 2364015, за 2364015 следует 2364051, затем идет 2364105.

Решение задачи 2 С. Маршрут

Тема: динамическое программирование

Обозначим исходную таблицу буквой T , а ее элемент на пересечении i -й строки и j -го столбца — $T(i, j)$. Для каждой клетки (i, j) найдем минимальную сумму цифр маршрута из клетки $(1, 1)$ в (i, j) . Обозначим полученную таким образом таблицу M , а ее элемент — $M(i, j)$.

Правила заполнения M .

- $M(1, 1) = T(1, 1)$ — в клетку $(1, 1)$ можно прийти только одним путем — остаться в ней.
- $M(i, 1) = M(i - 1, 1) + T(i, 1)$ — в клетку $(i, 1)$, $i > 1$, то есть клетку первого столбца, можно прийти только одним путем — из вышестоящей клетки. Для вычисления $M(2, 1)$ используется $M(1, 1)$, найденный ранее, для вычисления $M(3, 1)$ используется $M(2, 1)$ и т. д.
- $M(1, j) = M(1, j - 1) + T(1, j)$ — аналогично, в клетку $(1, j)$, $j > 1$, то есть клетку первой строки, можно прийти только одним путем — из клетки слева.
- Теперь общий случай: $i > 1, j > 1$. В клетку (i, j) можно прийти только двумя путями — либо слева из $(i, j - 1)$, либо сверху из $(i - 1, j)$. Очевидно, что $M(i, j) = \min(M(i, j - 1), M(i - 1, j)) + T(i, j)$ — для клеток $(i, j - 1)$ и $(i - 1, j)$ суммы цифр клеток, расположенных на оптимальных путях, уже известны, аналогичную сумму для клетки (i, j) можно получить, выбрав лучший из этих двух вариантов.

Параллельно с заполнением числовой матрицы M можно заполнять символьную матрицу F . $F(i, j)$ содержит: 'u' («from up», сверху) — если оптимальный путь из $(1, 1)$ приводит в клетку (i, j) сверху; 'l' («from left», слева) — если оптимальный путь из $(1, 1)$ приводит слева; 'i' («initial», начальная) — для клетки $(1, 1)$.

После построения матрицы F легко заполнить символьную матрицу R — выводимый результат. Сначала заполним матрицу R минусами. Затем, начав с клетки (N, N) матрицы F , будем двигаться вверх, если в текущей клетке F находится 'u', или влево, если в текущей клетке F находится 'l'. Движение завершается, когда доходим до 'i'. По ходу движения проходимые клетки отмечаются в R знаками '#'.

Детали реализации:

- Борьба с ограничениями Турбо Паскаля.

В условии задачи задано ограничение размера матрицы — 250×250 . Если объявить все описанные выше матрицы:

var

```
T : array [1..250, 1..250] of byte;  
M : array [1..250, 1..250] of integer;  
F : array [1..250, 1..250] of char;  
R : array [1..250, 1..250] of char;
```

то их общий размер составит $250 \cdot 250 \cdot (1 + 2 + 1 + 1) = 312\,500$ байт — при том, что основной памяти в Турбо Паскале 64 Кбайт. Очевидно, что при таком ограничении на память можно объявить только один массив 250×250 , да и тот или из байтов, или из символов.

Очевидно, массивы T, F и R можно объединить. Ведь мы проходим по массиву T с целью построения M всего один раз. Поэтому после того как цифра T[i, j] использована, можно в этот элемент массива записать какую-нибудь константу — аналог 'u', 'l' и 'i' массива F.

const

```
from_up = 10;  
from_left = 11;  
initial = 12;
```

Перейти в том же массиве от F к R тоже несложно: клетки опять проходятся один раз, поэтому не нужно помнить значение матрицы F для клетки маршрута. Клетки маршрута пометим каким-нибудь особым значением:

const

```
route = 13;
```

При выводе, если T[i, j] = route, нужно выводить знак #, в противном случае — минус.

Теперь рассмотрим, что можно сделать с массивом M. Очевидно, его элементы можно вычислять в естественном порядке двойным циклом:

```
for i := 1 to n do  
  for j := 1 to n do  
    ...
```

При таком обходе массива к моменту, когда вычисляется M[i, j], уже вычислены M[i, j - 1] и M[i - 1, j].

При заполнении строки используются значения только текущей и предыдущей строк, то есть эти значения можно запомнить не в двумерном массиве 250×250 , а в двух одномерных длиной 250. Более того, можно запомнить эти значения даже в одном массиве длиной 250 — назовем его M2. При вычислении значения в j-м столбце в элементах с j-го по n-й хранятся значения предыдущей строки, а в элементах с 1-го по (j-1)-й — значения текущей строки. Соответственно, в случае $i > 1$, $j > 1$ нужно выбирать минимум из M2[j] и M2[j - 1]. Ведь (j-1)-й элемент соответствует M[i, j - 1], то есть элементу текущей строки, а j-й элемент — M[i - 1, j], то есть элементу предыдущей строки.

В итоге глобальные переменные будут выглядеть так:

```
var
```

```
  T : array [1..250, 1..250] of byte;
```

```
  M2 : array [1..250] of integer;
```

Размер их составит $250 \cdot 250 + 250 \cdot 2 = 63\,000$ байт.

- Временная сложность этого решения небольшая, поэтому лучше использовать проверку $\{ \$R+.Q+ \}$ — целочисленных операций и границ массивов.
- В исходном файле цифры идут без пробелов, поэтому просто так прочитать цифру в целую переменную нельзя. Нужно делать это с помощью символьной переменной:

```
var
```

```
  c : char;
```

```
  b : byte;
```

```
begin
```

```
  ...
```

```
  read(c);
```

```
  b := ord(c) - ord('0');
```

```
  ...
```

```
end
```

Дело в том, что коды символов от '0' до '9' идут подряд, поэтому можно найти цифру, вычитая из ее кода код символа '0'. Обратите внимание на то, что выражение $\text{ord}(c) - \text{ord}(0)$ принимается компилятором, но работает неправильно, так как $\text{ord}(0)$ — это не код символа '0', а код целого числа 0, равный нулю. А код символа '0' совсем не равен нулю.

Не забудьте при чтении входных данных пропускать концы строк с помощью `readln`; — если этого не сделать, вместо первого символа следующей строки вы прочитаете в символьную переменную символ конца строки, а потом будете из него вычитывать код нулевого символа. В результате в лучшем случае вы получите сообщение об ошибке времени выполнения, если включена опция $\{ \$Q+ \}$.

Решение задачи 2 D. Пересечение отрезков

Тема: геометрия.

Вариант 1

Находим уравнение $Ax + By + C = 0$ прямой, проходящей через первый отрезок (см. вариант 1 решения задачи 1 D). Подставляем в него координаты концов второго отрезка. Если обе точки лежат на прямой, значит, оба отрезка лежат на одной прямой, и этот случай рассмотрим позже. Если обе точки лежат в одной полуплоскости (исключая прямую, разделяющую полуплоскости), то второй отрезок полностью лежит в этой полуплоскости, и отрезки не пересекаются.

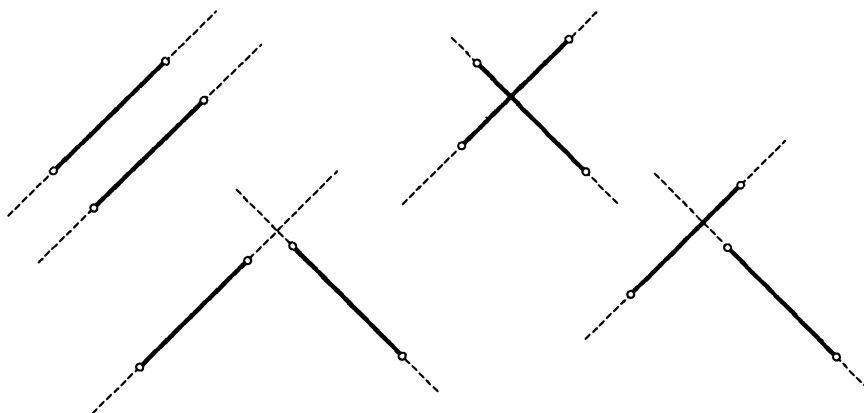


Рис. P2.1. Варианты взаимного расположения двух отрезков

Аналогично находим уравнение прямой, проходящей через второй отрезок. Если обе точки первого отрезка лежат в одной полуплоскости (исключая прямую, разделяющую полуплоскости), то и в этом случае отрезки не пересекаются.

Докажем, что если отрезки не находятся на одной прямой и концы каждого из отрезков не находятся в одной полуплоскости относительно прямой, проходящей через другой отрезок, то отрезки имеют общую точку.

Рассмотрим прямые, проходящие через данные отрезки. Прямые на плоскости могут или пересекаться, или совпадать, или быть параллельными. Случай совпадающих прямых по условию здесь не рассматривается, параллельными прямые быть не могут, так как тогда вся прямая, не говоря уже о лежащем на ней отрезке, лежит по одну сторону от другой прямой. Значит, прямые пересекаются. Покажем, что точка пересечения прямых принадлежит каждому из отрезков.

Рассмотрим прямую, проходящую через первый отрезок. Если какой-то из концов второго отрезка лежит на ней, значит, точка пересечения прямых принадлежит второму отрезку. Если ни один конец второго отрезка не лежит на прямой, проходящей через первый отрезок, то концы лежат строго в разных полуплоскостях. Функция $Ax + By + C$ дает результаты разных знаков для концов второго отрезка. Вследствие непрерывности функции на отрезке существует точка, в которой она равна 0. Это значит, что точка лежит на прямой. Итак, мы определили, что прямая, проходящая через первый отрезок, имеет общую точку со вторым отрезком.

Аналогично находим, что прямая, проходящая через второй отрезок, имеет общую точку с первым отрезком. Точка пересечения у прямых одна, она принадлежит каждому из отрезков. Значит, отрезки имеют общую точку. Утверждение доказано.

Осталось рассмотреть случай, когда отрезки лежат на одной прямой. Из перебора вариантов расположения отрезков следует, что отрезки имеют общую точку тогда и только тогда, когда конец одного из отрезков лежит внутри другого отрезка.

Проверка на принадлежность точки отрезку, если эта точка лежит на прямой, проходящей через этот отрезок, сводится к проверке принадлежности точки

прямоугольнику со сторонами, параллельными осям координат, диагональю которого является отрезок. Обратите внимание на то, что координаты концов во входных данных не упорядочены (во входных данных отрезок может быть задан как $x_1 = 0, y_1 = 0, x_2 = 2, y_2 = 2$, а может быть задан как $x_1 = 2, y_1 = 2, x_2 = 0, y_2 = 0$, так что сравнение

$(x1 \leq xt) \text{ and } (xt \leq x2) \text{ and } (y1 \leq yt) \text{ and } (yt \leq y2)$

не учитывает всех вариантов).

Детали реализации:

- В этом варианте очень много вычислений происходит по аналогичным формулам с различными параметрами. Чтобы минимизировать вероятность ошибки, нужно использовать структурный подход: объявлять типы данных и процедуры их обработки, например:

type

```
tPoint = record
  x, y : ...
end;
```

```
tLine = record
  a, b, c : ...
end;
```

```
tSegment = record
  p1, p2 : tPoint;
end;
```

```
procedure readPoint(var p : tPoint);
```

```
procedure readSegment(var s : tSegment);
```

```
procedure segment2line(s : tSegment; var l : tLine);
```

```
function pointSign(l : tLine; p : tPoint) : integer;
{подставляет координаты точки в уравнение прямой}
{и возвращает знак: +1, 0 или -1}
```

```
function onSegment(p : tPoint; s : tSegment) : boolean;
```

Обратите внимание на то, какой тип следует использовать для хранения начальных и промежуточных значений. Выбрать его поможет разбор задачи 1 D.

- Не помешает установить проверки переполнений — $\{\$R+.Q+\}$.

Вариант 2

Решение предложил Александр Сорокин.

Работать будем не с уравнением прямой вида $Ax + By + C = 0$, а с уравнением вида $y = kx + b$. Чтобы не рассматривать отдельно случай, когда уравнение вида $y = kx + b$

не существует (прямая вертикальна), повернем исходную систему координат на угол α . Делается это следующим образом:

```
xnew := x * cos(alpha) - y * sin(alpha);
ynew := y * cos(alpha) + x * sin(alpha);
```

Основная задача этого преобразования — исключить случай, когда x -координаты концов отрезка одинаковы. В тестах жюри наверняка есть случаи вертикальных отрезков, но жюри, естественно, не может сделать набор тестов, в котором есть вертикальные отрезки при повороте системы координат на *любой* угол. Поэтому выбор какого-нибудь случайного угла почти гарантирует отсутствие в повернутой системе координат вертикальных отрезков. В оригинальном решении был использован угол $\pi / 90$.

Найдем коэффициенты уравнения прямой по координатам концов отрезка:

```
k := (y2 - y1) / (x2 - x1);
b := y1 - k * x1;
```

Прямые параллельны, и отрезки не имеют общих точек, если $k_1 = k_2$ и $b_1 \neq b_2$.

Если $k_1 = k_2$ и $b_1 = b_2$, то прямые совпадают, и этот случай нужно рассмотреть отдельно. Так как прямая не вертикальна, то наличие общей точки у двух отрезков, лежащих на одной прямой, эквивалентно наличию общей точки у проекций этих отрезков на ось OX . Если мы обеспечим, что у каждого отрезка x -координата первого конца меньше x -координаты второго конца, то отсутствие общей точки можно определить из следующего простого условия:

```
notIntersect := (seg1x1 > seg2x2) or (seg2x1 > seg1x2);
```

Теперь рассмотрим случай, когда прямые пересекаются. Найдем точку пересечения прямых:

```
x := (b2 - b1) / (k1 - k2);
y := k1 * x + b1;
```

После этого остается только проверить принадлежность точки пересечения прямых каждому из отрезков. Так как она принадлежит прямой, на которой лежит отрезок, и отрезок не вертикальный, то достаточно проверить принадлежность ее проекции на ось OX проекции отрезка на ту же ось. Если мы заранее обеспечили то, что x -координата первого конца отрезка меньше x -координаты второго конца, то принадлежность точки отрезку записывается элементарно:

```
belongs := (x1 <= x) and (x <= x2);
```

Детали реализации:

- Повернув систему координат на угол α , мы избавились от вертикальных отрезков, но вместо целых координат у нас теперь вещественные. Работа с вещественными числами требует специальных приемов.

Сравнение целых чисел

$a = b$

$a < b$

$a <= b$

Сравнение вещественных чисел

$\text{abs}(a - b) < \text{eps}$

$a < b + \text{eps}$

$a < b + \text{eps}$

Сравнение целых чисел	Сравнение вещественных чисел
$a > b$	$a > b - \text{eps}$
$a \geq b$	$a > b - \text{eps}$
$a < b$	$\text{abs}(a - b) > \text{eps}$

Здесь eps — небольшое положительное число, выбираемое с учетом точности используемого вещественного типа и диапазона входных данных. Для этой задачи и типа `extended` подходит $1e-10$ (найденно экспериментально).

- Чтобы использовать тип `extended` вместо типа `real`, нужно в начале программы написать:

```
{ $N+.E- }
```

```
type
```

```
    real = extended;
```

- Нельзя автоматически заменять все приведенные в решении точные сравнения вещественными сравнениями, использующими eps . Пусть, например, отрезки лежат на одной прямой и имеют одну общую точку: $\text{seg1x1} < \text{seg1x2} = \text{seg2x1} < \text{seg2x2}$. Если переписать приведенную ранее формулу `notIntersect` с помощью вещественных сравнений, получится:

```
notIntersect := (seg1x1 > seg2x2 - eps) or
(seg2x1 > seg1x2 - eps);
```

Нетрудно заметить, что в данном случае эта формула даст значение `true` — отрезки не имеют общих точек. Однако и обратная формула —

```
intersect := (seg1x1 <= seg2x2) and (seg2x1 <= seg1x2);
```

с применением вещественных сравнений

```
intersect := (seg1x1 < seg2x2 + eps) and
(seg2x1 < seg1x2 + eps);
```

тоже даст значение `true` — отрезки имеют общие точки.

Что же делать? Прежде всего нужно заметить, что проблемы возникают только в случае, когда общая точка отрезков совпадает с одним из концов отрезков (ровно с одним концом она может совпадать, когда отрезки не лежат на одной прямой). Если точка находится очень близко к концу отрезка, лучше считать, что она скорее принадлежит отрезку, чем не принадлежит. Поэтому нужно производить вычисления по формулам `intersect` и `belongs`, а не по формулам-отрицаниям `notIntersect` и `notBelongs`.

Кстати, когда отрезки лежат на одной прямой и имеют одну общую точку, формулы поворота дадут одинаковые новые координаты для одинаковых точек, поэтому в формуле можно вообще обойтись без eps . В случае если прямые пересекаются и конец одного отрезка лежит на середине второго, обойтись без eps в формуле нельзя.

Решение задачи 2 Е. Длинная сумма

Тема: длинная арифметика.

Так как длинных чисел в задаче много — два слагаемых, а также сумма — и над ними нужно выполнять много разных операций — ввод, суммирование, вывод, — то удобно объявить тип для этих чисел:

```
const
    numlen = ...
type
    number = array [1..numlen] of byte;
```

Рассмотрим, как длинное число будет храниться в таком массиве. В 1-й ячейке будет храниться младший разряд числа, то есть единицы, во 2-й ячейке — десятки, в 3-й — сотни и т. д., по одной цифре в каждой ячейке. Если число имеет меньше разрядов, чем `numlen`, в оставшихся ячейках массива должны храниться нули.

Необходимый для этой задачи размер `numlen` составляет 30 001. Каждое из исходных чисел может иметь до 30 000 разрядов (см. нестрогое неравенство в условии), а результат сложения таких чисел может иметь до 30 001 разряда.

Первая проблема, с которой мы сталкиваемся в этой задаче, — число нужно прочитать. Турбо Паскаль не предоставляет прямой возможности ни прочитать в массив из целых строку, ни прочитать в целую переменную одну цифру, если за ней не идет пробельный символ. Поэтому нужно использовать символьный тип. Типичный цикл чтения символов до конца строки выглядит так:

```
var
    c : char;
...
while not eoln do begin
    read(c);
    ...{обработка символа}
end;
readln;
```

Нужно переводить читаемые из файла символы в целые числа, из которых состоит массив. Для получения цифры из символа используется следующая формула: `ord(c) - ord('0')`. Дело в том, что коды цифр '0'.. '9' являются последовательными числами, поэтому при вычитании из кода цифры кода символа '0' получается значение самой цифры. Обратите внимание на то, что писать `ord(c) - ord(0)` нельзя — будет вычтен не код символа '0', а код самого нуля.

Чтение числа происходит в три стадии:

1. Обнуление массива.
2. Чтение цифр числа в следующем порядке: в 1-й элемент массива помещается самый старший разряд, в элемент, соответствующий длине считываемого числа,

помещается самый младший разряд. Например, если читается число 12345, то в массиве будет содержаться

```
...a[7] a[6] a[5] a[4] a[3] a[2] a[1]
... 0   0   5   4   3   2   1
```

3. Обращение считанной части массива. Массив из предыдущего пункта должен быть преобразован в такой:

```
...a[7] a[6] a[5] a[4] a[3] a[2] a[1]
... 0   0   1   2   3   4   5
```

Это можно сделать циклом до половины считанной части массива, в котором элемент из первой половины обменивается с элементом из второй половины. Обратите внимание на то, что цикл по всей считанной части массива делать нельзя — половину цикла будет происходить переворот в одну сторону, половину цикла — обратно, так что после завершения цикла мы будем иметь то же, что и до него.

После завершения такой процедуры чтения в массиве будет находиться число в формате, который описывался в начале решения.

Лучше оформить чтение числа в качестве отдельной процедуры с параметром-переменной типа `number` — тогда в основной программе чтение двух чисел, записанных в двух строках, будет выполнено одинаково, а значит, будет меньше подвержено ошибкам.

Теперь рассмотрим вывод длинного числа (на стадии отладки не мешает с помощью процедуры вывода убедиться в правильности чтения исходных данных).

Процедура довольно проста — идем со стороны больших индексов массива к меньшим, пропуская нули. Как только встретится ненулевой элемент, выводим все от него до первого. Так как в каждой ячейке содержится ровно один разряд, то есть число от 0 до 9, то его можно просто писать в файл: `write(n[i]);`.

Особое внимание в процедуре вывода следует обратить на число 0, то есть на случай, когда во всех элементах массива содержатся нули. В этом случае нужно вывести в выходной файл один ноль.

При работе с длинными числами ошибка, состоящая в том, что число 0 выводится неправильно, является одной из самых распространенных.

Вывод длинного числа желательно оформить в виде отдельной процедуры, чтобы впоследствии при решении аналогичных задач воспроизводить ее по памяти, а не думать над этим чисто техническим элементом решения.

Теперь об основной части программы — сложении двух длинных чисел. Приведем процедуру сложения полностью:

```
procedure add(var n1, n2 : number);
var
  i, carry : integer;
begin
  carry := 0;
```

```
for i := 1 to numlen do begin
    carry := carry + n1[i] + n2[i];
    n1[i] := carry mod 10;
    carry := carry div 10;
end;
if carry <> 0 then
    halt(1);
end;
```

Процедура сложения записывается так просто благодаря выбранному формату представления длинных чисел. Здесь не нужно знать длину ни первого, ни второго числа. Да и результат будет получен в том же формате, что и исходные данные, без дополнительных переменных под возможно переполнившийся старший разряд.

В этой процедуре к первому числу добавляется второе, то есть сумма пишется в то же место, где было первое слагаемое. Это возможно потому, что каждый разряд слагаемого используется только один раз, и разряд суммы находится как раз тогда, когда разряд слагаемого больше не нужен.

Переменная `carry` содержит перенос из младших разрядов. Затем в ней вычисляется сумма i -х разрядов чисел `n1` и `n2` с учетом переноса из младших разрядов. В i -й разряд результата заносится последняя цифра суммы разрядов, а в новый перенос — все, кроме последней цифры.

После цикла идет проверка `carry` на ноль. Если длина результата превосходит максимальное число, представимое объявленным типом, то после выполнения основного цикла `carry` будет содержать ненулевое значение. Если вы неправильно рассчитали константу `numlen`, и программа не может правильно представить результат, то лучше от проверяющей системы получить «ошибку времени выполнения», чем «неверный ответ».

Детали реализации:

- Временная сложность алгоритма небольшая, поэтому желательно включить проверку на переполнения и границы массивов: `{R+.Q+}`.
- Параметры типа `number` следует передавать в процедуры по ссылке (`var`-параметр). Если такой параметр передается по значению (без `var`), то он копируется в стек. Размер стека по умолчанию — 16 Кбайт, у нас же размер одной переменной — 30 001 байт. В принципе, можно увеличить размер стека, но лучше этого не делать, а передавать длинные переменные по ссылке.

Решение задачи 2 F. Спираль

Глобальные переменные:

```
var
    a : array [0..101, 0..101] of integer;
    n, i, j : integer;
    di, dj : integer;
```

Сначала заполняем часть массива A единицами: $A[i, j]$ ($0 \leq i \leq N+1, 0 \leq j \leq N+1$). Затем заполняем часть массива A нулями: $A[i, j]$ ($1 \leq i \leq N, 1 \leq j \leq N$). Посмотрим, как будет заполнен массив для $N = 4$:

```

1 1 1 1 1 1 ...
1 0 0 0 0 1 ...
1 0 0 0 0 1 ...
1 0 0 0 0 1 ...
1 0 0 0 0 1 ...
1 1 1 1 1 1 ...
.....

```

Такое построение называется *барьером*. Будем изменять направление движения по матрице, когда наткнемся на ненулевой элемент. Благодаря использованию барьера первый проход не является особым случаем.

Теперь начинаем заполнение массива по спирали. Пара (i, j) будет означать координаты текущего числа (i — строка, j — столбец), а пара (di, dj) — направление. Координаты следующего числа будут вычисляться так:

```

i := i + di;
j := j + dj;

```

Начальные значения: $i = 1, j = 1, di = 0, dj = 1$.

Основной цикл состоит в том, что N^2 раз нужно поставить текущее число в ячейку $a[i, j]$. Если $a[i + di, j + dj]$ не равно нулю, то нужно изменить соответствующим образом di и dj — всего существует четыре направления, которые последовательно меняются. Затем следует обновить i и j .

Детали реализации: в условии задачи сказано, что выводимые числа должны быть разделены пробелами, но не сказано, каким количеством пробелов. Не следует добиваться того, чтобы вывод вашей программы совпадал с выводом, показанным в условии, и числа разделяло минимальное число пробелов. Наиболее простой способ получить требуемый результат, — выводить числа в формате `write(number : width)`, где `width` подобрать так, чтобы при максимальном N числа не «склеивались».

Тренировка 3

Решение задачи 3 А. Разложение на простые множители

Тема: теория чисел

Будем проверять делимость N на последовательные числа D : 2, 3... Пусть мы определили, что N делится на D . Заменяем N на $\frac{N}{D}$ и выведем D и символ $*$ в качестве части ответа.

Мы не проверяем простоту делителей. Не делаем этого потому, что все меньшие делители уже удалены из числа (как только находим делитель, делим N на него).

Нужно завершать перебор значений D , как только будет выполнено $D > \sqrt{N}$. Если нет простых делителей, не превосходящих корень из числа, то само число простое и может быть выведено в качестве последнего делителя, уже без знака умножения после него (доказательство приведено в разборе задачи 2 А).

Обратите внимание на то, что, если N разделилось на D , в следующий раз делимость нужно проверять опять на D , а не на $D + 1$. Это необходимо, чтобы учесть кратные делители (например, $24 = 2 \cdot 2 \cdot 2 \cdot 3$). Вложенный цикл («пока делится — делить») лучше в данном случае не применять, чтобы не рассматривать отдельно случай, когда число N окажется равным единице, и после последнего из делителей не нужно ставить знак умножения.

Обратите внимание на то, что каждое D не должно превосходить корня из *текущего* N (не начального). Например, для $9 = 3 \cdot 3$ корень из начального числа равен 3. Если поделить 9 на первый множитель 3, а корень оставить старым, то второй множитель тоже будет найден в основном цикле, а после цикла N станет равным 1, и результатом работы программы будет «3*3*1».

Использовать цикл `for` для реализации этого алгоритма нельзя, потому что в нем границы вычисляются один раз при входе в цикл и не обновляются при изменении значений переменных, входящих в выражения границ.

Решение задачи 3 В. Перестановки (2)

Эта задача отличается от задачи 2 В тем, что символы в исходной строке могут повторяться, а выводить по-прежнему нужно лишь различные варианты перестановок.

Вариант 1

Тема: перебор-построение.

Решение базируется на варианте 1 решения задачи 2 В. Единственное отличие — теперь не для каждого символа запоминается, использовался ли он, а для *различных* символов — сколько штук каждого из них еще не использовалось.

Общая идея решения такова: ставим на первое место все возможные *различные* символы, ставим на второе место все оставшиеся после заполнения первого места *различные* символы, на третье — все оставшиеся после заполнения первого и второго мест и т. д.

Например, в строке aab есть две различные буквы — а и b. То есть нет буквы а, которая стояла на первом месте в исходной строке, и нет буквы а, которая стояла на втором месте в исходной строке — запомнено только, что букв а две (а буква b — одна).

Глобальные объявления:

var

s : string;

count : array [char] of byte;

Обратите внимание на необычный стиль описания массива count. Вместо привычного диапазона вроде 1..10 записано просто имя типа. Имя типа в этом контексте эквивалентно диапазону от наименьшего возможного значения типа до наибольшего. Приведенное объявление эквивалентно array [#0..#255] of byte. Кроме имени типа в объявлении необычно еще и то, какой именно тип используется в качестве диапазона — тип char. Это означает, что нужно индексировать этот массив символами, а не целыми числами. Вообще-то в задаче сказано, что во входных данных могут встречаться только латинские буквы и арабские цифры. В кодовой таблице сначала идут коды цифр, затем (не непосредственно после них) коды заглавных латинских букв, затем (опять с пропуском) коды строчных латинских букв, причем коды и цифр, и букв идут в естественном порядке, то есть

'0'. '1'. '9'. 'A'. 'B'. 'Z'. 'a'. 'b'. 'z'

Очевидно, можно было бы объявить массив array ['0'..'z'] of byte, но это не сделано, чтобы программа могла переставлять даже русские буквы.

Итак, s — это исходная строка, в count[ch] хранится, сколько символов ch осталось для еще не построенной части строки. Сразу после чтения s заполняем на ее основе массив count, и после этого она больше не нужна. Заполнение происходит аналогично тому, как это описано в варианте 1 разбора задачи 1 F. Сначала обнуляем массив count. Для этого используется цикл по символьной переменной:

var

c : char;

...

for c := #0 to #255 do

count[c] := 0;

Затем проходим по строке s и увеличиваем элемент массива $count$, соответствующий текущему символу.

Как упоминалось ранее, после построения $count$ строка s уже не нужна. Будем использовать ее для хранения результата. В варианте 1 разбора задачи 2 В обсуждалась проблема поддержания длины строки при работе со строкой как с символьным массивом. Если использовать для построения и вывода результата исходную строку, эта проблема снимается.

Рекурсивная процедура перебора:

```
procedure p(current : integer);
```

Полное название процедуры может выглядеть так: «Перебрать все варианты расстановки символов s , $s = \#0\ldots\#255$, взятых в количестве $count[s]$, на места с $current$ -го по n -е строки s ».

При вызове этой процедуры уже определены символы строки s от первого до $(current-1)$ -го, и в массиве $count$ на соответствующих местах находятся оставшиеся количества символов.

Процедура работает так. Если $current = n + 1$, то перебирать ничего не надо, и процедура выводит построенную комбинацию. Если же $current \leq n$, то процедура ставит на место $current$ все допустимые символы по очереди (допустимым является символ из числа оставшихся) и для каждого варианта вызывает себя рекурсивно, чтобы расставить оставшиеся символы на места с $(current+1)$ -го по n -е.

Рассмотрим работу процедуры более подробно.

Конечный случай. Если $current = n + 1$, то все символы уже использованы, и можно выводить результат.

Продолжение рекурсии ($current \leq n$). Перебираем все символы ($\#0\ldots\#255$) уже упоминавшимся циклом `for`, если этот символ еще может использоваться (его количество ненулевое), то помещаем его на место $current$ строки s и уменьшаем для него $count$. Затем происходит рекурсивный вызов процедуры с параметром $current + 1$. После вызова увеличиваем обратно $count$ для этого символа.

Вызов рекурсии:

```
p(1);
```

В первый раз вызванная процедура будет устанавливать первый символ строки s .

Вариант 2

Решение предложил Никита Рыбак.

Тема: перебор-построение.

Каждому символу исходной строки сопоставляется число, означающее, сколько таких символов содержится в строке левее этого символа. Например, для строки `abbcbab`

массив этих чисел будет заполнен следующим образом:

0010213

Символ на очередное место разрешается ставить только в случае, если количество таких символов в построенном начале строки равно числу, сопоставленному этому символу.

Иными словами, делаем все символы разными:

(a0) (b0) (b1) (c0) (b2) (a1) (b3)

и (b2) разрешаем ставить только в том случае, когда перед ним уже находятся (b0) и (b1). В результате, одна и та же перестановка исходных символов не может быть получена более одного раза.

Вариант 3

Тема: лексикографический порядок.

Описанный в варианте 4 решения задачи 2 В алгоритм не требует никаких доработок, чтобы учитывать повторения символов. Нужно только в его описании слова «сортировка по возрастанию» заменить словами «сортировка по неубыванию», «концевые разряды в убывающем порядке» — словами «концевые разряды в невозрастающем порядке» и т. д.

Решение задачи 3 С. Копилка

Вариант 1

Тема: динамическое программирование.

Если известны вес пустой копилки и вес копилки с монетами, ничего не стоит найти вес монет как их разность.

Сначала решим задачу нахождения минимальной суммы денег в копилке.

Объявим таблицу M , где для каждого веса монет в копилке (0...9999) будем находить минимальную суммарную стоимость монет. Заполнять будем M последовательно от 0 к 9999. Если какую-то суммарную стоимость набрать нельзя, то соответствующее значение в таблице M равно -1 .

$M[0]$ известно — копилка пуста, суммарная стоимость монет — 0.

Пусть известны значения M для всех весов, меньших $weight$, и нужно найти $M[weight]$. Пусть в копилке с весом монет $weight$ есть монеты. Тогда там есть хотя бы одна монета какого-то вида. Про каждый вид монет по очереди предполагаем, что одна монета такого вида в копилке с весом монет $weight$ есть. Обозначим номер вида монет $kind$. Если в копилке есть одна монета вида $kind$, то должно соблюдаться неравенство $W[kind] \leq weight$, то есть общий вес монет должен быть не меньше веса монеты рассматриваемого типа. Кроме того, нужно, чтобы вес монет $weight - W[kind]$ можно было набрать монетами заданных в задаче типов. Вес $weight - W[kind]$ можно набрать, если $M[weight - W[kind]] \neq -1$ или $M[weight - W[kind]] \geq 0$ — минус единицами помечаются недостижимые веса.

Итак, пусть монета вида `kind` в копилке с весом монет `weight` есть. Тогда минимальная стоимость монет — $M[\text{weight} - W[\text{kind}]] + P[\text{kind}]$. Действительно, просто набираем наилучшим способом вес монет без веса этой монеты. А результат этого наилучшего способа уже известен, так как для всех меньших, чем рассматриваемый, весов уже известна минимальная стоимость.

Предполагаем про каждый вид монет по очереди, что он есть в копилке с данным весом монет. И для каждого получаем или минимальную стоимость (см. предыдущий абзац), или заключение, что вид отсутствует. Если в копилке вообще есть монеты, то какой-то их вид в ней присутствует. Значит, для нахождения $M[\text{weight}]$ нужно выбрать минимум из минимальных стоимостей монет, если тип `kind` в копилке есть.

Максимум суммарной стоимости ищется аналогично.

Выводить нужно или значение $M[F - E]$, если оно неотрицательное, или сообщение о невозможности набрать вес.

Детали реализации:

- Ограничения Турбо Паскаля. Массив `M` описывается так:

```
var
```

```
  M : array [0..9999] of longint;
```

его размер — 40 000 байт. Значит, два массива — для отдельного нахождения минимума и максимума — в память объемом 64 Кбайт не поместятся. Однако это не страшно — сначала в массиве `M` нужно найти минимум и запомнить его в отдельной переменной (или даже вывести), а потом в том же массиве найти максимум.

Вариант 2

Предложил Александр Меркулов.

Тема: динамическое программирование.

Найдем минимальную стоимость, максимальная ищется аналогично.

В этом решении также заполняем массив `M`, только основной цикл выполняется не по `weight` — набираемому весу, а по `kind` — количеству рассматриваемых монет.

Сначала заполняем `M` так, как будто видов монет 0. Массив будет выглядеть следующим образом: $M[0] = 0$, в остальных ячейках находятся -1 (см. обозначения варианта 1).

Затем предполагаем, что есть монеты только первого вида. Заполняем $M[\text{weight}]$ для `weight` от 1 до 9999. Для всех значений `M` с индексами, меньшими `weight`, уже найдена минимальная стоимость монет данного веса, если можно использовать монеты первого вида. Предположим, что монета вида 1 есть. Тогда должно выполняться $W[1] \leq \text{weight}$ и $M[\text{weight} - W[1]] \geq 0$ (см. пояснения в варианте 1). Если ранее (всеми предыдущими видами монет) набрать вес `weight` было нельзя, то есть $M[\text{weight}] = -1$, то теперь этот вес уже набрать можно, $M[\text{weight} - W[1]] + P[1]$ можно установить.

А если вес `weight` можно было набрать, то выбираем минимум из старого значения `M[weight]` и претендента.

Далее действуем аналогично. Пусть массив `M` заполнен для вида монет `kind - 1`. Добавляем в рассмотрение монеты вида `kind`. Получается, что если `W[kind] <= weight` и `M[weight - W[kind]] >= 0`, тогда или просто устанавливаем `M[weight]` равным `M[weight - W[kind]] + P[kind]`, если он ранее был равен `-1`, или находим минимум из `M[weight]` и `M[weight - W[kind]] + P[kind]`.

Решение задачи 3 D. Открытка и конверт

Тема: геометрия.

Вариант 1

Обозначим H_k — высота конверта, W_k — ширина конверта, H_o — высота открытки, W_o — ширина открытки.

Для отсеечения тривиального случая, когда открытка входит в конверт при параллельности сторон открытки и конверта, а также случая, когда открытка заведомо не входит в конверт, выполним следующие действия: отсортируем высоту и ширину открытки и конверта (должны выполняться неравенства $H_k \leq W_k$ и $H_o \leq W_o$). Сделать это несложно — если неравенство не выполняется, нужно обменять значения двух переменных.

Если после этого окажется, что $H_o > H_k$, то открытку поместить в конверт нельзя — в открытку помещается круг диаметром H_o , а в конверт круг такого диаметра не помещается. Значит, не поместится и открытка.

Итак, если $H_o > H_k$, то открытку в конверт поместить нельзя. Теперь рассмотрим случай $H_o \leq H_k$. Если $W_o \leq W_k$, то открытка входит в конверт параллельно сторонам конверта.

Единственный оставшийся случай — когда $H_o \leq H_k$ и $W_o > W_k$. Это как раз тот случай, когда нужно попытаться поместить открытку в конверт не параллельно сторонам конверта.

Утверждение: если выполняются четыре неравенства: $H_k \leq W_k$, $H_o \leq W_o$, $H_o \leq H_k$, $W_o > W_k$, — то открытку в конверт можно вставить тогда и только тогда, когда можно расположить один из противоположных концов открытки на нижнем основании конверта, а второй — на верхнем и при этом открытка помещается в конверт. Доказательство следует из варианта 2.

Из утверждения следует, что открытку нужно «поднять» на максимальный угол и посмотреть, входит ли открытка в таком положении в конверт, то есть меньше ли горизонтальное измерение «поднятой» открытки, чем ширина конверта (рис. P3.1).

На рисунке $ABCD$ — открытка, точка A лежит на нижнем основании конверта, точка C — на верхнем, все углы, кажущиеся прямыми, таковыми и являются, $AB = CD = W_o$, $BC = AD = H_o$, $AF = H_k$.

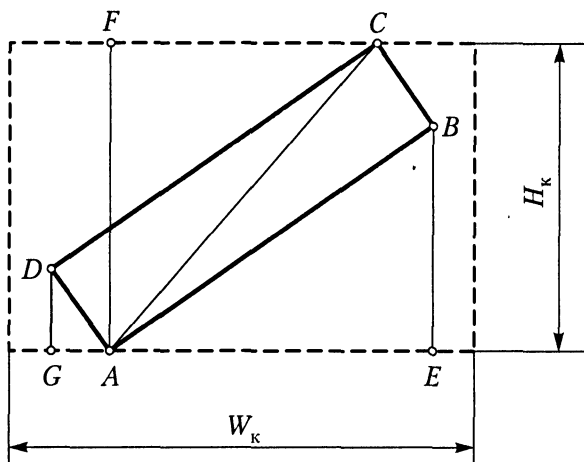


Рис. РЗ.1. Открытка, «поднятая» на максимальный угол

В прямоугольном треугольнике ABC по теореме Пифагора находим AC (AB и BC известны). В прямоугольном треугольнике ACF по теореме Пифагора находим CF (известны AF и AC).

$$\angle CAB = \arctg \frac{BC}{AB}; \angle FAC = \arctg \frac{FC}{AF}; \angle BAF = \angle BAC + \angle CAF;$$

$$\angle BAE = 90^\circ - \angle BAF; \angle DAF = 90^\circ - \angle BAF = \angle BAE; \angle GDA = \angle DAF = \angle BAE.$$

Итак, угол BAE найден. Теперь можем найти GE — проекцию открытки на основание конверта:

$$GE = GA + AE = AD \sin \angle GDA + AB \cos \angle BAE = H_0 \sin \angle BAE + W_0 \cos \angle BAE.$$

Когда значение GE получено, нужно всего лишь сравнить его с W_k . Если $GE \leq W_k$, то открытка входит в конверт в таком положении, иначе — не входит, а по утверждению и вообще не помещается.

Детали реализации:

- Не следует забывать, что углы в Турбо Паскале задаются в радианах, а не в градусах, так что прямой угол равен $\pi / 2$ (π — предопределенная константа Турбо Паскаля).
- Для вещественных вычислений лучше всего использовать вещественный тип с максимальной точностью — в Турбо Паскале это `extended`. Для минимального изменения программы, использующей тип `real`, нужно перед ней написать:

```
{ $N+.E- }
```

```
type
```

```
real = extended;
```

Если не записать директиву, будет выдана ошибка компиляции.

- При вещественных вычислениях значение может получиться неточным. В основном это относится к последнему сравнению $GE \leq W_k$. Может получиться так, что из-за ошибки вычислений GE окажется чуть-чуть больше W_k , в то время как в действительности эти два значения равны. Если в действительности GE меньше W_k , то вряд ли из-за ошибок GE окажется больше W_k .

Как же с этим бороться? Нужно объявить константу, которая будет учитывать ошибки в расчетах:

```
const
```

```
    eps = 1e-10;
```

Константу `eps` нужно выбирать, исходя из следующих соображений. Во-первых, нужно учитывать точность вещественного типа — для типа `extended` это 19 десятичных разрядов. То есть если взять `eps = 1e-20`, то будет выполняться $1 + \text{eps} = 1$. Таким образом, если взять слишком маленькую константу, будет все равно, участвует она в равенстве или нет. Во-вторых, нужно учитывать диапазон сравниваемых значений. К примеру, если x порядка $1e10$, то `eps = 1e-10` бесполезен — $x + \text{eps} = x$. А если x порядка $1e-10$, то `eps = 1e-10` пропускает слишком большие ошибки.

Таким образом, лучший выбор `eps` — порядков на 15 меньше, чем сравниваемые значения. То есть если сторона конверта — число от 1 до 1000 ($1e3$), то погрешность $1e-10$ подойдет.

С учетом этой константы последнее сравнение будет выглядеть так:
 $GE \leq W_k + \text{eps}$.

Можно аналогично ввести относительную погрешность — тогда уже не нужно думать о диапазоне сравниваемых значений: ее можно установить на порядок больше, чем предел точности типа:

```
const
```

```
    eps = 1e-18;
```

```
    ...
```

```
    if GE <= Wk * (1 + eps) then
```

```
    ...
```

Вариант 2

Пусть открытка каким-то образом входит в конверт. Передвинем открытку без изменения ее ориентации так, чтобы ее центр совпадал с центром конверта. Очевидно, если открытка ранее помещалась в конверт, то она помещается и теперь — ведь при движении были уменьшены максимальные расстояния по X и по Y от середины конверта до крайних точек открытки, положение открытки в конверте стало более симметричным, а значит, она занимает в нем меньше места.

Итак, можно теперь рассматривать только случаи, когда центр открытки совпадает с центром конверта. Если открытка не входит в конверт при таком условии, то она не входит в него вообще. Отсеем случаи, рассмотренные в начале варианта 1, когда высота открытки больше высоты конверта (тогда открытка не входит в кон-

верт) и когда высота открытки не превосходит высоты конверта, а ее длина не превосходит длины конверта (тогда открытка входит в конверт горизонтально).

Опишем вокруг открытки окружность. Посмотрим, как расположена эта окружность относительно конверта. Если конверт полностью в ней содержится, то, очевидно, открытку в нем разместить нельзя (диагональ открытки не помещается в конверте). Простое условие для проверки этого случая — диагональ открытки больше диагонали конверта.

Окружность не может полностью содержаться в конверте — тогда открытку можно было бы поместить в центр конверта с любой ориентацией, но этот случай мы отсекали, когда выдавали ответ для открытки, помещающейся в конверт горизонтально.

Итак, окружность должна пересекать хотя бы верхнюю и нижнюю стороны конверта. Посмотрим, пересекает ли она левую и правую стороны. Если она их не пересекает, то открытка, высота которой не превосходит высоту конверта (другие мы отсеяли как не входящие в конверт), входит в него горизонтально, а этот случай уже был рассмотрен. Итак, единственный способ расположения открытки, конверта и окружности, описанной вокруг открытки, изображен на рис. P3.2.

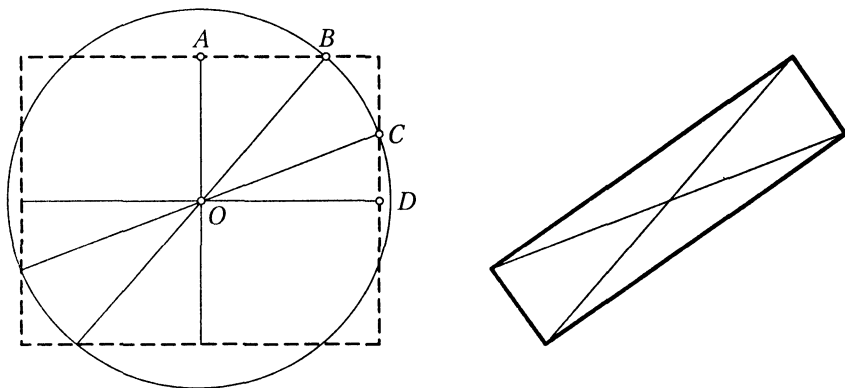


Рис. P3.2. Конверт и окружность с диаметром, равным диагонали открытки

Угол, под которым из центра открытки видна ее меньшая сторона, не должен превосходить угол BOC . Он может быть найден как $2 \arctg \frac{H_0}{W_0}$.

$$\angle BOC = 90^\circ - \angle AOB - \angle COD, \quad \angle AOB = \arctg \frac{AB}{AO}, \quad \angle COD = \arctg \frac{CD}{DO}.$$

Здесь AO — половина высоты конверта, DO — половина ширины конверта, AB находится из прямоугольного треугольника ABO (AO — половина высоты конверта, BO — половина диагонали открытки), CD находится из прямоугольного треугольника CDO (DO — половина ширины конверта, CO — половина диагонали открытки). Диагональ открытки находится из прямоугольного треугольника, катетами в котором являются высота и ширина открытки.

Детали реализации — см. вариант 1.

Решение задачи 3 Е. Длинное произведение

Вариант 1

Объявление типа длинного числа, ввод и вывод такие же как в решении задачи 2 Е. Отличие только в константе `numlen` — в этой задаче может получиться результат 10^{5000} , так что для него нужен 5001 разряд.

В процедуре длинного умножения будут использоваться дополнительные процедуры:

```
procedure set0(var n : number);           {обнуление числа}
```

```
function len(var n : number): integer;    {длина числа}
```

О процедуре обнуления говорить не стоит, а относительно функции получения длины числа следует отметить, что для массива, заполненного нулями, она должна выдавать 1.

Кстати, процедура обнуления может использоваться в процедуре чтения числа, а функция получения длины числа может использоваться в процедуре вывода числа.

Сама процедура умножения длинных чисел такова:

```
{A} procedure mul(var n1, n2, n3 : number);
{B} var
{C}   i1, i2, i3, len1, len2, carry : integer;
{D} begin
{E}   len1 := len(n1);
{F}   len2 := len(n2);
{G}   set0(n3);
{H}   for i1 := 1 to len1 do begin
{I}     i3 := i1;
{J}     carry := 0;
{K}     for i2 := 1 to len2 do begin
{L}       carry := carry + n3[i3] + n1[i1] * n2[i2];
{M}       n3[i3] := carry mod 10;
{N}       carry := carry div 10;
{O}       inc(i3);
{P}     end;
{Q}     n3[i3] := carry;
{R}   end;
{S} end;
```

Строка {A}: у процедуры два входных параметра (`n1` и `n2`) и один выходной (`n3`). При умножении длинных чисел нельзя один из входных параметров использовать для получения результата (в отличие от длинного сложения), так как каждая цифра первого множителя умножается на каждую цифру второго множителя, так что каждая исходная цифра используется много раз.

Строка {C}: переменные $i1, i2, i3$ — текущие места в исходных и получаемом массивах, $carry$ — перенос из младших разрядов, $len1$ и $len2$ — длины множителей.

Почему при сложении можно было идти до $numlen$, а теперь нужно идти до фактической длины числа? Потому что при сложении мы проходили по массиву, представляющему число, один раз, и число операций было пропорционально длине массива. Число операций, измеряемое тысячами, для компьютера не существенно — и 5000, и 30 000 операций будут выполнены за тысячные доли секунды. При умножении длинных чисел каждый разряд умножается на каждый, то есть число операций становится пропорциональным произведению длин чисел. Для 2500 разрядов (длина чисел в условии) получается более 6 млн операций, а это уже существенная работа для процессора, и увеличивать ее (и время работы программы) еще в четыре раза не следует (длина массива в решении — 5000 разрядов, каждый разряд умножается на каждый; если количество умножаемых разрядов возрастает вдвое, количество операций возрастает вчетверо).

В строке {F} один раз вычислена длина числа $n2$ — чтобы не находить ее каждый раз во вложенном цикле. Если ее находить каждый раз, затраты будут очень большими, ведь функция len проходит от старших разрядов числа, находя первый отличный от нуля элемент. Если числа *максимум* по 2500 разрядов, то функция len совершает *минимум* 2500 действий. Один раз выполнить ее в процедуре, где совершается 6 млн умножений, можно, а вот 2500 раз — не рекомендуется.

В строке {E} аналогично вычислена длина числа $n1$. Ее можно было не находить заранее, а подставить вызов функции прямо в качестве верхней границы цикла `for`. Дело в том, что значение границ цикла `for` вычисляется один раз — при входе в него. Но длина первого множителя занесена в отдельную переменную просто из соображений симметрии.

В строке {G} происходит обнуление результата. Кстати, поскольку все параметры процедуры — это параметры-переменные, то нельзя результат получить в одной из переменных, хранящих множители. Вызов `mul(n1, n2, n1)` приведет к получению ошибочного результата (какого?).

В строках {H}—{R} записан основной цикл умножения. В самом внешнем цикле перебираются цифры первого числа, в строках {I}—{Q} происходит умножение второго числа на фиксированную цифру $n1[i1]$ и добавление к результату.

Поддержка индекса $i3$ довольно прозрачна. В строке {I} указано, что при умножении на $i1$ -ю цифру к результату будет добавлено произведение $i1$ -й цифры первого числа на второе число, умноженное на 10^{i1-1} , то есть произведение $i1$ -й цифры первого числа на младшую цифру второго числа должно добавляться к $i1$ -й цифре результата.

При проходе по цифрам второго числа $i3$ постепенно увеличивается (строка {O}). Обнуление переноса в строке {J} в комментариях не нуждается. К моменту выполнения этой операции результат уже хранит произведение $(i1 - 1)$ -й цифры первого числа на второе число. В строках {K}—{P} будет производиться умножение на $i1$ -ю цифру, так что до этого в строке {J} переноса еще нет.

Строки $\{L\} - \{N\}$. Умножаем текущую цифру первого числа — $n1[i1]$ на текущую цифру второго числа — $n2[i2]$, добавляем значение текущего результата из ячейки, куда нужно записать результат умножения этих цифр, добавляем перенос. Последнюю цифру этой суммы записываем в нужную ячейку результата, а все остальное переносится в следующий разряд.

Со строкой $\{Q\}$ все не так очевидно. Чтобы она была корректна, должны выполняться два условия:

1. Переменная `carry` после цикла должна содержать корректную цифру, то есть число от 0 до 9.
2. Ячейка $n3[i3]$ должна содержать 0 до этого присваивания — в противном случае будет уничтожена часть уже полученного результата.

Сначала докажем, что `carry` содержит число от 0 до 9. До цикла по $i2$ значение переменной `carry` нулевое. При первом выполнении цикла максимальное значение `carry`

$$(0 + 9 + 9 * 9) \text{ div } 10 = 9$$

При последующих проходах этого цикла максимальное значение `carry`

$$(9 + 9 + 9 * 9) \text{ div } 10 = 9$$

Итак, доказано, что в `carry` после цикла по $i2$ содержится цифра (а не две, к примеру).

Теперь докажем, что $n3[i3]$ содержит 0. $i3$ после цикла по $i2$ равно $i1 + \text{len}2$. Действительно, перед циклом $i3 = i1$, затем $i3$ увеличивается в каждом цикле, то есть $\text{len}2$ раз. Переменная $i1$ во внешнем цикле увеличивается, и каждый очередной раз, когда управление попадает в строку $\{Q\}$, значение переменной $i3 = i1 + \text{len}2$ увеличивается на 1. Очевидно, во вложенном цикле затрагиваются только меньшие, чем в строке $\{Q\}$, индексы результата — там $i3$ меньше. Значит, увеличения $i3$ на 1 при каждом выполнении внешнего цикла достаточно для доказательства того, что ячейка $n3[i3]$ еще не посещалась. А раз она не посещалась, то в ней хранится 0 еще со времени обнуления массива в строке $\{G\}$.

Детали реализации: из-за квадратичной зависимости времени выполнения алгоритма от длины исходных чисел директивы проверки диапазонов $\{\$R+.0+\}$ следует использовать только во время отладки, а при отправке решения убирать их во избежание получения результата «Превышение предела времени».

Вариант 2

Можно записать и такое, более легкое для понимания (а главное — для запоминания), но *работающее в два раза дольше*, решение:

```
procedure mul(var n1, n2, n3 : number);
var
  i1, i2, i3, len1, len2, carry : integer;
begin
  len1 := len(n1);
  len2 := len(n2);
```

```
set0(n3):  
for i1 := 1 to len1 do begin  
  for i2 := 1 to len2 do begin  
    carry := n1[i1] * n2[i2];  
    i3 := i1 + i2 - 1;  
    while carry > 0 do begin  
      carry := carry + n3[i3];  
      n3[i3] := carry mod 10;  
      carry := carry div 10;  
      inc(i3);  
    end;  
  end;  
end;  
end;
```

Умножаем каждый разряд на каждый и добавляем в соответствующее место результата.

Вариант 3

Решение предложил Ярослав Музыкантов.

При умножении длинных чисел можно обойтись вообще без умножения, используя лишь сложение.

Пусть нужно умножить числа $n1$ и $n2$. Предварительно заполним массив $n1x$ вида

$n1x$: array [0..9, 1..2502] of byte;

$n1x[i]$ будет хранить число $n1$, умноженное на i . Его можно получить по формуле $n1x[0] = 0, n1x[i] = n1x[i - 1] + n1$.

Получение произведения двух длинных чисел теперь сводится к сложению $n1x[n2[i]] \cdot 10^i$. Умножение на степень десятки, очевидно, выполняется сдвигом числа на соответствующее число разрядов.

Сравнение алгоритмов

Достоинством варианта 3 является скорость. Оно работает в 1,5 раза быстрее, чем вариант 1.

Недостатком варианта 3 является отсутствие возможности выполнять вычисления не в десятичной системе счисления, а в системе счисления 10^N ($N = 2, 4, 9$), в то время как вариант 1 замечательно переводится в эти системы счисления и дает в некоторых из них увеличение скорости в полтора десятка раз.

Решение задачи 3 Ф. Змейка

Тема: элементы аналитической геометрии.

Заполнение массива числами происходит по прямым (рис. Р3.3).

			j			
		1	2	3	4	5
1						
2						
3						
4						
5						
i						

Рис. Р3.3. Прямые вида $i + j = \text{const}$

Уравнение такой прямой имеет вид: $i + j = \text{const}$. Для первой прямой это $i + j = 2$, для второй — $i + j = 3$ и т. д.

Нужно всего лишь выполнить цикл по целой переменной C от 2 до $2N$ — это будет параметр уравнения прямой. Для четных значений C нужно сделать цикл по i (номер строки матрицы) в порядке убывания от N к 1, а для нечетных значений C — сделать цикл по i в возрастающем порядке от 1 до N . Для каждого значения i нужно вычислить соответствующее значение $j = C - i$ (номер столбца матрицы). Если пара (i, j) находится внутри матрицы (то есть j от 1 до N), то на это место нужно ставить очередное число.

Тренировка 4

Решение задачи 4 А. Совершенные числа

Тема: теория чисел.

Как выглядит простейший алгоритм? Проверяем делимость каждого числа X из интервала от M до N на каждое из чисел от 1 до $X - 1$, в ходе проверки суммируем делители.

В этой задаче требуется только одно — проверять делимость не до $X - 1$, а до $\sqrt{X}$. Почему достаточно перебирать до корня, показано в разборе задачи 1 А — у каждого делителя, большего корня, есть соответствующий ему делитель, меньший корня. Однако в случае, когда число является точным квадратом, корню из числа будет соответствовать сам корень из числа, и чтобы не сосчитать его дважды, нужно принять специальные меры.

Обратите внимание на то, считает ли ваша программа совершенным число 1.

То, что этот алгоритм подойдет, видно из ограничений задачи. Первое ограничение ($1 \leq M \leq N \leq 10^9$) выглядит довольно устрашающе — за 5 с на указанном в предисловии компьютере с помощью подобного алгоритма можно перебрать менее 100 000 первых чисел, а находить сумму делителей каждого следующего числа все дольше и дольше. Все объясняет второе ограничение ($(N - M) \sqrt{N} \leq 10^7$) — если N большое, интервал не может быть слишком большим ($N - M \leq \frac{10^7}{\sqrt{N}}$), а при $M = 1$ верхняя граница интервала N может принимать значения только примерно до 40 000. Фактически, формула, данная в условии, является оценкой количества операций при описанном методе — для каждого числа (чисел $N - M + 1$, то есть примерно $N - M$) нужно произвести число действий, пропорциональное корню из него (N — самое большое число интервала, поэтому для оценки сверху корень извлекается из него). Произвести за 5 с 10 млн операций типа нахождения остатка от деления переменных типа `longint` — это вполне реально для указанного в предисловии компьютера.

Решение задачи 4 В. Разложение на слагаемые

Тема: перебор-построение.

Как вывести все разложения и не допустить повторений? Например, как вывести $1 + 2 + 1$ и не вывести при этом $1 + 1 + 2$? Нужно найти такое свойство, которым обладает только одно из разложений, одинаковых с точки зрения задачи.

Один из вариантов решения — выводить только разложения, в которых элементы идут в неубывающем порядке. Очевидно, что элементы любого разложения можно отсортировать так, что получится разложение с элементами в порядке неубывания. Также очевидно, что одинаковые с точки зрения задачи разложения имеют одинаковые отсортированные формы.

Общая идея решения такова: ставим на первое место все числа от 1 до N , на второе — все числа, удовлетворяющие двум условиям: а) оно больше либо равно первому; б) оно не превосходит того, что осталось от N после выбора первого числа; на третье место ставим все числа, не меньшие второго и не большие остатка, и т. д. Выводим разложение, если остаток равен нулю.

Обратите внимание на пример ответа задачи. Это типичный случай попытки составителя задачи увести решающего от решения, известного составителю. Если решающий поставит перед собой цель вывести для данных входных условий именно то, что показано в примере, он усложнит себе задачу в разы, если вообще сможет ее решить.

Глобальные объявления:

```
var
```

```
  n : integer;
```

```
  sum : array [1..40] of integer;
```

Здесь n — входные данные, sum — место, где записываются уже определенные слагаемые текущего варианта.

Рекурсивная процедура перебора:

```
  procedure p(step, min, left : integer);
```

Полное название процедуры может выглядеть так: «Перебрать все варианты разложения на слагаемые числа $left$ такие, что каждое из слагаемых не меньше min . Разложение должно быть записано в массив sum начиная со $step$ -го места в порядке неубывания слагаемых».

Параметры процедуры: $step$ — номер места, на котором перебирается слагаемое, min — значение предыдущего слагаемого, текущее не должно быть меньше, $left$ обозначает, сколько осталось от N после выбора элементов от 1-го до $(step-1)$ -го.

При вызове этой процедуры заполнены элементы массива sum от 1-го до $(step-1)$ -го.

Процедура устанавливает на место $step$ все допустимые слагаемые по очереди и для каждого из них вызывает себя рекурсивно с параметрами, учитывающими установку этого слагаемого. Если остатка нет, то комбинация выводится.

Рассмотрим работу процедуры более подробно.

Случай захода в тупик. Если $min > left$, то очередной элемент не может принимать никаких значений. Возвращаемся из процедуры, ничего не выводим.

Конечный случай. Если $min \leq left$, то весь остаток можно использовать в качестве последнего элемента. Изменяем массив sum для этого варианта и выводим его. Так-

же нужно учесть, что если $\text{step} = 1$, то ничего выводигь не надо — в разложении должно быть хотя бы два элемента.

Продолжение рекурсии (не исключается, когда что-то было выведено как конечный случай в данном вызове процедуры). Перебираются все значения от min до left включительно, $\text{sum}[\text{step}]$ устанавливается равным этому значению, затем процедура вызывает себя рекурсивно с параметрами $\text{step} + 1$, значением текущего слагаемого и обновленным остатком.

Можно, конечно, перебирать значения от min до $\text{left} \div 2$, имея в виду, что следующий элемент суммы должен быть больше текущего, поэтому текущий нельзя делать больше половины остатка. Тогда, похоже, даже не потребуется отдельно рассматривать случай захода в тупик.

Вызов рекурсии:

$p(1, 1, n)$;

В первый раз вызванная процедура будет устанавливать первый элемент массива sum , причем он может принимать все значения, большие или равные 1. В этот момент из N еще ничего не вычиталось, поэтому остаток — оно само.

Решение задачи 4 С. Гангстеры

Вариант 1

Решение предложил Александр Меркулов.

Тема: динамическое программирование.

Отсортируем гангстеров по неубыванию времени прихода. Если полнота гангстера больше времени его прихода, то он не может попасть в ресторан. Богатство таких гангстеров устанавливаем равным 0 — в этом случае от того, зашел он или нет, конечный результат не зависит.

Рассматриваем гангстеров в порядке неубывания времени прихода, и для каждого вычисляем, каким получится суммарное богатство, если этот гангстер станет последним пришедшим. Ничего не стоит найти эту характеристику, если известны характеристики гангстеров, пришедших раньше (они уже рассмотрены). Если только один этот гангстер попал в ресторан, то суммарное богатство будет равно его богатству. В противном случае перебираем, какой из уже рассмотренных к данному моменту гангстеров был предпоследним. Если в ресторан попало более одного гангстера, суммарное богатство будет равно суммарному богатству (характеристике) предпоследнего гангстера плюс богатство последнего. Конечно, не любой гангстер может быть предпоследним — должно выполняться следующее неравенство: модуль разности полноты должен не превосходить разности времени прихода. Выбирая максимум характеристик претендентов, получаем значение характеристики для текущего гангстера.

Какой-то из гангстеров является последним в оптимальном плане открытия двери. Нужно просто найти максимум из полученных характеристик.

Вариант 2

Тема: динамическое программирование.

Исключим из входных данных гангстеров, у которых полнота больше времени прихода.

Заполняем таблицу $M[0..30100, 0..100]$ — первый индекс означает текущее время, второй — текущую степень открытости двери. Элемент таблицы $M[t, i]$ означает, что к моменту времени t , если дверь оказалась в состоянии i , суммарное богатство может составить $M[t, i]$. Ответом задачи будет являться число $M[30100, 0]$ — даже если последний гангстер имел полноту 100 и пришел в момент времени 30 000, все равно к моменту времени 30 100 можно считать, что дверь закрыта.

Итак, заполняем таблицу. Для $M[0, i]$, $0 \leq i \leq 100$, значение таблицы 0. Гангстеров полноты 0 не существует, а остальные не могли попасть в нулевой момент времени.

Пусть для фиксированного t известны все $M[t-1, i]$, $0 \leq i \leq 100$. Найдем все $M[t, i]$. В состояние i в момент времени t можно попасть только из состояний $i, i-1$ или $i+1$ в момент времени $t-1$. Значит, выбираем из них оптимальное: $M[t, i] = \max(M[t-1, i-1], M[t-1, i], M[t-1, i+1])$. Также добавим к $M[t, i]$ богатство гангстеров, приходящих в момент времени t и имеющих полноту i , — если уж дверь открыта специально для них, то почему бы им не войти?

Когда таблица заполнена, осталось только вывести результат — $M[30100, 0]$.

Детали реализации:

- В Турбо Паскале нельзя объявить таблицу размером $30\,101 \cdot 101 = 3\,040\,201$, то есть более трех миллионов элементов. Но это и не нужно, ведь $M[t, i]$ вычисляется только по значениям таблицы с первым индексом, равным $t-1$, значит, достаточно всего двух линейных массивов размером 101. Первый будем заполнять как $M[t]$, из второго будем брать данные как из $M[t-1]$. После того как заполнен массив, соответствующий $M[t]$, нужно его присвоить массиву, соответствующему $M[t-1]$ и повторить всю процедуру для очередного t .
- Рассмотрим, как добавлять к $M[t, i]$ богатство гангстеров, пришедших в момент времени t и имеющих полноту i . Нельзя для каждого времени и для каждой степени открытости двери перебирать всех гангстеров и определять, соответствует ли полнота гангстера этой степени открытости. Получится число действий $30\,100 \cdot 101 \cdot 100 = 304\,010\,000$ — количество моментов времени, для которых происходит такое вычисление, умноженное на количество степеней открытости двери, умноженное на число гангстеров составляет более 30 млн. Триста миллионов таких операций как индексирование массива — это довольно много. Мы не можем позволить себе этого.

Решение этой проблемы может быть таким: для каждого времени t (то есть 30 100 раз) заполняем массив A размером 101 элемент — как состояний откры-

тости двери. Сначала обнуляем A . Затем сравниваем время прихода каждого гангстера (не более 100) с текущим t , и если оно совпадает, добавляем в элемент A , соответствующий полноте гангстера, его богатство. Итого получается $30 \cdot 100 \cdot 100$ дополнительных действий, выполнить которые реально. После нахождения элемента $M[t, i]$ без учета гангстеров, явившихся в момент времени t , просто добавляем к нему $A[i]$.

- Рассмотрим, как наиболее эффективно находить $M[t, i] = \max(M[t-1, i-1], M[t-1, i], M[t-1, i+1])$. Очевидно, для $i=0$ не существует $M[t-1, i-1]$, а для $i=100$ не существует $M[t-1, i+1]$. Чтобы не рассматривать эти случаи отдельно, нужно объявить массивы для степеней открытости двери с индексами не от 0 до 100, а от -1 до 101, причем -1 -й и 101-й элементы всегда оставлять нулевыми.

Решение задачи 4 D. Площадь многоугольника

Вариант 1

Решение сообщил Александр Степанович Сипин.

Пусть даны две точки: $T_1(x_1, y_1)$ и $T_2(x_2, y_2)$. Найдем площадь трапеции, образованной отрезком T_1T_2 , перпендикулярами, опущенными из этих двух точек на ось OX , и проекцией этого отрезка на ось OX . Она равна $S = (x_1 - x_2) \frac{y_1 + y_2}{2}$ — высоте трапеции $(x_1 - x_2)$, умноженной на полусумму оснований $\frac{y_1 + y_2}{2}$. На самом деле, если нужно найти именно площадь трапеции, то нужно еще взять найденную величину по модулю — площадь отрицательной не бывает. Но для дальнейших рассуждений потребуются именно формула площади трапеции со знаком.

Площадь многоугольника со знаком равна $S_m = S_T(1, 2) + S_T(2, 3) + \dots + S_T(N-2, N-1) + S_T(N-1, N) + S_T(N, 1)$, где $S_T(i, j)$ — площадь со знаком соответствующей трапеции для i -й и j -й точек многоугольника (рис. P4.1). Заметьте, знак $S_T(i, j)$ зависит от того, $x_i < x_j$ или $x_i > x_j$. Рисунок поможет убедиться в истинности формулы.

Нужно не забыть перед выводом ответа взять полученную площадь по модулю, ведь координаты многоугольника во входных данных могли быть заданы как в порядке обхода по часовой стрелке, так и против нее.

Вместо того чтобы выполнять деление на 2 каждый раз при вычислении площади трапеции, можно выполнить его один раз — для суммы. При этом количество операций деления в программе сократится с N до 1.

Детали реализации:

- Подберите подходящие типы данных.
- Включите проверки диапазонов для проверки того, правильные ли типы данных вы выбрали.

- Не нужно считать все данные в память — для приведенной формулы площади многоугольника нужны лишь координаты двух последовательных точек, также нужно запомнить координаты первой точки.

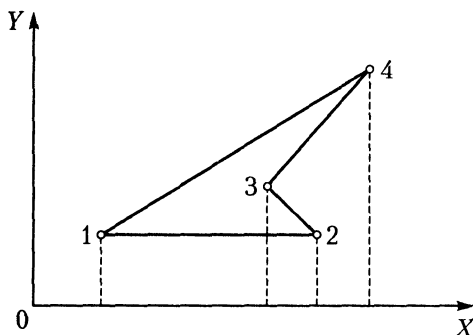


Рис. Р4.1. Вычисление площади многоугольника через площади трапеций

Вариант 2

Для выпуклого многоугольника очевидна формула $S_m = S_{\text{тр}}(1, 2, 3) + S_{\text{тр}}(1, 3, 4) + S_{\text{тр}}(1, 4, 5) + \dots + S_{\text{тр}}(1, N-2, N-1) + S_{\text{тр}}(1, N-1, N)$, где $S_{\text{тр}}(i, j, k)$ — площадь треугольника, образованного i -й, j -й и k -й точками.

Для площади невыпуклого многоугольника справедлива эта же формула с одной поправкой: при переходе от i -й точки к j -й по часовой стрелке относительно первой площадь треугольника нужно добавлять, а при переходе против часовой стрелки — вычитать. Этот подход иллюстрирует рис. Р4.2.

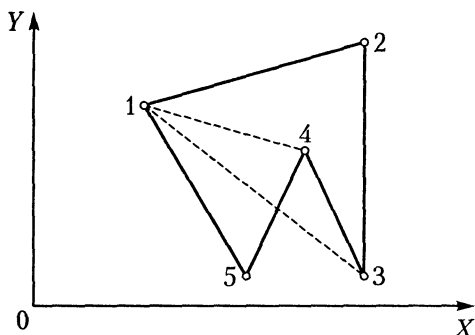


Рис. Р4.2. Вычисление площади многоугольника через площади треугольников

Для нахождения площади треугольника со знаком идеально подходит векторное произведение векторов, дающее формулу

$$S_{\text{тр}}(i, j, k) = \frac{(x_k - x_i)(y_j - y_i) - (x_j - x_i)(y_k - y_i)}{2}.$$

Так же, как и в варианте 1, нужно не забыть взять полученную площадь по модулю.

Так же, как и в варианте 1, можно деление на 2 вынести за сумму.

При решении этим способом формула площади треугольника со знаком вычисляется $N - 2$ раза, причем каждый раз выполняется две операции умножения и пять — вычитания. При больших N это более чем вдвое дольше, чем выполнять операции умножения, сложения и вычитания по N раз, как в варианте 1. Поэтому данную формулу нужно использовать только в случае, если она запоминается легче или способ ее получения дает простой способ решения каких-нибудь сходных задач.

Детали реализации:

- Подберите подходящие типы данных.
- Включите проверки диапазонов для проверки того, правильные ли типы данных вы выбрали.
- Не нужно считывать все данные в память — для приведенной формулы нужны лишь координаты первой и двух последовательных точек.

Решение задачи 4 Е. Деление длинного числа на короткое

Объявление типа длинного числа, ввод и вывод такие же, как в решении задачи 2 Е. В данном случае отличается только константа `numlen`. Из решений задач 2 Е и 3 Е можно видеть, что в качестве длины числа нужно использовать совсем не то число, которое написано в задании в качестве степени десятки — следует находить реальную длину чисел, которые могут быть даны и которые можно получить с такими ограничениями. Неправильный выбор длины числа является, пожалуй, наиболее распространенной ошибкой в задачах длинной арифметики. Конкурирующая с ней ошибка «неправильный вывод нуля» в практике человека, решающего олимпиадные задачи, встречается обычно не более одного раза — после этого он запоминает фрагмент программы, ответственный за правильный вывод числа. А вот выбирать ограничение на длину в каждой новой задаче приходится заново.

Сама процедура деления длинного числа на короткое:

```
procedure divshort(var n : number; divisor : longint; var rem : longint);
var
  i : word;
begin
  rem := 0;
  for i := numlen downto 1 do begin
    rem := rem * 10 + n[i];
    n[i] := rem div divisor;
    rem := rem mod divisor;
  end;
end;
```

Первый параметр процедуры — исходное длинное число, результат будем записывать туда же (результат сложения длинных чисел также записывался в переменную, представляющую первое слагаемое). Второй параметр — делитель. Третий, выходной параметр — остаток от деления.

Сама процедура деления ничем не отличается от обычного деления в столбик. Сначала остатка от деления старших разрядов нет. Для каждого разряда от старшего к младшему к остатку от деления предыдущих разрядов дописываем справа очередную цифру. Целая часть деления этого числа на делитель дает очередную цифру результата, а остаток используется в последующих вычислениях или, если это самый младший разряд, является частью ответа.

Обратите внимание на тип переменной *i*, используемой в качестве индекса *n*. В этой задаче длина исходного числа может быть больше, чем максимальное число типа *integer*. Также для типа *i* можно было использовать и *longint*.

Решение задачи 4 F. Скобки

Тема: скобочные выражения

Эта задача — первая в серии задач, где рассматриваются скобочные выражения. Она знакомит с понятием *правильного скобочного выражения*. Скобочное выражение является правильным, если у каждой открывающей скобки есть своя закрывающая скобка соответствующего типа, а внутри этой пары скобок находится правильное скобочное выражение.

При решении задачи нужно реализовать структуру данных, которая называется *стек*. С ней разрешено проводить следующие операции: занести элемент (*push*), взять элемент (*pop*). Также нужно проверять, есть ли в данный момент в стеке элементы.

Из стека элементы забираются в порядке, обратном порядку их занесения, то есть стек подчиняется правилу «последним пришел — первым ушел» (*Last In First Out*, *LIFO*).

Рассмотрим содержимое стека при разборе выражения $[[([{}])]]()$. В стеке (рис. P4.3) будем хранить открытые и до сих пор не закрытые скобки в порядке их открытия.

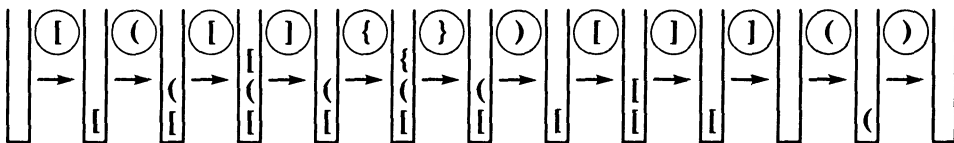


Рис. P4.3. Состояние стека при обработке выражения $[[([{}])]]()$

В схеме над стрелками показано, какая скобка обрабатывается в данный момент. Вертикальные столбики скобок показывают состояние стека, самая нижняя скобка — та, что попала в стек первой, она будет извлечена последней, а самая верхняя — та, которая попала последней к текущему моменту и будет извлечена первой.

Скобочное выражение является неправильным в следующих трех случаях.

1. Очередной является закрывающая скобка, а открытых скобок уже не осталось (стек пуст).
2. Тип очередной закрывающей скобки не совпадает с типом последней открытой скобки.
3. После обработки всего выражения в стеке остались открывающие скобки.

Иначе скобочное выражение является правильным.

Детали реализации:

- Легче всего реализовать стек в виде совокупности массива из символов и одного целого числа, которое обозначает размер стека на данный момент. Обратите внимание на то, какого типа должно быть число — размер текущего стека.
- Турбо Паскаль не позволяет объявить массив из символов размером 100 000 элементов. Но это и не нужно! Достаточно массива из 50 000 элементов. Ведь если число скобок максимальное — 100 000, то открывающих в *правильном* скобочном выражении — не более 50 000. Так что если открывающих скобок в какой-то момент стало 50 001, можно выводить ответ.

Тренировка 5

Решение задачи 5 А. Дружественные числа

Темы: теория чисел, метод вычисления заранее.

Казалось бы, эта задача ничем не отличается от задачи 4 А. Так же для каждого числа из интервала находим все его делители до корня из него вместе с парными делителями, большими корня, для получившейся суммы проделываем ту же процедуру, и если сумма ее делителей совпадет с первым числом — можно выводить пару.

Однако различия между задачами все-таки есть. Дело в том, что для указанного в предисловии компьютера подобный алгоритм на тесте, задающем максимальный интервал, будет работать чуть больше 4 мин вместо необходимой 1 с.

Предлагаемое решение будет основываться на том, что, во-первых, дружественные числа встречаются крайне редко (в промежутке до миллиона их менее сотни пар), а во-вторых, можно найти их все за разумное время (были упомянуты 4 мин).

Представим себе, что мы знаем все пары дружественных чисел. Тогда написать решение, работающее намного меньше 1 с, ничего не стоит — нужно просто в нужном порядке проверить, лежит ли каждая пара в требуемом интервале.

Итак, напишем программу, которая «честно» вычисляет дружественные числа (но на *нашем* компьютере, а не на компьютере тестирующей системы!) и записывает их в файл. Этот файл затем вашими усилиями превратится в исходный текст программы, которая и будет послана проверяющей системе.

Детали реализации:

- Лучший формат вывода пар дружественных чисел вычисляющей программой таков:

```
(220, 284).  
(1184, 1210).
```

то есть в каждой строке в скобках записана пара через запятую, а после скобок — еще одна запятая. Такой формат позволит легко преобразовать вывод вычисляющей программы в сдаваемую программу. В сдаваемой программе будет описана типизированная константа:

```
const  
  f : array [1..5, 1..2] of longint = (  
    (220, 284),  
    (1184, 1210),
```

(2620, 2924).

(5020, 5564).

(6232, 6368)

);

В этом примере показано объявление типизированной константы — двумерного массива для дружественных чисел в интервале до 10 000. Для интервала до 1 000 000 нужно соответствующим образом изменить размер массива с 5 до количества выданных вычисляющей программой строк.

- Время работы вычисляющей программы можно уменьшить, если находить сумму делителей не любой суммы делителей исходного числа, а только той, которая получилась в диапазоне от самого числа до верхней границы интервала.
- Обратите внимание на то, являются ли совершенные числа дружественными самим себе. Ответ на этот вопрос можно узнать, внимательно прочитав условие задачи.
- По традиции напоминаю, что нужно не забыть вывести соответствующее сообщение, если на заданном интервале дружественных чисел нет. Об этом легко забыть, наблюдая работу программы, пишущей другую программу.

ПРИМЕЧАНИЯ

1. «Метод вычисления заранее» нельзя было использовать в задаче 2 А, используя простой алгоритм из задачи 1 А, — простых чисел в интервале до миллиона слишком много (78 498), для них очень непросто, если вообще возможно, найти способ хранения в исходном тексте программы.

Также этот метод было нельзя применить при решении задачи 4 А — там основной интервал был слишком большим, и никаким стандартным методом найти в нем совершенные числа за разумное время было нельзя. Оценим, сколько пришлось бы ждать. В интервале от 1 до 40 000 совершенные числа находятся за 5 с. Если бы на определение того, является ли совершенным одно большое число, требовалось то же время, что и в среднем на интервале, то ждать пришлось бы $5 \cdot (1\,000\,000\,000 / 40\,000) = 125\,000$ с — больше суток (в сутках 86 400 с). И это еще без учета возрастания корня из числа с ростом числа (на интервале от 40 000 до 1 000 000 000 корень увеличивается более чем в 150 раз). Ну как, радует вас перспектива вычислять совершенные числа около года?

2. При использовании «метода вычисления заранее» нельзя использовать совсем неэффективные алгоритмы. Например, если при нахождении дружественных чисел перебирать для каждого числа делители не до корня, а до самого числа без единицы, то такой алгоритм на интервале от 1 до 1 000 000 вместо 4 мин будет работать двое суток, что абсолютно нереально в условиях олимпиады.

Решение задачи 5 В. Скобки (2)

Тема: перебор-построение, скобочные выражения.

При решении этой задачи предполагается умение работать со скобочными выражениями (задача 4 F).

Общая идея решения такова: ставим на первое место все скобки, которые могут присутствовать на первом месте в правильном скобочном выражении, на второе —

все скобки такие, что сочетание первой и второй скобок может быть началом правильного скобочного выражения, на третье место ставим все скобки такие, что сочетание первых трех скобок может быть началом правильного скобочного выражения, и т. д.

К примеру, пусть построено следующее начало:

`([[( )`

Какая скобка может быть следующей? Очевидно, можно закрыть последнюю открытую и не закрытую квадратную скобку. Теперь посмотрим, можно ли поставить открывающую скобку. В правильном скобочном выражении длиной N ровно $N/2$ открывающих скобок. Если уже использовано $N/2$ открывающих скобок, то еще одну поставить нельзя. Если же в построенном начале менее $N/2$ открывающих скобок, то еще одну открывающую (причем любого типа) поставить можно. Итак, для приведенного начала выражения ответ таков: если $N > 8$, то на следующее место наряду с `]` можно поставить и `(`, и `[`, а если $N = 8$, то выбора нет — следующая скобка обязана быть `]`. Стоит сказать, что $N < 8$ быть не может, поскольку, ставя все предыдущие открывающие скобки, мы заботились о том, чтобы их суммарное число не превысило $N/2$.

Глобальные объявления:

```
const
    maxn = 14;
type
    stack = array [1..maxn div 2] of char;
var
    n : integer;
    result : array [1..maxn] of char;
```

Здесь n — входные данные, `result` — переменная под строящийся результат. Определяем тип `stack` — для хранения порядка открытых и не закрытых к данному моменту скобок. Будем передавать этот массив как параметр-значение. Это несколько упрощает рекурсивную процедуру перебора скобочных выражений по сравнению с процедурой, использующей глобальный стек. В качестве упражнения можете реализовать приведенный далее алгоритм, а затем реализовать то же самое с глобальным стеком. Расходы на передачу стека незначительны — он занимает всего 7 байт. Это сравнимо с размером остальных параметров процедуры вместе взятых.

Рекурсивная процедура перебора:

```
procedure p(place, toOpen, sp : integer; s : stack);
```

Полное название процедуры может выглядеть так: «Перебрать все правильные скобочные выражения длиной n , у которых первая `place - 1` скобка определена в первых элементах массива `result`».

Параметры процедуры обозначают следующее: `place` — на какое место нужно поставить очередную скобку, `toOpen` — сколько еще осталось открывать скобок, `sp` — количество открытых, но не закрытых к данному моменту скобок; в элементах `s` от

1-го до sp -го хранятся сами открытые, но не закрытые скобки (в $s[sp]$ — последняя).

При вызове этой процедуры заполнены элементы массива `result` от 1-го до $(place-1)$ -го.

Процедура работает так. Если $place = n + 1$, то все скобки расставлены, и комбинация выводится. Если $place \leq n$, то на место $place$ массива `result` устанавливаются по очереди все скобки такие, что первые $place$ элементов массива `result` содержат допустимое начало правильного скобочного выражения длиной n . Для каждого варианта процедура вызывает себя рекурсивно, чтобы расставить скобки с $(place+1)$ -й до n -й.

Рассмотрим работу процедуры более подробно.

Конечный случай. Если $place = n + 1$, то поставлены все N скобок, и можно выводить очередную комбинацию, полученную в массиве `result`.

Продолжение рекурсии ($place \leq n$):

- Если открывать скобки можно, то есть $toOpen > 0$, то ставим на место $place$ открывающую круглую скобку, отмечаем ее в s и вызываем процедуру рекурсивно со следующими параметрами: $place + 1$ — ставить очередную скобку на следующее место; $toOpen - 1$ — количество скобок, которые предстоит открыть, уменьшилось; $sp + 1$ — стек незакрытых скобок увеличился; s — в этом вызове процедуры стек один. Затем проделываем то же самое для открывающей квадратной скобки, только нужно не забывать, что она ставится не после открытой ранее круглой скобки, а вместо нее.
- Если есть еще незакрытые скобки, то есть $sp > 0$, то ставим на место $place$ скобку, соответствующую последней незакрытой, и вызываем процедуру рекурсивно с параметрами $place + 1$ — ставить очередную скобку на следующее место, $toOpen$ — число открывающих скобок не изменилось, $sp - 1$ — размер стека уменьшился, s — передаем тот же стек.

Вызов рекурсии:

`p(1, n div 2, 0, s);`

В первый раз вызванная процедура будет устанавливать первый элемент массива `result`, предстоит открыть $N/2$ скобок, открытых и не закрытых скобок еще нет, четвертым параметром является объявленная глобально переменная-стек.

Решение задачи 5 С. Маршрут (2)

Темы: динамическое программирование, элементы аналитической геометрии.

Обозначим исходный массив A , его элемент в i -й строке и j -м столбце — $A[i, j]$. Объявим трехмерный массив M , его элемент $M[i, j, k]$ будет хранить максимальную сумму, которую можно получить, сложив числа в ячейках по пути длиной k , заканчивающемся в ячейке $[i, j]$. Если путь длиной k не может заканчиваться в $[i, j]$, $M[i, j, k]$ будет содержать 0.

При $k = 1$ заполнение массива очевидно: $M[1, 1, 1] = A[1, 1]$ — если остаться в клетке $[1, 1]$, то в сумме можно получить именно столько, остальные $M[i, j, 1]$ равны 0, так как путь длиной 1 не может привести в клетку $[i, j]$, если $i \neq 1$ или $j \neq 1$.

Заполним M при $k > 1$. Пусть заполнены все клетки M для предыдущих значений k и нужно вычислить $M[i, j, k]$. В клетку $[i, j]$ можно прийти из четырех клеток — $[i - 1, j]$, $[i + 1, j]$, $[i, j - 1]$, $[i, j + 1]$. Причем, если в клетку $[i, j]$ нужно попасть по пути длиной k , то в любую из этих четырех клеток нужно прийти по пути длиной $k - 1$. Для каждой из этих клеток для пути длиной $k - 1$ максимальная сумма чисел уже найдена, осталось только выбрать оптимальный ход из четырех. Итак, $M[i, j, k] = \max(M[i - 1, j, k - 1], M[i + 1, j, k - 1], M[i, j - 1, k - 1], M[i, j + 1, k - 1]) + A[i, j]$. Правда, если все четыре клетки недостижимы ровно за $k - 1$ шаг, то и в $M[i, j, k]$ нужно занести 0.

В качестве ответа нужно выдать максимальное число из $M[i, j, K]$, $1 \leq i \leq N, 1 \leq j \leq N$, — это лучшее, что можно получить за K ходов.

Детали реализации:

- Из примера определите, что именно подразумевается под длиной пути — число переходов или число посещенных клеток с учетом кратности посещения.
- Очевидно, в Турбо Паскале нельзя объявить массив M размером $100 \times 100 \times 2000$. Но это и не нужно. Часть массива для текущего шага строится только на основании информации о предыдущем шаге, значит, достаточно двух массивов 100×100 .

К сожалению, Турбо Паскаль не позволяет объявить две глобальные переменные типа `array [1..100, 1..100] of longint` — их размер 40 000 байт, а объем основной памяти в Турбо Паскале 64 Кбайт — как раз хватило бы на матрицу исходных значений 100×100 элементов типа `integer` и один массив 100×100 элементов типа `longint`. Так нельзя ли значения и текущего, и предыдущего шагов хранить в одном массиве?

Оказывается, можно. Воспользуемся *«уравнением шахматной доски»*. Пусть фигура стоит на черной клетке и может одним шагом перейти только вверх, вниз, вправо или влево, но не может остаться на месте. Очевидно, что за один шаг она может оказаться только на белой клетке. А с белой клетки одним шагом такая фигура может перейти только на черную. Значит, за четное число шагов, если начать с черной клетки, фигура может оказаться только на черной клетке. А за нечетное число шагов — только на белой.

Пусть есть номер строки i и номер столбца j . Как по ним определить, является ли клетка шахматной доски черной или белой? Оказывается, сумма $i + j$ четна для одного цвета клетки и нечетна — для другого.

Применим «уравнение шахматной доски» к данной задаче. Пройдя по пути длиной k , можно оказаться только в тех клетках, у которых четность $i + j$ отличается от четности k . Если четность $i + j$ не равна четности k для клетки $[i, j]$, значит, на текущем проходе массива четыре соседних клетки не менялись (меняются суммы в клетках «другого цвета»), значит, они остались от предыду-

щего прохода, и можно смело подставлять эти четыре значения в формулу перехода.

Для каждого k нужно сделать два прохода — на первом находим суммы для каждого элемента, куда, исходя из четности координат, можно прийти по пути длиной k , на втором проходе обнуляем суммы для элементов, куда, исходя из четности координат, нельзя прийти по пути длиной k . Совмещать эти проходы нельзя.

Заметьте, что если бы можно было ходить и по диагонали, применить «уравнение шахматной доски» было бы нельзя.

- Чтобы каждый раз не проверять, не на границе ли массива находится клетка и существуют ли клетки $[i - 1, j]$, $[i + 1, j]$, $[i, j - 1]$, $[i, j + 1]$, нужно объявить массив под суммы не как `array [1..100, 1..100] of longint`, а как `array [0..101, 0..101] of longint`, причем его элементы на границе (i или j равны 0 или $n + 1$) заполнить нулями.
- Индексирование двумерного массива — довольно продолжительная операция, поэтому проконтролируйте, чтобы фрагмент, отвечающий за нахождение максимума, содержал не слишком много индексирований. Кроме того, этот фрагмент — наиболее часто выполняющийся в программе, поэтому из соображений скорости желательно не выделять его в отдельную процедуру или функцию.

Дополнение 1

Предложил Никита Рыбак.

Достижимость клетки можно определить из неравенства $k > i + j - 2$. Здесь $i, j \geq 1$, а k означает количество пройденных клеток.

Однако трудно сказать, что будет быстрее — проверять для каждой пары координат это неравенство или сравнивать максимальное из значений четырех соседних клеток с нулем.

Дополнение 2

Нашли Никита Рыбак и Федор Меньшиков.

Не всегда является хорошей идеей писать в начале программы

`type`

```
integer = longint;
```

или, что то же самое, использовать только тип `longint` для целых переменных. Это плохо в двух случаях: когда есть жесткие ограничения на память (в этой задаче, например, два массива из 10 000 элементов типа `longint` объявить нельзя), и когда задача критична по времени (то есть в условии указано время работы программы 5 с, а у жюри есть решение, работающее 4 с на максимальном тесте).

Понятно, что в Турбо Паскале работа с переменными типа `integer` происходит быстрее, чем с переменными типа `longint`. Но во сколько раз? Проведем небольшое исследование для условия «четность $i + j$ не равна четности k » (см. таблицу). Время приведено для Intel Celeron 400.

Сравнение	Тип i, j, k	Время работы на максимальном тесте, с
$(i + j) \bmod 2 \lt \> k \bmod 2$	integer	5,50
$(i + j) \bmod 2 \lt \> k \bmod 2$	longint	14,90 (!)
$\text{odd}(i + j) \lt \> \text{odd}(k)$	integer	4,15
$\text{odd}(i + j) \lt \> \text{odd}(k)$	longint	4,60
$(i + j) \text{ and } 1 \lt \> k \text{ and } 1$	integer	3,55
$(i + j) \text{ and } 1 \lt \> k \text{ and } 1$	longint	4,20

Выводы:

1. При определении четности целесообразно использовать уж если не битовую арифметику, то хотя бы стандартную функцию `odd`. В сочетании с `longint` и то, и другое работает только чуть дольше, чем в сочетании с `integer`.
2. Нужно трижды подумать, прежде чем использовать `longint` в критичной по времени задаче. В этой задаче, если применять `mod`, программу с типом `integer` можно как-то изменить, чтобы она подходила по времени (например, вычислять $k \bmod 2$ каждый раз, когда меняется k , и помещать результат в отдельную переменную). Но используя в этой задаче `mod` вместе с `longint`, вы получите программу, у которой нет никаких шансов быть принятой.

Решение задачи 5 D. Выпуклая оболочка

Тема: геометрия.

Опишем некоторые свойства искомого объекта:

1. Искомый многоугольник является выпуклым. Если бы он был невыпуклым, можно было бы построить выпуклый многоугольник с вершинами в некоторых из вершин невыпуклого многоугольника так, чтобы невыпуклый полностью содержался в нем. Очевидно, все точки содержались бы в этом выпуклом многоугольнике, а периметр был бы меньше. Периметр был бы меньше потому, что между какими-то двумя вершинами контур выпуклого многоугольника прошел бы напрямую, а контур невыпуклого — по ломаной, длина которой очевидно больше длины отрезка (рис. P5.1).

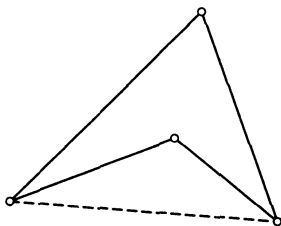


Рис. P5.1. Оптимальный многоугольник — выпуклый

2. Вершинами многоугольника являются какие-то из заданных точек, то есть нет вершин многоугольника, не совпадающих с исходными точками. Пусть есть вершины, не совпадающие с исходными точками. Выберем одну такую вершину. Обозначим ϵ минимум расстояний от этой точки до исходных точек и до всех вершин многоугольника, кроме данной. Отметим две точки, лежащие на сторонах, смежных с данным углом, и отстоящие от угла на $\epsilon/2$. Очевидно, что в построенном треугольнике не будет ни исходных точек, ни других вершин многоугольника. «Отрежем» этот треугольник от многоугольника. Согласно неравенству треугольника (любая сторона меньше суммы двух других), периметр многоугольника уменьшится. Значит, если в контуре многоугольника есть точка, не совпадающая ни с одной из исходных, то его периметр не минимален. Из этого следует, что если вообще можно построить многоугольник с минимальным периметром, то все его вершины — точки исходного множества (рис. P5.2).

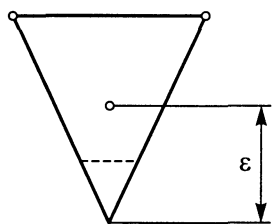


Рис. P5.2. Вершины оптимального многоугольника — исходные точки

Исходя из этих двух положений, получим процедуру генерации искомого объекта. Прежде всего найдем точку, гарантированно являющуюся вершиной искомого многоугольника (рис. P5.3). Подходит самая левая из самых нижних точек. Она не может быть строго внутри многоугольника — тогда сторона, относительно которой она лежит сверху, должна содержать точку, находящуюся ниже этой точки, а таких точек нет. Она не может лежать и на стороне — если сторона не горизонтальна, то одним из ее концов должна быть расположенная ниже точка. Если сторона горизонтальна, то вершиной должна быть точка, расположенная на одной высоте с выбранной и левее, а мы рассматриваем самую левую точку из нижних. Итак, одна вершина многоугольника найдена.

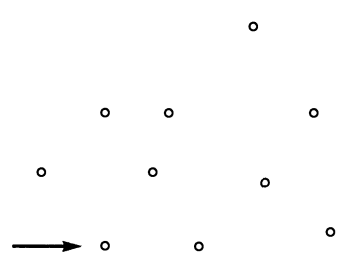


Рис. P5.3. Самая левая из самых нижних точек

Найдем остальные вершины многоугольника в порядке обхода по часовой стрелке. Если получать их последовательно, периметр находится элементарно — как сумма расстояний между соседними точками.

Обозначим вектором (dx, dy) единичной длины направление, по которому мы пришли в текущую вершину из предыдущей. Значение этого вектора для первой точки установим равным $(-1, 0)$. Теперь нужно найти точку, направление на которую из текущей вершины имеет самый маленький угол с вектором (dx, dy) , а из точек с минимальным углом — ту, которая находится на максимальном расстоянии от текущей. Действительно, если выбрать точку с не самым маленьким углом, то точка с самым маленьким углом окажется слева от построенной стороны, а граница строится так, чтобы все точки находились справа. Выбор самой далекой из точек с минимальным углом также очевиден — именно она является вершиной, остальные лежат на стороне.

Выбрав точку с минимальным углом, добавим к периметру расстояние от текущей вершины до этой точки, обновим вектор (dx, dy) — это теперь будет единичный вектор направления из текущей вершины в только что найденную, затем номером текущей вершины сделаем номер выбранной — очередная вершина многоугольника найдена. Описанные действия нужно повторять до тех пор, пока не окажемся в начальной вершине. Если на очередном шаге это произошло, можно выводить полученный периметр.

Детали реализации:

- Подберите подходящий тип данных, чтобы все промежуточные значения помещались в него. В геометрических задачах с вещественной арифметикой, возможно, даже исходные целые данные имеет смысл хранить в переменных вещественного типа — тогда любые переполнения будут исключены.
- Неориентированный угол между векторами легко найти через скалярное произведение векторов:

```
sp := dx * (x[i] - x[current]) + dy * (y[i] - y[current]);  
len := sqrt(sqr(x[i] - x[current]) + sqr(y[i] - y[current]));  
cos := sp / len;
```

Здесь $current$ — индекс последней найденной вершины на границе многоугольника, i — индекс предполагаемой новой вершины многоугольника.

Формула упрощается за счет того, что (dx, dy) — единичный вектор, и скалярное произведение не нужно делить на его длину.

Заметьте, что сам угол находить не обязательно. Вместо углов можно сравнивать косинусы: меньше угол — больше косинус.

Также заметьте, что в формуле используется деление на длину вектора из текущей вершины в предполагаемую следующую. Если длина окажется равной нулю, программа, не рассматривающая этот случай отдельно, завершится с ошибкой. Конечно, в задаче оговорено, что точки не совпадают, но совсем не обязательно так будет и в других подобных задачах. Но даже если точки не

совпадают, нужно проследить, чтобы текущая вершина не попала в эту формулу в качестве предполагаемой следующей. В любом случае, лучше сравнивать len с нулем, а не i с current .

В качестве исходного значения косинуса можно взять -2 . Любой найденный косинус будет больше. А если после выбора следующей вершины косинус так и останется равным -2 , значит, на самом деле задана только одна точка (или несколько совпадающих). Такой случай не может встретиться в этой задаче — здесь гарантируется ненулевая площадь, а если он встретится в других задачах, то к следующей вершине перейти нельзя и нужно выдавать в качестве длины выпуклой оболочки 0 .

- Получить единичный вектор можно следующим образом:

```
dx := (x[next] - x[current]) / bestlen;  
dy := (y[next] - y[current]) / bestlen;
```

Здесь next — индекс выбранной новой точки, current — индекс старой точки, bestlen — расстояние между этими точками (запоминается len , когда соответствующий косинус окажется больше текущего максимума).

Решение задачи 5 Е. Системы счисления

Первый из подводных камней этой задачи — использование в описании входных данных задачи имен, широко используемых как переменные цикла `for`. Чтобы не менять привычный вид циклов, рекомендуется для оснований систем счисления использовать имена переменных типа `ii` и `jj`.

Объявление типа данных для этой задачи ничем не отличается от других уже рассмотренных задач длинной арифметики. Действительно, цифры в задаче не превосходят 35 , поэтому они могут быть представлены переменными типа `byte`.

Вопрос, как всегда, заключается в том, какой выбрать константу numlen . Как отмечалось в разборе задачи 4 Е, при решении каждой очередной задачи нужно задумываться о длине возможных конечных и промежуточных значений, а не автоматически подставлять число из условия. К примеру, при переводе числа из системы с большим основанием в систему с меньшим длина числа увеличивается. Оценим, во сколько раз увеличится длина при переводе из 36 -ричной системы в двоичную. Если переводить из 32 -ичной системы, то на каждый разряд приходится ровно 5 , а если из 64 -ричной — то ровно 6 двоичных разрядов. Очевидно, для 36 -ричной системы длина числа увеличится более чем в 5 и не более чем в 6 раз, так что длины числа 6000 должно хватить. Точнее коэффициент увеличения длины можно оценить числом $\log_2 36$.

Чтение числа в системе счисления с основанием более десяти отличается от традиционного незначительно. Следует просто проверить, какому интервалу принадлежит прочитанный символ — `'0'.. '9'` или `'A'.. 'Z'`. Если первому, то соответствующее значение разряда находится по формуле $\text{ord}(c) - \text{ord}('0')$. А если встретилась буква, то используется формула $\text{ord}(c) - \text{ord}('A') + 10$. Нужно не забыть в этой форму-

ле «+ 10», ведь букве 'A' соответствует совсем не 0, а 10. Обратите внимание на регистр буквы 'A'. Если бы во входных данных задачи могли присутствовать буквы в нижнем регистре, то пришлось бы отдельно рассматривать интервалы 'A' .. 'Z' и 'a' .. 'z'.

Вывод числа в системе счисления с основанием более десяти отличается от традиционного более значительно: теперь нужно не просто выводить цифры — нужно преобразовывать цифры в символы, а потом выводить. Преобразование цифры в символ — функция, обратная описанной при вводе. Если цифра d принадлежит интервалу от 0 до 9, то соответствующий символ $\text{chr}(d + \text{ord}('0'))$, а если интервалу от 10 до 35, то соответствующий символ $\text{chr}(d - 10 + \text{ord}('A'))$.

Вариант 1

Если бы значения чисел могли быть абсолютно точно представлены каким-либо встроенным типом Турбо Паскаля, решение записывалось бы так:

```
readln(ii, jj);
value := 0;
while not eoln do begin
  readDigit(d);
  value := value * ii + d;
end;
len := 0;
repeat
  inc(len);
  result[len] := value mod jj;
  value := value div jj;
until value = 0;
for i := len downto 1 do
  writeDigit(d);
```

Здесь value — значение числа, которое нужно перевести. Получаем его, дописывая справа очередную цифру, — это эквивалентно умножению на ii и добавлению значения этой цифры.

Потом получаем остатки от деления value на jj . Значение $\text{value mod } jj$ — последняя цифра результата, $\text{value div } jj \text{ mod } jj$ — вторая с конца, $\text{value div } jj \text{ div } jj \text{ mod } jj$ — третья с конца и т. д.

Так как мы получаем разряды с конца, а выводить их нужно в обратном порядке, то сначала записываем их в массив result , а затем выводим так, как нужно.

Как теперь переделать это решение, чтобы оно работало с длинными числами? Если value будет длинным числом, то потребуются реализовать следующие операции с ним: умножение длинного на короткое ($\text{value} * ii$), добавление короткого к длинному ($+ d$), определение частного и остатка при делении длинного на короткое ($\text{value div } jj$, $\text{value mod } jj$) и, наконец, сравнение длинного числа с нулем ($\text{value} = 0$).

Деление длинного числа на короткое рассматривалось в разборе одноименной задачи. Сравнение длинного числа с нулем трудности не представляет. Приведем лишь текст процедур умножения длинного числа на короткое и добавления короткого к длинному:

```
procedure mulshort(var n : number; x : integer);
var
    i, carry : integer;
begin
    carry := 0;
    for i := 1 to numlen do begin
        carry := carry + n[i] * x;
        n[i] := carry mod 10;
        carry := carry div 10;
    end;
    if carry <> 0 then
        halt(1);
end;
```

```
procedure addshort(var n : number; x : integer);
var
    i : integer;
begin
    i := 1;
    while x > 0 do begin
        x := x + n[i];
        n[i] := x mod 10;
        x := x div 10;
        inc(i);
    end;
end;
```

Вариант 2

Решение предложил Арсений Сорокин.

В варианте 1 сначала из числа в исходной системе счисления получается его эквивалент в десятичной системе, а затем из числа в десятичной системе счисления — число в требуемой системе счисления. Фактически перевод из одной системы счисления в другую происходит два раза: сначала в десятичную, а затем из десятичной.

Очевидно, число 10 не является каким-то особенным. С точки зрения этой задачи, оно ничем не отличается от других чисел от 2 до 35. Поэтому можно обойтись только одной половиной варианта 1, если реализовать операции над длинными числами в недесятичной системе счисления.

В варианте 2 выбрана следующая схема: работа с числом производится в исходной системе счисления (*ii*). Остаток от деления исходного числа на *jj* является млад-

шим разрядом числа в требуемой системе счисления. Если число разделить на jj и опять найти остаток, получим второй с конца разряд. Будем продолжать так до тех пор, пока исходное число не станет нулем.

Очевидно, проверка равенства длинного числа нулю одинакова в любой системе счисления. Поэтому здесь все упирается в нахождение частного и остатка при делении длинного числа, записанного в ii -ричной системе счисления, на короткое.

Деление длинного числа на короткое в десятичной системе счисления рассматривалось в одноименной задаче. Деление числа, записанного в ii -ричной системе счисления, отличается от деления числа, записанного в десятичной системе счисления, только тем, что в процедуре деления все десятки заменены основанием нужной системы счисления — ii .

Вариант 3

Решение предложил Григорий Сизов.

Можно выбрать и другую часть варианта 1 — не хранить нигде разряды числа в исходной системе счисления, а по мере их чтения накапливать `value`, но не в десятичной, а в jj -ричной системе счисления. Тогда после чтения исходных данных число уже можно выводить.

Ничего не стоит написать процедуры умножения длинного числа на короткое и добавления короткого к длинному в jj -ричной системе счисления. Нужно просто написать процедуры из варианта 1 решения этой задачи и заменить в них все десятки на jj .

Сравнение алгоритмов

Варианты 2 и 3 имеют определенные преимущества перед вариантом 1: в них надо реализовывать меньше действий над длинными числами, они требуют меньше памяти и работают быстрее. Однако у варианта 1 есть два существенных преимущества:

1. Не нужно тратить время на то, чтобы до него додуматься, — если вы можете перевести короткое число из одной системы счисления в другую, то добавить длинную арифметику к этому решению можно настолько быстро, насколько быстро можно записать хорошо заученный текст этих процедур длинной арифметики.
2. Из-за того что промежуточные данные записаны в привычной для человека десятичной системе счисления, упрощается отладка, а значит, и общее время написания программы.

Решение задачи 5 F. День рождения

Тема: календарь

Основная проблема в решении этой задачи — выбрать алгоритм, который можно быстро реализовать и в котором вероятность ошибок минимальна.

Автору известен только один такой алгоритм: нужно написать процедуру вычисления следующих дня, месяца и года исходя из текущих дня, месяца и года, затем находить следующие дни от заданной даты до тех пор, пока день и месяц не совпадут с днем рождения.

Все известные автору попытки решить задачу с помощью ручного перебора всех комбинаций взаимного расположения текущей даты и дня рождения приводили к неудаче — никто не смог рассмотреть все возможные случаи и правильно найти два десятка констант. Кроме того, решение с вычислением следующего дня меньше по длине и быстрее пишется, так что альтернативы ему, похоже, нет.

Теперь о процедуре нахождения следующего дня. Предлагается такой вариант:

1. Найти число дней в феврале текущего года. Даже если текущий месяц не февраль — все равно найти число дней в феврале текущего года.
2. Найти число дней в текущем месяце. Если текущий месяц — февраль, то число дней в феврале уже найдено, и нужно использовать этот результат. Если текущий день месяца — пятый и ни при каких обстоятельствах не может быть последним днем месяца — все равно нужно найти число дней в текущем месяце.
3. Рассмотреть три ситуации:
 - 1) если день 31-й и месяц 12-й, нужно менять день, месяц и год;
 - 2) если день равен числу дней в месяце, нужно менять день и месяц;
 - 3) в любом ином случае нужно менять лишь день.

Эта задача требует от решающего знания числа дней в месяцах. Он будет просто обязан узнать количество дней во всех месяцах, если не знал этого раньше. Рекомендуются эти числа еще и запомнить — для решения других задач, связанных с календарем.

Тренировка 6

Решение задачи 6 А. Закраска прямой

Вариант 1

Темы: быстрая сортировка, динамическая память ТР.

С помощью быстрой сортировки упорядочим отрезки по неубыванию координат левого конца. То есть при обмене в ходе сортировки L_i и L_j нужно менять местами также R_i и R_j .

Будем рассматривать отрезки в порядке от 2-го к N -му и сравнивать с первым. Результатом сравнения может оказаться корректировка первого отрезка.

Итак, рассматриваем взаимное расположение 1-го и i -го отрезков. Благодаря сортировке гарантируется $L_1 \leq L_i$. Всего возможны три варианта расположения отрезков:

1. $L_1 \leq L_i \leq R_1 \leq R_i$ — i -й отрезок полностью содержится в первом. В этом случае делать ничего не надо — если бы i -го отрезка не было, результат остался бы прежним.
2. $L_1 \leq L_i \leq R_1 < R_i$ — отрезки имеют хотя бы одну общую точку и образуют один, больший отрезок. В этом случае изменим правый край первого отрезка: $R_1 := R_i$. После этого i -й отрезок может быть отброшен.
3. $L_1 \leq R_1 < L_i \leq R_i$ — отрезки разделяет некоторое расстояние. Если 1-й отрезок не пересекается с i -м, то он не пересекается и с любым отрезком с номером, большим i . Значит, можно добавить длину первого отрезка к длине окрашенной части прямой и вычислять независимо от первого отрезка длину части прямой, окрашенной отрезками с i -го по N -й. Чтобы не менять алгоритм (продолжать очередные отрезки сравнивать с первым), нужно первый отрезок изменить полностью: $L_1 := L_i$, $R_1 := R_i$.

Ответом программы будет являться сумма накопленной в п. 3 длины и длины первого отрезка, получившегося в результате обработки всех остальных.

Детали реализации:

- Описанный алгоритм предусматривает одновременное хранение всех входных данных в памяти компьютера: 15 000 переменных типа `longint` (4 байта) для координат левого края, столько же для координат правого края — итого 120 000 байт. В Турбо Паскале объем основной памяти составляет 64 Кбайт. Будем использовать динамическую память. Объявление динамических переменных записывается так:

```

type
  tarray = array [1..15000] of longint;
var
  L, R : ^tarray;

```

Сначала объявляется тип массива, размер которого не превосходит 64 Кбайт, затем это имя массива со знаком ^ («птичкой») впереди используется для описания переменных (суммарный размер динамических переменных не должен превышать 500–600 Кбайт).

В начале программы нужно предусмотреть создание динамических переменных:

```

begin
  new(L);
  new(R);
  ...

```

В дальнейшем их можно использовать как обычные массивы, только обращаться к их элементам придется не как обычно, $L[i]$, а так: $L^{\wedge}[i]$. При желании скопировать массив полностью вместо записи для обычных переменных $A := B$ для динамических следует записать $A^{\wedge} := B^{\wedge}$.

Вариант 2

Решение предложили Александр Меркулов и Алексей Моисеев.

Тема: быстрая сортировка.

С помощью быстрой сортировки отсортируем отдельно левые и правые концы отрезков.

Оказывается, если соединить наименьший левый конец с наименьшим правым, второй отсортированный левый конец со вторым отсортированным правым и так до соединения наибольшего левого конца с наибольшим правым, то получившиеся отрезки будут покрывать ту же часть прямой, что и исходные отрезки.

Сначала в общую сумму покрытой длины занесем длину первого отрезка, затем будем рассматривать пары отрезков: первый — со вторым, второй — с третьим, $(N-1)$ -й — с N -м. Существует всего два способа расположения двух соседних отрезков: или $L_{i-1} \leq L_i \leq R_{i-1} \leq R_i$ — тогда отрезки соединены, и длину окрашенной части нужно увеличить на $R_i - R_{i-1}$, или $L_{i-1} \leq R_{i-1} < L_i \leq R_i$ — тогда между отрезками есть расстояние, и длину окрашенной части нужно увеличить на $R_i - L_i$.

Решение задачи 6 В. Суммы

Тема: динамическое программирование.

Очевидно, нельзя перебрать все комбинации коэффициентов k_i . При максимальном N получится 2^{500} вариантов.

Заметим, что суммы не могут быть слишком большими. Максимальное значение суммы — 50 000, минимальное — 0. На этом и основывается решение. Найдем все возможные значения сумм и сосчитаем их количество.

Заполним таблицу $B(0...100, 0...50\,000)$ логических элементов. Элемент $B(L, S)$ означает, что можно подобрать коэффициенты $k_1, k_2, \dots, k_L$, каждый или 0, или 1, такие, что $k_1A_1 + k_2A_2 + \dots + k_LA_L = S$.

При $L = 0$ в сумме нет слагаемых, поэтому $B(0, 0) = \text{true}$, $B(0, S) = \text{false}$ ($1 \leq S \leq 50\,000$).

Найдем $B(L, S)$ при $L > 0$. В сумму S слагаемое A_L может или входить, или не входить. Если оно входит, тогда $k_1A_1 + k_2A_2 + \dots + k_{L-1}A_{L-1} = S - A_L$, если не входит, то $k_1A_1 + k_2A_2 + \dots + k_{L-1}A_{L-1} = S$. Получается, что сумму S с помощью L слагаемых можно получить в одном из двух случаев: или когда можно получить $S - A_L$ с помощью $L - 1$ слагаемого, или когда можно получить S с помощью $L - 1$ слагаемого, то есть $B(L, S) = B(L - 1, S) \text{ or } B(L - 1, S - A_L)$.

На самом деле двумерный массив не нужен — для каждого очередного L значения вычисляются на основе значений массива для $L - 1$. Поэтому достаточно двух массивов ($0...50\,000$) — суммы для L и для $L - 1$.

Оказывается, можно обойтись даже одним массивом ($0...50\,000$), только его нужно заполнять в правильном порядке. Если идти от нулевого к $50\,000$ -му элементу, то нельзя установить, оказался ли элемент с индексом $S - A_L$ равен true на текущем проходе или он и раньше был таким. Однако если изменять S от $50\,000$ до 0, то элемент с индексом $S - A_L$, меньшим S , очевидно, на текущем проходе не затрагивался.

Чтобы не проверять, существует ли в массиве индекс $S - A_L$, нужно изменять S не от $50\,000$ до 0, а от $50\,000$ до A_L .

Логическую операцию `or` можно записать и по-другому: вместо

```
b[s] := b[s] or b[s - a];
```

написать

```
if b[s - a] then
  b[s] := true;
```

Здесь a означает A_L , для хранения A_i не нужен массив, ведь они используются по порядку — сначала A_1 , затем A_2 и т. д., поэтому читать очередное значение A_i можно непосредственно перед использованием.

Решение задачи 6 С. Игра «Даты»

Темы: антагонистические игры, календарь, динамическое программирование.

Введем понятия «выигрышная позиция» и «проигрышная позиция». Назовем позицию (в этой задаче — сочетание дня и месяца) *проигрышной*, если при любом ходе игрока его противник, играющий правильно, выиграет. Например, 30 декабря — проигрышная позиция. Из нее можно перейти только к 31 декабря, что по условию задачи означает проигрыш. В данном случае противнику даже не потребовалось играть оптимальным образом.

Приведем более сложный пример. 27 декабря — проигрышная позиция. Если игрок назовет следующей датой 28 декабря, его оптимально играющий противник

назовет 30 декабря, дату, которая, как мы уже выяснили, является проигрышной для ходящего из нее. И если игрок перейдет из 27 декабря в 29 декабря, то противник опять может назвать 30 декабря.

Отсюда видно, что 28 и 29 декабря — *выигрышные* позиции. Игрок, ходящий из них, при правильной игре (и любой игре противника) выигрывает.

Сформулируем правила определения выигрышности позиции по соответствующей характеристике позиций, в которые можно совершить ход.

Позиция является проигрышной, если *любой* допустимый ход игрока приводит к выигрышной (теперь уже для противника) позиции.

Позиция является выигрышной, если существует *хотя бы один* допустимый ход, приводящий к проигрышной (теперь уже для противника) позиции.

Каждая позиция является или выигрышной, или проигрышной — описанная игра всегда за конечное число шагов завершается победой одного из игроков.

Теперь перейдем непосредственно к решению задачи. Определим массив

`var`

```
win : array [1..31, 1..12] of boolean;
```

Будем заполнять массив в следующем порядке: 30 декабря, 29 декабря, ..., 1 декабря, 30 ноября, ..., 1 ноября, 31 октября, ..., 1 октября и так далее до 31 января, ..., 1 января. При определении выигрышности очередной даты выигрышность дат, куда можно перейти допустимым ходом, уже определена.

Рекомендуется написать функцию

```
function isValid(day, month : integer) : boolean;
```

которая позволит легко отсеять даты, в которые ход запрещен.

Начальное заполнение: дату 31 декабря будем считать выигрышной, ведь проигрывает тот, кто в нее приходит, а не тот, кто из нее «должен ходить». Проконтролируйте, чтобы в ходе заполнения массива дата 31 декабря не высчитывалась по общим правилам (они могут дать сбой, если некуда ходить), а заполнялась отдельно.

Решение задачи 6 D. Площадь прямоугольников

Тема: геометрия.

Проведем прямые через каждую из сторон каждого прямоугольника. Получится $H \leq 2N$ горизонтальных и $V \leq 2N$ вертикальных прямых (ровно $2N$ получается не всегда, потому что прямые, порожденные разными прямоугольниками, могут совпадать). Эти H горизонтальных и V вертикальных прямых ограничивают $(H - 1)(V - 1)$ прямоугольников на плоскости. Каждый из полученных прямоугольников может либо целиком содержаться в каком-то из исходных, либо иметь с ним пересечение нулевой площади. На этом и будет основан описываемый далее алгоритм. Максимальное количество элементарных действий (определений при-

надлежности точки прямоугольнику) в этом алгоритме — $(2N - 1)(2N - 1)N$, что для $N = 100$ составляет чуть меньше 4 миллионов.

Выделим в линейный массив из целых x все X -координаты исходных прямоугольников, а в массив y , соответственно, — все Y -координаты. В $xcount$ будет храниться количество записей в массиве x , в $ycount$ будет храниться количество записей в массиве y .

Отсортируем массивы:

```

procedure sortX;
var
  i, j, t : integer;
begin
  for i := 1 to xcount do
    for j := 1 to xcount do
      if (i < j) and (x[i] > x[j]) then begin
        t := x[i];
        x[i] := x[j];
        x[j] := t;
      end;
    end;
  end;
end;
```

Удалим повторяющиеся элементы:

```

procedure killDupX;
var
  i, j : integer;
begin
  j := 1;
  for i := 2 to xcount do
    if x[i] <> x[i - 1] then begin
      inc(j);
      x[j] := x[i];
    end;
  end;
  xcount := j;
end;
```

После этого будем проверять принадлежность левого нижнего угла каждого построенного прямоугольника $(x[i], y[j])$ ($i = 1 \dots xcount - 1, j = 1 \dots ycount - 1$) каждому из исходных прямоугольников:

```

if
  (xmin[k] <= x[i]) and (x[i] < xmax[k]) and
  (ymin[k] <= y[j]) and (y[j] < ymax[k])
then
```

Здесь $(xmin[k], ymin[k])$ — координаты левого нижнего угла k -го исходного прямоугольника, а $(ymax[k], xmax[k])$ — координаты его правого верхнего угла. Обратите внимание на использование знаков $<=$ и $<$ в сравнениях. Если $(x[i], y[j])$ — координата

ты левого нижнего угла прямоугольника сетки, то их совпадение с максимальной координатой исходного прямоугольника ($x[i] = x_{\max}[k]$ или $y[j] = y_{\max}[k]$) означает нулевую площадь пересечения.

Площадь пересечения или нулевая, или равна площади построенного прямоугольника: $(x[i + 1] - x[i]) * (y[j + 1] - y[j])$. Проследите за тем, чтобы эта величина добавлялась в сумму не более одного раза для одного квадрата сетки.

Заметьте, что в задаче не сказано, что во входных данных задаются координаты левого нижнего и правого верхнего углов прямоугольника. Там говорится про два противоположных угла. Поэтому в $x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$ нужно помещать уже обработанные значения.

Детали реализации:

- Для увеличения скорости работы программы массивы x и y лучше создать из элементов типа `integer`. Суммарная же площадь переменной типа `integer` представлена может быть не всегда.

Проблема заключается в том, что, по правилам вычисления выражений, типом результата (в том числе и промежуточного) является максимальный из типов операндов, то есть оператор

```
square := square + (x[i + 1] - x[i]) * (y[j + 1] - y[j]);
```

будет работать неправильно, если тип `square` — `longint`, а x и y — массивы из элементов типа `integer`.

Подвыражения $(x[i + 1] - x[i])$ и $(y[j + 1] - y[j])$ будут вычислены правильно, и их тип, совпадающий с типом операндов, будет `integer`. Согласно условию задачи, каждая из этих величин по модулю может быть не больше 20 000. Очевидно, их произведение не поместится в тип `integer`, однако именно такой тип будет сопоставлен произведению двух подвыражений типа `integer`.

Решением является приведение в явном виде одного из выражений к типу `longint`:

```
square := square + longint(x[i + 1] - x[i]) * (y[j + 1] - y[j]);
```

Типом $(x[i + 1] - x[i])$ теперь является `longint`. При умножении выражения типа `longint` на выражение типа `integer` получается выражение типа `longint`.

Другим решением является объявление переменных соответствующих типов для промежуточных значений:

```
var
```

```
  dx, dy: longint;
```

```
  ...
```

```
  dx := (x[i + 1] - x[i]);
```

```
  dy := (y[j + 1] - y[j]);
```

```
  square := square + dx * dy;
```


Решение задачи 6 E. Lines

Тема: поиск в ширину.

Данные будем хранить в массиве из целых:

var

```
a : array [0..41, 0..41] of integer;
```

Размер массива увеличен на 1 во все стороны для того, чтобы не рассматривать отдельно пограничные клетки $a[i, j]$ ($i = 1$, или $j = 1$, или $i = N$, или $j = N$). Дополнительные клетки заполним препятствиями — шариками. Проще всего заполнить пограничные клетки после чтения размера таблицы перед чтением самой таблицы:

```
for i := 0 to n + 1 do
  for j := 0 to n + 1 do
    a[i, j] := wall;
```

Здесь `wall` — константа целого типа, соответствующая неподвижному шарiku. Определим также константу `free`, соответствующую пустому месту. Так как заполненные неотрицательными числами клетки таблицы будут иметь специальное значение, значения `wall` и `free` нужно выбрать отрицательными, например:

const

```
free = -1;
wall = -2;
```

При чтении заполняем таблицу следующим образом:

- в клетку, соответствующую свободному месту (символ точки), записываем значение `free`;
- в клетку, соответствующую шарiku-препятствию (символ 0), записываем `wall`;
- в клетку, куда нужно привести шарик (символ X), записываем `free` (ведь эта клетка свободна) и запоминаем в отдельных переменных x_i и x_j ее координаты;
- в клетку, соответствующую исходному положению шарика, записываем ноль.

Рассмотрим пример:

```
..0..
X.O.@
..0..
.....
.....
```

Ему будет соответствовать такая таблица из целых:

```
-1 -1 -2 -1 -1
-1 -1 -2 -1 0
-1 -1 -2 -1 -1
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1
```

кроме того, после чтения входных данных $x_i = 2$, $x_j = 1$ — символ X находился в клетке (2, 1).

Пройдем по массиву и заменим значения всех клеток, содержащих free и граничащих с нулевой клеткой, единицами:

-1	-1	-2	-1	1
-1	-1	-2	1	0
-1	-1	-2	-1	1
-1	-1	-1	-1	-1
-1	-1	-1	-1	-1

Таким образом, единицами оказались отмечены клетки, достижимые за одно перемещение шарика.

Следующим проходом заменим значения всех клеток, содержащих free и граничащих с единичными клетками, двойками:

-1	-1	-2	2	1
-1	-1	-2	1	0
-1	-1	-2	2	1
-1	-1	-1	-1	2
-1	-1	-1	-1	-1

Двойками оказались помечены клетки, достижимые ровно за 2 шага (за меньшее число шагов их достичь нельзя).

Продолжаем заполнять таблицу:

-1	-1	-2	2	1
-1	-1	-2	1	0
-1	-1	-2	2	1
-1	-1	-1	3	2
-1	-1	-1	-1	3

-1	-1	-2	2	1
-1	-1	-2	1	0
-1	-1	-2	2	1
-1	-1	4	3	2
-1	-1	-1	4	3

...

-1	8	-2	2	1
8	7	-2	1	0
7	6	-2	2	1
6	5	4	3	2
7	6	5	4	3

```

9 8 -2 2 1
8 7 -2 1 0
7 6 -2 2 1
6 5 4 3 2
7 6 5 4 3

```

Заполнение таблицы завершается, когда на очередном проходе не удастся найти свободных клеток, соседствующих с измененными на предыдущем проходе.

Сразу же после завершения заполнения таблицы можно решить вопрос, существует ли искомый путь. Если клетка $a[x_i, x_j]$ осталась равной free, значит, пути не существует, можно выводить N и завершать работу. В противном случае ($a[x_i, x_j] > 0$) путь существует, далее описывается его построение.

Будем обозначать клетки, принадлежащие пути, какой-нибудь константой, например

const

```
way = -3;
```

Выберем соседнюю с $a[x_i, x_j]$ клетку со значением на единицу меньшим, чем значение $a[x_i, x_j]$. Поместим в $a[x_i, x_j]$ значение way и заменим x_i и x_j координатами выбранной соседней клетки.

В построенном примере первый шаг можно сделать вправо или вниз. Выберем шаг вниз (жирным шрифтом отмечена клетка $a[x_i, x_j]$):

```

9 8 -2 2 1
-3 7 -2 1 0
7 6 -2 2 1
6 5 4 3 2
7 6 5 4 3

```

Следующий шаг тоже можно сделать или вправо, или вниз. Выберем шаг вправо:

```

9 8 -2 2 1
-3 7 -2 1 0
-3 6 -2 2 1
6 5 4 3 2
7 6 5 4 3

```

Далее выбора нет, нужно идти вниз:

```

9 8 -2 2 1
-3 7 -2 1 0
-3 -3 -2 2 1
6 5 4 3 2
7 6 5 4 3

```

Таким же образом делаются последующие шаги до тех пор, пока не окажется $a[x_i, x_j] = 0$:

```

9 8 -2 2 1
-3 7 -2 1 0
-3 -3 -2 2 -3
6 -3 -3 -3 -3
7 6 5 4 3

```

После этого таблицу можно выводить:

- положительные числа и free соответствуют символу точки;
- wall соответствует #;
- way соответствует +;
- ноль соответствует @.

Перед таблицей нужно не забыть вывести Y.

Решение задачи 6 F. Покраска лабиринта

Тема: заливка.

Стандартная процедура заливки 4-связной области¹ выглядит так:

```
procedure paint(i, j : integer);
begin
    if a[i, j] <> '.' then
        exit;
    a[i, j] := '$';
    paint(i + 1, j);
    paint(i - 1, j);
    paint(i, j + 1);
    paint(i, j - 1);
end;
```

Здесь a — двумерный массив из символов. Область, которую нужно «залить», обозначена символами '.', в ходе заливки символы '.' области заменяются символами '\$'.

Параметрами процедуры являются координаты любой клетки области. Процедура заменяет значение $a[i, j]$ с '.' на '\$', затем вызывает себя рекурсивно с параметрами — координатами четырех соседних клеток. Если соседняя клетка занята стеной (символ '#') или уже посещалась (символ '\$'), вызванная процедура немедленно завершается (оператор exit — выход из процедуры). Если же соседняя клетка свободна (символ '.'), то заливка продолжается.

Предполагается, что для любого символа области существуют четыре соседних. Применительно к нашей задаче это требование означает, что нужно объявлять массив не как `array [1..33, 1..33] of char`, а как `array [0..34, 0..34] of char`, причём клетки на границе $a[i, j]$ ($i = 0$, или $j = 0$, или $i = N + 1$, или $j = N + 1$) нужно заполнить символами '#', непроходимыми для процедуры заливки. Проще всего заполнить границу после чтения размера карты лабиринта N перед чтением самой карты лабиринта:

¹ В математике N -связным называется граф, который остается связным при удалении любых $(N - 1)$ ребер. В компьютерной графике 4-связной называется клетчатая область, которую можно обойти ходом ладьи, а 8-связной — которую можно обойти ходом ферзя. В этой книге принята терминология компьютерной графики.

```
for i := 0 to n + 1 do  
  for j := 0 to n + 1 do  
    a[i, j] := '#';
```

Перейдем непосредственно к решению задачи. Нужно произвести заливку 4-связной области, содержащей клетку $(1, 1)$, и области, содержащей клетку (N, N) . Это может быть одна область, а могут быть две, потому что в условии задачи не сказано, что гарантируется существование пути от одного выхода к другому.

После этого в массиве a символами '\$' оказываются помечены все свободные клетки, в которые посетители лабиринта могут попасть. Теперь нужно только посчитать, сколько существует пар соседних по стороне клеток, одна из которых — достижимая свободная (помечена символом '\$'), а вторая занята стеной (стены обозначены символами '#').

После того как подсчитано количество таких пар, можно выводить ответ. Следует только учесть, что площадь сегмента стены — 9 м^2 , кроме того мы посчитали лишние четыре сегмента — сверху и слева от верхнего левого угла, а также справа и снизу от правого нижнего угла.

Тренировка 7

Решение задачи 7 А. Упорядоченные дроби

Вариант 1

Темы: алгоритм Евклида, быстрая сортировка.

Двойным циклом (по знаменателю и числителю) переберем все дроби, у которых знаменатель не превосходит N , а числитель строго меньше знаменателя. Если числитель и знаменатель окажутся взаимно простыми, занесем дробь в массив. В результате в массиве будут содержаться те и только те дроби, которые нужно вывести. Отсортируем массив по отношению числителя и знаменателя быстрой сортировкой, и можно выводить результат.

Детали реализации:

- Определить, что числитель и знаменатель взаимно просты, можно с помощью алгоритма Евклида — алгоритма нахождения наибольшего общего делителя — НОД (greatest common divisor, GCD):

```
function gcd(x, y : integer) : integer;  
begin  
    if y = 0 then  
        gcd := x  
    else  
        gcd := gcd(y, x mod y);  
    end;
```

Если НОД равен 1, то числа взаимно просты.

- Числитель и знаменатель не превосходят 255, поэтому их можно представить типом `byte`. Чтобы уменьшить вероятность ошибки, рекомендуется определить тип данных «дробь»:

```
type  
    fraction = record  
        numerator;  
        denominator : byte;  
    end;
```

- В задаче проводится не слишком много вычислений, проверки диапазонов лучше включить: $\{\mathbb{R}^+, \mathbb{Q}^+\}$.

Идея решения найдена в книге [Грэхем], стр.141.

Построим последовательность Фарея — последовательность всех несократимых дробей. Искомая последовательность будет частью последовательности Фарея. Начнем с двух фиктивных значений, $\frac{0}{1}$ и $\frac{1}{1}$, и на каждом следующем шаге будем между двумя соседними дробями писать дробь, числитель которой равен сумме числителей соседних дробей, а знаменатель — сумме их знаменателей:

[illegible]

```
procedure show(num left, den left, num right, den right : integer);
```

Здесь `num_left` и `den_left` — числитель и знаменатель дроби — левой границы того участка, который должна вывести эта процедура, `num_right` и `den_right` — числитель и знаменатель дроби — правой границы.

Процедура работает так. Если дробь посередине заданного участка $(\text{num_left} + \text{num_right}) / (\text{den_left} + \text{den_right})$ имеет знаменатель больше, чем N , то и все дроби участка будут иметь знаменатели больше N , а значит, на всем участке ничего выводить не надо — можно выходить из процедуры.

Если же знаменатель средней дроби не превосходит N , то выведем все интересующие нас дроби из левой части участка (рекурсивным вызовом процедуры с тем же левым краем и средней дробью в качестве правого края), затем выведем среднюю дробь, затем все интересующие нас дроби из правой части участка (рекурсивным вызовом со средней дробью в качестве левого края и с тем же правым краем).

Решение задачи 7 В. Сообщение

Темы: динамическое программирование, длинная арифметика.

Обозначим $C(i)$ количество исходных сообщений, из которых можно было получить первые i цифр заданной последовательности.

Тривиальный случай: $C(0) = 1$ — пустое сообщение порождает пустую последовательность цифр, $C(1) = 1$ — любая одиночная цифра соответствует какой-то одной букве.

Найдем, чему равно $C(i)$ при $i \geq 2$. Если две последние цифры ($(i - 1)$ -я и i -я) образуют число от 10 до 33, то есть могут соответствовать букве, кодируемой двумя цифрами, то $C(i) = C(i - 2) + C(i - 1)$ — можно к сообщению, порождающему последовательность $i - 2$ цифр, дописать букву с кодом, соответствующим комбинации $(i - 1)$ -й и i -й цифр, или к сообщению, порождающему последовательность из $i - 1$ цифр, дописать букву с кодом, соответствующим i -й цифре.

Если же две последние цифры не образуют число от 10 до 33, то $C(i) = C(i - 1)$ — последняя буква исходного сообщения обязана иметь код, соответствующий i -й цифре.

Детали реализации:

- Максимальный ответ задачи не может быть точно представлен ни одним из встроенных типов Турбо Паскаля. Поэтому нужно применять длинную арифметику, а именно: процедуры сложения двух длинных чисел и вывода длинного числа (см. разбор задачи 2 Е). Настоятельно рекомендуется вынести работу с длинной арифметикой в отдельные процедуры.
- В задаче проводится не слишком много вычислений, проверки диапазонов лучше включить: $\{\$R+, Q+\}$.

Решение задачи 7 С. Игра умножения

Тема: антагонистические игры.

Вариант 1

Темы: теория чисел, динамическое программирование, тип `comp`.

Предполагается, что читатель знаком с терминами антагонистических игр «выигрышная позиция» и «проигрышная позиция» и с тем, как они связаны друг с другом (см. решение задачи 6 С).

Позицией в этой игре будем считать число, которое может быть получено из 1 умножением (0, 1 или несколько раз) на целые числа от 2 до 9 включительно.

Прочитав ограничения задачи ($N \leq 4\,294\,967\,295$), можно подумать, что количество позиций игры очень велико, и нельзя за разумное время определить, выигрышная ли каждая из позиций или нет. Однако оказывается, что на интервале до 4 294 967 295 чисел, разложение которых на простые множители содержит только четыре простых числа: 2, 3, 5 и 7 — довольно мало — менее 7000.

Найдем все числа до 4 294 967 295, которые можно получить, умножая 1 на целые числа от 2 до 9. Числа можно хранить в отсортированном виде. Обработка каждого числа (начиная с меньших) заключается в следующем: оно умножается на каждое из чисел от 2 до 9. Если результат не превосходит N и отсутствует в массиве, то он заносится в массив так, чтобы отсортированность не нарушалась. Занесенный элемент больше текущего, поэтому часть массива слева от текущего не изменилась, и теперь очередным можно рассматривать элемент с индексом, большим на 1.

Получить все такие числа можно и другим способом — например, используя хеш-таблицу с открытым перемешиванием и последующую сортировку. Но этот способ читателям предлагается реализовать самостоятельно.

В результате массив будет содержать все игровые позиции. Пройдем по этому массиву от больших индексов к меньшим и заполним массив выигрышности позиций. Позиция выигрышная, если соответствующее ей число, умноженное на 9, удовлетворяет условию выигрыша $P \geq N$.

Позиция является проигрышной, если из нее можно попасть только в выигрышные (теперь уже для противника) позиции. Позиция является выигрышной, если из нее можно попасть хотя бы в одну проигрышную (теперь уже для противника) позицию. Если числу 1 будет соответствовать выигрышная позиция, результатом игры будет выигрыш игрока, начинающего игру, в противном случае выигрывает его противник.

Детали реализации:

- Во входных данных задачи и при вычислениях встречаются значения, которые нельзя представить переменными типа `longint`. Наиболее подходящим для работы с такими значениями является тип `comp` — переменные этого типа могут принимать целые значения от -2^{63} до $2^{63} - 1$, то есть целые значения, по модулю не превосходящие $9 \cdot 10^{18}$.
- Для использования типа `comp`, как и любого типа, поддерживаемого компилятором (`single`, `double`, `extended`), нужно в начале программы написать `{ $N+.E- }`. В противном случае проверяющая система выдаст ошибку компиляции.
- Чтобы решение уложилось в отведенное время, нужно использовать не линейный, а двоичный поиск в массиве:

```
var
    numbers : array [1..7000] of comp;
    tablen : integer;
```

```
function indexOf(n : comp) : integer;
var
  l, r, mid : integer;
begin
  l := 1;
  r := tablen;
  while l <= r do begin
    mid := (l + r) div 2;
    if numbers[mid] = n then begin
      indexOf := mid;
      exit;
    end;
    if numbers[mid] < n then
      l := mid + 1
    else
      r := mid - 1;
    end;
  end;
  indexOf := -1;
end;
```

Если значение в элементах массива с первого по `tablen`-й отсутствует, то результатом функции будет `-1`, в противном случае результатом будет индекс числа. На этапе формирования таблицы нужно только выяснять, есть ли уже число в таблице, и если есть, то ничего не делать, а если нет, то добавлять. На этапе определения выигрышности позиций индекс потребуется, чтобы определить выигрышность позиции, в которую можно перейти из рассматриваемой.

- Добавление элемента в массив с сохранением отсортированности:

```
procedure put(n : comp);
var
  i : integer;
begin
  if n > 4294967295.0 then
    exit;
  if indexOf(n) <> -1 then
    exit;
  i := tablen;
  while (i >= 1) and (numbers[i] > n) do begin
    numbers[i + 1] := numbers[i];
    dec(i);
  end;
  numbers[i + 1] := n;
  inc(tablen);
end;
```

Обратите внимание на первое сравнение. Константа 4294967295 вызывает ошибку, а 4294967295.0 — нет.

Вариант 2

Решение предложил Никита Рыбак.

В варианте 1 уже было замечено, что позиции от $\lceil N/9 \rceil$ до $N-1$ являются выигрышными (знак $\lceil \rceil$ означает округление вверх, то есть $\lceil 5,2 \rceil = 6$, $\lceil 5 \rceil = 5$). Продолжим аналитически определять выигрышность позиций.

Из позиции от $\lceil \lceil N/9 \rceil / 2 \rceil$ до $\lceil N/9 \rceil - 1$ игрок будет вынужден перейти в выигрышную позицию от $\lceil N/9 \rceil$ до $N-1$ (так как $\lceil \lceil N/9 \rceil / 2 \rceil \cdot 2 \geq \lceil N/9 \rceil$ и $(\lceil N/9 \rceil - 1) \cdot 9 \leq N - 1$). Следовательно, такие позиции являются проигрышными.

Из позиции от $\lceil \lceil \lceil N/9 \rceil / 2 \rceil / 9 \rceil$ до $\lceil \lceil N/9 \rceil / 2 \rceil - 1$ игрок может сходить в проигрышную позицию от $\lceil \lceil N/9 \rceil / 2 \rceil$ до $\lceil N/9 \rceil - 1$, следовательно, такие позиции являются выигрышными.

Можно продолжать получать выигрышные и проигрышные интервалы и далее, попеременно деля нижнюю границу на 9 и на 2 до тех пор, пока нижней границей очередного интервала не окажется 1. Тогда можно выводить ответ.

Детали реализации:

- В Турбо Паскале есть функция округления к нулю — `int`, но нет функции округления вверх. Ее ничего не стоит написать:

```
function roundUp(x : real) : real;
var
  t : real;
begin
  t := int(x);
  if t < x then
    t := t + 1;
  roundUp := t;
end;
```

Замечательно, что функция будет выдавать правильный ответ, если `int` заменить любой другой функцией округления (в Турбо Паскале есть еще `trunc` и `round`). Однако использовать функцию округления, возвращающую целое значение (`trunc` или `round`), не рекомендуется — она будет вызывать ошибку времени выполнения, если аргумент больше `maxlongint`.

- Подходящим типом для вещественных вычислений является `extended`. Чтобы по всей программе ссылаться на него как `real`, нужно в начале программы написать:

```
{ $N+, E- }
type
  real = extended;
```

Решение задачи 7 D. Прямая и квадраты

Тема: геометрия.

Найдем уравнение прямой вида $Ax + By + C = 0$ по двум заданным точкам, через которые она проходит (см. вариант 1 разбора задачи 1 D).

Двойным циклом ($i, j = 1, 2, \dots, N$) переберем все квадраты и для каждого определим, имеет ли он общие точки с этой прямой. Квадрат не имеет общих точек с прямой, если для всех четырех его вершин выполняется одновременно или $Ax + By + C < 0$, или $Ax + By + C > 0$, то есть квадрат лежит строго в одной полуплоскости относительно этой прямой.

Детали реализации:

- Чтобы не запутаться в вычислениях, рекомендуется объявить тип данных, соответствующий понятию «прямая», и процедуры работы с ним: получение коэффициентов прямой по двум точкам и определение знака уравнения при подстановке произвольной точки.
- В задаче производится не слишком много вычислений, проверки диапазонов лучше включить: $\{\$R+, Q+\}$.

Решение задачи 7 E. Lines (2)

Темы: поиск в ширину с очередью, динамическая память ТР.

Эта задача ничем не отличается от 6 E, за исключением ограничения $N \leq 250$ вместо $N \leq 40$. Рассмотрим, почему в этой задаче нельзя применять алгоритм, использовавшийся в варианте 1 разбора задачи 6 E.

Одним из худших случаев для описанного алгоритма является «змейка»:

```
X.....
00000000.
.....
.00000000
.....
00000000.
.....
.00000000
.....@
```

Свободны чуть больше половины клеток, путь пролегает через все свободные клетки, то есть его длина примерно $N^2/2$. Если применить вариант 1 решения задачи 6 E, будет сделано примерно $N^2/2$ проходов по матрице и при каждом проходе просмотрена каждая клетка (N^2 клеток). Всего получается примерно $N^4/2$ элементарных действий. Для $N = 40$ это чуть больше миллиона, то есть вполне приемлемо. Для $N = 250$ количество элементарных действий — около 2 млрд, что абсолютно невыполнимо выполнить на указанном в предисловии компьютере за 2 с.

Почему вариант 1 решения задачи 6 Е неэффективен на этом примере? Там затрачивается слишком много действий на поиск клеток, которые изменились на предыдущем шаге. Если бы координаты этих клеток хранились в отдельной таблице, то общее количество действий было бы пропорционально количеству достижимых из исходного положения свободных клеток ($\leq N^2$), потому что каждая клетка (вместе с четырьмя соседними) рассматривалась бы только один раз.

Для хранения координат клеток, подлежащих рассмотрению, будем использовать *очередь* (queue, [kju:]). Разница между очередью и стеком в том, что из стека забирается последний занесенный элемент, а из очереди — первый занесенный и еще не востребованный элемент.

Рассмотрим пример:

```
..@
...
X..
```

Заменяем X точкой, ведь место, куда нужно привести шарик, пустое. Будем обозначать элементы, находившиеся или находящиеся в очереди, числами — количеством шагов, за которые их можно достичь из начальной клетки. Изначально в очереди содержится только исходное положение шарика. Саму очередь будем обозначать парами координат клеток, находящихся в ней: первая координата — строка, вторая — столбец. Добавлять координаты будем в конец очереди (справа), а брать из начала (слева). Рассматриваемый элемент будем выделять жирным шрифтом. Итак, исходное положение:

```
..0 Очередь: (1, 3)
```

```
...
...
```

Извлекаем из очереди первый (и пока единственный) элемент — (1, 3) и рассматриваем соседей этой клетки. Две соседние клетки пусты — заносим их координаты в конец очереди:

```
..10 Очередь: (1, 2) (2, 3)
```

```
..1
...
...
```

Извлекаем из очереди первый элемент — (1, 2) и рассматриваем соседей этой клетки. Координаты пустых соседних клеток заносим в очередь:

```
210 Очередь: (2, 3) (1, 1) (2, 2)
```

```
..21
...
...
```

Извлекаем элемент (2, 3) и заносим в очередь координаты единственной пустой соседней клетки:

```
210 Очередь: (1, 1) (2, 2) (3, 3)
```

```
..21
..2
```

Извлекаем элемент (1, 1) и заносим в очередь координаты единственной пустой соседней клетки:

210 Очередь: (2, 2) (3, 3) (2, 1)

321

..2

Извлекаем элемент (2, 2) и заносим в очередь координаты единственной пустой соседней клетки:

210 Очередь: (3, 3) (2, 1) (3, 2)

321

.32

Извлекаем элемент (3, 3) и ничего не заносим в очередь — пустых соседних клеток нет:

210 Очередь: (2, 1) (3, 2)

321

.32

Извлекаем элемент (2, 1) и заносим в очередь координаты единственной пустой соседней клетки:

210 Очередь: (3, 2) (3, 1)

321

432

Извлекаем элемент (3, 2) и ничего не заносим в очередь — пустых соседних клеток нет:

210 Очередь: (3, 1)

321

432

Извлекаем элемент (3, 1) и ничего не заносим в очередь — пустых соседних клеток нет. После этого процесс завершен, очередь пуста:

210 Очередь: пуста

321

432

Возникает вопрос, почему в результате использования очереди таблица заполняется корректно — в каждой клетке находится число шагов, за которые до этой клетки можно дойти из начальной.

В начальный момент в очереди находятся все клетки, достижимые за 0 шагов (одна клетка). Затем эта клетка убирается, и в очередь попадают все клетки, достижимые за 1 шаг. Затем последовательно рассматриваются все клетки, достижимые за 1 шаг, а клетки, достижимые за 2 шага, добавляются в конец очереди. Нетрудно заметить, что очередь всегда содержит клетки с количеством шагов, различающимся не более чем на 1, причем все клетки с меньшим числом шагов находятся в начале очереди, а с большим — в конце. Получается, что клетки рассматриваются в нужном порядке: сначала все достижимые за M шагов, и только затем достижимые за $M + 1$ шаг.

Детали реализации:

- Максимальное количество клеток пути при ограничениях задачи — чуть более $\frac{250 \cdot 250}{2} = 31\,250$. Это число может поместиться в переменную типа `integer`, но не типа `byte`. Однако в Турбо Паскале объявить массив 250×250 из элементов типа `integer` нельзя — не хватит памяти (требуется $250 \cdot 250 \cdot 2 = 125\,000$ байт, а есть только 64 Кбайт).

Решить проблему можно следующим образом: в клетке хранить не число шагов, за которые она достижима, а обозначение направления, с которого в нее пришли. Матрица в приведенном примере будет выглядеть так:

```
rr@  
uuu  
uuu
```

Здесь 'r' означает, что в клетку пришли справа (*right*), а 'u' — что сверху (*up*). В отправной точке остался символ '@' — в нее ниоткуда не пришли.

Итак, требуется только массив из символов — практически исходные данные, разве что 'X' заменен точкой. При заполнении таблицы некоторые точки будут заменены символами 'r', 'l', 'u' или 'd'. Если в клетке, где был символ 'X', осталась точка, можно выводить отрицательный ответ, в противном случае нужно идти из клетки, в которой был 'X', по направлениям, указанным в клетках, заменяя по пути значения клеток символами '+'. Перед выводом таблицы нужно заменить все 'r', 'l', 'u' и 'd' точками.

Не следует забывать, что использование барьера помогает упростить программу.

- В этой задаче очередь наиболее эффективно реализуется в виде совокупности линейного массива и двух переменных целого типа. Одна переменная — `first` — обозначает индекс первого неиспользованного элемента очереди, а вторая — `last` — индекс последнего элемента очереди.

Взять данные из очереди не представляет труда — получить элемент массива с индексом `first` и увеличить `first`, чтобы не забрать тот же элемент еще раз.

Данные в очередь добавляются так: увеличивается `last`, затем в элемент массива с индексом `last` заносятся данные.

Определить, что очередь не пуста, можно из условия `first <= last`.

Очевидно, что при такой организации очереди длина массива должна быть равна максимальному количеству элементов, которые могут быть занесены в очередь.

Для этой задачи длину массива нужно задать как $250 \cdot 250 = 62\,500$. Почти вся основная память уже занята картой поля. Поместим массивы, соответствующие очереди, в динамическую память (см. также детали реализации варианта 1 решения задачи 6 А). Массивов будет два — один для `x`, другой для `y`. Координаты могут быть представлены числом типа `byte`. Итак, работа с очередью будет выглядеть так:

```
type
  queue = array [1..62500] of byte;
var
  qx, qy : ^queue;
  first, last : word;

  procedure initQueue;
  begin
    new(qx);
    new(qy);
    first := 1;
    last := 0;
  end;

  procedure put(x, y : byte);
  begin
    inc(last);
    qx^[last] := x;
    qy^[last] := y;
  end;

  procedure get(var x, y : byte);
  begin
    x := qx^[first];
    y := qy^[first];
    inc(first);
  end;

  function notEmpty : boolean;
  begin
    notEmpty := first <= last;
  end;
```

Процедура `initQueue` должна быть вызвана один раз до любых манипуляций с очередью.

Решение задачи 7 F. Удаление клеток

Темы: заливка, ограничения ТР.

Казалось бы, эта задача принципиально ничем не отличается от задачи 6 F. Нужно в любом порядке пройти по всем клеткам и, если клетка содержит символ '#', залить всю четырехсвязную область, содержащую эту клетку, символами '.'. Ответом будет количество вызовов процедуры заливки извне ее.

Основная сложность задачи — написать процедуру заливки, которая будет работать при заданных ограничениях ($M, N \leq 100$). Худший случай для заливки — когда размер поля максимален ($M = N = 100$) и ничего (или почти ничего) не вырезано. У приведенной в решении задачи 6 F процедуры глубина рекурсии при таких условиях будет равна числу клеток, то есть 10 000.

Рассчитаем, какого размера стек требуется этой процедуре. При вызове процедуры в стек заносится адрес возврата (2 байта по умолчанию, 4 байта с директивой `{ $F+ }` или `far`), значение регистра BP (указатель на параметры и локальные переменные вызывающей процедуры, 2 байта) и по 2 байта на каждый параметр процедуры типа `integer`. Итак, всего при одном вызове процедуры в стек заносится 8 байтов. Значит, при глубине рекурсии 10 000 этой процедуре требуется размер стека 80 000 байтов.

В Турбо Паскале размер стека по умолчанию — 16 384 байта, его можно увеличить до 65 520 байтов. Получается, что при заданных ограничениях ($M, N \leq 100$) процедуру, описанную в решении задачи 6 F, применять нельзя.

Нужно переписать процедуру так, чтобы при ее вызове в стек заносилось не более 6 байтов. Очевидное решение — сделать тип параметров `byte`, тогда они будут занимать по одному байту, и всего на вызов будет тратиться 6 байтов. Решение *неверное*, потому что в Турбо Паскале один параметр-значение занимает не менее 2 байта стека. Параметр-переменная всегда занимает 4 байта.

Второе решение — сгруппировать два параметра типа `byte` в один типа `record`. Параметр-значение типа `record` размером 2 байта занимает в стеке ровно 2 байта, то есть если установить размер стека максимальным, следующая процедура будет работать:

```
type
  location = record
    i, j : byte;
  end;

...

procedure paint(place : location);
begin
  if a[place.i, place.j] <> '#' then
    exit;
  a[place.i, place.j] := '.';

  place.i := place.i + 1;
  paint(place);
  place.i := place.i - 1;

  place.i := place.i - 1;
  paint(place);
  place.i := place.i + 1;
```

```

place.j := place.j + 1;
paint(place);
place.j := place.j - 1;

place.j := place.j - 1;
paint(place);
place.j := place.j + 1;
end;
```

Нетрудно заметить, что процедура `paint` *всегда* вызывается с одним и тем же параметром — `place`. Если его сделать глобальной переменной, процедура будет работать. Более того, при одном вызове процедуры будет расходоваться только 4, а не 6 байт стека.

Заметим также, что как только переменная `place` стала глобальной, отпала необходимость в экономии памяти на нее. Теперь нет необходимости комбинировать две переменные целого типа в одну типа запись. Итак, окончательный вариант процедуры:

```

{$M 65520.0.655360}
var
  painti, paintj : integer;

  ...

procedure paint;
begin
  if a[painti, paintj] <> '#' then
    exit;
  a[painti, paintj] := '.';

  inc(painti);
  paint;
  dec(painti);

  dec(painti);
  paint;
  inc(painti);

  inc(paintj);
  paint;
  dec(paintj);

  dec(paintj);
  paint;
  inc(paintj);
end;
```

Переменные, используемые в процедуре, названы не просто i и j для того, чтобы они не были ошибочно использованы не по назначению другими процедурами. В этой задаче других процедур нет, но в других задачах заливка может быть только частью алгоритма.

Написанная в начале программы директива $\$M$ имеет три параметра. Первый из них — размер стека. Остальные два — минимальный и максимальный размеры кучи — лучше оставлять заданными по умолчанию: 0 и 655360.

Не следует забывать, что приведенные процедуры заливки требуют использования барьера.

Тренировка 8

Решение задачи 8 А. Несоставляемое число

Темы: быстрая сортировка, тип `comp`.

Пусть K натуральных чисел $A_1, A_2, \dots, A_K$ таковы, что любое число от 1 до их суммы S можно представить суммой некоторых из этих чисел. Покажем, что если новое число A_{K+1} удовлетворяет условию $A_{K+1} \leq S + 1$, то в этом случае уже для $K + 1$ числа справедливо то же самое: любое число от 1 до их суммы $S + A_{K+1}$ может быть представлено суммой некоторых из этих чисел.

Разобьем промежуток от 1 до $S + A_{K+1}$ на три: от 1 до $A_{K+1} - 1$, одно число A_{K+1} и от $A_{K+1} + 1$ до $S + A_{K+1}$.

Любое число на интервале от 1 до $A_{K+1} - 1$ можно получить с помощью первых K чисел, так как $A_{K+1} - 1 \leq S$, а все числа от 1 до S можно представить суммой первых K чисел.

Число A_{K+1} , очевидно, может быть представлено суммой, в которую входит только оно само.

Если число находится на интервале от $A_{K+1} + 1$ до $S + A_{K+1}$, то одним из членов суммы возьмем A_{K+1} . Останется получить еще какое-то число от 1 до S с помощью первых K чисел. По определению это возможно.

Перейдем к решению исходной задачи. Изначально в наборе чисел нет ($K = 0$), и ими можно представить любое число от 1 до их суммы $S = 0$ (то есть никакое число ими представить нельзя). Если среди нерассмотренных чисел есть число от 1 до $S + 1$, то можно включить это число в набор и соответствующим образом изменить сумму набора. Если же все нерассмотренные числа больше $S + 1$, то число $S + 1$ нельзя представить в виде суммы никаких из заданных чисел. Действительно, в наборе есть числа, меньшие $S + 1$ и большие $S + 1$. Понятно, что большие в сумму, образующую $S + 1$, входить не могут. А сумма всех меньших чисел равна S . Таким образом, $S + 1$ получить нельзя.

Детали реализации:

- Очевидно, если выбирать самое маленькое из нерассмотренных чисел, алгоритм будет работать. Значит, числа должны обрабатываться в порядке их роста. Применим быструю сортировку. После сортировки останется только пройти по массиву от меньших индексов к большим и для каждого числа определить, что оно не превосходит $S + 1$ (тогда его нужно добавлять в S), или оно больше $S + 1$ (тогда можно выдавать ответ $S + 1$).

- Оценим максимальный ответ. Если взять первые 30 чисел степенями двойки ($2^0 = 1, 2^1 = 2, \dots, 2^{29} = 536\,870\,912$), то с помощью суммы некоторых из них можно представить любое число от 1 до 1 000 000 000. Если взять оставшиеся 9970 чисел равными 1 000 000 000, то ответом будет сумма всех чисел плюс один. Это число чуть меньше 10^{13} . Для точного представления таких целых чисел подходит тип `comp`. При его использовании нужно в начале программы написать `{ $N+, E- }`. В противном случае проверяющая система выдаст ошибку компиляции.

Решение задачи 8 В. SMS

Темы: динамическое программирование, длинная арифметика.

По полученному сообщению восстановим последовательность нажатий кнопок телефона. Так как ошибки могут возникать только в расстановке пауз, исходная последовательность нажатий восстанавливается однозначно. Информацию о том, какой букве соответствует какая кнопка и сколько нажатий этой кнопки, лучше всего хранить в константных массивах:

```
const
  button : array ['A'..'Z'] of integer = (
    2, 2, 2,
    3, 3, 3,
    4, 4, 4,
    5, 5, 5,
    6, 6, 6,
    7, 7, 7, 7,
    8, 8, 8,
    9, 9, 9, 9
  );
  presses : array ['A'..'Z'] of integer = (
    1, 2, 3,
    1, 2, 3,
    1, 2, 3,
    1, 2, 3,
    1, 2, 3,
    1, 2, 3, 4,
    1, 2, 3,
    1, 2, 3, 4
  );
```

Встретив букву L принятого сообщения, нужно добавить к последовательности нажатий кнопок `presses[L]` штук кнопок `button[L]`.

Обозначим B полученную последовательность нажатий клавиш, $B(1)$ — первая нажатая кнопка, $B(2)$ — вторая и т. д. Обозначим N_B длину последовательности.

Обозначим $C(P, L)$, количество сообщений длиной L , которые могут соответствовать последовательности нажатий клавиш $B(1)...B(P)$. Положительные значения C могут быть только в диапазоне $P, L \geq 0$, остальные значения будем считать равными нулю.

Ответом задачи является $C(N_B, N)$ — то есть количество сообщений длиной N , из которых может получиться полная последовательность нажатий клавиш.

Найдем значения $C(P, L)$ при $P = 0$. Если нажатий кнопок не было, то этому может соответствовать только одно сообщение — пустое, то есть $C(0, 0) = 1$, $C(0, L) = 0$ при $L > 0$.

Кроме того, если сообщение пустое, то ему может соответствовать только одна последовательность нажатий кнопок — пустая. Следовательно, $C(P, 0) = 0$ при $P > 0$.

Найдем значения $C(P, L)$ при $P, L > 0$. Последняя нажатая кнопка — $B(P)$. Рассмотрим, какой могла быть последняя буква сообщения, соответствующего такой последовательности нажатий клавиш.

В любом случае последней могла быть буква, соответствующая одному нажатию на кнопку $B(P)$. Например, если последней была нажата кнопка 2, то последней буквой сообщения могла быть «А». Если последней была нажата кнопка 9, последней буквой сообщения могла быть «W». Итак, в позицию (P, L) можно прийти из $(P - 1, L - 1)$.

Если $P > 1$ и две последние нажатые кнопки совпадают ($B(P) = B(P - 1)$), то последней могла быть буква, соответствующая двум нажатиям на кнопку $B(P)$. Например, если два последних нажатия были на кнопку 5, то последней буквой сообщения могла быть «К». Получается, что при условии $B(P) = B(P - 1)$ в позицию (P, L) можно прийти из $(P - 2, L - 1)$.

Аналогично, если $P > 2$ и три последние нажатые кнопки совпадают ($B(P) = B(P - 1) = B(P - 2)$), то последней могла быть буква, соответствующая двум нажатиям на кнопку $B(P)$. Например, если три последних нажатия были на кнопку 8, то последней буквой сообщения могла быть «V». Получается, что при условии $B(P) = B(P - 1) = B(P - 2)$ в позицию (P, L) можно прийти из $(P - 3, L - 1)$.

Случай, когда совпадают последние четыре нажатые кнопки, сложнее. Если на кнопке нарисованы три буквы, то совпадение четырех последних кнопок нам ничего не дает: в позицию (P, L) можно прийти только из $(P - 1, L - 1)$, из $(P - 2, L - 1)$ и из $(P - 3, L - 1)$. Если же на кнопке нарисованы четыре буквы, то в (P, L) можно прийти также и из $(P - 4, L - 1)$.

Случай совпадения последних пяти нажатых кнопок ничего нового не дает.

Итак, получаются следующие формулы:

- если $P \geq 4$, $B(P) = B(P - 1) = B(P - 2) = B(P - 3)$, а также $B(P) = 7$ или $B(P) = 9$, то формула

$$C(P, L) = C(P - 1, L - 1) + C(P - 2, L - 1) + C(P - 3, L - 1) + C(P - 4, L - 1);$$

- иначе, если $P \geq 3$ и $B(P) = B(P - 1) = B(P - 2)$, то формула

$$C(P, L) = C(P - 1, L - 1) + C(P - 2, L - 1) + C(P - 3, L - 1);$$

- иначе, если $P \geq 2$ и $B(P) = B(P - 1)$, то формула

$$C(P, L) = C(P - 1, L - 1) + C(P - 2, L - 1);$$

- в прочих случаях формула

$$C(P, L) = C(P - 1, L - 1).$$

Детали реализации:

- Максимальный ответ задачи не может быть точно представлен никаким встро-
енным типом Турбо Паскаля. Нужно использовать длинную арифметику,
а именно: процедуры сложения длинных чисел и вывода длинного числа (см.
разбор задачи 2 Е). Проблемой является подбор длины числа. Один из возмож-
ных способов — экспериментальный. Увидеть худший случай довольно легко,
рассмотрев формулы, — больше всего сложений получается в том случае, ког-
да все буквы сообщения получены нажатиями одной «четырёхбуквенной»
кнопки (все с 7 или все с 9). Очевидно также, что N нужно взять наибольшим
допустимым в задаче — 80. Остается только подбирать длину и содержание
полученного сообщения и смотреть, какой получается результат. В условиях
неизвестной заранее длины числа особую важность приобретает объявление
именованной константы — размера длинного числа.

Другим возможным способом подбора длины числа является метод половин-
ного деления. Использовали слишком маленькую длину числа — получили от
проверяющей системы ответ «Runtime error» благодаря самодиагностирующей
процедуре сложения (см. разбор задачи 2 Е). Использовали слишком большую
длину числа — получили ответ «Time limit exceeded».

- Размер полученного сообщения — до 80 символов, ему соответствуют до 320 на-
жатий кнопок. Размер исходного сообщения — до 80 символов. Следова-
тельно, для реализации алгоритма потребуется массив 320×80 , состоящий из длин-
ных чисел. В память Турбо Паскаля такой массив не поместится. Но это и не
страшно. Легко заметить, что во всех формулах вычисления $C(P, L)$ использу-
ются только значения со вторым индексом $L - 1$. Значит, можно обойтись дву-
мя линейными массивами размером 320 из длинных целых. При этом основной
памяти Турбо Паскаля хватит на то, чтобы длинные числа имели до 100 десятич-
ных разрядов.

Решение задачи 8 С. Дерево игры

Тема: антагонистические игры.

Вариант 1

Темы: косвенная рекурсия, деревья.

Описание дерева прочитаем в такой массив:

var

a : array [1..1000] of record

```

isLeaf : boolean;
parent : integer;
leafResult : integer;
end;
```

Здесь `isLeaf` показывает, является ли узел листом. Ложное значение `isLeaf` означает, что узел является внутренним. Это поле заполняется на основе первого символа строк входного файла. Поле `parent` указывает предка узла. Поле `leafResult` имеет смысл только при истинном значении `isLeaf` и означает результат игры, закончившейся в этом узле.

Для первого узла фиктивные значения таковы: `isLeaf = false`, `parent = 0`.

Напишем две взаимно-рекурсивные функции:

```
function best2(node : integer) : integer; forward;
```

```
function best1(node : integer) : integer;
```

```
...
```

```
end;
```

```
function best2(node : integer) : integer;
```

```
...
```

```
end;
```

Нужно, чтобы функция `best1` могла вызывать функцию `best2`, а функция `best2` — функцию `best1`. В Паскале разрешено использование только описанных выше имен. Поэтому как бы ни были размещены функции, одна из них не будет видеть другую. Решением проблемы является использование опережающего объявления. Заголовок процедуры нужно продублировать в начале программы со словом `forward` вместо тела процедуры.

Функции `best1` и `best2` устроены практически одинаково и выполняют почти одинаковые действия. Их параметр `node` — номер узла, для которого нужно найти лучший результат. Только `best1` должна искать результат при свободе выбора первого игрока (он будет выбирать максимум из результатов дочерних узлов), а `best2` — при свободе выбора второго игрока (он будет выбирать минимум из результатов дочерних узлов).

Если узел — лист дерева, то свободы действий нет, и нужно выдавать в качестве результата функции то, что записано в этом листе:

```

if a[node].isLeaf then
  best1 := a[node].leafResult
  {или "best2 := " в функции best2}
```

Если же узел внутренний, то игрок может выбрать наиболее благоприятный для себя вариант. Нужно перебрать все узлы, куда можно попасть из текущего:

```

for i := 1 to n do
  if a[i].parent = node then
    ...
```


и для каждого такого узла смоделировать поведение соперника — из `best1` вызывается `best2`, а из `best2` вызывается `best1`.

Ответом задачи является `best1(1)` — первый игрок начинает с первого узла.

Почему подобные процедуры, имитирующие действия игроков, нельзя было использовать, скажем, при решении задачи 6 С? Дело в том, что там выигрышность одной и той же даты, например, 5 сентября, используется при нахождении выигрышности разных дат — в 5 сентября можно попасть из 3 и 4 сентября, из 5 августа и из 5 июля. Если высчитывать результат игры для одной даты снова и снова, время работы программы будет расти экспоненциально.

В данной же задаче у каждого узла есть только один предок, так что в сумме будет сделано N вызовов функций `best1` и `best2`. Внутри каждой из них есть цикл от 1 до N . Таким образом, общее число действий будет пропорционально N^2 .

Вариант 2

Решение предложил Эдуард Гудков.

Это решение основывается на предположении, что родительский узел имеет меньший номер, чем дочерний. В условии задачи этого не сказано, но на реальном соревновании тестов, не подпадающих под этот случай, не встретилось.

Данные будем хранить в массиве:

```
var
  a : array [1..1000] of record
    parent : integer;
    player : integer;
    result : integer;
  end;
```

Здесь `parent` — номер родительского узла. Поле `player` обозначает, какой игрок ходит из этого узла. При чтении входных данных заполняем поле `player` следующим образом: из узла ходит противник того, кто ходил из родительского узла:

```
a[i].player := 3 - a[a[i].parent].player;
```

Перед чтением нужно заполнить поле `player` первого элемента массива: `a[1].player := 1` — из первого узла ходит всегда первый игрок.

Поле `result` — результат игры. Для листьев значение читаем из файла. Для внутренних узлов будем присваивать полю `result` значение в зависимости от значения поля `player`. Для первого игрока худшим случаем будет `result = -1`, для второго игрока хуже всего `result = 1`. Эти значения худших случаев и присвоим полям `result` для внутренних узлов. В дальнейшем эти значения будут подкорректированы.

Определение выигрышей для узлов происходит так: проходим массив от N -го ко 2-му элементу. Значение `result` при таком проходе у текущего элемента определено. Модифицируем соответствующим образом поле `result` родителя.

Если у родителя $\text{player} = 1$, его значение result нужно изменить на result дочернего узла, если у дочернего узла result больше, чем у родителя. И наоборот, если у родителя $\text{player} = 2$, его значение result нужно изменить на result дочернего узла, если у дочернего узла result меньше, чем у родителя.

Таким образом, когда рассматривается внутренний узел, все дочерние узлы уже рассмотрены, и поле result установлено равным максимуму из значений result дочерних узлов для хода первого игрока или минимуму из значений result дочерних узлов для хода второго игрока.

Ответом является $a[1].\text{result}$ — результат игры, начатой с 1-го узла.

Решение задачи 8 D. Дуга на сфере

Тема: геометрия.

Вариант 1

Обозначим *longitude* долготу точки на поверхности планеты, *latitude* — ее широту. Заметьте, во входных данных широта и долгота даны в градусах, а в Турбо Паскале все тригонометрические функции работают со значениями в радианах. Поэтому сразу после чтения широты и долготы нужно преобразовать их в радианы (разделить на 180 и умножить на π).

Переведем сферические координаты в трехмерные декартовы:

$$X = R \cos(\text{latitude}) \cos(\text{longitude});$$

$$Y = R \cos(\text{latitude}) \sin(\text{longitude});$$

$$Z = R \sin(\text{latitude}).$$

Найдем расстояние между точками по прямой:

$$D = \sqrt{(X_1 - X_2)^2 + (Y_1 - Y_2)^2 + (Z_1 - Z_2)^2}.$$

Рассмотрим треугольник, образованный двумя заданными точками и центром планеты. Две его стороны равны R и одна D . Если найти α — угол при вершине, являющейся центром планеты, то минимальный путь по поверхности планеты легко найти по формуле αR (α в радианах).

Найдем угол α в этом треугольнике. Опустим высоту на сторону длиной D . Получатся два равных прямоугольных треугольника. В прямоугольном треугольнике гипотенуза будет равна R , а сторона, противолежащая углу $\alpha/2$, будет равна $D/2$. Следовательно, $\alpha/2 = \arcsin((D/2)/R)$.

В Турбо Паскале нет функций $\arcsin$ и $\arccos$, есть только $\arctan$. Воспользуемся ею. Если известна гипотенуза и один из катетов прямоугольного треугольника, то найти второй катет ничего не стоит по теореме Пифагора: $H = \sqrt{R^2 - (D/2)^2}$. После чего угол находится через арктангенс: $\alpha/2 = \arctg((D/2)/H)$.

Детали реализации:

- При использовании в этой задаче типа `real` требуемая точность достигнута не будет — следует применять `double` или `extended`.
- Тип `extended` позволяет представить вещественное значение наиболее точно. Однако для него более, чем для других вещественных типов, характерны ошибки округления. Пусть, например, точки лежат на противоположных сторонах планеты. По формулам должно выполняться $D = 2R$. В реальных вычислениях D может оказаться чуть-чуть больше $2R$. Это «чуть-чуть» проявит себя при вычислении H : возникнет ошибка времени выполнения — вычисление корня из отрицательного числа. Чтобы этого избежать, перед вычислением H нужно написать:

```
if d > 2 * r then
  d := 2 * r;
```

Если не позаботиться об этом явно, при вычислении по формуле $\alpha = 2 \arctg((D/2)/H)$ может произойти деление на 0. Случай $H = 0$ нужно выделить отдельно, в этом случае точки лежат на противоположных сторонах планеты и $\alpha = \pi$.

Вариант 2

Решение предложил Никита Рыбак.

Найдем трехмерные декартовы координаты точек на поверхности сферы, как в варианте 1.

Центр сферы является центром системы координат, поэтому координаты точек являются также координатами векторов, направленных из центра сферы в заданные точки.

Найдем косинус угла между векторами с помощью скалярного произведения векторов: $\cos \alpha = \frac{X_1 X_2 + Y_1 Y_2 + Z_1 Z_2}{R^2}$.

Теперь нужно бы найти арккосинус, но в Турбо Паскале нет функции `arccos`, есть только `arctan`. Найдем синус угла из основного тригонометрического тождества: $\sin^2 \alpha + \cos^2 \alpha = 1$. Получается формула $\sin \alpha = \sqrt{1 - \cos^2 \alpha}$. Синус неотрицательный, потому что нас интересует угол из интервала $[0; \pi]$.

Тангенс находится по определению: $\operatorname{tg} \alpha = \frac{\sin \alpha}{\cos \alpha}$. От этой величины уже можно взять арктангенс, только нужно всегда помнить, что функция `arctan` возвращает значение из интервала $\left(-\frac{\pi}{2}; \frac{\pi}{2}\right)$, а нам в этой задаче нужно значение из интервала $[0; \pi]$. Поэтому, если угол лежит во второй четверти ($\cos \alpha < 0$), то к углу, полученному с помощью `arctan`, нужно добавить π .

Нельзя для определения того, добавлять или не добавлять π к результату `arctan`, использовать знак самого результата `arctan` — если угол близок к 0 или к π , значе-

ние $\arctan$ становится равным 0, и различить эти два случая по знаку результата арктангенса невозможно.

Детали реализации:

- Как и в варианте 1, точности типа `real` не хватит.
- Как и в варианте 1, следует иметь в виду ошибки округления. Например, полученный с помощью скалярного произведения косинус может оказаться чуть больше 1 или чуть меньше -1 . При нахождении синуса с помощью такого косинуса возникнет ошибка вычисления корня из отрицательного числа. Значит, если значение косинуса больше 1, нужно его сделать равным 1, если меньше -1 — равным -1 .
- Как и в варианте 1, следует перед делением убедиться в том, что делитель не нулевой. Например, при нахождении тангенса по синусу и косинусу нужно отдельно рассмотреть случай $\cos \alpha = 0$ — в этом случае угол равен $\pi/2$.

Вариант 3

Решение предложил Никита Рыбак.

Найдем трехмерные декартовы координаты точек на поверхности сферы, как в варианте 1.

Найдем косинус угла между векторами, как в варианте 2.

В этом решении используется тот факт, что функция $\arctan$ возвращает угол из интервала $\left(-\frac{\pi}{2}; \frac{\pi}{2}\right)$. Если косинус угла получился отрицательный, то угол принадлежит интервалу $\left[\frac{\pi}{2}; \pi\right]$. Используем равенство $\cos \alpha = -\cos(\pi - \alpha)$. То есть если косинус получился отрицательным, нужно запомнить его знак в отдельную переменную и взять его по модулю. А когда в результате вычислений получится угол из интервала $\left[0; \frac{\pi}{2}\right]$, вспомнить, был ли минус, и если был, то настоящее значение угла получить так:

```
alpha := pi - alpha;
```

Детали реализации — см. вариант 2: опасности в этих способах решения одинаковы.

Решение задачи 8 Е. Путь коня

Тема: поиск в ширину.

Решение этой задачи аналогично решению задачи 6 Е. Какое-нибудь одно из двух отмеченных символом @ мест будем считать начальной клеткой, второе — клеткой, куда надо прийти.

В матрице из целых обозначим начальное положение нулем, препятствия — значениями -2 , пустое место и место, куда нужно прийти, — значениями -1 . Например, для входных данных

5

@..@.

..##.

.....

.....

.....

матрица из целых будет выглядеть так:

```

0 -1 -1 -1 -1
-1 -1 -2 -2 -1
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1

```

На первом проходе пометим единицами пустые клетки, доступные из начального положения за 1 ход коня. Такая клетка одна (выделена жирным шрифтом):

```

0 -1 -1 -1 -1
-1 -1 -2 -2 -1
-1 1 -1 -1 -1
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1

```

На втором проходе пометим двойками клетки, содержащие -1 , доступные из клеток, содержащих 1 , за один ход коня. Такие клетки будут достижимы из исходного положения за два хода:

```

0 -1 2 -1 -1
-1 -1 -2 -2 -1
-1 1 -1 -1 -1
-1 -1 -1 2 -1
2 -1 2 -1 -1

```

На третьем проходе пометим тройками клетки, все еще содержащие -1 , в которые можно ходом коня попасть из клеток, содержащих двойки. Такие клетки будут достижимы из исходного положения за три хода:

```

0 -1 2 -1 -1
3 -1 -2 -2 3
-1 1 -1 3 -1
3 -1 3 2 3
2 3 2 -1 -1

```

На четвертом проходе пометим числами 4 пустые клетки, в которые можно попасть из клеток, содержащих числа 3:

```

0 -1 2 -1 4
3 4 -2 -2 3
4 1 4 3 4
3 4 3 2 3
2 3 2 -1 4

```

На пятом проходе пометим числами 5 пустые клетки, в которые можно попасть из клеток, содержащих числа 4:

```

0  5  2  5  4
3  4 -2 -2  3
4  1  4  3  4
3  4  3  2  3
2  3  2  5  4

```

На шестом проходе убеждаемся в том, что пустых клеток, в которые можно попасть из клеток, содержащих числа 5, нет. На этом заполнение матрицы заканчивается.

«Вспоминаем», где находилась клетка, куда нужно было привести коня. Если бы она содержала -1 , нужно было бы выводить сообщение о невозможности построения пути.

В рассматриваемом примере в этой клетке находится число 5.

```

0  5  2  5  4
3  4 -2 -2  3
4  1  4  3  4
3  4  3  2  3
2  3  2  5  4

```

Будем строить обратный путь: найдем клетку, в которую можно перейти из текущей ходом коня и содержащую на единицу меньшее значение. Сделаем ее текущей. В клетку, из которой ушли, занесем -3 — признак того, что путь коня проходит через эту клетку.

```

0  5  2 -3  4
3  4 -2 -2  3
4  1  4  3  4
3  4  3  2  3
2  3  2  5  4

```

Будем повторять это действие до тех пор, пока не окажемся в клетке, содержащей 0 — в начале пути коня:

```

0  5  2 -3  4
3  4 -2 -2  3
4  1 -3  3  4
3  4  3  2  3
2  3  2  5  4

```

```

0  5  2 -3  4
-3  4 -2 -2  3
4  1 -3  3  4
3  4  3  2  3
2  3  2  5  4

```

```

0  5 -3 -3  4
-3  4 -2 -2  3
4  1 -3  3  4
3  4  3  2  3
2  3  2  5  4

```

```

0  5 -3 -3  4
-3  4 -2 -2  3
4 -3 -3  3  4
3  4  3  2  3
2  3  2  5  4

```

Из соображений симметрии и в исходную клетку тоже занесем -3 :

```

-3  5 -3 -3  4
-3  4 -2 -2  3
4 -3 -3  3  4
3  4  3  2  3
2  3  2  5  4

```

Нетрудно заметить, что эта матрица является ответом — числа -3 соответствуют пути — символам @, числа -2 соответствуют препятствиям — символам #, все остальные соответствуют пустому месту — символам точки.

Детали реализации:

- Использование барьера упрощает реализацию алгоритма, однако вид барьера в этой задаче будет отличаться от вида барьера в решении задачи 6 Е. В той задаче шарик передвигался на соседнюю клетку, и поэтому достаточно было барьера шириной 1. Здесь же конь может изменять любую из своих координат на две единицы, поэтому по краям поля нужно создать кольцо непроходимых клеток шириной 2.

Объявление массива:

```

var
  a : array [-1..52, -1..52] of integer;

```

Создание барьера происходит после чтения N и перед чтением матрицы:

```

readln(n);
for i := -1 to n + 2 do
  for j := -1 to n + 2 do
    a[i, j] := -2;

```

- В задаче 6 Е шарик мог перейти в 4 соседних клетки. В этой задаче у коня 8 «соседних» клеток. Чтобы не ошибиться при использовании формул координат соседних клеток, настоятельно рекомендуется объявить константные массивы:

```

const
  di : array [1..8] of integer = (1, 2,  2,  1, -1, -2, -2, -1);
  dj : array [1..8] of integer = (2, 1, -1, -2, -2, -1, 1,  2);

```

и для обработки соседних клеток использовать цикл:

```

for i := 1 to n do
  for j := 1 to n do
    if a[i, j] = ... then
      for k := 1 to 8 do
        if a[i + di[k], j + dj[k]] = -1 then
          ...

```

Решение задачи 8 F. Грядки

Темы: заливка с явным стеком, динамическая память ТР.

Эта задача не отличается от задачи 7 F ничем, кроме ограничений — размер поля может составлять до 200×200 вместо 100×100 . В задаче 7 F приводились оценки, согласно которым глубина рекурсии в худшем случае примерно равна количеству клеток поля. Очевидно, что рекурсивную процедуру, описанную в решении той задачи, в данном случае применить нельзя.

Здесь нужно создавать стек в явном виде. Если в основной памяти Турбо Паскаля хранить саму карту садового участка, явный стек в основную память не поместится. Будем использовать динамическую память.

type

```
stack = array [1..40000] of byte;
```

var

```
ii, jj : ^stack;
```

```
sp : word;
```

Сразу после начала работы программы нужно создать динамические переменные:

begin

```
new(ii);
```

```
new(jj);
```

Сама процедура заливки выглядит очень просто — в стек заносятся координаты первой клетки области, а затем, пока стек не пуст, из него извлекаются очередные координаты и в стек заносятся координаты еще не покрашенных соседей:

```
procedure paint(i, j : integer);
```

```
begin
```

```
  sp := 0;
```

```
  push(i, j);
```

```
  while sp > 0 do begin
```

```
    pop(i, j);
```

```
    push(i + 1, j);
```

```
    push(i - 1, j);
```

```
    push(i, j + 1);
```

```
    push(i, j - 1);
```

```
  end;
```

```
end;
```


Чтобы в основной процедуре четыре раза не проверять, что соседняя клетка пуста, эта проверка вынесена в процедуру занесения в стек — push.

```
procedure push(i, j : integer);  
begin  
  if c[i, j] = '#' then begin  
    inc(sp);  
    ii^[sp] := i;  
    jj^[sp] := j;  
    c[i, j] := '.';  
  end;  
end;
```

В задаче проводится не слишком много вычислений, проверки диапазонов лучше включить: $\{\$R+, Q+\}$.

ПРИМЕЧАНИЕ

Для заливки можно применить и очередь (см., например, разбор задачи 7 E). Обратное неверно — использовать стек при нахождении минимального пути нельзя.

Тренировка 9

Решение задачи 9 А. Диалог компьютеров

Тема: быстрая сортировка.

Прежде всего нужно понять описываемый в условии процесс. В каком случае клиент, знающий все полные имена файлов и, скажем, расширение выбранного файла, может сказать, что он не знает полного имени файла? Когда существует несколько полных имен с таким расширением. Рассмотрим пример из условия:

```
LICENCE.TMP
WIN32.LOG
FILEID.
PSTOTEXT.TXT
GSVIEW32.EXE
GSVIEW32.ICO
GSVIEWDE.HLP
LICENCE
GSVIEWEN.HLP
GSVW32DE.DLL
FILEID.TMP
GSVW32EN.DLL
PSTOTXT3.DLL
PSTOTXT3.EXE
GSV16SPL.EXE
GVWGS32.EXE
ZLIB32.DLL
PRINTER.INI
README.TXT
```

Если клиент, знающий расширение, сказал, что он не знает полного имени файла, значит, расширение файла не могло быть LOG, ICO, INI — иначе он знал бы полное имя файла, потому что есть только по одному файлу с этими расширениями. Итак, после получения такой информации мы (как и второй клиент) можем сделать вывод, что выбран один из следующих файлов:

```
LICENCE.TMP
FILEID.
PSTOTEXT.TXT
```

```

GSVIEW32.EXE
GSVIEWDE.HLP
LICENCE
GSVIEWEN.HLP
GSVW32DE.DLL
FILEID.TMP
GSVW32EN.DLL
PSTOTXT3.DLL
PSTOTXT3.EXE
GSV16SPL.EXE
GVWGS32.EXE
ZLIB32.DLL
README.TXT

```

Аналогично, если теперь клиент, знающий имя, сказал, что он не знает полного имени файла, то имя файла не может быть уникальным — PSTOTEXT, GSVIEW32, GSVIEWDE, GSVIEWEN, GSVW32DE, GSVW32EN, GSV16SPL, GVWGS32, ZLIB32, README. Значит, после такого заявления клиента мы (как и другой клиент) можем сделать вывод, что загаданный файл содержится в следующем наборе:

```

LICENCE.TMP
FILEID.
LICENCE
FILEID.TMP
PSTOTXT3.DLL
PSTOTXT3.EXE

```

Этот набор и является ответом задачи. Если бы в примере было $M = 3$, то нужно было бы опять убрать файлы с уникальным расширением:

```

LICENCE.TMP
FILEID.
LICENCE
FILEID.TMP

```

При больших M в этом примере процесс стабилизируется — уникальных имен и уникальных расширений не осталось.

Основная сложность задачи заключается в том, чтобы найти алгоритм, который позволяет *быстро* убрать из набора уникальные имена (или уникальные расширения). Используем быструю сортировку. После ее применения одинаковые имена (или одинаковые расширения — смотря по чему сортировали) будут сгруппированы в одном месте, и легко определить, уникальное имя или нет.

Процедура, удаляющая из отсортированного массива уникальные элементы (на примере массива из целых), такова:

```

var
  a : array [1..maxn] of integer;
  n : integer;           {число используемых элементов}

```

```
procedure killSingle;
var
  i, j : integer;
begin
  j := 0;
  for i := 1 to n do
    if
      (i < n) and (a[i] = a[i + 1]) or
      (i > 1) and (a[i] = a[i - 1])
    then begin
      inc(j);
      a[j] := a[i];
    end;
  n := j;
end;
```

Теперь перейдем непосредственно к решению задачи. Входные данные рекомендуется занести в следующую структуру:

```
type
  filename = record
    number : integer;
    fullname : string[8 + 1 + 3];
    name : string[8];
    extension : string[3];
  end;

var
  files : array [1..1000] of filename;
  n : integer;
```

При чтении данных в `fullname` заносится полное имя файла. Затем из него выделяются `name` и `extension` — части, расположенные до и после точки. Если точки в имени файла нет, в поле `extension` нужно занести пустую строку. В `number` нужно занести порядковый номер этого имени файла во входных данных (зачем это нужно, объясним в дальнейшем).

После прочтения входных данных нужно выполнить M шагов: если шаг нечетный (нумерация шагов начинается с единицы), то нужно удалить уникальные расширения, если четный — уникальные имена.

После этого будет получен набор имен файлов, который нужно выводить. Но в условии сказано, что имена должны идти в том порядке, в котором они перечислялись во входных данных. Для этого отсортируем полученный массив по возрастанию значения поля `number`, после чего можно выводить запомненные полные имена `fullname`.

Получается, что нужно написать три варианта быстрой сортировки: по имени, по расширению и по номеру во входных данных. Кроме того, нужно написать две про-

цедуры «сжатия» отсортированных данных: для уникальных имен и для уникальных расширений. Очевидно, что если процедуры различаются незначительно, нужно написать одну, а потом скопировать ее текст и в копию внести минимальные изменения.

Детали реализации: в процедуре быстрой сортировки не следует типом `key` делать `string`, так как размер этого типа довольно большой — 256 байт. Наряду с увеличением размера локальных переменных увеличивается и вероятность переполнить стек, поэтому нужно точно указывать длину `key` — `string[3]` или `string[8]`.

Решение задачи 9 В. Бросание кубика

Тема: динамическое программирование.

Обозначим $P(K, S)$ вероятность того, что при бросании кубика K раз сумма выпавших чисел окажется равна S .

При $K = 0$ значения функции таковы: $P(0, 0) = 1$ — бросив кубик 0 раз, мы обязательно получим 0 очков. Любое другое количество очков получить нельзя: $P(0, S) = 0, S \neq 0$.

Найдем, чему равно $P(K, S)$ при $K > 0$. Рассмотрим, сколько очков выпало при последнем бросании кубика. Это целое число от 1 до 6. Следовательно, в позицию (K, S) можно попасть только из позиций $(K - 1, S - 1)$, $(K - 1, S - 2)$, $(K - 1, S - 3)$, $(K - 1, S - 4)$, $(K - 1, S - 5)$ и $(K - 1, S - 6)$.

Рассмотрим переход от $(K - 1, S - 1)$ к (K, S) . Чтобы переход состоялся, требуется, чтобы на кубике выпало 1. Это происходит с вероятностью $1/6$. Следовательно, если $P(K - 1, S - 1)$ — вероятность попасть в позицию $(K - 1, S - 1)$, то переход из позиции $(K - 1, S - 1)$ в (K, S) состоится с вероятностью $\frac{P(K - 1, S - 1)}{6}$.

Аналогично, переход от $(K - 1, S - 2)$ к (K, S) состоится с вероятностью $\frac{P(K - 1, S - 2)}{6}$ и т. д.

Получили формулу расчета вероятности:

$$P(K, S) = \frac{P(K - 1, S - 1)}{6} + \frac{P(K - 1, S - 2)}{6} + \frac{P(K - 1, S - 3)}{6} + \frac{P(K - 1, S - 4)}{6} + \frac{P(K - 1, S - 5)}{6} + \frac{P(K - 1, S - 6)}{6}.$$

Детали реализации:

- Самый маленький положительный ответ задачи получается при входных данных $N = 500, Q = 500$. Убедитесь в том, что выбранный вами вещественный тип даст на этом тесте ненулевой ответ.
- Если бы памяти было много, естественно было бы объявить массив для хранения вероятностей так:

`var`

`p : array [0..500, -5..3000] of real;`

Диапазон первых индексов начинается с нуля, чтобы можно было провести начальную инициализацию: $P(0, 0) = 1$, $P(0, S) = 0$ при $S \neq 0$. Диапазон вторых индексов начинается с -5 , чтобы можно было применять полученную формулу для вычисления $P(K, 1)$ без опасения обратиться к несуществующему элементу массива.

- Памяти в Турбо Паскале для такого массива недостаточно, так что придется обойтись двумя одномерными массивами, благо в формуле $P(K, S)$ используются вероятности только для предыдущего, $(K - 1)$ -го, броска. Таким образом, один линейный массив будет хранить значения $P(K, *)$, а второй — $P(K - 1, *)$, и первый будет заполняться на основе второго.
- Чтобы уменьшить количество операций, следует в формуле расчета вероятности деление на 6 вынести за скобки.
- Чтобы уменьшить количество действий, можно заполнять таблицу по формуле только для тех ячеек, для которых она заведомо ненулевая. При K бросках сумма не может быть меньше K или больше $6K$, поэтому для всех $S < K$ и $S > 6K$ вероятность $P(K, S)$ можно не считать по формуле, а сразу установить равной нулю.
- Проверки диапазонов массивов $\{R+\}$ при написании и отладке программы лучше включить, а при отправке решения на проверку лучше эту директиву убрать.
- При использовании вещественных типов `single`, `double`, `extended` в начале программы следует записать директиву $\{\$N+, E-\}$.

Модификация решения

Можно еще уменьшить количество действий. Пусть нужно найти

$$P(K, S) = \frac{P(K-1, S-1)}{6} + \frac{P(K-1, S-2)}{6} + \frac{P(K-1, S-3)}{6} + \\ + \frac{P(K-1, S-4)}{6} + \frac{P(K-1, S-5)}{6} + \frac{P(K-1, S-6)}{6}.$$

Заметим, как было найдено значение $P(K, S-1)$:

$$P(K, S-1) = \frac{P(K-1, S-2)}{6} + \frac{P(K-1, S-3)}{6} + \frac{P(K-1, S-4)}{6} + \\ + \frac{P(K-1, S-5)}{6} + \frac{P(K-1, S-6)}{6} + \frac{P(K-1, S-7)}{6}.$$

Получим формулу

$$P(K, S) = P(K, S-1) - \frac{P(K-1, S-7)}{6} + \frac{P(K-1, S-1)}{6}$$

или, уменьшая количество действий,

$$P(K, S) = P(K, S-1) + \frac{P(K-1, S-1) - P(K-1, S-7)}{6}.$$

В этой формуле одно вещественное сложение, одно вещественное вычитание и одно вещественное деление вместо пяти вещественных сложений и одного вещественного деления в формуле основного решения.

При использовании этой формулы нужно позаботиться о том, чтобы для любой вычисляемой вероятности $P(K, S)$ существовал элемент массива, соответствующий позиции $(K - 1, S - 7)$.

Решение задачи 9 С. Игра с монетами

Темы: антагонистические игры, динамическое программирование.

Обозначим $S(R)$ суммарное количество монет в стопках с R -й по N -ю. Эти значения часто будут использоваться не только в описании алгоритма, но и в программе, поэтому их нужно заранее вычислить и хранить в массиве.

Обозначим $G(R, M)$ количество монет, которое может получить игрок, если сейчас его ход, при условии, что остались стопки с R -й по N -ю, и он не может взять больше M стопок.

Будем заполнять G . Если игрок может взять все оставшиеся стопки, то он должен взять их все, иначе противник что-то получит из этих стопок, и выигрыш игрока, чей сейчас ход, не будет максимальным. Таким образом оказываются определенными $G(R, M)$, $M \geq N - R + 1$, в том числе все $G(N, M)$.

Будем находить $G(R, M)$ в порядке убывания R при условии $M < N - R + 1$. По условию задачи в такой ситуации игрок может взять от 1 до M стопок. Обозначим I количество взятых стопок. Если игрок возьмет I стопок, то противник окажется в ситуации $(R + I, I)$ — его ход, разыгрываются стопки с $(R + I)$ -й по N -ю, можно взять не более I стопок. По условию задачи он извлечет из этой ситуации максимальную выгоду — получит $G(R + I, I)$. А вообще в стопках с R -й по N -ю было $S(R)$ монет. Значит, первый игрок, взяв I стопок, получит $S(R) - G(R + I, I)$.

Первому игроку остается выбрать максимальное значение своего выигрыша:

$$G(R, M) = \max_{1 \leq I \leq M} (S(R) - G(R + I, I)).$$

Очевидно, ответом задачи является $G(1, K)$ — столько может получить первый игрок, если разыгрываются стопки с 1-й по N -ю и можно взять не более K монет.

Число действий в задаче пропорционально NK^2 — находим каждое значение $G(R, M)$, $1 \leq R \leq N$, $1 \leq M \leq K$ и при условии $M < N - R + 1$ выбираем максимум из M вариантов. Если бы $S(R)$ не было вычислено заранее, сложность составляла бы N^2K^2 .

Решение задачи 9 D. Бассейн реки

Тема: геометрия.

Следует обратить внимание на то, как в этой задаче заданы ограничения на входные данные: число рек до 10, суммарное число точек во всех описаниях рек до 1000.

То есть, выбирая представление входных данных, следует учитывать, что может быть 10 рек, из которых 9 заданы одной точкой, а одна задана 991 точкой. Рассмотрим первый приходящий в голову способ хранения данных — двумерный массив, в котором первый индекс означает номер реки, второй — номер точки. Получится, что нужно резервировать память для 10 000 точек. Если точка представлена двумя координатами типа `extended`, то потребуется 200 Кбайт.

В принципе, если иметь в виду динамическую память Турбо Паскаля, то ничего невозможного в таком объеме памяти нет. Но есть лучший способ представления исходных данных. Глобальные объявления:

`var`

```
n : integer;  
x1, y1 : array [1..1000] of real;  
source, mouth : array [1..10] of integer;
```

Здесь `n` — число рек. В массивах `x1` и `y1` хранятся координаты точек в том порядке, в каком они встречаются во входном файле. Массивы `source` и `mouth` хранят индексы точек истока и устья рек. Например, если первая река задана десятью точками, а вторая река — двадцатью, то `source[1] = 1, mouth[1] = 10, source[2] = 11, mouth[2] = 30`.

При необходимости перебрать все точки i -й реки используется цикл

```
for j := source[i] to mouth[i] do begin  
    ... x1[j] ...  
    ... y1[j] ...  
end;
```

Нетрудно заметить, что при таком способе организации данных даже при использовании самого точного вещественного типа — `extended` — потребуется всего 20 Кбайт памяти. Так что помимо того, что все данные помещаются в основную память Турбо Паскаля, там остается место для координат еще 2000 точек, и мы этим в дальнейшем воспользуемся.

Итак, входные данные прочитаны в память. Следующее действие — объединить связанные друг с другом реки. Для хранения данных о том, притоками чего являются реки, объявим массив

`var`

```
tributaryOf : array [1..10] of integer; {приток чего}
```

Поначалу занесем в этот массив сведения о том, что каждая из рек является своим притоком:

```
for i := 1 to n do  
    tributaryOf[i] := i;
```

Затем в `tributaryOf[i]` занесем информацию о том, в какую реку i -я река впадает непосредственно. Если река не впадает ни в одну, останется `tributaryOf[i] = i`. Чтобы получить такую информацию, нужно перебрать все пары рек $i1, i2 = 1...N, i1 \neq i2$, и посмотреть, не совпадает ли точка устья реки $i1$ с какой-нибудь точкой реки $i2$. Если есть совпадение, то можно устанавливать `tributaryOf[i1] := i2`.

Итак, теперь `tributaryOf[i]` содержит или `i`, если река `i` не впадает ни в какую, или `j`, если река `i` впадает в реку `j`. Мы знаем, какие реки связаны между собой непосредственно. Теперь определим, какие реки вообще связаны. Например, в случае «река 1 впадает в реку 2, а река 2 впадает в реку 3» нужно сделать так, чтобы `tributaryOf[1]` было равно не 2, а 3. После выполнения такого преобразования связанные реки будут иметь одинаковое значение `tributaryOf`.

Сделаем следующее: переберем все пары рек `i1, i2 = 1...N, i1 ≠ i2`. Если река `i1` впадает в реку `i2` (`tributaryOf[i1] = i2`) и река `i2` тоже куда-то впадает (`tributaryOf[i2] <> i2`), то запишем, что река `i1` впадает не в реку `i2`, а туда же, куда и `i2` (`tributaryOf[i1] := tributaryOf[i2]`). Станем повторять эти операции до тех пор, пока в очередной раз не найдется тройка рек, где первая река впадает во вторую, а вторая — в третью.

Теперь найдем выпуклую оболочку для каждого связного набора рек. Как выделить все точки связанных рек? Находим реку, которая никуда не впадает (`tributaryOf[i] = i`), и находим все реки `i2`, которые впадают в нее (`tributaryOf[i2] = i`). Все точки реки (от `source[i2]` до `mouth[i2]`) занесем в массивы `x2` и `y2`. Выделяем точки одного бассейна в отдельные массивы для того, чтобы алгоритм нахождения точек выпуклой оболочки (см. разбор задачи 5 D) записывался естественным образом:

`var`

```
x2, y2 : array [1..1000] of real;
points2 : integer; {счетчик точек в массивах x2 и y2}

...
for i := 1 to n do
  if tributaryOf[i] = i then begin
    points2 := 0;
    for i2 := 1 to n do
      if tributaryOf[i2] = i then
        for j := source[i2] to mouth[i2] do begin
          inc(points2);
          x2[points2] := x1[j];
          y2[points2] := y1[j];
        end;
    ...      {вызов процедур нахождения выпуклой оболочки и площади}
    ...      {нахождение максимальной из площадей}
  end;
```

После нахождения точек-вершин выпуклой оболочки нужно найти площадь образованного ими многоугольника (см. разбор задачи 4 D). Рекомендуется вершины, находимые в алгоритме построения выпуклой оболочки, заносить в массивы `x3` и `y3`, чтобы алгоритм нахождения площади записывался наиболее естественно и привычно.

Детали реализации: обратите внимание на то, чтобы первая найденная точка выпуклой оболочки не заносилась в массив вершин дважды — как первая и как последняя. Если это произойдет, то при некоторых входных данных (1 река — 1000 точек, расположенных по кругу) произойдет переполнение массива, в котором хранятся точки для нахождения площади. Другой способ решения этой проблемы — сделать размер массивов $\times 3$ и $у 3$ не 1000, а 1001.

Решение задачи 9 Е. Ближайшее число

Основная сложность этой задачи — уменьшить количество действий. Алгоритм решения «в лоб» — для каждой клетки просматривать всю матрицу в поисках ненулевых элементов — потребует порядка N^4 действий, что для $N = 200$ составляет 1,6 миллиарда.

Вариант 1

Темы: подобие волнового алгоритма, динамическая память ТР.

Определим тип «координаты» (ячейки). Значение этого типа будет представлено парой целых чисел. Записываться оно будет (i, j) . В этой задаче значения компонентов пары лежат в интервале от 1 до 200. Также определим фиктивные значения $(0, 0)$ и $(201, 201)$. Значения типа «координаты» разрешено сравнивать на равенство.

По входным данным заполним матрицу координат $K_0(1...N, 1...N)$ следующим образом:

- если $A(i, j) = 0$, то $K_0(i, j) = (0, 0)$;
- если $A(i, j) \neq 0$, то $K_0(i, j) = (i, j)$.

То есть если число ненулевое, то в K_0 попадают его координаты, иначе значение $(0, 0)$.

По таблице K_0 заполним таблицу K_1 следующим образом:

- если $K_0(i, j) \neq (0, 0)$, то $K_1(i, j) = K_0(i, j)$;
- если $K_0(i, j) = (0, 0)$ то рассматриваются четыре соседние ячейки — $K_0(i + 1, j)$, $K_0(i - 1, j)$, $K_0(i, j + 1)$, $K_0(i, j - 1)$. Если все значения этих ячеек равны $(0, 0)$, то в $K_1(i, j)$ также заносится $(0, 0)$. Если все ненулевые значения соседних ячеек совпадают, то в $K_1(i, j)$ заносится это ненулевое значение, а если среди значений соседних ячеек есть разные ненулевые, то в $K_1(i, j)$ заносится значение $(201, 201)$.

По таблице K_1 аналогично заполним таблицу K_2 , по таблице K_2 — таблицу K_3 , и так далее до тех пор, пока полученная на очередном шаге таблица не окажется равна предыдущей.

Рассмотрим, что же происходило в ходе описанного процесса. В K_1 значения получали ячейки, соответствующие нулевым ячейкам исходной матрицы A , которые

соседствуют (в смысле определенного в условии задачи расстояния) с ненулевыми ячейками. Если нулевая ячейка соседствовала с двумя разными ненулевыми ячейками, то в соответствующую ячейку K_1 заносилось значение (201, 201).

В K_2 заполнялись нулевые ячейки, находящиеся на расстоянии 2 от ненулевых. Очевидно, они соседствуют с нулевыми, находящимися на расстоянии 1 от ненулевых. Также очевидно, что если у всех ненулевых соседей ближайшей является одна и та же нулевая ячейка, то она же будет единственной ближайшей и для рассматриваемой клетки. А если соседи доступны из разных ненулевых клеток за одинаковое число шагов, то и текущая клетка тоже будет доступна из нескольких ненулевых клеток за одинаковое число шагов. Такие клетки заполняем значениями (201, 201).

Аналогично в K_t заполняются нулевые ячейки, находящиеся на расстоянии t от ненулевых.

Процесс очень похож на волновой алгоритм, описанный в решении задачи 6 Е. Только там пустые ячейки заполнялись вокруг одного места — клетки начала движения. А в этой задаче ячейки заполняются вокруг каждой ненулевой клетки. В отличие от решения задачи 6 Е, здесь нельзя обойтись одной матрицей, потому что не запоминается шаг, на котором мы пришли в ячейку, и существует опасность использовать ее уже на текущем, а не на следующем шаге. Нельзя использовать значение ненулевой ячейки вместо ее координат — если нулевая ячейка находится на одинаковом расстоянии от двух ненулевых ячеек с одинаковыми значениями, то, по условию задачи, она должна остаться нулевой.

Ответ задачи выводится следующим образом. Если $K(i, j) = (201, 201)$, то в соответствующей позиции нужно выводить 0 — расстояние от клетки до двух ненулевых клеток одинаково. В противном случае нужно выводить исходное число, ближайшее к клетке. Его координаты $K(i, j)$, оно само $A(K(i, j))$.

Количество действий в этом алгоритме пропорционально N^3 . Действительно, максимальное расстояние от нулевой клетки до ненулевой может составить $2N - 2$ (клетки расположены в противоположных углах матрицы), и на каждом проходе рассматривается N^2 клеток.

Детали реализации:

- Для значений исходной матрицы A подошел бы тип `longint`, но матрица 200×200 элементов типа `longint` не может существовать ни в основной, ни в динамической памяти Турбо Паскаля.

Предлагается такое решение: создать массив из указателей на массивы. Массив 200×200 элементов типа `longint` заменяется массивом из 200 указателей на массивы из 200 элементов типа `longint`. Таким образом, в динамической памяти будет 200 массивов по 800 байт. Турбо Паскаль ограничивает размер одного массива 65 520 байтами, но суммарный размер данных в динамической памяти ограничен только размером динамической памяти (500–600 Кбайт). Замена двумерного массива массивом из указателей на массивы — стандартный способ преодоления ограничения на размер одного массива.

Структуры данных таковы:

```

type
  long200 = array [1..200] of longint;
var
  a : array [1..200] of ^long200;

  {
  Процедура инициализации массива A
  Создание линейных массивов в динамической памяти
  }
  procedure initA;
  var
    i : integer;
  begin
    for i := 1 to 200 do
      new(a[i]);
    end;
  ...
begin
  initA;
  ...

```

Там, где для двумерного массива пишется $a[i, j]$, для массива из указателей на массивы нужно писать $a[i]^{[j]}$.

- Достаточно двух матриц K — для текущего и предыдущего шага. Эти матрицы не поместятся в основную память Турбо Паскаля, поэтому нужно объявлять массивы в динамической памяти:

```

var
  ki, kj, ki_prev, kj_prev : ^byte200x200;

```

- Не нужно после построения новой матрицы K сравнивать ее с предыдущей — нужно просто в логической переменной запоминать, изменилось ли какое-нибудь нулевое значение предыдущей матрицы на ненулевое текущей.

Вариант 2

Решение предложил Ярослав Музыкантов.

Обозначим расстояние до ближайшего ненулевого элемента $D(i, j)$. Если $A(i, j) \neq 0$, то $D(i, j) = 0$.

Пусть нужно найти ближайший к нулевому $A(i, j)$ ненулевой элемент и известно расстояние до ближайшего ненулевого элемента какого-то соседа $A(i, j)$ — например, $A(i, j - 1)$.

Утверждается, что $D(i, j)$ может быть только одним из трех: $D(i, j - 1) - 1$, $D(i, j - 1)$ или $D(i, j - 1) + 1$, — то есть отличается от той же характеристики соседа не более чем на 1. Без доказательства.

Что нам дает эта информация? Теперь для одного элемента (i, j) нужно перебирать не все 200×200 ячеек исходной матрицы, а только ячейки, образующие вокруг него ромб (квадрат, повернутый на 45°) шириной 3. Рассмотрим входные данные:

7

```

0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 1 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

```

Пусть нам известно, что для клетки $(4, 3)$ — четвертая сверху строка, третий столбец — значение функции $D(4, 3) = 2$. Тогда для клетки $(4, 4)$ значение функции может быть 1, 2 или 3. При этом геометрическое место ближайшей к $(4, 4)$ ненулевой клетки — ромб:

```

0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 1 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

```

Ромб может содержаться в исходной матрице лишь частично. При тех же входных данных для клетки $(1, 6)$ значение функции $D(1, 6) = 8$. Исходя из этого, для клетки $(1, 7)$ значение функции может быть 7, 8 или 9:

```

0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 1 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

```

Рассмотрим уравнение «квадратного» ромба: $|x - x_0| + |y - y_0| = C$. Если убрать знаки модуля, получатся четыре уравнения прямых:

$$(x - x_0) + (y - y_0) = C, x_0 \leq x \leq x_0 + C, y_0 \leq y \leq y_0 + C;$$

$$(x_0 - x) + (y - y_0) = C, x_0 - C \leq x \leq x_0, y_0 \leq y \leq y_0 + C;$$

$$(x_0 - x) + (y_0 - y) = C, x_0 - C \leq x \leq x_0, y_0 - C \leq y \leq y_0;$$

$$(x - x_0) + (y_0 - y) = C, x_0 \leq x \leq x_0 + C, y_0 - C \leq y \leq y_0;$$

Каждой x -координате этих прямых однозначно соответствует своя y -координата. Значит, перебрав $C + 1$ значение для каждого из четырех уравнений (всего $4C + 4$ значения), можно получить все координаты точек ромба (координаты точек, в которых прямые пересекаются, будут получены дважды).

Если нужно рассмотреть три вложенных ромба, количество действий будет около $12C$, где C — значение функции D для соседней клетки. $C \leq 2N$, так что общее число действий в этом решении должно быть пропорционально N^3 — для каждой из N^2 клеток нужно проверить не более $24N$ клеток. На самом деле в среднем операций получается значительно меньше. Алгоритм работает быстро, так как для больших значений C чаще всего все сводится к проверке $1 \leq x \leq N$ и $1 \leq y \leq N$.

Чтобы воспользоваться этим решением, нужно знать как минимум одно значение D . Предлагается находить его «в лоб» для первого элемента каждой строки.

Детали реализации — см. вариант 1.

Решение задачи 9 F. Прямоугольное деление

Вариант 1

Темы: палетка, заливка с явным стеком.

Пусть даны прямоугольники со сторонами, параллельными осям координат. Палеткой называется следующее построение: через стороны каждого прямоугольника проводятся прямые. Если взять любой образовавшийся в результате этого прямоугольник и любой исходный, то можно утверждать следующее: новый прямоугольник или полностью содержится в исходном, или имеет с ним пересечение нулевой площади (они не пересекаются, или соприкасаются сторонами, или соприкасаются углами). Это построение уже было сделано при решении задачи 6 D, правда, термин «палетка» там не использовался.

Что дает палетка в этой задаче? Очевидно, что внутри прямоугольников палетки линий нет. Линии могут идти только по границам прямоугольников палетки.

Утверждается следующее: между двумя соседними по стороне прямоугольниками палетки нет линии тогда и только тогда, когда совпадают множества исходных прямоугольников, которым принадлежат эти соседние прямоугольники палетки. Без доказательства.

Занесем в массив x все уникальные X -координаты исходных прямоугольников, в массив y — все уникальные Y -координаты исходных прямоугольников. Добавим в оба массива по два значения: $-10\,001$ и $10\,001$. Отсортируем массивы по возрастанию. Если изначально было не более 100 прямоугольников, то уникальных координат не более 200 и размер массивов x и y нужно взять 202. Обозначим n_x количество координат в массиве x и n_y — количество координат в массиве y .

Получается поле, состоящее из $(n_x - 1) \times (n_y - 1)$ клеток. Про любые две соседние по стороне клетки можно узнать, есть между ними граница или нет — если множества исходных прямоугольников, покрывающих эти клетки, совпадают, то границы нет, а если есть исходный прямоугольник, который покрывает одну из клеток, но не покрывает другую, то граница между клетками есть.

Применим заливку. Найдем не покрашенную клетку и закрасим всю 4-связную область, покрываемую тем же набором прямоугольников. Найдем очередную не

покрашенную клетку и снова сделаем эту операцию, и т. д. Ответом задачи является количество областей, для которых применялась заливка.

Детали реализации:

- В условии не сказано, что во входных данных даны координаты именно левого нижнего и правого верхнего углов. Если это не так, рекомендуется обработать такой случай при вводе.
- Можно подобрать пример: 100 исходных прямоугольников, 201×201 прямоугольников палетки, число прямоугольников палетки в одной 4-связной области более 30 000. Отсюда следует выбор алгоритма закраски. К сожалению, в этой и без того непростой задаче придется применять самый сложный из рассмотренных алгоритмов — из решения задачи 8 F.
- Не нужно хранить множества исходных прямоугольников, покрывающих каждый прямоугольник палетки, — для этого не хватит памяти Турбо Паскаля. Нахождение этих множеств заново при определении возможности перехода границы займет немного времени. Действительно, для каждого прямоугольника палетки это множество будет вычисляться не более пяти раз: один раз, если заливка области начинается с этого прямоугольника, и четыре раза, если проверяется наличие каждой из четырех границ.

Вариант 2

Решение предложил Ярослав Музыкантов.

Темы: палетка, заливка с явным стеклом.

Можно хранить сведения о том, существует ли каждая из четырех границ каждого прямоугольника палетки. Структура данных будет выглядеть так:

type

```
cell_data = set of (painted, isMinI, isMaxI, isMinJ, isMaxJ);
```

var

```
a : array [1..201, 1..201] of cell_data;
```

Множество из пяти элементов (`cell_data`) помещается в 1 байт, поэтому массив 201×201 элементов такого типа может быть объявлен в Турбо Паскале. Занести информацию о том, что клетка $[i, j]$ имеет левую границу, можно так:

```
include(a[i, j], isMinJ);
```

Узнать ту же информацию можно так:

```
if isMinJ in a[i, j] then
```

Чтобы инициализировать ячейку — не покрашенная, без всех четырех границ, — нужно записать:

```
a[i, j] := [];
```

В варианте 1 требуется порядка N^3 действий — палетка $2N \times 2N$, для каждой ячейки проверяется принадлежность всем N исходным прямоугольникам.

В этом варианте сложность составляет всего лишь N^2 . В нем нужно пройти по прямоугольникам палетки, лежащим на границе каждого из исходных прямоугольников, и установить элемент множества, есть ли верхняя, нижняя, левая или правая границы соответствующей ячейки массива. После этого при закраске для каждого прямоугольника палетки уже не требуется проверять принадлежность каждому из исходных прямоугольников, а можно использовать значение соответствующей ему ячейки.

Вариант 3

Решение нашли Сергей Шиловский и Федор Меньшиков.

Темы: теория графов, быстрая сортировка, заливка не на плоскости.

Для связного плоского (размещенного на плоскости без пересечений ребер) графа справедлива формула $V - E + F = 2$, где V — число вершин графа, E — число ребер графа, F — число частей, на которые граф разбивает плоскость. Для плоского графа, имеющего C компонент связности, формула выглядит так: $V - E + F - C = 1$. Таким образом, ответ задачи (F) будет найден, как только будут найдены три числа: V , E и C .

Будем считать вершинами графа вершины исходных прямоугольников, а также точки пересечения сторон исходных прямоугольников, не совпадающие с вершинами исходных прямоугольников. Точки пересечения несложно найти следующим образом. Прямоугольник рассматривается как совокупность четырех отрезков — его сторон: двух горизонтальных и двух вертикальных. Рассматриваются точки пересечения каждого горизонтального отрезка, полученного из исходных прямоугольников, с каждым вертикальным, при этом точка пересечения не должна лежать на конце отрезка.

Некоторые из полученных таким образом вершин графа могут совпадать, а нам нужно найти количество различных вершин. Оценим, сколько точек может получиться.

Существует пример, когда 100 прямоугольников пересекаются каждый с каждым, и у пары прямоугольников четыре точки пересечения. В этом случае всего в таблицу будет занесено $4 \cdot 100 + \frac{4 \cdot 100 \cdot 99}{2} = 20\,200$ вершин: $4 \cdot 100$ — количество вершин исходных прямоугольников, $\frac{4 \cdot 100 \cdot 99}{2}$ — количество пересечений каждого из

100 прямоугольников с остальными 99 в четырех точках (делим на 2 потому, что таким образом каждое пересечение учитывается два раза — со стороны каждого из прямоугольников).

Отсортируем массив с помощью быстрой сортировки по совокупности координат точки. Чтобы воспользоваться быстрой сортировкой, нужно чтобы сортируемые ключи можно было сравнивать. Но как сравнить *пары* координат? Введем лексикографический порядок. Сравним пары (x_1, y_1) и (x_2, y_2) . Если $x_1 < x_2$, то $(x_1, y_1) < (x_2, y_2)$. Если $x_1 > x_2$, то $(x_1, y_1) > (x_2, y_2)$. Если $x_1 = x_2$, то результат сравнения будет равен результату сравнения y_1 и y_2 .

Задавая отношение «равно», нужно убедиться в следующем:

1. Для всех a выполняется $a = a$.
2. Из $a = b$ следует $b = a$.

Задавая отношение «меньше», нужно убедиться в том, что выполняются следующие условия:

1. Ни для какого a не выполняется $a < a$.
2. Из $a < b$ следует, что не выполняется $b < a$.
3. Если $a < b$ и $b < c$, то $a < c$.

Предоставляем читателю самостоятельно убедиться в том, что отношение лексикографического порядка удовлетворяет требуемым аксиомам.

Функция сравнения выглядит так:

```
function cmp(x1, y1, x2, y2 : integer) : integer;  
begin  
    if x1 < x2 then  
        cmp := -1  
    else if x1 > x2 then  
        cmp := 1  
    else if y1 < y2 then  
        cmp := -1  
    else if y1 > y2 then  
        cmp := 1  
    else  
        cmp := 0;  
end;
```

Эта функция выдает -1 , если первая пара меньше второй, 1 — если она больше, и 0 — если пары равны. Используется она следующим образом: подразумевая сравнение $(x1, y1)$ отношение $(x2, y2)$, нужно написать: $\text{cmp}(x1, y1, x2, y2)$ отношение 0 .

После того как точки будут отсортированы, очень просто найти количество различных точек — оно равно количеству пар соседних в отсортированном массиве различных точек плюс один.

Итак, количество вершин V найдено. Найдем число ребер E . Каждое ребро соединяет две вершины. Очевидно, что из всех вершин выходит в сумме $2E$ ребер (каждое ребро учитывается дважды). Число ребер, выходящих из любой вершины полученного графа, которая не являлась вершиной исходных прямоугольников, 4 — такие вершины расположены на пересечении горизонтального и вертикального отрезков, причем не на конце ни того, ни другого.

Найдем число ребер, выходящих из вершин, которые были вершинами исходных прямоугольников. Из набора $4N$ вершин исходных прямоугольников удалим повторяющиеся. После этого для каждой точки найдем, выходит ли из нее ребро в каждом из четырех направлений: вверх, вниз, влево и вправо. Это можно сделать, пересчитав $2N$ горизонтальных и $2N$ вертикальных отрезков — сторон исходных прямоугольников.

Обозначим E_0 полученное таким образом количество ребер, выходящих из вершин исходных прямоугольников. Обозначим V_0 количество различных вершин исходных прямоугольников. Тогда получится следующая формула для нахождения количества ребер:

$$E = \frac{4(V - V_0) + E_0}{2}.$$

Осталось найти количество областей связности C . Представим следующий граф: вершины соответствуют исходным прямоугольникам, между вершинами есть ребро тогда и только тогда, когда прямоугольники имеют общую точку. Применим к этому графу заливку. Найдем первую не покрашенную точку, покрасим ее и все точки, связанные с ней. Найдем очередную не покрашенную точку и т. д. Значением C будет являться количество вызовов процедуры заливки извне ее.

var

colored : **array** [1..100] of boolean;

procedure paint(i : integer);

var

j : integer;

begin

if colored[i] then

exit;

colored[i] := true;

for j := 1 to rect_num do

if rect_common(rects[i], rects[j]) then

paint(j);

end;

procedure coherent(var count : integer);

var

i : integer;

begin

for i := 1 to rect_num do

colored[i] := false;

count := 0;

for i := 1 to rect_num do

if not colored[i] then **begin**

inc(count);

paint(i);

end;

end;

Процедура, определяющая, есть ли у прямоугольников общая точка, такова:

function rect_common(r1, r2 : rectangle) : boolean;

var

outside, inside12, inside21 : boolean;

```
begin
  outside :=
    (r2.xmax < r1.xmin) or (r1.xmax < r2.xmin) or
    (r2.ymax < r1.ymin) or (r1.ymax < r2.ymin);
  inside12 :=
    (r1.xmin < r2.xmin) and (r1.xmax < r2.xmax) and
    (r2.ymin < r1.ymin) and (r1.ymax < r2.ymax);
  inside21 :=
    (r1.xmin < r2.xmin) and (r2.xmax < r1.xmax) and
    (r1.ymin < r2.ymin) and (r2.ymax < r1.ymax);
  rect_common := not outside and not inside12 and not inside21;
end;
```

Общей точки у двух прямоугольников не будет, если один прямоугольник находится внутри или вне другого. Если прямоугольник находится строго внутри другого, его проекции на оси X и Y должны лежать строго внутри соответствующих проекций второго прямоугольника. Прямоугольники не пересекаются и расположены вне друг друга, если не пересекаются их проекции на ось X или Y .

Детали реализации:

- Объема основной памяти Турбо Паскаля для двух массивов длиной 20 200 из элементов типа `integer` недостаточно, поэтому нужно будет использовать динамическую память.
- В задаче производится не слишком много вычислений, проверки диапазонов лучше включить: $\{\$R+, Q+\}$.

Тренировка 10

Решение задачи 10 А. Анти-QuickSort

Темы: перебор $n!$, динамическая память ТР.

Чтобы сгенерировать худший случай для быстрой сортировки, нужно как минимум представлять себе работу этого алгоритма.

Алгоритм работает так. На отрезке массива, подлежащем сортировке, выбирается значение произвольного элемента (в процедуре, приведенной в условии, выбирается элемент из середины отрезка). После этого элементы массива переупорядочиваются так, что все значения, меньшие выбранного, оказываются в начале отрезка, а все значения, большие выбранного, — в конце. Таким образом, отрезок оказывается частично упорядоченным. После этого процедура быстрой сортировки применяется к каждому из этих отрезков.

Эффективность быстрой сортировки основана на том, что с очень высокой вероятностью отрезок делится на две почти равные части. Если бы отрезок делился ровно пополам, то длина сортируемых отрезков на каждом шаге уменьшалась бы в два раза. Таким образом, случай тривиального отрезка — из одного элемента — достигался бы за $\log_2 N$ делений.

В худшем случае одна из частей, на которые делится отрезок, имеет длину 1. Если при каждом делении отрезка одна из частей будет иметь длину 1, то количество делений для сортировки массива вместо $\log_2 N$ будет порядка N .

Отрезок длины 1 получается при делении, если выбранный ключ является самым маленьким или самым большим на интервале.

Приведенной информации достаточно, чтобы сгенерировать довольно-таки плохие случаи для быстрой сортировки, однако чтобы сгенерировать худший случай, нужно поэкспериментировать и найти закономерности, не предсказуемые теоретически.

Итак, чтобы разобраться в способе генерации худшего случая для произвольного N , для начала найдем решения для маленьких значений N . Сколько их нужно найти? Мое личное мнение, что для обнаружения закономерности вполне достаточно найти все худшие случаи до $N = 7$ (а перебрать худшие случаи только до $N = 4$ недостаточно).

Можно попытаться вручную перебрать все перестановки, для каждой вручную применить быструю сортировку и посчитать количество сравнений. К сожалению,

количество перестановок растет как $N!$. Последнее N , при котором еще возможен ручной перебор, — это 4. При $N=5$ получается уже 120 перестановок. Такое количество вручную перебирать очень долго, да и ошибиться можно.

К чему я веду? К тому, что для выработки идеи нужно написать программу. Программу, которая применяет быструю сортировку к каждой перестановке чисел от 1 до N ($N \leq 7$) и выдает перестановки, которые требуют наибольшего числа сравнений.

Алгоритм генерации всех перестановок обсуждался в решении задачи 2 В. Объявим два массива:

var

```
a, b : array [1..7] of integer;
```

В массиве b будем генерировать перестановки, потом копировать содержимое массива b в массив a , потом сортировать. Фактически, копирование массива b в массив a и вызов сортировки массива a нужно поставить на место, где в решении задачи 2 В находился вывод сгенерированной перестановки.

Теперь о том, как подсчитывать количество сравнений. Нужно объявить переменную целого типа, обнулять ее каждый раз, когда массив b копируется в a , и увеличивать на единицу при каждом выполнении цикла `while` и при каждом выходе из цикла `while`:

```
while a[i] < key do begin
    inc(count);
    inc(i);
end;
inc(count);
```

Счетчик внутри цикла учитывает, сколько раз условие цикла стало истинным, счетчик после цикла учитывает, сколько раз условие стало ложным, вследствие чего цикл завершился. В сумме как раз и получается количество сравнений.

Детали реализации:

- Мало найти идею генерации худшего случая быстрой сортировки. Эту идею еще нужно реализовать. Если программу генерации худшего случая нужно написать на Турбо Паскале (в условиях ограничений на размер массивов), реализация идеи еще больше увеличивает сложность задачи.

При нахождении перестановки худшего случая потребуется массив длиной N , каждый элемент которого может содержать значения от 1 до N . При $N = 70\,000$ все становится очень плохо. В Турбо Паскале один массив не может занимать более 64 Кбайт памяти, даже если он расположен в динамической памяти. И это уже не говоря о том, что под элемент нужно отводить более 2 байт.

Решением может быть следующая нетривиальная конструкция из нескольких массивов, лежащих в динамической памяти:

type

```
L15k = array [0..14999] of longint;
```

```

var
  a : array [0..4] of ^L15k;

  procedure put(index, value : longint);
  begin
    a[index div 15000]^[index mod 15000] := value;
  end;

  function get(index : longint) : longint;
  begin
    get := a[index div 15000]^[index mod 15000];
  end;

begin
  for i := 0 to 4 do
    new(a[i]);
  
```

Используя такую структуру, вместо записи $a[\text{index}] := \text{value}$ нужно писать $\text{put}(\text{index}, \text{value})$, а для получения значения $a[\text{index}]$ нужно писать $\text{get}(\text{index})$.

Решение задачи 10 В. Строки Фибоначчи

Темы: динамическое программирование, функции работы со строками.

Обозначим $C(K)$ количество вхождений искомой строки S в строку Фибоначчи $F(K)$, $K \geq 1$.

Следует ли из равенства $F(K) = F(K-1) + F(K-2)$ равенство $C(K) = C(K-1) + C(K-2)$? Нет, не следует. Не учтены вхождения S на границе $F(K-1)$ и $F(K-2)$.

Учесть их не слишком сложно — нужно для каждой строки Фибоначчи хранить достаточно длинный кусок ее начала и достаточно длинный кусок ее конца. На сколько длинным должен быть кусок? Когда S накладывается на соединение двух строк Фибоначчи, в каждой из них лежит не более $\text{length}(S) - 1$ символа. Значит, зная $\text{length}(S) - 1$ символ конца $F(K-1)$ и $\text{length}(S) - 1$ символ начала $F(K-2)$, можно найти количество вхождений S в $F(K)$ на стыке $F(K-1)$ и $F(K-2)$.

Обозначим $\text{Head}(K)$ начало $F(K)$ длиной $\text{length}(S) - 1$. Обозначим $\text{Tail}(K)$ конец $F(K)$ длиной $\text{length}(S) - 1$. Обозначим $\text{Entries}(\text{substr}, \text{str})$ количество вхождений substr в str .

Формула перехода в общем виде такова:

$$\begin{aligned} \text{Entries}(S, F(K)) &= \text{Entries}(S, F(K-1)) + \\ &+ \text{Entries}(S, \text{Tail}(K-1) + \text{Head}(K-2)) + \text{Entries}(S, F(K-2)). \end{aligned}$$

$\text{Entries}(S, F(K))$ ранее было обозначено $C(K)$. С учетом этого формула перехода будет выглядеть так:

$$C(K) = C(K-1) + C(K-2) + \text{Entries}(S, \text{Tail}(K-1) + \text{Head}(K-2)).$$

Head и *Tail* из этой формулы находятся так:

$$Head(K) = Head(K - 1);$$

$$Tail(K) = Tail(K - 2).$$

Итак, формулы перехода получены. Но с каких значений начать? В ходе рассуждений предполагалось, что *Head(K)* и *Tail(K)* — строки длиной $\text{length}(S) - 1$ — целиком содержатся в *F(K)*. Следовательно, строка *F(K)* не может быть слишком маленькой.

По условию задачи $\text{length}(S) \leq 25$. К сведению: $\text{length}(F(9)) = 34$, $\text{length}(F(10)) = 55$ (значения получены экспериментально). Значит, если в условии задано $N \leq 10$, нужно построить строку по определению и найти количество вхождений явно. А если $N > 10$, то нужно построить строки *F(9)* и *F(10)* по определению, найти количество вхождений в них строки *S*: *C(9)* и *C(10)*, — а также найти начало и конец соответствующей длины строк *F(9)* и *F(10)*, после чего по приведенным формулам строить *C(K)*, *Head(K)* и *Tail(K)* для последующих *K*.

Детали реализации:

- Получить начало строки *s* длиной *len* в Турбо Паскале очень просто при помощи функции *copy*:

```
function getHead(s : string; len : integer) : string;
begin
    getHead := copy(s, 1, len);
end;
```

- Получить конец строки *s* длиной *len* в Турбо Паскале сложнее, потому что в *copy* указывается, с какого места нужно начинать вырезать, а не где заканчивать:

```
function getTail(s : string; len : integer) : string;
begin
    getTail := copy(s, length(s) - len + 1, len);
end;
```

Можно написать еще такой, более «ошибкоустойчивый», вариант: удалять первый символ строки до тех пор, пока длина не окажется равной требуемой.

```
function getTail(s : string; len : integer) : string;
begin
    while length(s) > len do
        delete(s, 1, 1);
    getTail := s;
end;
```

Обратите внимание на следующий момент: для того чтобы эта функция не изменяла исходные данные, нужно, чтобы параметр *s* был передан по значению, а не по ссылке.

- Найти количество вхождений можно следующим образом. В Турбо Паскале есть функция *pos* — позиция первого вхождения подстроки. Будем находить

первое вхождение и удалять часть строки от начала до первой буквы вхождения включительно. Так будем продолжать делать до тех пор, пока очередное вхождение не будет найдено (результат `pos` равен 0):

```
function entries(substr, str : string) : integer;
var
    count, p : integer;
begin
    count := 0;
    while true do begin
        p := pos(substr, str);
        if p = 0 then begin
            entries := count;
            exit;
        end;
        inc(count);
        delete(str, 1, p);
    end;
end;
```

- Не рекомендуется в этой задаче обходиться без массивов — например, при построении строк Фибоначчи $F(K)$, $K \leq 10$, не нужно пытаться обойтись тремя переменными. Использование массивов максимально приближает запись программы к теоретическим выкладкам, а расход памяти на массивы небольшой длины вполне приемлем. Итак, рекомендуемые глобальные объявления:

```
var
    f : array [1..10] of string;
    c : array [1..45] of longint;
    head, tail : array [1..45] of string;
```

- В задаче производится не слишком много вычислений, так что проверки диапазонов лучше включить: $\{\$R+.Q+\}$.
- Одна из самых распространенных ошибок, допускаемых при решении этой задачи, — перепутанный порядок сложения строк в формуле $F(K)$.

Решение задачи 10 С. Игра в зачеркивание

Тема: антагонистические игры.

Попытаемся применить здесь тот же подход, что и к решению других задач, описывающих антагонистические игры, — выделим *все* позиции игры и на основе того, из каких позиций можно перейти в какие, установим, является позиция выигрышной или проигрышной.

Оценим количество разных позиций рассматриваемой игры. Прежде всего следует учесть, что игра полностью определяется набором длин полосок пустых смежных клеток, неважно, в каком порядке идут эти наборы. Сумма длин полосок

не превосходит N . Неожиданно приходим к почти такой же задаче, как задача 4 В. Если запустить ее решение на максимальном тесте ($N = 40$), то можно найти, что в этом случае существует около 40 000 разложений на слагаемые.

Итак, одна оценка у нас уже есть. Позиций в игре не миллионы и не миллиарды, их вполне реально рассмотреть все. Другое дело, что если их десятки тысяч, их будет трудно поместить в память Турбо Паскаля. Попытаемся улучшить оценку.

Заметим, что больше всего разложений получается, когда $K = 1$. Почему? Когда $K = 2$, можно считать, что полосок длиной 1 нет — все равно их нельзя вычеркнуть, они так и останутся до конца игры. В общем случае можно считать, что нет полосок длиной меньше K . Еще одна оптимизация: если полоска имеет длину от K до $2K - 1$, то она эквивалентна полоске длиной K — после первого же вычеркивания придем к полоске длиной меньше K . Нетрудно заметить, что эти оптимизации можно применить только при $K > 1$.

Рассмотрим случай $K = 1$ отдельно. Каждый игрок зачеркивает по одной клетке. Очевидно, неважно, в каком месте он зачеркивает — имеет значение только количество клеток. Если пустых клеток нет, нужно выводить 0, если их нечетное число, нужно выводить 1, если четное — 2.

Итак, случай $K = 1$ рассмотрен отдельно. Сколько же существует игровых ситуаций при $K = 2$? Менее 3500. Если бы автор этого не сообщил, вы могли бы найти то же самое экспериментально, но общие тенденции уже прослеживались — разложений с единицами десятки тысяч, единицы в разложениях встречаются очень часто.

Выработаем способ хранения игровой ситуации:

type

```
situation = record
  amount : byte;
  len : array [1..13] of byte;
end;
```

Поле `amount` содержит количество участков пустых смежных клеток, массив `len` в элементах с 1-го по `amount`-й содержит их длины.

Рассмотрим, почему была выбран размер массива 13. Худший случай — $N = 40$, в группах по две пустые клетки, группы разделяются одной зачеркнутой клеткой. Если длину такой комбинации (2 пустые + 1 зачеркнутая клетка) умножить на 13, то получится, что не задействована еще одна клетка. Но группа из одной клетки эквивалентна отсутствию группы.

Чтобы две ситуации можно было сравнивать, нужно их привести к единому виду. Для того чтобы это сделать, нужно, во-первых, заменить значения элементов `len` от 1 до $K - 1$ нулем, а значения от $K + 1$ до $2K - 1$ — значением K . Во-вторых, нужно отсортировать значения, скажем, в порядке невозрастания — в `len[1]` должно содержаться самое большое значение, в `len[2]` меньшее и т. д. В-третьих, нужно привести `amount` в соответствие с количеством ненулевых элементов `len` (нулевые элементы могли появиться при замене чисел от 1 до $K - 1$ нулем).

В решении потребуется функция сравнения двух ситуаций на равенство:

```
function equal(var s1, s2 : situation) : boolean;
```

Так как в Паскале сравнивать записи нельзя, нужно описать понятие равенства самостоятельно: две ситуации равны, когда равно количество полосок пустых клеток, а также равны значения соответствующих элементов упорядоченных массивов.

Сгенерируем все ситуации игры. Для их хранения воспользуемся массивом:

```
var
```

```
  table : array [1..3500] of situation;  
  tabptr, tablen : integer;
```

Переменная `tablen` содержит количество занесенных в массив `table` ситуаций.

Генерировать ситуации будем следующим образом. Сначала занесем в `table` заданную во входных данных ситуацию. Затем будем подряд рассматривать ситуации таблицы — `table[1]`, `table[2]` и т. д., для каждой из них генерировать все ситуации, в которые можно попасть непосредственно из нее, и добавлять в массив те, которых там еще не было. Переменная `tabptr` хранит индекс текущей рассматриваемой ситуации.

Проверять наличие ситуации в таблице будем линейным поиском — сравнивать новую ситуацию со всеми находящимися в таблице. Такой алгоритм нельзя назвать эффективным, однако в требуемое время он укладывается, а более быстрый алгоритм, использующий двоичное дерево, требует в полтора раза больше времени на реализацию.

Заканчивать генерировать ситуации нужно при условии `tabptr > tablen`.

После того как все ситуации сгенерированы, заполним таблицу выигрышности ситуаций:

```
var
```

```
  wins : array [1..3500] of integer;  
  {0 - неизвестно, 1 - делающий ход выигрывает, -1 - проигрывает}
```

Заполним ее с помощью рекурсивной процедуры

```
  function canWin(index : integer) : boolean;
```

Если к моменту вызова функции `canWin` значение `wins[index]` определено, то есть не равно нулю, она возвращает соответствующий результат — при `wins[index] = 1` — true, при `wins[index] = -1` — false.

Пусть значение выигрышности позиции на момент вызова функции не определено (`wins[index] = 0`). Рассмотрим тривиальный случай. Если есть одна полоска пустых клеток (`amount = 1`) и длина этой полоски не превосходит $3K - 2$, то можно зачеркнуть K клеток из середины и выиграть игру.

Нетривиальный случай. Генерируются все непосредственные потомки ситуации, и функция вызывается рекурсивно с параметром, соответствующим каждой из ситуаций-потомков. Выигрышность вычисляется обычным способом — если все ситуации, в которые можно перейти из данной, являются выигрышными, то данная — проигрышная, а если есть хотя бы одна проигрышная ситуация, в которую можно перейти из данной, то данная является выигрышной.

Ответ задачи — выигрывает игрок 1 или 2 — определяется по значению `canWin` с параметром 1 — исходная ситуация была занесена в таблицу первой. Ответ 0 определяется сразу же после чтения исходной ситуации и ее нормализации — поле `amount` в этом случае должно быть равно нулю.

Решение задачи 10 D. Граница многоугольника

Темы: алгоритм Евклида, тип `comp`.

Прочитав слова «алгоритм Евклида», вы, паверно, уже поняли, как решать эту задачу. Рассмотрим прямоугольник со сторонами, параллельными осям координат, и целочисленными координатами вершин. Число точек с целочисленными координатами на его диагонали равно $1 + \text{НОД}$ (наибольший общий делитель, `greatest common divisor`, `gcd`) длин сторон. Если рассмотреть замкнутую ломаную без самопересечений, то в ней таким способом каждая из вершин будет посчитана дважды, поэтому конечная формула для многоугольника — просто сумма НОД.

Детали реализации:

- Все данные одновременно считать в память не получится, но это и не страшно — нужны всего лишь пары соседних точек, точно так же, как в варианте 1 решения задачи 4 D.
- Следует учитывать ограничения на входные данные алгоритма Евклида. Этот алгоритм находит НОД только для положительных чисел, так что в качестве аргумента нельзя подставлять просто разности координат соседних точек — нужно разность еще взять по модулю. Также, вообще говоря, особым является случай, когда какая-то сторона многоугольника является горизонтальной или вертикальной. Очевидно, найти число точек на этой стороне не составляет труда — оно равно длине отрезка (опять же $+1$, если рассматривать один отрезок, или без прибавления единицы, если рассматривать общую формулу). Однако оказывается, что стандартный алгоритм нахождения НОД (см. детали реализации варианта 1 решения задачи 7 A) выдает правильный для решения этой задачи ответ, даже когда один из аргументов равен 0.
- В то время как для вычисления количества целых точек на одной стороне многоугольника типа `longint` вполне достаточно, это не является справедливым для ответа задачи. Найдем максимальное значение, которое может стать ответом задачи. Обозначим dx и dy модули разности соответственно X - и Y -координат соседних точек. Из ограничений на величину координат следует, что dx и dy не могут превышать $2 \cdot 10^9$. НОД не превосходит любой из своих аргументов, таким образом, одна сторона может внести не более $2 \cdot 10^9$ точек в общую сумму. Сторон у многоугольника, по условию задачи, не более 100 000. Таким образом, ответ задачи можно оценить сверху числом $2 \cdot 10^{14}$. Очевидно, это число не может быть представлено переменной типа `longint`, а вот тип `comp` подойдет в самый раз.

- Не следует относить все переменные задачи к типу `comp`, так как для переменных этого типа не определена операция `mod`, и самостоятельно ее реализовать труднее, чем сделать типом суммы `comp`, а типом остальных переменных — `longint`.

Решение задачи 10 Е. Путь спелеолога

Тема: поиск в ширину.

Рассмотрим, чем эта задача отличается от задач 6 Е и 7 Е. Да почти ничем. Вместо двумерного поля — трехмерное, вместо одной конкретной ячейки, в которую нужно привести спелеолога, — верхний уровень ячеек. Ну и самое приятное — не нужно строить путь. Вот, собственно говоря, и все отличия, поэтому неудивительно, что задача решается аналогично вышеупомянутой.

Карту пещеры представим трехмерным массивом из целых. В этом массиве в ячейки, соответствующие пустотам, занесем значение `free` (-1), в ячейки, соответствующие камням, — значение `rock` (-2), в ячейку, соответствующую исходному положению спелеолога, — 0 . Неотрицательное число в таблице будет означать число шагов на кратчайшем пути до исходного положения спелеолога.

После чтения исходных данных все свободные клетки, соседние с доступными из исходного положения за 0 шагов, помечаем значением 1 . Потом все свободные клетки, соседние с доступными за 1 шаг, помечаем значением 2 . Так продолжаем до тех пор, пока не исчезнут свободные клетки, в которые можно попасть из начального положения. Ответом задачи будет минимальное из неотрицательных значений клеток, соответствующих верхнему уровню пещеры.

Детали реализации:

- Выберем тип элементов трехмерного массива. Чтобы не искать путь назад, придется в каждой ячейке записывать количество движений, которое содержит минимальный путь от нее до начального положения спелеолога. Думаю, вы сами можете представить карту пещеры, в которой существует путь на поверхность длиной более 255 , то есть 8 -разрядные целые не подходят. Всего в пещере $30^3 = 27\,000$ ячеек, так что размера 16 -разрядного целого типа со знаком вполне достаточно.
- К сожалению, барьер в этой задаче использовать не удастся. Дело в том, что массив $30 \times 30 \times 30$ из 16 -разрядных целых входит в основную память Турбо Паскаля, а массив $32 \times 32 \times 32$ — уже нет. Поэтому, проверяя соседей клетки, придется писать проверки:

```
if (i > 1) and (a[i - 1, j, k] = free) then
...
...
if (k < n) and (a[i, j, k + 1] = free) then
...
```

- Какой алгоритм предпочесть — из задачи 6 Е или 7 Е — с очередью или без? К сожалению, придется использовать более сложный алгоритм с очередью.

Одним из худших случаев входных данных этой задачи является горизонтальная «змейка» через уровень. Длина пути примерно равна количеству клеток, деленному на 4. При использовании алгоритма без очереди будет выполнено $\frac{27\,000^2}{4}$ (примерно 200 млн) элементарных действий. Учитывая то, что элементарное действие — это несколько индексирований трехмерных массивов, можно прийти к выводу о необходимости использования алгоритма с очередью. В случае же применения очереди количество действий пропорционально числу клеток.

Решение задачи 10 F. Дырявая ткань

Тема: заливка.

Рассмотрим более простую задачу. Гарантируется, что во входных данных задан один кусок ткани. Требуется найти количество дыр в нем. Например:

```

.....
..*****..
..**.*.**..
..**.*.**..
*****
..**.*.**..
..**.*.**..
..*****..
.....

```

Что мешает пройти по этой матрице и, встретив точку, закрасить всю 4-связную область точек и увеличить счетчик дыр? Ответ: области в углах. Они не являются дырами, но при описанном методе подсчета увеличивают счетчик.

Избавиться от этой проблемы очень просто — нужно слить все эти области в одну. Требуется только гарантия, что фигура отстоит по крайней мере на одну клетку от границы поля, например:

```

.....
..*****..
..**.*.**..
..**.*.**..
*****
..**.*.**..
..**.*.**..
..*****..
.....

```

Теперь все значительно лучше, не правда ли? Количество дыр можно найти как количество областей точек минус один.

Итак, подзадача решена. Осталось только свести основную задачу к этой подзадаче. Это тоже не слишком сложно. Нужно вместо одного двумерного массива работать с двумя. В первом будет содержаться исходная карта поля. Обходим его в любом

порядке и, встретив звездочку, копируем всю 4-связную область звездочек во второй массив, в первом заполняем это место точками. Теперь гарантируется, что во втором массиве ровно один кусок ткани. Обойти все его дыры труда не составляет. После обхода дыр находим следующую область звездочек в первом массиве, очищаем второй массив для копирования этой области (детали см. далее), копируем область во второй массив, обходим дыры и так далее для каждой области звездочек первого массива.

Детали реализации:

- Размер поля по условию задачи не превышает 100×100 . Следовательно, может быть кусок ткани размером 100×100 ; может быть дыра 98×98 . Отсюда следует выбор алгоритма закраски — рекурсией без параметров (см. решение задачи 7 F).
- Копировать связную область из массива `a1` в массив `a2` можно рекурсивно параллельно с заливкой:

```
var
  paint2i, paint2j : integer;

procedure paint2;
begin
  if a1[paint2i, paint2j] <> '*' then
    exit;
  a1[paint2i, paint2j] := '.';
  a2[paint2i, paint2j] := '*';

  inc(paint2i); paint2; dec(paint2i);
  dec(paint2i); paint2; inc(paint2i);
  inc(paint2j); paint2; dec(paint2j);
  dec(paint2j); paint2; inc(paint2j);
end;
```

- Следует иметь в виду, что нельзя при определении количества дыр всегда использовать второй массив целиком. Рассмотрим такой случай. Размер поля 100×100 , клетки закрашены в шахматном порядке. Получается 5000 кусков ткани. Каждый из них перенесем во второй массив и для каждого выполним заливку всего пустого места во втором массиве. Во втором массиве каждый раз будет около 10 000 свободных клеток. Итого получается порядка 50 миллионов элементарных действий. Одно элементарное действие будет включать рекурсивный вызов процедуры и несколько индексирований двумерных массивов. В отведенное время такое решение не уложится.

Решить проблему можно следующим образом. К каждому куску ткани нужно подходить индивидуально, учитывая его размер. Если кусок большой, то и места ему отводить много, а если маленький — отводить ему мало места и строить барьер вокруг этого маленького места. Например, возьмем такие входные данные:

```
. * .
. . .
****
****
```

Чтобы лучше соотнести эту схему со схемой второго массива, дополним ее двумя рядами фиктивных значений с каждой стороны:

```
XXXXXXXX
XXXXXXXX
XX . * . XX
XX . . . XX
XX **** XX
XX **** XX
XXXXXXXX
XXXXXXXX
```

Итак, переносим верхнюю область во второй массив и строим вокруг нее барьер:

```
. ***** .
. * . * .
. * * * .
. * . * .
. ***** .
. . . . .
. . . . .
. . . . .
```

Все операции заливки с целью нахождения числа дыр будем производить не во всем втором массиве, а только в выбранном огороженном прямоугольнике. Таким образом, даже если областей будет много, но они будут мелкими, количество действий в целом будет не слишком большим.

А вот как выглядит второй массив при перенесении нижней области:

```
. . . . .
. . . . .
*****
* . . . . *
* . **** . *
* . **** . *
* . . . . *
*****
```

Осталось выяснить один вопрос — как эффективно определять размер куска ткани. Делается это рекурсивно в ходе заливки:

```
var
  paintli, paintlj: integer;
  mini, minj, maxi, maxj: integer;
```

```
procedure paint1;  
begin  
  if a1[paint1i, paint1j] <> '*' then  
    exit;  
  a1[paint1i, paint1j] := '#';  
  
  if paint1i < mini then  
    mini := paint1i;  
  if paint1i > maxi then  
    maxi := paint1i;  
  if paint1j < minj then  
    minj := paint1j;  
  if paint1j > maxj then  
    maxj := paint1j;  
  
  inc(paint1i); paint1: dec(paint1i);  
  dec(paint1i); paint1: inc(paint1i);  
  inc(paint1j); paint1: dec(paint1j);  
  dec(paint1j); paint1: inc(paint1j);  
end;
```

Эта процедура перекрашивает область звездочек в область решеток, параллельно находя минимальные и максимальные координаты области. Перед вызовом процедуры нужно инициализировать переменные максимума и минимума:

```
mini := paint1i;  
maxi := paint1i;  
minj := paint1j;  
maxj := paint1j;  
paint1;
```

После нахождения минимальных и максимальных координат области нужно очистить во втором массиве прямоугольник пространства необходимого размера и создать вокруг него барьер:

```
for i := mini - 2 to maxi + 2 do  
  for j := minj - 2 to maxj + 2 do  
    a2[i, j] := '*';  
for i := mini - 1 to maxi + 1 do  
  for j := minj - 1 to maxj + 1 do  
    a2[i, j] := '.';
```

И в эту чистую область необходимого размера можно переносить область решеток.

Тренировка 11

Решение задачи 11 А. Последовательность

Казалось бы, в этой задаче нет ничего сложного. Всё решение пишется в десять строчек:

```
var
  a, n, i : longint;
...
readln(a, n);
for i := 2 to n do
  a := sqr(a) mod 10000;
writeln(a);
```

Вся сложность задачи заключена в совокупности ограничений: N до 2 000 000 000, а времени всего 2 с. Цикл до 2 миллиардов на указанном компьютере за 2 с не выполнится. Сомнительно даже, что он выполнится за 20 с.

Вариант 1

Каждый элемент последовательности имеет значение от 0 до 9999, то есть всего 10 000 различных значений. Рассмотрим начало последовательности длиной 10 001. По принципу Дирихле¹, среди этих 10 001 элемента есть два одинаковых.

Пусть $a_j = a_k$ ($j < k$, j и k от 1 до 10 001). По условию задачи, значение a_{i+1} зависит только от значения a_i . Значит, из того, что $a_j = a_k$, следует, что $a_{j+1} = a_{k+1}$, $a_{j+2} = a_{k+2}$ и вообще $a_{j+q} = a_{k+q}$ для любого натурального q (строгое доказывается по индукции).

Применим полученное равенство: $a_k = a_{j+(k-j)} = a_{k+(k-j)}$. Заметим, что $k-j > 0$. Значит, найден еще один элемент с индексом, бóльшим, чем j и k , значение которого равно значению a_j и a_k . По индукции можно доказать, что все элементы вида $a_{j+p(k-j)}$ равны a_j для любого натурального p .

Таким образом, значения a_j присутствуют в последовательности как минимум через каждые $k-j$ членов, начиная с j -го. Опять, используя то, что значение a_{i+1} зависит только от значения a_i , получаем $a_{j+p(k-j)+q} = a_{j+q}$ для любых натуральных p и q . Для любого индекса N , бóльшего, чем j , разложение $N = j + p(k-j) + q$ единственно, если на q наложить условие $0 \leq q < k-j$.

¹ Встретив в техническом тексте незнакомый термин, просто пропустите его — текст сохраняет свой смысл и без него.

Получаем следующий алгоритм: если заданный индекс N достаточно большой, то нужно найти в начале последовательности два одинаковых элемента — $a_j = a_k$ ($j < k$), из равенства $N = j + p(k - j) + q$ получить q ($0 \leq q < k - j$), после чего можно в качестве ответа выдавать a_{j+q} .

Простейший способ нахождения двух одинаковых элементов среди первых 10 001 состоит в сравнении каждого элемента с каждым. Это около 50 миллионов сравнений. Рассмотрим более быстрый способ.

Для начала убедимся, что не всегда можно взять $j = 1$. Например, пусть $a_1 = 100$, тогда $a_i = 0$ ($i > 1$). Среди элементов последовательности с индексами, большими 1, не найдется элемента со значением a_1 . Что же нам помешало? То, что в начале последовательности может быть ациклическая часть.

Заметим, что ациклическая часть не может быть слишком большой. Как было показано ранее, циклическая часть последовательности начинается уже среди первых 10 000 элементов. Возьмем $j = 10\,001$. По определению последовательности $a_i = (a_{i-1})^2 \bmod 10\,000$ найдем 10 001-й ее член. Затем запомним значение $a_{10\,001}$ и все значения следующих элементов последовательности будем сравнивать с этим запомненным значением. Когда значение совпадет, его индекс и обозначим k . Так как период последовательности не превосходит 10 000, то $k \leq 20\,001$.

Итак, алгоритм в целом таков. Если $N \leq 10\,000$, то найдем a_N по определению. В противном случае найдем по определению $a_{10\,001}$ и, вычисляя значения следующих членов последовательности, минимальный индекс $k > 10\,001$ такой, что $a_k = a_{10\,001}$. После этого нужно найти решение уравнения $N = 10\,001 + p(k - 10\,001) + q$ такое, что $0 \leq q < k - 10\,001$. Решение находится элементарно: $q = (N - 10\,001) \bmod (k - 10\,001)$. Найденное таким образом значение q используем для нахождения по определению значения $a_{10\,001+q}$, которое равно искомому a_N .

Вариант 2

Прежде всего отбросим случай, когда искомое число лежит в ациклической части последовательности. Если $N \leq 10\,000$, вычислим a_N по определению последовательности и завершим работу.

В массиве

var

pos : array [0..9999] of integer;

будем хранить номер места в последовательности, на котором впервые встретилось число. Если i — число, то $\text{pos}[i]$ — позиция его первого вхождения (нумерация членов последовательности с единицы). Сначала заполним массив значениями -1 , затем будем заносить в него данные, пока не окажется, что информация о значении очередного построенного элемента уже была занесена в массив. Пусть k — номер этого элемента, a_k — его значение, тогда в ациклической части последовательности содержится $\text{pos}[a_k] - 1$ значение, а длина периода равна $k - \text{pos}[a_k]$.

Приведем задачу к следующему виду. «Вычтем» из всех индексов последовательности число $\text{pos}[a_k]$. Таким образом, у новой последовательности период начина-

ется с нулевого элемента (не с первого, чтобы удобно было использовать `mod`), и нужно найти значение элемента с номером $N - \text{pos}[\text{ak}]$. Так как искомый элемент лежит в периодической части, то его значение равно значению элемента «сдвинутой» последовательности с номером $(N - \text{pos}[\text{ak}]) \bmod (k - \text{pos}[\text{ak}])$, а это значение можно найти по определению.

Решение задачи 11 В. Провода

Тема: метод половинного деления в целых числах.

В задаче требуется по количеству равных отрезков найти их длину. Воспользуемся тем, что обратная задача (определение количества равных отрезков заданной длины) решается очень просто. Применим метод половинного деления.

Для того чтобы применить метод половинного деления, достаточно монотонности функции. Убедимся в наличии монотонности, а именно: покажем, что если длина равных отрезков, на которые нужно порезать исходные, увеличится, то их количество не увеличится. Утверждение почти очевидное. Докажем от противного: пусть количество равных отрезков увеличилось. Тогда можно получить ровно то же количество меньших отрезков, сначала разрезав исходные отрезки на длинные, а потом укоротив их.

Итак, монотонность функции количества отрезков от их длины показана. Используем метод половинного деления. Возьмем такие первоначальные значения: `good := 0`; — K отрезков длиной 0 нарезать можно всегда, `bad := 10000001`; — K отрезков длиной 10 000 001 нарезать нельзя, потому что даже одного отрезка такой длины нарезать нельзя — все исходные отрезки имеют меньшую длину.

Цикл половинного деления:

```
while good + 1 < bad do begin
    mid := (good + bad) div 2;
    if count(mid) < k then
        bad := mid
    else
        good := mid;
end;
```

Здесь функция `count` выдает количество отрезков равной заданной длины, на которые можно порезать исходные отрезки. Если можно нарезать K отрезков длиной `mid`, то увеличиваем нижнюю границу, если нельзя — уменьшаем верхнюю. Когда цикл завершится, будет выполняться `good + 1 = bad`, то есть K отрезков длиной `good` нарезать можно, а на единицу большей длины — нельзя.

Решение задачи 11 С. Палиндромы

Тема: динамическое программирование.

Обозначим i -й символ исходной строки $S[i]$, подстроку с i -го по j -й символ обозначим $S[i..j]$. Заполним таблицу $C(1...60, 1...60)$. $C(F, L)$ означает, сколько может по-

лучиться палиндромов из подстроки исходной строки, в которую входят буквы с F -й по L -ю включительно. Нетрудно заметить, что значения некоторых элементов массива (у которых $F > L$) не имеют смысла. Значения таких элементов использоваться не будут, поэтому на их существование можно не обращать внимания.

Будем заполнять таблицу в порядке увеличения $L - F$. Для $L = F$ заполняем $C(F, L) = 1$ — из однобуквенной строки может получиться только один палиндром. Небольшим перебором можно убедиться в том, что для строк длиной 2 (у которых $L = F + 1$) выполняется или $C(F, L) = 2$ (если $S[F] \neq S[L]$), или $C(F, L) = 3$ (если $S[F] = S[L]$).

Таблицу для больших длин будем заполнять на основе информации о значениях для меньших длин.

Пусть концевые буквы рассматриваемой подстроки разные: $S[F] \neq S[L]$. Тогда они обе не могут использоваться при получении палиндрома (первая и последняя буква палиндрома одинаковы). Если вычеркнуть $S[F]$, то получится подстрока $S[F + 1 \dots L]$. Очевидно, число способов получить из нее палиндром равно $C(F + 1, L)$. Аналогично, если вычеркнуть $S[L]$, количество полученных палиндромов будет равно $C(F, L - 1)$. Однако если из $S[F + 1 \dots L]$ вычеркнуть последнюю букву, то получится точно такая же строка, как если бы из $S[F \dots L - 1]$ вычеркнули первую букву. Таким образом, количество палиндромов, получающихся из $S[F + 1 \dots L - 1]$, подсчитывается два раза.

Итак, если $S[F] \neq S[L]$, то $C(F, L) = C(F + 1, L) + C(F, L - 1) - C(F + 1, L - 1)$.

Если же концевые буквы рассматриваемой подстроки равны: $S[F] = S[L]$, то возможны все варианты вычеркивания, какие возможны при $S[F] \neq S[L]$, кроме того, возможен вариант, когда концевые буквы не вычеркиваются. Количество палиндромов, получающихся в этом случае, равно $C(F + 1, L - 1) + 1$ — все варианты получить палиндром из подстроки $S[F + 1 \dots L - 1]$ плюс вариант ее полного вычеркивания.

Итак, если $S[F] = S[L]$, то $C(F, L) = C(F + 1, L) + C(F, L - 1) + 1$.

Ответом задачи является $C(1, N)$.

Детали реализации:

- Чтобы рассматривать подстроки в порядке возрастания длин (начиная с трех), рекомендуется использовать следующую конструкцию:

```
for len := 3 to n do
  for f := 1 to n - len + 1 do begin
    l := f + len - 1;
    ...
  end;
```

- Если строка состоит из одинаковых букв, палиндром получается из нее при вычеркивании любого множества букв (кроме вычеркивания всех букв). Таким образом, для строки длиной N из одинаковых букв ответом задачи является $2^N - 1$. Для $N = 60$ ответ будет равен $2^{60} - 1$, так что результат может быть представлен переменной типа `comp` (переменные типа `comp` могут принимать целые значения до $2^{63} - 1$).

- Следует учитывать одну особенность реализации Турбо Паскаля. Проблема заключается в том, что процедура `writeln` выводит не более 18 первых цифр значений вещественных типов (а `comp` формально является вещественным). Например, число $2^{60} - 1 = 1152921504606846975.0$ при использовании формата «: 0: 0» будет выведено как 1152921504606846980. Решением является «ручной» перевод числа в строку. Использовать `str` нельзя, так как, похоже, и `writeln`, и `str` используют один и тот же метод получения изображения числа.

Под словами «ручной перевод» подразумевается следующее: цифры числа получаются, начиная с младшей, с помощью аналогов `div 10` и `mod 10` для чисел типа `comp` (см. детали реализации решения задачи 11 F), цифры преобразуются в символы, а символы добавляются слева к строке.

Решение задачи 11 D. Круговая площадь

Тема: геометрия.

Вариант 1

По координатам центров окружностей найдем расстояние d между ними. После этого координаты центров окружностей не нужны — задача решается однозначно, если известны r_1 , r_2 и d .

Для начала рассмотрим (и исключим из дальнейшего рассмотрения) случаи, когда площадь пересечения нулевая или равна площади меньшего круга. Круги имеют нулевую площадь пересечения из-за того, что находятся слишком далеко друг от друга, в случае $d \geq r_1 + r_2$. Центры кругов находятся слишком близко друг к другу, так что площадь пересечения равна площади меньшего круга, в случае $d \leq |r_1 - r_2|$. Во всех остальных случаях *окружности* имеют ровно две точки пересечения (рис. P11.1).

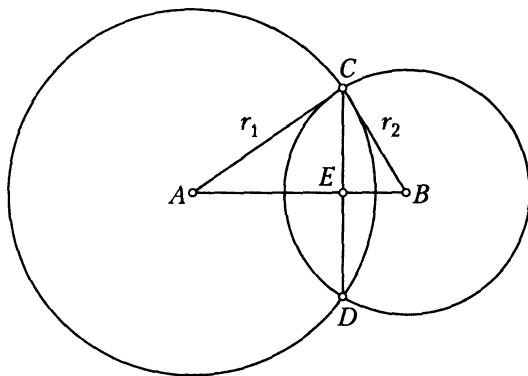


Рис. P11.1. Две пересекающиеся окружности

Обозначим A — центр первой окружности, B — центр второй окружности, C, D — точки пересечения окружностей, E — точка пересечения отрезков AB и CD . Соблюдаются следующие соотношения: $AC = AD = r_1$, $BC = BD = r_2$, $AB = d$, $CE = ED$, AB и CD пересекаются под прямым углом.

Площадь пересечения кругов равна сумме площадей сегментов ACD и BCD . Площадь сегмента находится как разность площадей соответствующих сектора и треугольника. Площадь сектора находится из пропорции $\frac{S_c}{\pi r^2} = \frac{\alpha}{2\pi}$. Площадь равнобедренного треугольника со стороной r и углом при вершине α находится по формуле $S_{\text{тр}} = \frac{1}{2} r^2 \sin \alpha$. Таким образом, площадь сектора выражается через радиус круга и угол.

Радиусы кругов известны, осталось только найти углы ($\angle CAD = 2\angle CAB$ и $\angle CBD = 2\angle CBA$).

У прямоугольных треугольников ACE и BCE общая сторона, так что для них справедливо равенство (применяется теорема Пифагора) $AC^2 - AE^2 = BC^2 - BE^2$. В этом равенстве известны $AC = r_1$ и $BC = r_2$. Также известно, что $AE + BE = AB$. Пусть $AB = d$. Обозначив x длину отрезка AE , получаем уравнение $r_1^2 - x^2 = r_2^2 - (d - x)^2$, приводящееся к линейному.

Итак, AE и BE известны. По теореме Пифагора находим CE , $\angle CAE = \arctg \frac{CE}{AE}$, $\angle CBA = \arctg \frac{CE}{BE}$.

Детали реализации:

- Стандартная функция $\arctan$ Турбо Паскаля возвращает угол в диапазоне $\left(-\frac{\pi}{2}; \frac{\pi}{2}\right)$, а в этой задаче углы CAB и CBA могут лежать в диапазоне $(0; \pi)$. Формула приведения следует из равенства $\text{tg } \alpha = \text{tg}(\alpha + \pi)$. Таким образом, если функция $\arctan$ выдала отрицательное значение, нужно к нему добавить π .
- Случай прямого угла ($AE = 0$ или $BE = 0$) следует рассмотреть отдельно, в противном случае при вычислении по общей формуле произойдет деление на ноль.

Вариант 2

Решение предложил Никита Рыбак.

Углы CAB и CBA (см. рис. 11.1) можно найти и по-другому. В треугольнике ABC известны три стороны, так что по теореме косинусов можно найти косинус любого угла:

$$BC^2 = AB^2 + AC^2 - 2 \cdot AB \cdot AC \cdot \cos \angle CAB;$$

$$AC^2 = AB^2 + BC^2 - 2 \cdot AB \cdot BC \cdot \cos \angle CBA.$$

Детали реализации:

- В Турбо Паскале нет функции $\arccos$, есть только $\arctan$. Можно сначала по значению косинуса найти значение синуса (синус угла из диапазона $(0; \pi)$ положителен), затем по этим двум значениям найти значение тангенса, к которому и применить функцию $\arctan$.
- Как и в варианте 1, результат $\arctan$ нужно привести к нужному интервалу.
- Как и в варианте 1, отдельно нужно рассмотреть случай прямого угла (косинус равен нулю).

Решение задачи 11 Е. Гомер Симпсон

Решение предложил Алексей Ивченко.

Тема: сокращение перебора.

Количество гамбургеров, съеденных за время T , может лежать в пределах от 0 до $T \div N$, а количество чизбургеров — от 0 до $T \div M$. Ограничения задачи (T — до миллиона, M и N — от единицы) таковы, что перебрать все пары количества гамбургеров и чизбургеров за разумное время нельзя.

Но перебирать все пары и не требуется. Пусть количество гамбургеров зафиксировано и равно H . Тогда на поедание чизбургеров и дополнительное время остается всего $T - HN$ миллисекунд. В условии сказано, что меньшее количество дополнительного времени предпочтительнее, чем большее суммарное количество съеденных гамбургеров и чизбургеров. Поэтому, если на чизбургеры и свободное время приходится всего $T - HN$ секунд, то количество чизбургеров и свободного времени отсюда находится однозначно — как $(T - HN) \div M$ и $(T - HN) \bmod M$ соответственно.

Итак, пришли к следующему решению: перебирать все возможные количества гамбургеров от 0 до $T \div N$, для каждого количества гамбургеров определять количество чизбургеров и свободного времени, затем сравнивать найденные значения с текущими оптимальными.

Детали реализации: для перебора значений H лучше всего использовать цикл `for`, так как в нем значения границ вычисляются один раз.

Решение задачи 11 F. Дробная арифметика

В том, чтобы выполнить операцию над двумя дробями, нет ничего сложного или необычного. Основная сложность задачи заключается в правильном чтении исходных данных и записи полученных результатов.

Рекомендуется считать строку с дробью в переменную типа `string`. Если первый символ — минус, запомнить это и удалить его из строки. Далее нужно рассмотреть три варианта: 1) у дроби есть и целая, и дробная части; 2) есть только целая часть; 3) есть только дробная часть. Эти случаи легко разделить, используя в качестве первого аргумента стандартной функции поиска подстроки `pos` пробел (раздели-

тель целой и дробной частей) и знак деления (разделитель числителя и знаменателя). Из исходной строки нужно выделить все числа с помощью стандартной функции получения подстроки `сору`, используя результаты поиска символов `' '` и `'/'`. Сразу же после прочтения следует дробь сделать неправильной, то есть содержащей только числитель и знаменатель. Знак удобно хранить в отдельной целой переменной, принимающей значения $+1$ и -1 .

Следует заметить, что операция вычитания эквивалентна операции сложения с числом противоположного знака, а операция деления эквивалентна операции умножения на обратное. Таким образом, если задана операция « $-$ », можно символ операции заменить символом « $+$ », одновременно сменив знак второго операнда. А если задана операция « $/$ », можно символ операции заменить символом « $*$ », обменяв числитель и знаменатель второго операнда.

Учесть знак операндов при реализации операции сложения легче всего, если перед выполнением операции числители домножить на знаки операндов, а после операции выделить знак результата, сравнивая получившийся числитель с нулем. При сложении не нужно приводить дроби к минимальному общему знаменателю — легче всего привести их к знаменателю, равному произведению знаменателей операндов.

Умножение производится естественным образом: перемножаются отдельно числители, отдельно знаменатели, отдельно знаки.

При выводе нужно сначала проверить, не равна ли получившаяся дробь нулю. Как правило, при выводе данных в естественном формате (дроби, многочлены) случай равенства нулю является особым. Если числитель получившейся неправильной дроби равен нулю, нужно вывести соответствующий ответ и завершить работу. После проверки на ноль нужно проверить дробь на отрицательность и, если необходимо, вывести знак минуса.

Удовлетворяя требованию задачи, нужно сделать дробь несократимой, применяя алгоритм Евклида (см. детали реализации варианта 1 решения задачи 7 A). Обратите внимание на то, что алгоритм Евклида рассчитан на положительные операнды, именно поэтому числитель и знаменатель полученной неправильной дроби должны быть положительными, а знак дроби должен содержаться в отдельной переменной. Сделать дробь правильной, выделив целую часть, не составляет труда с помощью `div` и `mod`. Еще один аргумент в пользу того, что знак результата должен храниться в отдельной переменной, — это то, что у числителя правильной дроби не должно быть знака, а он появился бы, если бы он был у числителя неправильной дроби.

При выводе опять нужно рассмотреть случаи наличия у дроби только целой части, только дробной части, а также и целой, и дробной частей. Обратите внимание на то, что лишние символы (в том числе и пробелы) недопустимы, то есть пробел должен стоять, только если есть и целая, и дробная части.

Детали реализации:

- Рассмотрим, какой тип необходим для корректного представления промежуточных результатов. При преобразовании входных данных в неправильную

дробь может получиться числитель до $30\,000^2 + 30\,000$, знаменатель не превосходит 30 000. При выполнении операции сложения в числителе может оказаться значение немногим более $2 \cdot 30\,000^3$. При выполнении операции умножения в числителе может оказаться значение немногим более $30\,000^4$. Таким образом, размера типа `comp` должно хватить.

- Единственная сложность, связанная с использованием типа `comp`, состоит в том, что для него (как формально вещественного типа) не определены операции `div` и `mod`, необходимые как для эффективной реализации алгоритма Евклида, так и для получения целой части и числителя правильной дроби-результата. С помощью вещественных операций `div` и `mod` можно выразить так:
 - Операция `a div b` для операндов типа `comp` может быть записана следующим образом: `int(a / b)`. Поясним: `a / b` дает точный результат деления, `int` дает целую часть этого результата, в совокупности получается целочисленное деление — `div`.
 - Соотношение между операциями `div` и `mod` такое: `a = (a div b) * b + (a mod b)`. Отсюда легко получить формулу для `a mod b`: `a - int(a / b) * b`.

Заметьте, что использовать `trunc` вместо `int` нельзя — если результат будет больше `maxlongint`, то произойдет ошибка времени выполнения.

Тренировка 12

Решение задачи 12 А. Последовательность (2)

Вариант 1

Решение в целом подобно решению задачи 11 А. Различие заключается в том, что в той последовательности периодически повторялись сами значения элементов, а в этой — значения троек элементов.

Если каждая цифра может принимать значения от 0 до 9 независимо от других, то разных троек цифр может быть 1000. Таким образом, среди первой 1001 тройки цифр найдутся две одинаковые: $(d_j, d_{j+1}, d_{j+2}) = (d_k, d_{k+1}, d_{k+2})$ такие, что $1 \leq j < k \leq 1001$.

Как и в решении задачи 11 А, можно вывести формулу $d_{j+p(k-j)+q} = d_{j+q}$, где $0 \leq q < k - j$. Таким образом, для решения задачи нужно из уравнения $N = j + p(k - j) + q$ найти $q = (N - j) \bmod (k - j)$ и по определению найти d_{j+q} , которое равно искомому d_N .

Еще одно отличие от задачи 11 А состоит в том, что в последовательности троек цифр нет ациклической части. Покажем, что если известны значения трех последовательных цифр d_{i+1}, d_{i+2} и d_{i+3} , то по ним значение предыдущей цифры d_i определяется однозначно. Прямая формула $(d_i + d_{i+1} + d_{i+2}) \bmod 10 = d_{i+3}$ задана в условии задачи. С учетом того, что d_{i+3} — десятичная цифра, эта формула эквивалентна $(d_i + d_{i+1} + d_{i+2}) \bmod 10 = (d_{i+3}) \bmod 10$. К левому операнду $\bmod$ каждой части добавим $10 - d_{i+1}$ и $10 - d_{i+2}$. Получится $(d_i + d_{i+1} + d_{i+2} + 10 - d_{i+1} + 10 - d_{i+2}) \bmod 10 = (d_{i+3} + 10 - d_{i+1} + 10 - d_{i+2}) \bmod 10$ или, что то же самое, $d_i \bmod 10 = (d_{i+3} + 20 - d_{i+1} - d_{i+2}) \bmod 10$. Учитывая, что d_i — десятичная цифра, можно убрать $\bmod$ из левой части: $d_i = (d_{i+3} + 20 - d_{i+1} - d_{i+2}) \bmod 10$. Итак, если известны значения d_{i+1}, d_{i+2} и d_{i+3} , значение d_i определяется однозначно.

В терминах арифметики остатков доказательство записывается еще проще: формулы $d_i + d_{i+1} + d_{i+2} = d_{i+3} \pmod{10}$ и $d_i = d_{i+3} - d_{i+1} - d_{i+2} \pmod{10}$ эквивалентны. В формуле, записанной в терминах Турбо Паскаля, требуется использовать число 20, так как по правилам Турбо Паскаля $-13 \bmod 10 = -3$, а совсем не 7, как в арифметике остатков.

Как можно использовать отсутствие ациклической части? Очень просто: можно зафиксировать $j = 1$ и находить минимальное $k \geq 2$ такое, что $(d_1, d_2, d_3) = (d_k, d_{k+1}, d_{k+2})$. Так как $k \leq 1001$ (то есть довольно малó), то соответствующее начало последовательности можно найти по ее определению.

Детали реализации (тип `comp`):

- Ограничения задачи таковы, что значение N не может быть представлено ни одним из целочисленных типов Турбо Паскаля, но может быть точно представлено типом `comp`. С этим связана проблема: для типа `comp` не определена операция `mod`, используемая в формуле $q = (N - 1) \bmod (k - 1)$.

Реализация `div` и `mod` для операндов типа `comp` описана в деталях реализации решения задачи 11 F.

- Проследите за тем, чтобы ваша программа выдавала правильный ответ при $1 \leq N \leq 3$.

Вариант 2

Решение предложил Виктор Сергеевич Губа.

Если бы требовалось много раз быстро находить значение элемента по заданным трем первым цифрам последовательности и номеру элемента, то особенно эффективным было бы следующее решение.

Рассмотрим три последовательности: $1\ 0\ 0\dots, 0\ 1\ 0\dots$ и $0\ 0\ 1\dots$. Обозначим N -е элементы этих последовательностей соответственно d_{100N}, d_{010N} и d_{001N} . Пусть требуется найти N -й элемент последовательности, начинающейся цифрами a, b, c . Утверждается, что его значение равно $(ad_{100N} + bd_{010N} + cd_{001N}) \bmod 10$. Без доказательства.

Как быстро найти значения d_{100N}, d_{010N} и d_{001N} ? В варианте 1 решения доказано, что последовательность, описанная в задаче, не имеет ациклической части. Значит, достаточно просто запомнить в трех массивах элементы первого цикла последовательностей с началом $1\ 0\ 0, 0\ 1\ 0$ и $0\ 0\ 1$. Размер каждого из циклов не может превышать 1000 по принципу Дирихле. Если известно содержимое циклов последовательностей с началом $1\ 0\ 0, 0\ 1\ 0$ и $0\ 0\ 1$, то значение d_{100N}, d_{010N} или d_{001N} находится очень просто — как элемент соответствующего массива с индексом $N \bmod \text{период_последовательности}$.

Следует только иметь в виду, что операция `mod` может выдать результат 0, так что для обработки этого случая массивы должны содержать элемент с индексом 0, значение которого должно быть равно значению элемента с номером `период_последовательности`.

Решение задачи 12 В. Гирлянда

Изучая ответ для примера задачи ($N = 8, A = 15$), можно подумать, что в этом случае построенная гирлянда касается земли ($H_5 = 0$). На самом деле 9,75 — это ответ, округленный до двух знаков после запятой. То есть если получится значение 9,750 000 001, то оно будет округлено до 9,75. А в случае ответа 9,750 000 001 ни одна из лампочек земли не касается.

Из этих рассуждений можно понять, что ограничения на высоту лампочек $H_i > 0$ и $H_i \geq 0$ эквивалентны. Действительно, пусть какая-то лампочка касается земли ($H_i = 0$). Чтобы удовлетворялись условия исходной задачи, поднимем эту лампочку

ку на 10^{-100} мм. В результате ответ изменится на какую-то малую долю миллиметра. Конечно, он может быть округлен до следующего числа с двумя знаками после запятой, но погрешность ответа 0,01 в этой задаче допустима.

Вариант 1

Тема: метод половинного деления в вещественных числах.

Заметим, что высоту каждой лампочки гирлянды можно узнать очень просто, если известны высоты двух соседних лампочек. Высота первой лампочки дана ($H_1 = A$). Соседствует с ней только вторая лампочка.

Заметим монотонность такой функции: чем ниже расположена вторая лампочка, тем ниже расположена самая нижняя из лампочек. И наоборот, чем выше расположена вторая лампочка (при зафиксированной высоте первой), тем выше расположена самая нижняя из лампочек. Без доказательства.

Возьмем два положения второй лампочки: $bad := 0$; — если вторая лампочка расположена на высоте 0, то вся гирлянда касается земли, $good := a$; — если вторая лампочка расположена на высоте первой, то высота каждой следующей лампочки будет все больше, таким образом, гирлянда земли не касается.

Цикл половинного деления:

```
while true do begin
  mid := (good + bad) / 2;
  if (mid = good) or (mid = bad) then
    break;
  if isAboveGround(mid) then
    good := mid
  else
    bad := mid;
end;
```

Если соседние целые числа различаются на единицу, то как же определить, что вещественные числа являются соседними? Нужно найти их среднее. Если оно будет округлено до какого-то из исходных (к которому именно, зависит от режима округления сопроцессора), то между исходными нет других вещественных чисел.

Следует заметить, что приведенная запись цикла является абсурдной с точки зрения математики абсолютно точных значений. Если $good$ и bad различны, то их полусумма никогда не будет равна ни тому, ни другому. Однако в условиях конечного множества вещественных чисел (а у встроенных вещественных типов компьютера множество значений конечное) полусумма двух чисел оказывается равной одному из этих чисел, если числа соседние.

Функция `isAboveGround` должна определять, правда ли то, что гирлянда с такой заданной высотой второй лампочки полностью висит над землей.

По окончании цикла в переменной `good` будет содержаться высота второй лампочки искомого варианта. Найти высоту последней лампочки не составит труда.

Детали реализации: очевидно, предпочтителен вещественный тип с максимальной точностью — extended.

Вариант 2

Решение предложил Никита Рыбак.

Тема: метод половинного деления в целых числах.

Если известна высота двух лампочек (не обязательно соседних), то можно решить систему уравнений и найти высоту всех лампочек.

Половинным делением будем искать номер лампочки, касающейся земли (почему условия $H_i > 0$ и $H_i \geq 0$ эквивалентны, говорилось ранее). Изначально границы поиска таковы: $\text{left} := 2$; — первая лампочка закреплена на ненулевой высоте, поэтому только вторая может касаться земли, $\text{right} := n$; — последняя лампочка может касаться земли. Основной цикл:

```
while true do begin
  mid := (left + right) div 2;
  if HL(mid) < 0 then begin {лампочка слева от mid ниже уровня земли}
    right := mid - 1;
    continue;
  end;
  {с лампочкой слева все в порядке}

  if mid = n then
    break; {лампочка самая правая - ответ найден}
  if HR(mid) < 0 then begin {лампочка справа от mid ниже уровня земли}
    left := mid + 1;
    continue;
  end;
  {лампочки и слева, и справа в порядке - ответ найден}

  break;
end;
```

Функции HL и HR по номеру лампочки, касающейся земли, определяют высоту соседней от нее соответственно слева или справа лампочки. Если лампочка слева находится ниже уровня земли, то в оптимальном варианте расположения гирлянды лампочка, касающаяся земли, имеет номер меньший, чем mid, а если лампочка справа находится ниже уровня земли, то в оптимальном варианте лампочка, касающаяся земли, имеет номер больший, чем mid. Отдельным случаем является $\text{mid} = n$ — если земли касается самая правая лампочка, то не нужно проверять высоту лампочки справа от нее.

Вариант 3

Тема: метод половинного деления в вещественных числах.

В предыдущем варианте решения было замечено, что по номерам и высотам двух любых лампочек можно, решив систему уравнений, найти высоты всех

остальных. Основываясь на этом факте, можно поиск вести по высоте B последней лампочки.

Значение нижней границы поиска предложить легко — лампочка не может быть ниже уровня земли, а вот с верхней границей все не так просто. Предлагается найти верхнюю границу (high) следующим образом:

```
low := 0;
high := 1;
while not isAboveGround(high) do begin
    low := high;
    high := high * 2;
end;
```

Следует заметить, что функция `isAboveGround` отличается от функции с тем же названием из варианта 1. В варианте 1 параметром была высота второй лампочки, здесь же параметром является высота последней лампочки.

Верхняя граница все время увеличивается в два раза, так что даже для очень больших значений B верхняя граница поиска будет найдена всего за несколько десятков шагов. Параллельно с изменением верхней границы корректируем и нижнюю. Вообще-то этого можно и не делать.

Цикл половинного деления записывается точно так же, как и в варианте 1.

Это решение, конечно же, хуже, чем первое, — и систему уравнений решать надо, и верхняя граница неопределенная. Но оно показывает, что делать в случае неопределенной верхней границы.

Решение задачи 12 С. Головоломка умножения

Тема: динамическое программирование.

Обозначим $A[i]$ число на i -й ($1 \leq i \leq N$) карте. Обозначим $A[l...r]$ последовательность чисел на картах с l -й по r -ю включительно. В частности, $A[1...N]$ — вся исходная последовательность.

Будем решать исходную задачу для всех подпоследовательностей $A[l...r]$ в порядке увеличения их длины. Обозначим $M[l, r]$ минимальные очки, которые можно получить, оставив от последовательности $A[l...r]$ две карты, а остальные убрав по правилам, заданным в условии. В частности, $M[1, N]$ — ответ задачи.

Для последовательностей длиной 2 ($r = l + 1$) результат $M[l, r]$ равен 0 (ничего убирать нельзя).

Для последовательностей большей длины будем перебирать, какая карта была взята последней. Обозначим ее номер m . При удалении последней карты в сумму добавляется $A[l] A[m] A[r]$, так как перед этим в ряду оставались только 3 карты — l -я, m -я и r -я именно в таком порядке. До m -й карты убираются карты последовательностей $A[l...m]$ и $A[m...r]$. Заметим, что эти последовательности убираются независи-

модруг от друга, при этом минимально возможными суммами являются $M[l, m]$ и $M[m, r]$ соответственно.

Таким образом, если последней на интервале $A[l...r]$ была убрана карта с номером m , то минимальная набираемая сумма составляет $A[l] A[m] A[r] + M[l, m] + M[m, r]$. Перебрав возможные положения последней карты ($m = l + 1...r - 1$) и для каждого вычислив это выражение, можно найти $M[l, r]$ как минимум из этих значений.

Решение задачи 12 D. Точки в многоугольнике

Тема: геометрия.

Число точек I , лежащих строго внутри многоугольника, просто выражается через площадь многоугольника S и число точек с целочисленными координатами на его границе B . Формула¹ имеет вид $I = k_1 S + k_2 B + k_3$, где k_1, k_2 и k_3 — вещественные числа. Эти числа можно найти, решив систему уравнений, получающуюся при подстановке данных практически любых трех многоугольников.

О том, как найти площадь многоугольника и число точек с целочисленными координатами на его границе, можно узнать из разборов задач 4 D и 10 D соответственно.

Решение задачи 12 E. Водопровод

Определим, в скольких местах может заканчиваться труба, если можно использовать не более 10 (максимум C_i) отрезков одинаковой длины. Если графически представить все возможные места окончания трубы, получится ромб с 21 точкой на каждой диагонали. Всего в таком ромбе 221 точка, то есть набор из 10 одинаковых отрезков дает 221 возможное положение места окончания трубы.

Если есть K таких наборов отрезков, то, перебирая все комбинации, получим 221^K мест. Некоторые из них могут совпасть, но для перебора это не имеет значения, так как способ заранее легко отбросить повторяющиеся варианты автору неизвестен.

Таким образом, для $K = 3$ объем перебора составит $221^3 = 10\,793\,861$ вариант, что находится на границе числа действий, которые указанный компьютер способен выполнить за отведенное время. А вот уже для $K = 4$ объем перебора составит более 2 миллиардов, что совершенно немыслимо выполнить в течение заданного времени.

Вариант 1

Тема: сокращение перебора.

Будем перебирать только использование отрезков наборов с первого по $(K - 1)$ -й. Пусть с помощью первого $K - 1$ набора трубу, начинающуюся в (x_1, y_1) , удалось довести до точки (x_i, y_i) . Тогда для завершения строительства потребуется $|x_2 - x_i| \div \text{div } L_K + |y_2 - y_i| \div \text{div } L_K$ отрезков труб K -го вида. Если это число не превосходит C_K ,

¹ Формула Пика.

то найдено одно из возможных решений, но только при условии, что $|x_2 - x_t| \bmod L_K = 0$ и $|y_2 - y_t| \bmod L_K = 0$.

Вариант 2

Тема: сокращение перебора.

Сначала произведем перебор по x -координате, то есть переберем количество и направление (влево или вправо) горизонтальных отрезков. Как и в варианте 1, количество горизонтальных отрезков K -го вида определяется однозначно по количеству взятых отрезков видов с 1-го по $(K - 1)$ -й. Если получить горизонтальную составляющую трубы нельзя, то незачем перебирать и вертикальную составляющую.

Для фиксированного варианта набора горизонтальных отрезков переберем все варианты расположения вертикальных отрезков, имея при этом в виду то, что количество вертикальных отрезков вида i ограничено не числом C_i , а числом C_i за вычетом числа горизонтальных отрезков того же вида. Как и для x -координаты, для y -координаты не нужно перебирать положения отрезков K -го вида — их число и расположение следуют из выбора расположения отрезков всех предыдущих видов.

Решение задачи 12 F. Химические реакции

Тема: метод рекурсивного спуска.

Расширенная форма Бэкуса–Наура (РБНФ) — это форма записи правил того, каким может быть выражение. Рассмотрим ее на примерах.

`<слово1> ::= "a" ["b"|"c"]`

Слева от знака `::=` находится название понятия, справа — вид выражений, удовлетворяющих этому понятию. Вертикальная черта означает выбор одного из вариантов, квадратные скобки означают необязательность того, что в них заключено. Таким образом, понятию `<слово1>` соответствуют три слова: «a» (если то, что описано в квадратных скобках, отсутствует), «ab» (если то, что описано в квадратных скобках, присутствует, и из «b» и «c» выбрано первое) и, наконец, «ac».

`<слово2> ::= {"a"|"b"}`

Фигурные скобки означают повторение 0 или более раз. Таким образом, понятию `<слово2>` соответствуют все слова (включая пустое), состоящие из последовательностей букв «a» и «b», например: «aabb», «aba», «a», «».

`<псв> ::= ["(" <псв> ")"] | <псв> <псв>]`

Слово в угловых скобках, встретившееся в правой части правила, означает, что вместо него можно подставить любую последовательность символов, соответствующую этому понятию. Понятие `<псв>` описывает все правильные скобочные выражения. Действительно, пустая строка является правильным скобочным выражением (возможна благодаря квадратным скобкам), кроме того, правильное скобочное выражение может быть получено заключением в скобки правильного скобочного выражения или записыванием подряд двух правильных скобочных выражений.

Возьмем выражение $((()))((()))()$. Его можно рассматривать как записанные подряд два скобочных выражения: $((()))$ и $((()))()$. В свою очередь, $((()))$ можно представить как выражение $(())$ в скобках, а $((()))()$ можно представить как записанные подряд $(())()$ и $()$, и т. д.

Более удобная для практического применения эквивалентная формула выглядит так:

```
<псв> ::= {" "<псв> " }
```

Символы в кавычках называются *терминалами* («окончательными»), так как они содержатся в разбираемой строке, а символы в угловых скобках называются *нетерминалами*, так как вместо них подставляется соответствующее правило.

Перейдем к вопросу о том, как разобрать последовательность символов, удовлетворяющую заданному набору правил. Символы строки рассматриваются по одному слева направо, так что имеет смысл хранить текущий символ в глобальной переменной s и использовать вызов процедуры `nextChar` для получения значения следующего символа:

```
var
  s : string;      {строка, подлежащая разбору}
  pos : integer;   {текущая позиция в строке}
  c : char;        {текущий символ}

procedure init(var s0 : string);
begin
  s := s0;
  pos := 0;
end;

procedure nextChar;
begin
  if pos < length(s) then begin
    inc(pos);
    c := s[pos];
  end
  else
    c := #0;      {признак того, что строка закончилась}
  end;
end;
```

В программе разбора все конструкции РБНФ довольно прямолинейным образом переводятся в конструкции языка программирования.

Для каждого правила пишется своя процедура. Например, в этой задаче потребуются написать процедуры `formula` (формула), `sequence` (последовательность), `element` (элемент), `chem` (химический элемент) и `number` (число). Процедур для понятий прописной и строчной буквы создавать не нужно, так как это просто множества символов. Когда процедура вызывается, текущим символом является первый сим-

вол разбираемого понятия. Когда процедура завершает работу, текущим становится первый символ после разобранного понятия.

Квадратным скобкам РБНФ соответствует оператор `if`, фигурным — `while`, вертикальной черте соответствует `if — else`. Записанному подряд соответствует последовательность операторов.

Напишем процедуры, только разбирающие искомое понятие, но не производящие никаких вычислений.

Правилу `<число> ::= "1".."9" {"0".."9"}` соответствует процедура

```
1: procedure number;
2: begin
3:   if not (c in ['1'..'9']) then
4:     halt(1);
5:   nextChar;
6:   while c in ['0'..'9'] do
7:     nextChar;
8:   end;
```

Если текущий символ — не цифра от 1 до 9, эта процедура вызываться не должна. Для самоконтроля написана проверка, вызывающая аварийное завершение при несоблюдении этого условия (строки 3–4). В строке 5 пропускается первая цифра числа (здесь уже известно, что она от 1 до 9). Повторение 0 или более раз в РБНФ обозначается фигурными скобками, а в программе на Паскале — циклом `while`.

Правилу `<химический элемент> ::= <прописная буква> [<строчная буква>]` соответствует процедура

```
1: procedure chem;
2: begin
3:   if not (c in ['A'..'Z']) then
4:     halt(1);
5:   nextChar;
6:   if c in ['a'..'z'] then
7:     nextChar;
8:   end;
```

Если эта процедура вызвана, то текущим символом должна быть прописная латинская буква (строки 3–4). Первая буква пропускается (строка 5), затем, если за ней следует строчная латинская буква (строка 6), то пропускается и она (строка 7). Выполнение 0 или 1 раз в РБНФ обозначается квадратными скобками, а в программе на Паскале — оператором `if`.

Правилу `<элемент> ::= <химический элемент> | "(" <последовательность> ")"` соответствует процедура

```
procedure element;
begin
  if c = '(' then begin
    nextChar;
```

```

sequence;
if c <> ')' then
    halt(1);
nextChar;
end
else
    chem;
end;

```

Выбор из двух вариантов в РБНФ обозначается вертикальной чертой, а в программе на Паскале — с помощью `if — else`. Если текущий символ — открывающая круглая скобка, то должен быть выбран вариант последовательности в скобках, в противном случае перед нами химический элемент. Упоминание нетерминала в формуле РБНФ в программе на Паскале соответствует вызову процедуры разбора этого нетерминала.

Правилу РБНФ `<формула> ::= [<число>] <последовательность> { "+" [<число>] <последовательность> }` соответствует процедура

```

procedure formula;
begin
    if c in ['1'..'9'] then
        number;
    sequence;
    while c = '+' do begin
        nextChar;
        if c in ['1'..'9'] then
            number;
        sequence;
    end;
end;

```

В этой процедуре также все стандартно — квадратные скобки РБНФ соответствуют оператору `if`, фигурные — `while`, упоминание нетерминала соответствует вызову процедуры, отвечающей за это понятие.

Заметим, что в формулах РБНФ может присутствовать рекурсия:

```

<последовательность> ::= <элемент> [<число>] {<элемент> [<число>]}
<элемент> ::= <химический элемент> | "(" <последовательность> ")"

```

то есть из процедуры `sequence` (последовательность) нужно вызывать процедуру `element` (элемент), и наоборот, из процедуры `element` нужно вызывать процедуру `sequence`. Для описания взаимно рекурсивных процедур в Паскале используются опережающие (`forward`) объявления:

```

procedure sequence; forward;

procedure element;
begin
    ...
end;

```

```

procedure sequence;
begin
    ...
end;

```

Итак, процедуры синтаксического анализа понятия «формула» написаны, осталось только вставить в них семантическую обработку тех данных, которые они пропускают через себя.

Объявим тип для хранения количества каждого из химических элементов, встретившихся в формуле или ее части:

```

type
    Table = ....

```

Для этого типа нужно реализовать следующие операции:

- Создание таблицы, содержащей один химический элемент, заданный одной (hi) или двумя (hi и lo) буквами:

```

procedure create1(hi : char; var t : Table);

```

```

procedure create2(hi, lo : char; var t : Table);

```

Процедура chem получает имя химического элемента и создает с помощью create1 или create2 таблицу, содержащую один этот элемент.

- Увеличение количества каждого из элементов, содержащихся в таблице t, в n раз:

```

procedure mul(var t : Table; n : integer);

```

Используется в процедурах formula и sequence для применения соответственно, коэффициентов и индексов.

- Добавление элементов, содержащихся в таблице t2, к элементам, содержащимся в таблице t1:

```

procedure add(var t1, t2 : Table);

```

Используется в процедурах formula и sequence для сложения наборов элементов двух фрагментов формулы.

- Проверка равенства двух таблиц:

```

function equal(var t1, t2 : Table) : boolean;

```

Используется в основной программе, когда уже найдены количества элементов в двух формулах и нужно выдать ответ, совпадают ли эти количества.

Теперь напишем процедуры с семантической обработкой. Перед этим приведем заданный в условии набор правил к эквивалентному, чуть более удобному, виду:

<формула> ::= <число> <последовательность> {"+" <число> <последовательность>}

<последовательность> ::= <элемент> <число> {<элемент> <число>}

<элемент> ::= <химический элемент> | "(" <последовательность> ")"

<химический элемент> ::= "A".."Z" ["a".."z"]

<число> ::= ["1".."9" {"0".."9"}]

По сравнению с начальным набором правил изменилось следующее: исчезли понятия прописной и строчной латинских букв, а квадратные скобки, всегда окружавшие нетерминал <число>, переместились в само правило.

Приведем процедуры с семантической обработкой:

```
{
  Процедура пропускает число и возвращает его значение
  в выходном параметре n. Если числа нет, выдает 1
}
procedure number(var n : integer);
begin
  if c in ['1'..'9'] then begin
    n := ord(c) - ord('0');
    nextChar;
    while c in ['0'..'9'] do begin
      n := n * 10 + (ord(c) - ord('0'));
      nextChar;
    end
  end
  else
    n := 1;
end;

{
  Процедура пропускает обозначение химического элемента
  и возвращает в выходном параметре t таблицу, содержащую
  один этот элемент
}
procedure chem(var t : Table);
var
  hi, lo : char;
begin
  if not (c in ['A'..'Z']) then
    halt(1);
  hi := c;
  nextChar;
  if c in ['a'..'z'] then begin
    lo := c;
    nextChar;
    create2(hi, lo, t);
  end;
  else
    create1(hi, t);
end;
```

```
{
  Процедура пропускает "последовательность" и возвращает
  в выходном параметре t таблицу с количеством элементов
  в этой "последовательности"
}
procedure sequence(var t : Table);
var
  t2 : Table;
  n : integer;
begin
  element(t);
  number(n);
  mul(t, n);
  while c in ['A'..'Z', '('] do begin
    element(t2);
    number(n);
    mul(t2, n);
    add(t, t2);
  end;
end;
```

Процедуры `formula` и `element` записываются в аналогичном стиле.

Обратите внимание на то, что химические формулы, которые будут даны вашей программе во входных данных, должны удовлетворять описанным правилам, однако в них не обязаны содержаться обозначения реальных химических элементов. В частности, разных элементов, использованных во всех формулах, может быть больше, чем количество известных сейчас химических элементов.

Тренировка 13

Решение задачи 13 А. Двойная решетка

Тема: последовательности.

Решение предложил Александр Меркулов.

Рассмотрим последовательность $\{D_{x,i}\}$ — расстояние от последовательных вертикальных линий второй решетки до ближайшей слева вертикальной линии первой решетки. Первый член последовательности задан в условии задачи: $D_{x,0} = D_x$. Формула перехода $D_{x,i} = (D_{x,i-1} + x_2) \bmod x_1$, то есть к предыдущему смещению добавляем шаг второй решетки x_2 и приводим смещение к интервалу $[0; x_1)$ — если смещение больше или равно x_1 , значит, та линия первой решетки, от которой это смещение отсчитывается, не ближайшая слева.

Покажем, что у этой последовательности нет ациклической части. Вообще-то это следует из соображений симметрии: мы взяли какую-то конкретную линию, решетка имеет бесконечное число параллельных ей линий и слева, и справа — так откуда же в бесконечной решетке взяться особенности именно на той линии, которую мы выбрали?

Можно, конечно, доказать отсутствие ациклической части и более строго. Рассмотрим смещение x_1 -й линии. Все размеры решеток — натуральные числа, поэтому x_1 — натуральное число, и x_1 -я линия существует. Ее смещение относительно нулевой линии равно $x_1 x_2$ — ее номер, умноженный на шаг решетки. Смещение x_1 -й линии относительно ближайшей к ней слева линии первой решетки то же, что и у нулевой: $(D_{x,0} + x_1 x_2) \bmod x_1 = D_{x,0}$.

Получается, что все расстояния между вертикальными линиями составной решетки можно получить, рассмотрев только участок между нулевой и x_1 -й линиями второй решетки. Это значительно лучше, чем рассматривать бесконечность линий, не правда ли? Аналогично, все расстояния между горизонтальными линиями составной решетки можно получить, рассмотрев только участок между нулевой и y_1 -й горизонтальной линиями второй решетки.

Расстояния между линиями составной решетки будем записывать в массивы:

`var`

```
sx, sy : array [0..100] of boolean;
```

Если в составной решетке есть расстояние по горизонтали i , то `sx[i]` истинно, если нет — ложно. Перед нахождением расстояний инициализируем оба массива ложными значениями.

Заполняем массивы. Пусть D_x — смещение какой-то вертикальной линии второй решетки относительно ближайшей слева вертикальной линии первой решетки. Очередная линия второй решетки будет иметь смещение $D_x + x_2$ относительно той же линии первой решетки.

- Если $D_x + x_2 \leq x_1$, то в sx можно занести истинное значение в x_2 -й элемент — это случай, когда между двумя соседними линиями второй решетки нет линий первой решетки.
- Если $x_1 < D_x + x_2 < 2x_1$, то ровно одна вертикальная линия первой решетки находится между соседними вертикальными линиями (или совпадает с одной из них). В массиве sx можно отмечать элементы $(x_1 - D_x)$ -й и $(D_x + x_2 - x_1)$ -й.
- Если $D_x + x_2 \geq 2x_1$, то между последовательными вертикальными линиями второй решетки находится более чем одна линия первой решетки. Значит, в массиве sx можно отмечать $(x_1 - D_x)$ -й, x_1 -й и $((D_x + x_2) \bmod x_1)$ -й элементы. Не следует особо выделять случай, когда $(D_x + x_2) \bmod x_1 = 0$, — впоследствии можно просто не учитывать $sx[0]$.

Чтобы получить все возможные расстояния по горизонтали, нужно выполнить эти действия для x_1 последовательных вертикальных линий второй решетки.

Массив sy заполняется аналогично sx . Теперь нужно найти количество различных произведений и сами произведения. Переберем все пары элементов sx и sy , и если оба члена пары истинны, занесем истинное значение в элемент массива s с индексом, равным произведению индексов элементов sx и sy .

var

```
s : array [0..10000] of boolean;
```

Если массив sx содержал все возможные смещения по горизонтали, а sy — все возможные смещения по вертикали, то массив s , полученный в результате перемножения их элементов, будет содержать все возможные площади ячеек составной решетки. После этого нетрудно сосчитать количество истинных элементов в массиве s и вывести сами значения площадей в возрастающем порядке.

Следует обратить внимание на то, что нулевая площадь может попасть в s , но выводить ее не следует.

Решение задачи 13 В. Последовательность Фибоначчи

Вариант 1

Тема: метод половинного деления в целых числах.

Будем считать, что в исходных данных $i < j$. Если это условие не выполняется, обменяем значения i и j , а также F_i и F_j .

Заметим, что по значениям двух соседних членов последовательности Фибоначчи легко находится любой член последовательности. Значение i -го члена известно — это F_i . Подберем значение F_{i+1} методом деления пополам. Применение этого мето-

да возможно, так как если F_{i+1} увеличить, то увеличатся и значения всех элементов с номерами, большими $i+1$.

Таким образом, если из предположения, что элемент с номером $i+1$ имеет значение X , получено, что j -й элемент должен иметь значение, большее чем F_j , то реальное значение F_{i+1} меньше, чем X . А если получено, что j -й элемент должен иметь значение, меньшее чем F_j , то реальное значение F_{i+1} больше, чем X .

В качестве границ поиска F_{i+1} возьмем границы возможных значений F на указанном интервале, то есть $-2\,000\,000\,000$ и $+2\,000\,000\,000$. Основной цикл имеет вид

```
while true do begin
  mid := trunc((0.0 + high + low) / 2);
  if valueJ(mid) < Fj then
    low := mid + 1
  else if valueJ(mid) > Fj then
    high := mid - 1
  else
    break;
  if low > high then
    halt(1);
end;
```

Функция `valueJ` определяет, каким должно быть значение j -го элемента, если значение $(i+1)$ -го элемента равно `mid`. Проверка `low > high` введена как элемент диагностики. Если в результате выполнения цикла оказалось, что F_{i+1} не может быть никаким числом из интервала $[-2\,000\,000\,000; 2\,000\,000\,000]$, то где-то есть ошибка — или в алгоритме, или в тесте. Дело в том, что в качестве тестовых данных нельзя подставлять абсолютно любые значения F_i и F_j — некоторые пары являются недопустимыми.

По окончании цикла значение F_{i+1} находится в переменной `mid`. А если известны значения двух соседних членов последовательности Фибоначчи, найти F_n не составляет труда.

Детали реализации:

- Важно избежать переполнений при вычислении промежуточных значений. Конечно, для искомой последовательности гарантируется, что на рассматриваемом отрезке чисел вне диапазона $[-2\,000\,000\,000; 2\,000\,000\,000]$ нет. Но это же не обязано быть справедливым, если в качестве F_{i+1} взять любое число из интервала $[-2\,000\,000\,000; 2\,000\,000\,000]$.

Таким образом, вычисляя значение элемента по формуле $F_k = F_{k-1} + F_{k-2}$, нужно помнить, что результат может быть представлен числом типа `longint` далеко не всегда, то есть нужно или использовать тип `comp` и проверять результат на принадлежность интервалу $[-2\,000\,000\,000; 2\,000\,000\,000]$ уже после выполнения операции, или, используя только тип `longint`, применять специальные методы:

```
{Нужно вычислить a := b + c}
if (b > 0) and (c > 0) then begin
```

```

{условие успешного сложения:  $b + c \leq 2\,000\,000\,000$ }
{эквивалент без переполнений:  $b \leq 2\,000\,000\,000 - c$ }
if b <= 2000000000 - c then
    a := b + c
else
    a := 2000000001;
end
else if (b < 0) and (c < 0) then begin
    {условие успешного сложения:  $b + c \geq -2\,000\,000\,000$ }
    {эквивалент без переполнений:  $b \geq -2\,000\,000\,000 - c$ }
    if b >= -2000000000 - c then
        a := b + c
    else
        a := -2000000001;
    end
else
    {знаки слагаемых разные, переполнения быть не может}
    a := b + c;

```

Как только при вычислениях в функции `valueJ` значение очередного члена последовательности оказалось более $2\,000\,000\,000$, можно в качестве найденного значения j -го элемента выдавать $2\,000\,000\,001$ — это больше, чем любое реальное значение F_j . Аналогично, как только значение оказалось менее $-2\,000\,000\,000$, можно в качестве значения j -го элемента выдавать $-2\,000\,000\,001$.

- Использование «хитрого» способа получения среднего значения

```
mid := trunc((0.0 + high + low) / 2);
```

вместо обычного

```
mid := (high + low) div 2;
```

тоже связано с желанием избежать переполнения. Когда и `low`, и `high` находятся вблизи двух миллиардов, их сумма будет около четырех миллиардов, то есть за пределами допустимых значений типа `longint`. Решением проблемы является использование вещественного типа для промежуточных значений. Сложение с вещественным нулем (`0.0 +`) — это способ преобразования типа выражения в вещественный тип. Сложение значений вещественного (`0.0 + high`) и целого (`low`) типа дает результат вещественного типа. Заметьте, значение $(0.0 + high + low) / 2$ лежит между значениями `low` и `high`, поэтому перевод его в целый тип (с помощью `trunc`) безопасен.

Вариант 2

Решение предложил Эдуард Гудков.

Если известны значения двух (не обязательно соседних) членов последовательности Фибоначчи, то можно решить систему уравнений и найти значения всех членов последовательности.

Вообще говоря, при ограничениях задачи могла бы потребоваться длинная арифметика, но для прохождения тестов на реальном соревновании хватило размера и точности типа `extended`.

Решение задачи 13 С. Скобки (3)

Вариант 1

Тема: динамическое программирование.

Обозначим S исходную последовательность скобок, $S[i]$ — ее i -й символ, $S[L...R]$ — последовательные символы исходной последовательности с L -го до R -го ($L \leq R + 1$). Если $L = R + 1$ — интервал пуст. Обозначим N длину исходной последовательности. Тогда $S[1...N]$ — вся исходная последовательность.

Будем решать исходную задачу для всех интервалов $S[L...R]$ в порядке увеличения их длины, начиная с 1. Обозначим $M[L, R]$ минимальную длину правильной последовательности скобок, в которой $S[L...R]$ содержится в качестве подпоследовательности. В частности, $M[1, N]$ — длина ответа задачи. Если $L = R + 1$, то $M[L, R] = 0$ — пустая последовательность скобок является правильной.

Если первая скобка интервала $S[L...R]$ — закрывающая ($S[L] = ')'$ или $S[L] = ']'$), то к этой скобке нужно дописать слева открывающую, а задача сводится к поиску решения исходной задачи для подстроки $S[L + 1...R]$. Таким образом, если $S[L]$ — закрывающая скобка, то $M[L, R] = 2 + M[L + 1, R]$. Подстроки рассматриваются в порядке увеличения длины, поэтому к моменту вычисления $M[L, R]$ уже известно значение $M[L + 1, R]$.

Аналогично, если последняя скобка интервала $S[L...R]$ — открывающая ($S[R] = '('$ или $S[R] = '['$), то к этой скобке нужно дописать справа закрывающую, а задача сводится к поиску решения исходной задачи для подстроки $S[L...R - 1]$. Таким образом, если $S[R]$ — открывающая скобка, то $M[L, R] = M[L, R - 1] + 2$.

В частности, если $L = R$ (то есть интервал состоит из одной скобки), то $M[L, R] = 2$.

Осталось рассмотреть случаи, когда интервал имеет длину не менее 2, первая скобка интервала ($S[L]$) — открывающая, а последняя ($S[R]$) — закрывающая.

Сначала рассмотрим случай, когда типы скобок не совпадают ($S[L] = '('$ и $S[R] = ']'$ или $S[L] = '['$ и $S[R] = ')'$). Тогда можно не рассматривать дополнения этого интервала до последовательности вида $(...)$ или $[...]$. Действительно, тогда для дополнения последовательности до правильной поставлена или открывающая скобка на первое место, или закрывающая — на последнее. Очевидно, выражение осталось бы правильным, если бы закрывающая скобка была поставлена не на последнее место, а на второе: $()...$, а открывающая — не на первое, а на предпоследнее: $...[]$.

Итак, если типы первой и последней скобок не совпадают, можно считать, что исходная последовательность имеет вид $(...)$ или $[...]$ (то есть первая и последняя скобки исходной последовательности остаются первой и последней). Всего существует два правила получения правильных скобочных выражений: «если S правиль-

ное, то (S) и $[S]$ также правильные» и «если A и B правильные, то AB также правильное». Очевидно, правильное скобочное выражение вида $(\dots)$ или $[\dots]$ могло быть получено только по второму правилу как конкатенация (запись подряд) двух непустых правильных скобочных выражений. Осталось перебрать, какая часть символов интервала $S[L\dots R]$ была дополнена до первого выражения, а какая — до второго.

Если до первого правильного выражения были дополнены символы с L -го по E -й, то до второго — с $(E + 1)$ -го по R -й ($L \leq E \leq R - 1$). Получается, что отрезок разбивается на два интервала меньшей длины, для каждого из которых нужно решить исходную задачу. Но они уже решены, так как подстроки рассматриваются в порядке возрастания длин.

Итак, если $S[L]$ — открывающая скобка, а $S[R]$ — закрывающая, и их типы различны, то $M[L, R]$ равно минимуму из сумм $M[L, E] + M[E + 1, R]$, где E изменяется от L до $R - 1$. Если же $S[L]$ — открывающая скобка, а $S[R]$ — закрывающая одного и того же типа, то правильное скобочное выражение минимальной длины может быть также получено и по правилу вывода «если S правильное, то (S) и $[S]$ также правильные». В этом случае для нахождения $M[L, R]$ нужно перебрать не только все суммы $M[L, E] + M[E + 1, R]$, но еще и значение $1 + M[L + 1, R - 1] + 1$ — когда первая и последняя скобки образуют пару, а все выражение между ними дополняется до правильной скобочной последовательности.

Итак, сформулированы правила, позволяющие для любого интервала исходной последовательности скобок получить минимальную длину правильной скобочной последовательности, содержащей этот интервал в качестве подпоследовательности.

Детали реализации:

- Нетрудно заметить, что решение конструктивно: если бы можно было наряду с минимальной длиной скобочной последовательности хранить и саму последовательность, ответ задачи можно было бы вывести сразу после нахождения всех $M[L, R]$. К сожалению, в Турбо Паскале памяти достаточно для двумерного массива 100×100 из байтов или целых, но недостаточно для массива 100×100 из строк.

Решением проблемы является использование дополнительного массива — W . $W[L, R]$ имеет или значение E , если $M[L, R]$ было получено как $M[L, E] + M[E + 1, R]$, или значение 0, если $M[L, R]$ было получено как $1 + M[L + 1, R - 1] + 1$.

- Нетрудно написать рекурсивную функцию

```
function result(L, R : integer) : string;
```

которая выдает правильную скобочную последовательность минимальной длины для отрезка $S[L\dots R]$, когда массив W заполнен.

В функции рассматривается несколько случаев:

- 1) $L = R + 1$ — интервал пуст, результат тоже пустой;
- 2) $L = R$ — интервал состоит из одной скобки, результатом является правильная последовательность двух скобок (пужно дописать парную);

- 3) $W[L, R] = 0$ — результат получается по правилу «если S правильное, то (S) и [S] также правильные» и равен $S[L] + \text{result}(L + 1, R - 1) + S[R]$;
- 4) $W[L, R] > 0$ — результат получается по правилу: «если A и B правильные, то AB также правильное», и равен $\text{result}(L, W[L, R]) + \text{result}(W[L, R] + 1, R)$.
- Ответом задачи является $\text{result}(1, N)$.

Вариант 2

Решение предложил Александр Меркулов.

Тема: сокращение перебора.

Сначала рассмотрим, как выглядело бы переборное решение без попыток сокращения перебора.

Глобальные объявления:

type

TStack = array [1..100] of char;

var

source : string;

current, result : array [1..200] of char;

resultLen : integer;

Здесь source — исходная последовательность скобок, current служит для построения последовательностей скобок, содержащих исходную в качестве подпоследовательности, result — лучшая из таких последовательностей (с минимальной длиной), resultLen — длина текущей лучшей последовательности.

Рекурсивная процедура:

procedure p(sourcePtr, currentPtr, stackPtr : integer; stack : TStack);

При вызове этой процедуры скобки с 1-й по (sourcePtr - 1)-ю исходной последовательности уже обработаны, в результате построены символы с 1-го по currentPtr-й последовательности, содержащей данную (current), а открытыми и еще не закрытыми скобками заполнены элементы с 1-го по stackPtr-й стека (stack).

Итак, при вызове процедура должна рассмотреть sourcePtr-й символ исходной последовательности скобок source, как-то изменить current, currentPtr, stack и stackPtr, а затем вызвать себя рекурсивно, чтобы расставить следующие символы.

Если sourcePtr > length(source), то все символы исходной последовательности уже обработаны. Если при этом currentPtr + stackPtr < resultLen, то найден очередной наилучший вариант: его начало записано в current[1]... current[currentPtr], а конец составляют скобки, парные stack[stackPtr]... stack[1]. Так как идет сравнение с resultLen, до вызова рекурсивной процедуры этой переменной должно быть присвоено заведомо большое значение — например 201 (даже если исходных скобок 100, и к каждой исходной скобке приписать свою парную, длина будет только 200).

Осталось рассмотреть случай $\text{sourcePtr} <= \text{length}(\text{source})$.

Если $\text{source}[\text{sourcePtr}]$ — открывающая скобка, то она заносится в *stack*, и процедура вызывает себя рекурсивно для расстановки следующих скобок.

Осталось рассмотреть случай, когда $\text{source}[\text{sourcePtr}]$ — закрывающая скобка.

Если стек открытых и еще не закрытых скобок пуст ($\text{stackPtr} = 0$), то нет другого выбора, кроме как написать перед встретившейся закрывающей скобкой соответствующую открывающую. В *current* заносится пара скобок, *stack* остается пустым, процедура вызывает себя рекурсивно для расстановки следующих скобок.

Осталось рассмотреть случай, когда $\text{source}[\text{sourcePtr}]$ — закрывающая скобка, и стек не пуст.

Если скобка на вершине стека соответствует встретившейся, то скобка с вершины стека убирается ($\text{dec}(\text{stackPtr})$), в *current* заносится $\text{source}[\text{sourcePtr}]$, и процедура вызывает себя рекурсивно для расстановки следующих скобок.

Последний случай — когда $\text{source}[\text{sourcePtr}]$ — закрывающая скобка, стек не пуст и скобка на вершине стека ей не соответствует.

Во всех предыдущих случаях действия были однозначны. Здесь же нужно перебрать два варианта действий. Если в стеке есть скобка, парная $\text{source}[\text{sourcePtr}]$, то можно закрыть все неподходящие скобки на вершине стека (пока там не окажется подходящая скобка), а затем действовать так же, как в случае, когда скобка на вершине стека соответствовала $\text{source}[\text{sourcePtr}]$. Второй вариант действий — дописать открывающую скобку к $\text{source}[\text{sourcePtr}]$, тогда действия такие же, как и при пустом стеке.

Итак, алгоритм без сокращения перебора описан. Когда он выполняет заведомо бесполезные действия? Рассмотрим пример:

```
[[[[[([)(])([)([)
```

При рассмотрении каждой из закрывающих квадратных скобок нужно выбрать: или дописать квадратную открывающую скобку непосредственно перед закрывающей, или закрыть все открытые круглые скобки. Причем тот же выбор происходит для *каждой* закрывающей квадратной скобки независимо от выбора для всех предыдущих. Таким образом, количество рассматриваемых вариантов равно 2^K , где K — количество закрывающих квадратных скобок (а также количество открывающих круглых и открывающих квадратных). Всего скобок может быть до 100, так что максимальное K равно 33. Очевидно, 2^{33} даже элементарных действий нельзя выполнить за указанное время.

Какие лишние действия были совершены? Заметим следующее: если про какую-то из открывающих круглых скобок было решено, что вписывается не парная ей, а парная открывающей квадратной, то в дальнейшем нет смысла в подобной ситуации выбирать противоположное. Пусть, например, обработаны 8 первых скобок приведенной последовательности, причем перед первой закрывающей квадратной было решено вписать открывающую квадратную:

```
[[[[[([ ( ? )(])([)([)
```

Какой может быть очередная скобка? Выбор такой: или поставить открывающую квадратную, или закрыть две открытые круглые. Рассмотрим, почему можно не рассматривать случай закрытия двух круглых скобок. Пусть поставлены две закрывающие круглые:

```
[[[[[([() ) ])([()]
```

Тогда правильность начала не изменится, если первую из этих закрывающих скобок перенести к соответствующей открывающей:

```
[[[[[()]([() ) ])([()] .
```

Тут нужно вспомнить, что в свое время, когда нужно было выбрать, что поставить между первой парой () — скобку) или [, выбор был сделан в пользу [. Фактически, постановка)) отменяет прежний выбор, и заново рассматривается вариант поставки) после первой (.

Таким образом, если однажды в ситуации `stack[stackPtr] = '('` и `source[sourcePtr] = ']'` было решено вставить '[' , то эта скобка '(' может уйти из стека только в одном случае — когда в очередной раз встретится `source[sourcePtr] = ')' .`

Аналогично, если однажды в ситуации `stack[stackPtr] = '['` и `source[sourcePtr] = ']'` было решено вставить '(' , то эта скобка '[' может уйти из стека только в одном случае — когда в очередной раз встретится `source[sourcePtr] = ']' .`

Предлагается следующая структура данных для хранения такой информации:

type

 TStackItem = record

 bracket : char;

 closeByActual : boolean;

 end;

 TStack = array [1..100] of TStackItem;

В стеке хранятся не только скобки, но и признак того, может ли скобка быть закрыта *только* встретившейся в исходной последовательности парной ей скобкой (`closeByActual = true`) или также можно вписать дополнительную закрывающую скобку (`closeByActual = false`).

При занесении скобки в стек в поле `closeByActual` заносится значение `false`. Как только в ситуации «`source[sourcePtr]` — закрывающая скобка, не парная скобке на вершине стека» было выбрано вписать открывающую скобку к `source[sourcePtr]`, нужно устанавливать поле `closeByActual` вершины стека равным `true`.

Детали реализации: глубина рекурсии равна количеству скобок плюс один (максимум 101). Параметр типа `TStack` передается по значению. Размер массива из записей `TStack` — 200 байт. Таким образом, только для этого параметра требуется около 20 Кбайт стека. А по умолчанию размер стека в Турбо Паскале — 16 Кбайт. Чтобы программа работала при количестве скобок, близком к максимальному, размер стека нужно увеличить. Чтобы не думать, насколько нужно увеличить стек, его можно увеличить до максимума: 65 520.

Решение задачи 13 D. Центр тяжести

Тема: геометрия.

Пусть плоскую фигуру A ненулевой площади можно составить из двух фигур: B и C . Более строго, пересечение B и C имеет нулевую площадь, а объединение B и C дает A . Из этого, в частности, следует выражение для площадей $S_A = S_B + S_C$.

Пусть для фигур B и C известны площади (S_B и S_C) и координаты центров тяжести — (x_B, y_B) и (x_C, y_C) соответственно. Тогда можно найти координаты центра тяжести фигуры A по формулам

$$x_A = \frac{x_B S_B + x_C S_C}{S_B + S_C}, y_A = \frac{y_B S_B + y_C S_C}{S_B + S_C}.$$

Аналогичные формулы справедливы, если исходную фигуру разбить на большее число фигур: в числителе будет стоять сумма произведений координат центров тяжести фигур на их площади, а в знаменателе — сумма площадей фигур. Таким образом, осталось только разбить исходный многоугольник на элементарные фигуры, координаты центров тяжести которых находятся каким-нибудь тривиальным способом.

Координаты центра тяжести треугольника находятся элементарно: как $1/3$ суммы соответствующих координат его вершин. Площадь треугольника также найти просто. Если бы удалось простым способом разбить исходный многоугольник на треугольники, задача была бы решена.

Для выпуклого многоугольника такое разбиение предложить легко. Пусть точки многоугольника пронумерованы от 1 до N в порядке обхода контура. Тогда многоугольник является объединением треугольников $(1, 2, 3)$, $(1, 3, 4)$, $(1, 4, 5)$, ..., $(1, N-2, N-1)$, $(1, N-1, N)$ — в скобках указаны номера вершин исходного многоугольника.

Оказывается, если вычислять площади треугольников с помощью векторного произведения (см. вариант 2 решения задачи 4 D), то формула координат центра тяжести, выведенная для выпуклого многоугольника, будет работать также и для невыпуклого многоугольника. Это возможно потому, что при вычислениях с помощью векторного произведения может получиться площадь треугольника как с плюсом, так и с минусом.

Можно применить и вариант 1 решения задачи 4 D — он основан на вычислении площади трапеции со знаком. К сожалению, координаты центра тяжести трапеции найти несколько сложнее, чем координаты центра тяжести треугольника. Следует учитывать также, что трапеция или треугольник могут быть вырожденными в отрезок. В случае треугольника вычисление координат центра тяжести безопасно, а при вычислении координат центра тяжести вырожденной трапеции может возникнуть переполнение, поэтому случай вырожденной трапеции нужно рассматривать отдельно.

Решение задачи 13 Е. Сумма произведений

Темы: сокращение перебора, решение уравнений в целых числах.

Очевидно, нельзя в явном виде перебирать все комбинации значений переменных, так как тогда количество действий будет равно 3^N , что для $N = 10\,000$ составляет гигантское число. К счастью, с точки зрения задачи переменные одинаковы, поэтому достаточно перебрать только все возможные количества значений $+1$, 0 и -1 среди них.

К сожалению, сделать это за отведенное время также невозможно. Если количество нулей z будет изменяться от 0 до N , а количество единиц p — от 0 до $N - z$, то потребуется примерно $N^2/2$ элементарных действий. Для $N = 10\,000$ это порядка 50 млн действий, а каждое действие включает в себя несколько операций сложения и умножения чисел типа `longint`.

Будем перебирать количество нулей z и для каждого z определять, решая уравнение, каким должно быть количество единиц p , чтобы сумма произведений оказалась равна S . Если p окажется целым числом из диапазона $[0, N - z]$, то можно увеличивать счетчик найденных решений.

Детали реализации:

- Уравнение имеет вид $S = \frac{p(p-1)}{2} + \frac{m(m-1)}{2} - pm$, где p — количество единиц, m — количество минус единиц. Умножив его на 2 и заменяя m на $N - z - p$, получим: $2S = p(p-1) + (N-z-p)(N-z-p-1) - 2p(N-z-p)$.

Для решения нужно уравнение привести к виду $ap^2 + bp + c = 0$, где a , b и c — целочисленные выражения, состоящие из констант (чисел, N , S и z).

- Для решения уравнения в целых числах требуются специфические приемы. Выполняя каждую из промежуточных операций, нужно убеждаться в том, что она дает целочисленный результат. Например, проверка того, что число является полным квадратом, может выглядеть так:

```
d := b * b - 4 * a * c;
...
if sqr(round(sqrt(d))) = d then
    {продолжаем вычисления}
else
    {это уравнение в целых числах не решается}
```

Решение задачи 13 F. Статическая сложность

Тема: метод рекурсивного спуска.

Решение этой задачи в целом аналогично решению задачи 12 F: для каждого правила РБНФ пишется процедура с выходным параметром, описывающим многочлен, характеризующий сложность участка. Поэтому остановимся только на различиях. Основное различие заключается в том, что в задаче 12 F в правилах РБНФ элемен-

тарными единицами являлись одиночные символы, а в этой задаче элементарными являются группы символов — LOOP, OP и даже числа, кроме того, в любом месте могут встретиться пробелы. Чтобы не усложнять основной алгоритм разбора, нужно ввести дополнительный уровень обработки исходной информации: процедуру nextLex, которая будет пропускать пробелы и группировать символы в «слова» — лексемы:

type

integer = longint;

const {виды лексем}

lexBEGIN = 1;

lexEND = 2;

lexLOOP = 3;

lexOP = 4;

lexN = 5;

lexNumber = 6;

lexEOF = 7; {отдельная лексема - конец текста}

var

c : char; {текущий символ - заполняется nextChar, используется nextLex}

lex : integer; {текущая лексема (виды см. ранее)}

value : integer; {значение числа, если текущая лексема lexNumber}

{

Чтение очередного символа из файла.

Переводы строк эквивалентны пробелам.

При встрече конца текста выдает #255.

}

procedure nextChar;

begin

if eof then

c := #255

else if eoln then begin

readln;

c := #32;

end

else

read(c);

end;

{

Разбор идентификатора.

Последовательность букв и цифр заносится в пале,

затем сравнивается с определенными в РБНФ словами.

Результат заносится в глобальную переменную lex.

```

}
procedure ident:
var
    name : string;
begin
    name := '';
    while c in ['A'..'Z', 'a'..'z', '0'..'9'] do begin
        name := name + c;
        nextChar;
    end;
    if name = 'BEGIN' then
        lex := lexBEGIN
    else if name = 'END' then
        lex := lexEND
    else if name = 'LOOP' then
        lex := lexLOOP
    else if name = 'OP' then
        lex := lexOP
    else if name = 'n' then
        lex := lexN
    else begin
        writeln('ident error ', name, '<');
        halt(1);
    end;
end;

```

```

{
Разбор числа (последовательности цифр).
В lex заносится lexNumber, в value - значение числа.
}

```

```

procedure number:
var
    d : integer;
begin
    value := 0;
    while c in ['0'..'9'] do begin
        d := ord(c) - ord('0');
        if value > (maxlongint - d) div 10 then begin
            writeln('number error');
            halt(1)
        end
        else
            value := value * 10 + d;
    end

```

```
    nextChar;
end;
lex := lexNumber;
end;

{
Получение следующей лексемы.
Пропускает все пробельные символы, а затем,
если текущий символ - буква, вызывает ident,
если текущий символ - цифра, вызывает number,
есть еще случай конца текста.
}
procedure nextLex;
begin
    while c <= #32 do
        nextChar;
    case c of
        'A'..'Z', 'a'..'z':
            ident;
        '0'..'9':
            number;
        #255:
            lex := lexEOF;
        else
            writeln('nextLex error');
            halt(1);
        end;
    end;
end;
```

Тренировка 14

Решение задачи 14 А. Марковский цикл

Тема: последовательности, строки.

ПРИМЕЧАНИЕ

Случай отсутствия циклической части в решениях не рассматривается по причине того, что его легко обработать в процедуре получения следующей строки. Если очередную строку получить нельзя, нужно вывести число шагов с начала работы программы, затем число 0 и завершить программу.

Вариант 1

Темы: кодирование, динамическая память ТР.

Если отвлечься от реализации, то задача решается очень просто. Нужно в последовательности строк $\{S_i\}$, $i \geq 0$ найти два равных элемента, $S_j = S_k$, $j < k$, таких, что k минимально. Проще говоря, нужно выделить минимальное начало последовательности, в котором есть два одинаковых элемента. Пусть S_j и S_k — эти самые равные элементы. Тогда ответ задачи будет следующий: ациклических шагов j , длина цикла $k - j$.

Нужно строить элементы последовательности и запоминать уже построенные. Для каждой очередной полученной строки нужно смотреть: не построена ли она ранее. Как только будет найдена строка, встречающаяся ранее, будет найдено k — индекс второго вхождения. Индекс первого вхождения можно хранить вместе с запомненной строкой — тогда j будет найдено немедленно. Если же индекс вхождения не запоминать, то можно или повторить путь сначала (теперь уже известно, с какого элемента начинается цикл), или пройти еще один виток цикла (тогда будет найдена длина цикла, и для определения j ее нужно будет вычесть из k).

Оставшаяся часть разбора посвящена реализации алгоритма в условиях жестких ограничений на размер памяти.

Детали реализации:

- Ключевым моментом является подсчет того, сколько всего может быть строк описанного вида. В строке содержится не более 12 символов. Длина строки при преобразованиях не увеличивается — и в левой, и в правой части правил по

N символов. Каждый из символов строки может принимать любое из трех значений: А, В, С. Всего вариантов 3^L , где L — длина исходной строки. В худшем случае, при $L = 12$, получается $3^{12} = 531\,441$ вариант.

Оценим, как именно ограничивает нас число 531 441. Скорость работы процессора измеряется миллионами и десятками миллионов элементарных операций в секунду. Значит, алгоритм, где требуется порядка 531 441 действия можно себе позволить, а вот $531\,441^2$ — нет. Объем доступной динамической памяти Турбо Паскаля составляет, как правило, не менее 400 Кбайт, иногда — до 600 Кбайт, так что если действовать на грани, то можно выделить 531 441 байт, а 531 441 бит выделить вообще не проблема. Выберем реализацию алгоритма, где требуется 1 бит на вариант строки.

- Каждой строке длиной L взаимно однозначно сопоставим целое число от 0 до $3^L - 1$, вычисленное, например, таким образом:

```
function encode(s : string) : longint;  
var  
  i : integer;  
  result : longint;  
begin  
  result := 0;  
  for i := 1 to length(s) do  
    result := result * 3 + (ord(s[i]) - ord('A'));  
  encode := result;  
end;
```

Фактически, когда из кодов букв 'А'...'С' вычитается код буквы 'А', получается число в троичной системе счисления, состоящее из цифр 0, 1 и 2. В результате выполнения алгоритма

результат := 0;

для каждого разряда от старшего к младшему

результат := результат * 3 + разряд;

получается значение числа, записанного в троичной системе счисления.

- Создадим таблицу из логических с ключами от 0 до $3^{12} - 1$. На каждый ключ отведем 1 бит. Тогда размер таблицы составит более 64 Кбайт ($531\,441/8 = 66\,430,125$). Придется использовать динамическую память. Представим таблицу в виде массива из указателей на массивы, размещенные в динамической памяти. Если использовать тип `boolean`, то одно логическое значение будет представлено одним байтом, хотя требуется только бит. Более экономно использовать память позволяют множества (`set`). В случае множеств тоже есть маленькая хитрость: для наиболее эффективного использования памяти нижняя граница элементов множества должна делиться на 8, и количество элементов множества тоже должно делиться на 8. Приведем описание структур данных и процедур работы с ними:

```

const
  len3 = 8:      {размер множества}
                  {из соображений экономии памяти должен быть кратен 8}

  len2 = 1024:   {размер массива из множеств}
                  {для повышения скорости работы должен быть степенью двойки}

  len1 = 65:     {размер массива из указателей на массивы из множеств}
                  {должно выполняться: len1 * len2 * len3 >= 3^12 = 531 441}

```

type

```

arr = array [0..len2 - 1] of set of 0..len3 - 1;
      {указатель можно объявить только на тип, имеющий имя}

```

var

```

table : array [0..len1 - 1] of ^arr;

```

```

function was(index : longint) : boolean;

```

var

```

  index1, index2, index3 : integer;

```

begin

```

  index3 := index mod len3;
  index2 := index div len3 mod len2;
  index1 := index div len3 div len2; {mod len1}
  was := index3 in table[index1]^[index2];

```

end;

```

procedure put(index : longint);

```

var

```

  index1, index2, index3 : integer;

```

begin

```

  index3 := index mod len3;
  index2 := index div len3 mod len2;
  index1 := index div len3 div len2; {mod len1}
  include(table[index1]^[index2], index3);

```

end;

```

procedure initTable;

```

var

```

  index1, index2 : integer;

```

begin

```

  for index1 := 0 to len1 - 1 do begin
    new(table[index1]);

```

```

    for index2 := 0 to len2 - 1 do
        table[index1]^[index2] := [];    {пустое множество}
    end;
end;
end;

```

Перед использованием `index` раскладывается на три части. При приведенных значениях констант `index3` — это младшие 3 бита числа, `index2` — следующие 10 битов числа, а `index1` — то, что перед ними. Переменная `index1` индексирует массив из указателей, переменная `index2` индексирует массив из множеств, а `index3` определяет элемент множества.

Чтобы занести элемент в множество, используется стандартная процедура `include`; чтобы проверить, есть ли элемент в множестве, используется операция `in`.

- Нужно выполнять описанные в условии действия над строкой, получающиеся строки кодировать и заносить в таблицу. Когда выяснится, что очередная построенная строка уже встречалась (was выдаст истинное значение), значение k (см. начало решения) будет определено, а для определения j нужно будет или пройти еще один виток цикла, пока не встретится значение S_k , или построить последовательность с начала до тех пор, пока не будет получено значение S_k (при втором построении последовательности таблица уже не заполняется). В случае, если алгоритм заканчивается за конечное число шагов, вывести ответ не составляет труда. Не следует только забывать про этот случай.
- Выполнять описанные в условии задачи действия над строками легче всего с использованием стандартных функций и процедур работы со строками: `pos` — для определения позиции или отсутствия подстроки, `delete` — для удаления подстроки, которая была найдена, `insert` — для вставки подстроки, которой она должна быть заменена.
- При чтении входных данных пробелы легко удалить, используя стандартные строковые процедуры и функции:

```

procedure killSpaces(var s : string);
begin
    while pos(' ', s) > 0 do
        delete(s, pos(' ', s), 1);
    end;
end;

```

Вариант 2

Рассмотрим решение задачи в условиях, когда нежелательно хранение значительного объема данных. Запомним S_0 — начальную строку. Найдем строку, заведомо находящуюся в циклической части. Среди первых $3^{12} + 1 = 531\,442$ строк есть две одинаковые. Значит, строка, полученная за 531 440 шагов из исходной, точно находится в циклической части.

После получения $S_{531\,440}$ найдем длину цикла обычным способом — строя очередные строки и сравнивая их с $S_{531\,440}$. Количество шагов до первого повторения $S_{531\,440}$ и будет равно длине цикла. Обозначим длину цикла C .

Найдем какой-нибудь элемент в циклической части последовательности с индексом, кратным C . Например, можно начать с $S_{531\,440}$ и делать шаги до тех пор, пока индекс очередного элемента не окажется кратным C . Обозначим этот элемент S_K .

Если бы ациклической части не было, значение S_K было бы равно значению S_0 . Будем получать строки, следующие за S_K и S_0 : $\{S_{K+A}\}$ и $\{S_{0+A}\}$. Минимальное целое отрицательное число A , при котором $S_{K+A} = S_{0+A}$, и будет искомым числом ациклических шагов.

Покажем, что всегда существует A , для которого выполняется $S_{K+A} = S_{0+A}$. Возьмем в качестве A число 531 440. Равенство превратится в $S_{K+531\,440} = S_{531\,440}$. Равенство следует из того, что оба элемента находятся в циклической части, где элементы повторяются с периодом C , а K делится на C .

Детали реализации:

- Это решение, как правило, работает дольше, чем вариант 1, из-за слишком грубой оценки длин ациклической и периодической частей. В ответах к тестам задачи числа более чем на порядок меньше этих оценок. Число действий в варианте 1 пропорционально длине реального периода, в этом же варианте число действий всегда не менее 531 440.

Время работы этого решения настолько больше времени работы решения варианта 1, что есть шанс не уложиться в отведенное время. Если от проверяющей системы получен ответ «Time limit exceeded», можно ускорить решение, потеряв, однако, при этом гарантию, что оно вообще будет работать. Идея ускорения элементарна — уменьшить число 531 440 в решении в несколько раз. На реальном соревновании рискнуть имело бы смысл, так как в случае благоприятного исхода эксперимента задача была бы сдана, а при неблагоприятном исходе было бы потрачено не слишком много времени на эту попытку изменения программы.

Решение задачи 14 В. Д-44

Темы: метод Эйлера, ручной метод половинного деления.

Будем считать, что в течение времени dt (скажем, 0,001 с) снаряд летит прямолинейно и равномерно. А спустя dt будем пересчитывать его скорость и направление движения, предполагая, что на снаряд в течение всего времени dt действовали силы точно такие же, как и в начале этого интервала времени.

Скорость будем характеризовать вертикальной v_y и горизонтальной v_x составляющими. Найти начальные значения не составляет труда:

$$v_{x0} = v_0 \cos \alpha; \quad v_{y0} = v_0 \sin \alpha.$$

Здесь v_0 — начальная скорость снаряда, а α — угол к горизонту. Ось OX расположим параллельно поверхности земли в направлении выстрела, ось OY направим вверх, дуло пушки разместим в начале координат. Тогда $x_0 = 0, y_0 = 0$.

Пусть известны координаты (x, y) и скорость снаряда (v_x, v_y) в момент времени t . Найдем координаты снаряда и скорость в момент времени $t + dt$, предполагая, что снаряд в течение dt двигался прямолинейно и равномерно (для расчета координат), а силы на него действовали так же, как и в момент t (для расчета новой скорости). Координаты находятся элементарно:

$$x_{t+dt} = x_t + v_{x,t} dt; \quad y_{t+dt} = y_t + v_{y,t} dt.$$

Находить новые скорости чуть сложнее. Для их расчета требуется учитывать все силы, действующие на снаряд. Таких сил две: сила тяжести mg , направленная вниз, и сила сопротивления воздуха. Модуль силы сопротивления воздуха пропорционален квадрату скорости: kv_t^2 , направление обратно вектору скорости, то есть ее проекция на ось OX пропорциональна $v_{x,t}$, а проекция на ось OY пропорциональна $v_{y,t}$. Скорость в следующий момент времени находим по таким формулам. Модуль скорости в предыдущий момент времени: $v_t = \sqrt{v_{x,t}^2 + v_{y,t}^2}$; модуль силы сопротивления воздуха: $F_{R,t} = kv_t^2$. Проекцию силы сопротивления воздуха на ось OX найдем из выражения $F_{R,x,t} = -\frac{F_{R,t}v_{x,t}}{v_t}$. Знак «минус» означает, что она направлена противоположно скорости снаряда.

Аналогично определяется проекция силы сопротивления воздуха на ось OY :

$$F_{R,y,t} = -\frac{F_{R,t}v_{y,t}}{v_t}. \text{ Проекцию ускорения на ось } OX \text{ определим по формуле } a_{x,t} = \frac{F_{R,x,t}}{m},$$

а проекцию ускорения на ось OY — по формуле $a_{y,t} = \frac{F_{R,y,t}}{m} - g$. «Минус» перед g значит, что ускорение свободного падения направлено оси OY .

По формуле $v_{x,t+dt} = v_{x,t} + a_{x,t} dt$ определяется новая проекция скорости на ось OX , а по $v_{y,t+dt} = v_{y,t} + a_{y,t} dt$ — новая проекция скорости на ось OY .

Нужно повторять процедуру определения состояния снаряда в очередной момент времени до тех пор, пока очередное y не окажется меньше нуля или равно ему. Тогда ответом задачи будет x .

В этом решении использованы две неизвестные величины: dt — размер кванта времени и k — коэффициент сопротивления воздуха. Найдем их. Сначала найдем подходящее dt . Для этого зафиксируем $k = 0$.

Когда сопротивление воздуха отсутствует, задача о дальности полета снаряда решается аналитически. Пусть начальная вертикальная скорость снаряда $v_{y,0}$. В течение всего полета эту вертикальную скорость изменяет только ускорение свободного падения. Из соображений симметрии снаряд в момент падения на землю имеет такую же по модулю вертикальную скорость, что и в момент выстрела, но с другим знаком. Ускорению свободного падения нужно изменить скорость на $2v_{y,0}$. Значит,

снаряд проведет в воздухе время $\frac{2v_{y,0}}{g}$. Когда сопротивление воздуха отсутствует, горизонтальная скорость снаряда остается постоянной в течение всего полета. Значит, дальность полета составит $\frac{2v_{x,0}v_{y,0}}{g}$. При стрельбе под углом 45° $v_{x,0} = \frac{v_0}{\sqrt{2}}$

и $v_{y0} = \frac{v_0}{\sqrt{2}}$, так что дальность полета составляет $\frac{v_0^2}{g}$. То есть без сопротивления воздуха снаряд улетел бы на $\frac{800^2}{9,8} = 65\,306$ м.

Итак, дальность стрельбы при отсутствии сопротивления воздуха известна. Нужно подбирать такое dt , при котором погрешность составляет менее 10 м, как и требуется в задаче. Конечно, чем меньше взять dt , тем лучше. С этим связан такой хитрый прием: можно взять минимальное dt , при котором решение укладывается в отведенное время. Этот прием экономит много времени, но вы не будете уверены в том, что необходимая точность достигнута.

Итак, dt определено. Теперь определим k . Придется находить его «ручным» методом половинного деления, то есть подставить одно значение и посмотреть, как далеко улетел снаряд при стрельбе под углом 45° . Если дальше требуемых 15 650 м, то увеличить коэффициент, если ближе — уменьшить коэффициент. Можно, конечно, для поиска коэффициента написать дополнительную процедуру (и тогда метод не будет «ручным»), но, думаю, на написание такой процедуры уйдет больше времени, чем на подбор коэффициента.

Детали реализации:

- Следует перевести угол, заданный во входных данных, из градусов в радианы.
- Не стоит использовать тип `real`. Операции над переменными любого другого вещественного типа выполняются на порядок быстрее. Из соображений точности лучше выбрать `extended` или `double`.

Решение задачи 14 С. Упаковка символов

Тема: динамическое программирование.

Обозначим S исходную последовательность, $S[i]$ — ее i -й символ, $S[L...R]$ — последовательные символы исходной последовательности с L -го до R -го ($L \leq R$). Обозначим N длину исходной последовательности. Тогда $S[1...N]$ — вся исходная последовательность.

Будем решать исходную задачу для всех интервалов $S[L...R]$ в порядке увеличения их длины, начиная с 1. Обозначим $M[L, R]$ минимальную длину упакованной последовательности $S[L...R]$. В частности, $M[1, N]$ — длина ответа задачи.

Очевидно, что лучший способ упаковать *одну* букву — это представить ее ей же самой. Понятно, что упаковать ее пустой последовательностью нельзя, а способ упаковки последовательностью длиной 1 очевиден.

Если длина интервала $S[L...R]$ больше 1, то упакованная последовательность для него может быть получена одним из двух способов: или как конкатенация двух упакованных последовательностей, или в виде $X(S)$.

Перебрать все способы представления упакованной последовательности как конкатенации двух довольно просто. Пусть отдельно пакуются $S[L...E]$ и $S[E+1...R]$, $L \leq E < R$. Для меньших интервалов уже известны длины наименьших упакован-

ных последовательностей. Таким образом, кандидатами на $M[L, R]$ становятся все суммы $M[L, E] + M[E + 1, R]$, $L \leq E < R$.

Чтобы интервал $S[L...R]$ мог быть упакован в виде $X(S)$, необходимо, чтобы он представлял собой X одинаковых интервалов, записанных подряд. Переберем все значения X от 2 до $R - L + 1$. Проверить наличие X одинаковых интервалов можно с помощью следующей функции:

```
function isPeriod(left, right, count : byte) : boolean;
var
  p1, p2 : byte;
begin
  isPeriod := false;
  if (right - left + 1) mod count <> 0 then
    exit;
  p1 := left;
  p2 := left + (right - left + 1) div count;
  while p2 <= right do begin
    if S[p1] <> S[p2] then
      exit;
    inc(p1);
    inc(p2);
  end;
  isPeriod := true;
end;
```

Если $R - L + 1$ не делится на X , то, естественно, X одинаковых интервалов получить нельзя. Далее для проверки используется следующее свойство: X одинаковых интервалов существует тогда и только тогда, когда равны любые две буквы, места которых различаются ровно на длину интервала, то есть на $\frac{R - L + 1}{X}$.

Итак, если интервал $S[L...R]$ состоит из X одинаковых интервалов, то кандидатом на $M[L, R]$ является также $\text{len}(X) + 1 + M\left[L, L + \frac{R - L + 1}{X} - 1\right] + 1$. Функция len должна выдавать количество цифр в десятичной записи целого числа. Для нахождения $M[L, R]$ нужно выбрать меньшее из значений-кандидатов.

Детали реализации:

- Нетрудно заметить, что решение конструктивно — если бы можно было наряду с минимальной длиной упакованной последовательности хранить и саму последовательность, ответ задачи можно было бы вывести сразу после нахождения всех $M[L, R]$. К сожалению, в Турбо Паскале памяти достаточно для шести двумерных массивов 100×100 , состоящих из целых или логических переменных, но ее недостаточно для массива 100×100 , состоящего из строковых переменных.

Решением проблемы является использование двух дополнительных двумерных массивов — P из логических и W из байтов. Истинное значение $P[L, R]$ является признаком того, что $M[L, R]$ было получено как

$$\text{len}(X) + 1 + M \left[L, L + \frac{R - L + 1}{X} - 1 \right] + 1,$$

а значение X хранится в $W[L, R]$. Ложное значение $P[L, R]$ является признаком того, что $M[L, R]$ было получено как $M[L, E] + M[E + 1, R]$, в этом случае в $W[L, R]$ хранится значение E .

Нетрудно написать рекурсивную функцию

```
function result(L, R : integer) : string;
```

которая выдает упакованную последовательность минимальной длины для отрезка $S[L...R]$, когда заполнены массивы P и W .

В функции рассматривается несколько случаев:

- 1) $L = R$ — интервал из одной буквы; результатом является строка, содержащая одну эту букву;
- 2) $P[L, R]$ ложно — результат равен $\text{result}(L, E) + \text{result}(E + 1, R)$, где E равно $W[L, R]$;
- 3) $P[L, R]$ истинно — результат равен $\text{int2str}(X) + '(' + \text{result}(L, L + (R - L + 1) \div X - 1) + ')'$, где X равно $W[L, R]$, а функция int2str выдает строковое представление числа.

Ответом задачи является $\text{result}(1, N)$.

Решение задачи 14 D. Пирамиды

Тема: геометрия.

Введем прямоугольную декартову систему координат в пространстве. Точку A разместим в начале координат (координаты $(0, 0, 0)$). Точку B разместим на оси OX (координаты $(AB, 0, 0)$). Точку C разместим на плоскости XOY так, что ее y -координата будет положительной. Координаты точки C можно найти, решив систему уравнений

$$\begin{cases} (x_C - x_A)^2 + (y_C - y_A)^2 = AC^2 \\ (x_C - x_B)^2 + (y_C - y_B)^2 = BC^2 \end{cases}$$

или, после подстановки уже известных значений,

$$\begin{cases} x_C^2 + y_C^2 = AC^2 \\ (x_C - AB)^2 + y_C^2 = BC^2. \end{cases}$$

Значение x_C находится из линейного уравнения, получаемого при вычитании первого уравнения системы из второго. Значение y_C получается из первого уравнения при подстановке найденного значения x_C .

Координаты точки D находятся из аналогичной системы, решаемой так же просто.

Объем пирамиды может быть найден как $1/3$ площади основания, умноженная на высоту, то есть $\frac{x_{в}y_{с}z_{д}}{6}$.

Решение задачи 14 Е. Поле для крикета

Тема: сокращение перебора.

Пусть найдены оптимальные расположение и размер поля для крикета. Будем перемещать это поле на юг (уменьшать y -координату) до тех пор, пока оно не достигнет дерева или южной границы парка. Аналогично, после этого будем перемещать поле на запад (уменьшать x -координату) до тех пор, пока оно также не достигнет дерева или западной границы парка.

Таким образом показано, что в поисках оптимального расположения достаточно рассматривать только поля, y -координата южной границы которых совпадает с какой-то из y -координат деревьев или равна нулю, а x -координата западной границы совпадает с какой-то из x -координат деревьев или равна нулю.

Без учета этого предложения объем перебора положений юго-западного угла поля составил бы $W \cdot H$, то есть 100 миллионов при максимальных значениях W и H . С учетом же этого предложения объем перебора составляет всего $(N + 1)^2$, то есть чуть более 10 тысяч при максимальном значении N .

Для каждого положения юго-западного угла поля нужно выбрать максимальный допустимый размер поля, учитывая размеры парка (W и H), а также расположение всех деревьев, у которых и x -, и y -координаты больше соответствующих координат юго-западного угла поля.

Решение задачи 14 F. Электронная таблица

Тема: метод рекурсивного спуска.

Входные данные занесем в следующие структуры:

```
var
  expr : array ['A'..'Z', '1'..'9'] of string;
  found : array ['A'..'Z', '1'..'9'] of (no, yes, during);
  value : array ['A'..'Z', '1'..'9'] of longint;
```

Массив `expr` содержит выражение для ячейки из входных данных. Если во входных данных выражение для конкретной ячейки отсутствует, соответствующий элемент массива равен пустой строке. При чтении входных данных и занесении их в массив `expr` рекомендуется удалять пробелы, чтобы не усложнять основной алгоритм.

Массив `found` отражает состояние ячейки: значение (в нашей программе) вычислено, не вычислено или находится в стадии вычисления. Если значение ячейки вычислено, то соответствующий элемент массива `value` содержит значение этой ячейки. Для всех ячеек, не определенных во входных данных, элемент массива `found`

равен yes, а элемент массива value равен 0. Для ячеек, определенных во входных данных, сначала устанавливаем элемент found равным no как знак того, что их значение еще предстоит вычислить.

Далее алгоритм работает рекурсивно. Вначале находится значение ячейки A1. Если в формуле встретилось имя какой-то ячейки, то нужно проверить соответствующий элемент массива found. Если значение ячейки уже вычислено (элемент found равен yes), то значение этой ячейки уже находится в массиве value, и заново вычислять его не надо. Если используемая в формуле ячейка находится в стадии вычисления (элемент found равен during), то налицо циклическая ссылка — можно вывести число 1 000 000 и завершать работу. Если же в формуле встретилось обозначение ячейки, для которой есть формула, но значение еще не подсчитано (элемент found равен no), то нужно приостановить вычисление текущего выражения и рекурсивно вызвать разбор формулы для встретившейся ячейки.

Одним из основных элементов решения является нахождение значения выражения, содержащего имена переменных, скобки и знаки арифметических операций. Применим метод рекурсивного спуска. Отличие этой задачи от, скажем, задачи 12 F — в том, что в условии не приведены правила РБНФ для арифметического выражения.

Для выражения без унарных операций правила выглядят так:

Выражение = Слагаемое { ("+" | "-") Слагаемое }.

Слагаемое = Множитель { ("*" | "/") Множитель }.

Множитель = число | имя | "(" Выражение ")".

Обратите внимание на то, что следующие правила:

Выражение = Подвыражение { ("+" | "- " | "*" | "/") Подвыражение }.

Подвыражение = число | имя | "(" Выражение ")".

использовать *нельзя* — в них не учтен приоритет операций.

Использованная нотация немного отличается от нотации задачи 12 F. Нетерминалы пишутся не в угловых скобках, а с прописной буквы, вместо ::= пишется просто =, в конце правила ставится точка.

Тренировка 15

Решение задачи 15 А. Игра с калькулятором

Тема: последовательности.

Вариант 1

Тема: остатки.

Понятно, что нет никакого смысла моделировать то, что описано в задаче. Прежде всего, не очевидно, при каких значениях K число из одинаковых цифр не будет построено никогда. Но даже если научиться по значению K узнавать, можно ли построить число из одинаковых цифр, далеко не всегда двух секунд будет достаточно для его построения. Существуют примеры, когда длина числа из одинаковых цифр более 100.

Сформулируем задачу в эквивалентном виде: найти минимальное число вида $dd\dots d$ (d — цифра от 1 до 9), которое делится на K без остатка.

Зафиксируем цифру d . Рассмотрим последовательность остатков по модулю K чисел, состоящих из i цифр d : $\{m_{d,i}\}$. Найти первый член последовательности не представляет труда — $m_{d,1} = d \bmod K$. Как выразить число, состоящее из i цифр d , через число, состоящее из $i - 1$ цифры d ? Например, так: $\underbrace{dd\dots d}_{i \text{ раз}} = \underbrace{dd\dots d}_{i-1 \text{ раз}} \cdot 10 + d$. Как

выразить $m_{d,i}$ через $m_{d,i-1}$? Точно так же: $m_{d,i} = (m_{d,i-1} \cdot 10 + d) \bmod K$.

Получена последовательность, элементы которой не превосходят K . Значит, среди первых $K + 1$ первого элемента последовательности есть два одинаковых. Так как значение следующего элемента зависит только от значения предыдущего, то после второго из одинаковых элементов последуют те же значения, что и после первого. Таким образом, все возможные значения элементов последовательности можно найти среди значений первых K элементов, то есть если значения 0 среди первых K элементов не было, то его и в дальнейшем не будет.

Возвращаемся к основной задаче. Получается, что достаточно рассмотреть только $9K$ чисел: $m_{d,i}$, $d = 1\dots 9$, $i = 1\dots K$. Если среди них нет нулей — значит, цели достичь нельзя. Если есть нули, нужно выбрать нулевое $m_{d,i}$ с минимальным i , а если i у двух нулевых элементов совпадают — тогда с минимальным d . Ответом задачи будет пара d и i этого элемента.

Вариант 2

Тема: остатки.

К получению числа, состоящего из i цифр d , через число, состоящее из $i - 1$ цифры d , можно подойти и по-другому: $\underbrace{dd\dots d}_{i \text{ раз}} = \underbrace{dd\dots d}_{i-1 \text{ раз}} + d \cdot 10^{i-1}$.

Как можно использовать такую формулу? Рассмотрим последовательность пар $(m_{d,i}, t_i)$. Здесь $m_{d,i}$ по-прежнему означает остаток по модулю K числа, состоящего из i цифр d . А запись t_i означает остаток по модулю K числа 10^i . Пара с индексом i получается из пары с индексом $i - 1$ следующим образом: $t_i = (t_{i-1} \cdot 10) \bmod K$, $m_{d,i} = (m_{d,i-1} + d \cdot t_{i-1}) \bmod K$.

Получается, что следующая пара зависит только от предыдущей. Всего различных значений пар не может быть более K^2 . Таким образом, анализируя эту последовательность, можно прийти к выводу, что достаточно рассмотреть только первые K^2 элементов $m_{d,i}$ в поисках нулевого.

Конечно, этот вариант решения хуже первого — длина периода оценена числом K^2 вместо K . Но он показывает, что элементами последовательности могут быть не только числа, но и векторы.

Решение задачи 15 В. Площадь треугольника

Сначала разберемся с особыми случаями: 1) треугольник построить нельзя; 2) его площадь определяется неоднозначно. Треугольник построить нельзя, когда две из трех заданных длин отрезков совпадают, а третья длина отличается от заданной. Площадь треугольника определяется неоднозначно, если все три длины совпадают. Тогда равнобедренный треугольник с любым основанием может иметь заданные медиану, высоту и биссектрису. Если же все три заданные длины различны, то площадь треугольника определяется однозначно.

Самая маленькая длина у высоты, средняя — у биссектрисы, а самая большая — у медианы. Без доказательства.

Вариант 1

Тема: метод половинного деления в вещественных числах.

Площадь треугольника равна половине произведения высоты на длину стороны, на которую она опущена. Высота известна. Если будет найдена длина стороны, задачу можно будет считать решенной.

Заметим, что при заданных длинах высоты и медианы длина биссектрисы монотонно зависит от длины основания. Когда длина основания стремится к нулю, длина биссектрисы стремится к длине медианы. Когда длина основания стремится к бесконечности, длина биссектрисы стремится к длине высоты. Эти замечания, конечно, не являются доказательством монотонности.

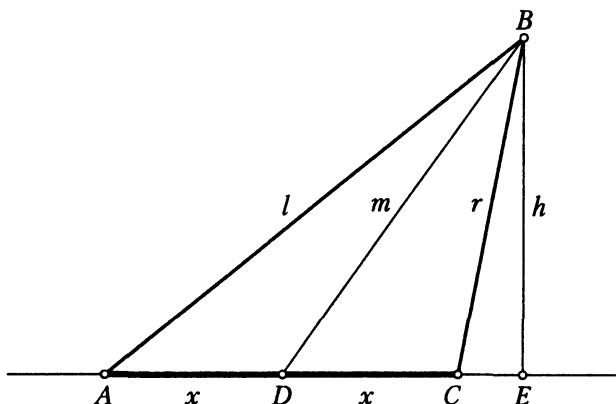


Рис. Р15.1. Треугольник с высотой и медианой, опущенными из одной точки

Обозначим длину высоты h , длину медианы m , длину биссектрисы b (рис. Р15.1). Длину отрезка DE обозначим d . Ее легко определить по теореме Пифагора:

$$d = \sqrt{m^2 - h^2}.$$

Поиск будем вести по половине длины основания x . Для фиксированного x можно найти длины сторон AB и BC треугольника — l и r соответственно. Они определяются по теореме Пифагора:

$$l_x = \sqrt{h^2 + (d + x)^2}; \quad r_x = \sqrt{h^2 + (d - x)^2}.$$

Отношение длин отрезков, на которые биссектриса делит основание, равно отношению соответствующих боковых сторон. Отсюда проекция биссектрисы на прямую AC определяется как

$$bp_x = d - x + \frac{2xr_x}{l_x + r_x}.$$

А сама длина биссектрисы при таком выборе половины длины основания будет равна

$$b_x = \sqrt{bp_x^2 + h^2}.$$

Итак, функция длины биссектрисы от половины длины основания найдена. Теперь нужно определить границы поиска. Минимальная длина основания равна 0. А как оценить максимальное значение половины длины основания? Установим его сначала равным 1, а затем будем умножать на 2 до тех пор, пока найденная для этого значения биссектриса не окажется меньше заданной во входных данных. Попутно можно и минимальное значение половины длины основания корректировать:

```

min := 0;
max := 1;
while getB(max) >= b do begin
    min := max;

```

```

max := max * 2;
end;

```

После этого выполняется $\text{getB}(\min) \geq b$ и $\text{getB}(\max) < b$. Как и в варианте 1 решения задачи 12 В, будем сужать интервал до предела точности вещественного типа:

```

while true do begin
  mid := (min + max) / 2;
  if (mid = min) or (mid = max) then
    break;
  if getB(mid) < b then
    max := mid
  else
    min := mid;
end;

```

По окончании цикла в переменной $\min$ будет содержаться искомая половина длины основания, в переменных mid и $\max$, впрочем, тоже — значения в этих трех переменных будут различаться крайне незначительно.

В качестве ответа задачи можно выдавать $\min * h$, причем лучше не задавать формат вывода : 0: 3, так как требуется обеспечить точность 0,001, а не вывести число с тремя знаками после запятой. Точность обеспечить легче всего, если вывести ответ как есть — в формате по умолчанию с примерно 20 десятичными знаками.

Детали реализации: в качестве базового вещественного типа лучше всего выбрать как можно более точный.

Вариант 2

Решение предложил Никита Рыбак.

Можно вывести формулу, выражающую длину основания треугольника через длины биссектрисы, медианы и высоты, опущенных из одной точки.

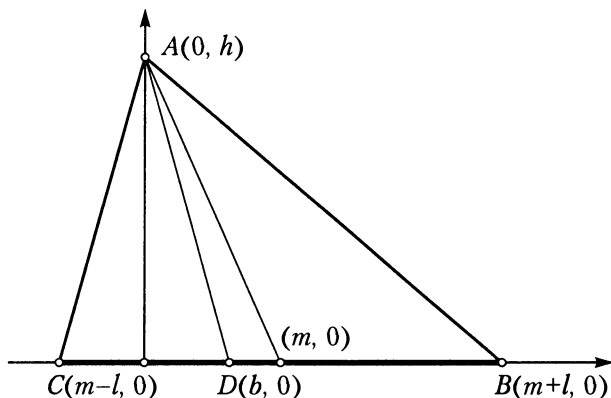


Рис. Р15.2. Треугольник с высотой, медианой и биссектрисой, опущенными из одной точки

Обозначим h высоту, l — половину длины основания, m — длину проекции медианы на основание (определяется из длины медианы и высоты), b — длину проекции биссектрисы на основание (определяется из длины биссектрисы и высоты) (рис. P15.2).

Введем декартову систему координат. Начало координат находится на пересечении основания и высоты, ось OX направлена вдоль основания в сторону пересечения с медианой, ось OY направлена в сторону вершины, из которой опущена высота. Вершины исходного треугольника имеют координаты: $A(0, h)$, $B(m + l, 0)$, $C(m - l, 0)$. Точка D — общая точка биссектрисы и основания — имеет координаты $(b, 0)$.

Длины отрезков находим по следующим формулам:

$$AC = \sqrt{(m - l)^2 + h^2}, \quad AB = \sqrt{(m + l)^2 + h^2}, \quad CD = b - (m - l), \quad BD = (m + l) - b.$$

Отношение длин отрезков, на которые биссектриса делит основание, равно отношению соответствующих боковых сторон: $\frac{AC}{CD} = \frac{AB}{BD}$, или, в обозначениях l, m, b и h :

$$\left((m - l)^2 + h^2\right)(m + l - b)^2 = \left((m + l)^2 + h^2\right)(b - m + l)^2.$$

Из этого уравнения выполнением элементарных, но очень громоздких преобразований можно получить выражение l^2 через m, b и h (замечу еще раз, что m и b — это длины *не* медианы и биссектрисы, а их проекций на основание).

Окончательная формула не приводится, чтобы читатель, желающий реализовать это решение, ощутил на себе все его достоинства и недостатки.

Сравнение алгоритмов

Олимпиады могут проводиться по разным правилам — например, они могут быть личными (один человек, один компьютер) или командными (несколько человек, один компьютер).

На командной олимпиаде, вполне возможно, имеет смысл экономить время у компьютера, и тогда одного из членов команды можно усадить за аналитическое решение задачи даже без надежды на успех.

Если же олимпиада личная, то имеет значение как время ввода исходного текста, так и время теоретического решения. Если аналитическое решение не приходит в голову сразу, или если оно приводит к системам уравнений второй и более степени, или если получаются уравнения высоких степеней, то задачу нужно решать численно.

Решение задачи 15 С. Формирование поезда

Вариант 1

Тема: динамическое программирование.

Обозначим $L_{лев}$ — тип левой сцепки локомотива (А или В), $L_{пр}$ — тип правой сцепки локомотива. Будем рассматривать только последовательности вагонов, у которых

левая сцепка самого левого вагона имеет тип $L_{\text{лп}}$. Обозначим $\text{count}(l, r)$ количество вагонов во входных данных с типом левой сцепки l и правой r (l и r — или А, или В). Обозначим $C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, E)$ количество различных последовательностей вагонов, которые можно получить из $N_{\text{АА}}$ вагонов типа АА, $N_{\text{АВ}}$ вагонов типа АВ, $N_{\text{ВА}}$ вагонов типа ВА и $N_{\text{ВВ}}$ вагонов типа ВВ, причем левой сцепкой самого левого вагона последовательности должна быть $L_{\text{лп}}$, а правой сцепкой самого правого вагона должна быть E . Также предполагается, что $0 \leq N_{\text{АА}} \leq \text{count}(\text{А}, \text{А})$, $0 \leq N_{\text{АВ}} \leq \text{count}(\text{А}, \text{В})$, $0 \leq N_{\text{ВА}} \leq \text{count}(\text{В}, \text{А})$ и $0 \leq N_{\text{ВВ}} \leq \text{count}(\text{В}, \text{В})$.

Будем находить значения $C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, E)$ в порядке увеличения $N_{\text{АА}} + N_{\text{АВ}} + N_{\text{ВА}} + N_{\text{ВВ}}$, начиная с 0.

Если длина состава равна 0, будем считать, что правая сцепка состава такая же, как и левая, а левая фиксирована — $L_{\text{лп}}$. Таким образом, $C(0, 0, 0, 0, E) = 1$, если $E = L_{\text{лп}}$, и $C(0, 0, 0, 0, E) = 0$, если $E \neq L_{\text{лп}}$.

Найдем $C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, E)$ при $N_{\text{АА}} + N_{\text{АВ}} + N_{\text{ВА}} + N_{\text{ВВ}} > 0$, считая, что для всех меньших сумм первых четырех параметров значения C известны.

Пусть есть состав, содержащий $N_{\text{АА}}$ вагонов типа АА, $N_{\text{АВ}}$ вагонов типа АВ, $N_{\text{ВА}}$ вагонов типа ВА и $N_{\text{ВВ}}$ вагонов типа ВВ, причем правая сцепка состава — А. Очевидно, что самым правым вагоном состава мог бы быть или АА, или ВА (если, конечно, таких вагонов ненулевое количество). Если убрать самый правый вагон, останется состав на единицу меньшей длины, в котором количество вагонов, соответственно, или АА, или ВА меньше на единицу, а самой правой сцепкой является, соответственно, или А, или В. Количество таких составов известно (значения C получаются в порядке увеличения длины состава). Из двух разных составов на единицу меньшей длины добавлением справа вагона получаются разные составы. Поэтому $C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, \text{А}) = C(N_{\text{АА}} - 1, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, \text{А}) + C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}} - 1, N_{\text{ВВ}}, \text{В})$. Если какое-то из слагаемых не существует ($N_{\text{АА}} = 0$ или $N_{\text{ВА}} = 0$ соответственно), то оно в сумму не добавляется. Аналогично, $C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, \text{В}) = C(N_{\text{АА}}, N_{\text{АВ}} - 1, N_{\text{ВА}}, N_{\text{ВВ}}, \text{А}) + C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}} - 1, \text{В})$.

Ответом задачи является сумма всех $C(N_{\text{АА}}, N_{\text{АВ}}, N_{\text{ВА}}, N_{\text{ВВ}}, L_{\text{лсв}})$, у которых $N_{\text{АА}} + N_{\text{АВ}} + N_{\text{ВА}} + N_{\text{ВВ}} = K$, а также, естественно, выполняются ограничения на количество вагонов каждого типа.

Детали реализации:

- Максимальный ответ задачи не может быть представлен значением типа `longint`, нужно использовать `comr`. Оценить максимальный результат можно следующим образом. Предположим, что количество вагонов каждого вида неограниченно. Если фиксировать левую часть состава определенной длины, к ней справа можно добавить вагоны двух видов. А если фиксирован $K - 1$ левый вагон, то самый правый и вообще определяется однозначно — по правой сцепке $(K - 1)$ -го вагона и по левой сцепке локомотива. Таким образом, количество составов не превосходит 2^{K-1} . Для $K = 40$ это 2^{39} , что меньше максимального числа типа `comr`: $2^{63} - 1$.
- В сумме вагонов всех видов 40. Вагонов каждого вида может быть до 40. Если реализовать C как пятимерный массив `array [0..40, 0..40, 0..40, 0..40,`

'A' .. 'B'] of comp, потребуется более 40 Мбайт памяти. Далее приводятся две реализации, позволяющие обойтись значительно меньшим объемом памяти.

Реализация 1

Предложил Никита Рыбак.

Пусть известна длина состава $N_{AA} + N_{AB} + N_{BA} + N_{BB}$, начальная и конечная сцепки состава ($L_{\text{нп}}$ и E), а также количество вагонов AA и AB (N_{AA} и N_{AB}). Утверждается, что по этим данным количество вагонов BA и BB (N_{BA} и N_{BB}) определяется однозначно.

Будем рассматривать состав слева направо. При прохождении каждого вагона тип сцепки или меняется на противоположный (вагоны типа AB и BA), или остается прежним (вагоны типа AA и BB). Отсюда легко получить, что если тип сцепок на концах какого-то участка одинаков, то на этом участке количества вагонов типов AB и BA совпадают. Если тип левой сцепки участка A, а правой — B, то на участке вагонов типа AB на единицу больше, чем вагонов BA. Если же тип левой сцепки участка B, а правой — A, то наоборот, на участке вагонов типа BA на единицу больше, чем вагонов AB.

Итак, если $L_{\text{нп}} = E$, то $N_{BA} = N_{AB}$; если $L_{\text{нп}} = A$, а $E = B$, то $N_{BA} = N_{AB} - 1$; и наконец, если $L_{\text{нп}} = B$, а $E = A$, то $N_{BA} = N_{AB} + 1$.

Если известно суммарное количество вагонов $N_{AA} + N_{AB} + N_{BA} + N_{BB}$ и количество вагонов каждого из трех видов N_{AA} , N_{AB} и N_{BA} , то количество вагонов оставшегося четвертого вида N_{BB} находится элементарно.

Получается, что для хранения значений C (см. обозначения варианта 1) для одной длины состава нужен только трехмерный массив `array [0..40, 0..40, 'A'..'B'] of comp` размером всего лишь 26 896 байт. Чтобы вычислить значения C для состава очередной длины требуются только значения C для состава на единицу меньшей длины, так что можно обойтись всего двумя такими массивами.

Реализация 2

Предположим, что количество четверок ($N_{AA}, N_{AB}, N_{BA}, N_{BB}$) при фиксированной длине состава $N_{AA} + N_{AB} + N_{BA} + N_{BB}$ не может быть слишком большим (скажем, оно не превосходит тысячи).

Можно использовать следующую структуру:

```
type
  TTable = record
    k : integer;
    len : integer;
    a : array [1..1000] of record
      aa, ab, ba, bb : byte;
      count : comp;
    end;
  end;
```

Поле `k` фиксирует для всей таблицы то, какой должна быть сумма `aa + ab + ba + bb`. Поле `len` указывает, сколько элементов массива `a` заполнено. Поля `aa`, `ab`, `ba` и `bb`

служат для хранения количества вагонов соответствующего вида. В поле count хранится $C(aa, ab, ba, bb, E)$ — в обозначениях варианта 1. Заметим: в структуре нет поля для фиксации того, какой должна быть правая сцепка состава E , так что для каждой длины состава нужно использовать две переменные типа TTable — для составов с правой сцепкой A и для составов с правой сцепкой B.

Заносить очередную четверку в таблицу можно с помощью следующей процедуры:

```
procedure put(var t : TTable; aa, ab, ba, bb : byte; count : comp);
begin
    if aa + ab + ba + bb <> t.k then
        halt(1);
    inc(t.len);
    t.a[t.len].aa := aa;
    t.a[t.len].ab := ab;
    t.a[t.len].ba := ba;
    t.a[t.len].bb := bb;
    t.a[t.len].count := count;
end;
```

Перед занесением первого элемента таблица должна быть проинициализирована:

```
procedure initTable(var t : TTable; k : integer);
begin
    t.k := k;
    t.len := 0;
end;
```

Функция линейного поиска может выглядеть так:

```
function get(var t : TTable; aa, ab, ba, bb : integer) : comp;
var
    i : integer;
begin
    if aa + ab + ba + bb <> t.k then
        halt(1);

    if (aa < 0) or (ab < 0) or (ba < 0) or (bb < 0) then begin get := 0;
        exit;
    end;

    for i := 1 to t.len do
        if
            (t.a[i].aa = aa) and (t.a[i].ab = ab) and
            (t.a[i].ba = ba) and (t.a[i].bb = bb)
        then begin
            get := t.a[i].count;
```

```

    exit:
  end:
  halt(1):
end:

```

При отрицательном количестве вагонов какого-то типа выдается нулевой ответ, чтобы в основном алгоритме отдельно не рассматривать случай отсутствия вагонов какого-то типа. Например, в формуле $C(N_{AA}, N_{AB}, N_{BA}, N_{BB}, A) = C(N_{AA} - 1, N_{AB}, N_{BA}, N_{BB}, A) + C(N_{AA}, N_{AB}, N_{BA} - 1, N_{BB}, B)$ первое слагаемое не имеет смысла при $N_{AA} = 0$, а второе — при $N_{BA} = 0$.

Обратите внимание на то, что в TTable используется тип byte в целях экономии места, а в функции get соответствующие параметры имеют тип integer. Это сделано для того, чтобы они могли принимать отрицательные значения. Также не рекомендуется делать byte типом одиночных глобальных переменных, чтобы вычитание единицы из нуля дало отрицательное число (а не 255 при выключенных проверках диапазонов).

К сожалению, решение, использующее таблицы с линейным поиском, на некоторых тестах ($N = K = 40$, вагонов всех типов поровну) работает в 2–3 раза дольше отведенного времени. Используем двоичный поиск.

Чтобы двоичный поиск можно было применить, нужно чтобы ключи можно было сравнивать не только на равенство, но и на меньше (более подробно см. вариант 3 решения задачи 9 F). Введем на четверках лексикографический порядок: сначала нужно сравнивать количество вагонов AA, при равенстве — количество AB, затем BA и BB. Можно, конечно, написать функцию сравнения четверок, как в варианте 3 решения задачи 9 F, но здесь мы применим другой способ сравнения — с помощью наложения представления одних данных на другие (понятие «запись с вариантами» языка Паскаль):

```

type
  Titem = record
    count : comp;
    case boolean of
      true : (
        bb, ba, ab, aa : byte;
      );
      false : (
        key : longint;
      );
    end;
  end;

  TTable = record
    k : integer;
    len : integer;
    a : array [1..1000] of Titem;
  end;

```


Для хранения четырех полей, bb, ba, ab, aa, и поля key используется одно и то же место в памяти. Таким образом, если полю key присвоить значение \$12345678, то поле bb примет значение \$78, поле ba — \$56, поле ab — \$34, а поле aa — \$12. И наоборот, если присвоить значения полям bb, ba, ab и aa, потом комбинацию этих данных можно будет считать из поля key.

Обратите внимание на то, что порядок описания полей изменился с aa, ab, ba, bb на противоположный: bb, ba, ab, aa. Это связано с тем, что не все равно, в каком порядке накладывать поля типа byte на поле типа longint. Порядок на четверках был установлен так, что четверки сначала сравниваются по количеству вагонов AA. Именно это поле должно быть наложено на самый старший байт поля key. В последнюю очередь происходит сравнение по количеству вагонов BB, и соответствующее поле должно быть наложено на самый младший байт поля key.

Если бы ограничение на число вагонов каждого вида было не 40, а более 127, схема с использованием наложения не сработала бы. Если самый старший байт числа типа longint 128 или больше, то число является отрицательным. Например, (\$80, 0, 0, 0) > (\$7F, 0, 0, 0), но \$80000000 < \$7F000000.

Функция двоичного поиска может выглядеть следующим образом:

```
function get(var t : TTable; aa, ab, ba, bb : integer) : comp;
var
  need : TItem;
  high, low, mid : integer;
begin
  if aa + ab + ba + bb <> t.k then
    halt(1);

  if (aa < 0) or (ab < 0) or (ba < 0) or (bb < 0) then begin
    get := 0;
    exit;
  end;

  need.aa := aa;
  need.ab := ab;
  need.ba := ba;
  need.bb := bb;

  low := 1;
  high := t.len;

  while true do begin
    mid := (low + high) div 2;
    if t.a[mid].key = need.key then begin
      get := t.a[mid].count;
      exit;
    end;
  end;
```

```

end
else if t.a[mid].key < need.key then
    low := mid + 1
else
    high := mid - 1;

    if low > high then
        halt(1);
    end;
end;

```

Чтобы двоичный поиск можно было применять, требуется, чтобы массив был отсортирован. В этой задаче есть возможность *не* сортировать массив явно. Можно просто заносить элементы в нужном порядке. Например, цикл перебора четверок может выглядеть следующим образом:

```

for aa := 0 to count['A', 'A'] do
    for ab := 0 to count['A', 'B'] do
        for ba := 0 to count['B', 'A'] do
            for bb := 0 to count['B', 'B'] do
                if aa + ab + ba + bb = ... then begin
                    ...
                end;
            end;
        end;
    end;
end;

```

Так четверки перебираются как раз в лексикографическом порядке, и их можно вносить в таблицу с помощью той же процедуры put, что была написана для линейного поиска.

Вариант 2

Тема: комбинаторика, перебор.

Пусть фиксированы правая и левая сцепки состава — $L_{\text{пр}}$ и $L_{\text{лев}}$ соответственно. Если известно количество вагонов АВ, то можно найти количество вагонов ВА (см. реализацию 1 варианта 1). А если также известно и количество вагонов АА, то можно найти и количество вагонов ВВ.

Будем перебирать по количеству вагонов АВ и по количеству вагонов АА. Если состав с таким количеством этих вагонов может существовать (учитывая ограничения на количество вагонов разных видов), то по комбинаторным формулам найдем количество различных составов с таким набором вагонов.

Пусть, например, состав должен состоять из двух вагонов АВ, одного вагона ВА, двух АА и трех ВВ, причем задан локомотив ВА. Выпишем «остов» из вагонов АВ и ВА (слева и справа показаны сцепки локомотива):

А] [АВ] [ВА] [АВ] [В

Очевидно, остов можно получить только одним способом: в нем должны чередоваться вагоны АВ и ВА, причем сцепки начала и конца фиксированы сцепками локомотива.

Теперь нужно найти число способов разместить в места А [А остова вагоны АА, а в места В] [В — вагоны ВВ. Очевидно, любой способ размещения вагонов АА может комбинироваться с любым способом размещения вагонов ВВ. Все комбинации перечислять не будем, ограничимся только перечислением отдельно способов расставить вагоны АА и отдельно — ВВ:

А] [АА] [АА] [АВ] [ВА] [АВ] [В

А] [АА] [АВ] [ВА] [АА] [АВ] [В

А] [АВ] [ВА] [АА] [АА] [АВ] [В

А] [АВ] [ВВ] [ВВ] [ВВ] [ВА] [АВ] [В

А] [АВ] [ВВ] [ВВ] [ВА] [АВ] [ВВ] [В

А] [АВ] [ВВ] [ВА] [АВ] [ВВ] [ВВ] [В

А] [АВ] [ВА] [АВ] [ВВ] [ВВ] [ВВ] [В

Обозначим N_{AA} количество вагонов АА в составе, N_{BB} — количество вагонов ВВ, P_{AA} — количество мест А [А остова, на которые можно разместить вагоны АА (в примере два таких места), P_{BB} — аналогично, количество мест, на которые можно разместить вагоны ВВ. Количество способов сформировать состав равно произведению количества способов разместить N_{AA} одинаковых вагонов по P_{AA} разным местам и количества способов разместить N_{BB} одинаковых вагонов по P_{BB} разным местам.

Итак, пришли к следующей задаче. Есть N одинаковых предметов и P разных мест. Требуется найти количество способов размещения этих N предметов по этим P местам, причем в каждом месте может находиться 0 или более предметов. Два способа считаются разными, если в каком-то месте расположено разное число предметов. Пусть предметы как-то распределены. Закодируем это распределение в виде последовательности строчных букв a и b . Сначала напишем столько букв a , сколько предметов в первом месте. Затем напишем разделитель — букву b . Затем напишем столько букв a , сколько предметов во втором месте. Затем опять напишем b в качестве разделителя и т. д. Только после последней последовательности букв a разделитель писать не надо.

В результате распределение оказалось закодировано последовательностью N букв a и $P - 1$ буквы b . Нетрудно убедиться в том, что разным распределениям соответствуют разные последовательности, и что любой последовательности N букв a и $P - 1$ буквы b соответствует какое-то распределение.

Таким образом, задача сведена к подсчету числа строк, состоящих из N букв a и $P - 1$ буквы b . А эта задача эквивалентна задаче о том, сколькими способами можно выбрать N предметов из $N + P - 1$ одинакового, а это уже определение количества сочетаний — C_{N+P-1}^N .

Рекомендуется значения C_N^K ($0 \leq K \leq N \leq 40$) вычислить один раз заранее и занести в массив, а потом пользоваться этими значениями.

Вычислять C_N^K следует по формулам $C_N^0 = 1$, $C_N^N = 1$, $C_N^K = C_{N-1}^{K-1} + C_{N-1}^K$.

Следует обратить внимание на случай, когда нужно найти количество способов разместить N предметов по $P = 0$ местам. Такое случается, когда не используется ни одного вагона АВ или ВА, а типы сцепок локомотива совпадают, например:

A] [A

Если в этот остов нужно разместить 0 вагонов ВВ, то логически задача вполне разрешима — это количество способов можно принять равным 1. Если же в этот остов нужно разместить 1 или более вагонов ВВ, то, очевидно, этого сделать нельзя.

Итак, формула C_{N+P-1}^N не рассчитана на случай $P = 0$, его нужно рассматривать отдельно.

Решение задачи 15 D. Стена

Тема: геометрия.

На самом деле это задача на нахождение длины обычной выпуклой оболочки. Требование, чтобы стена отстояла от стен замка как минимум на L футов, отражается только на длине кривых в углах. А длины этих кривых в сумме дают длину окружности радиусом L (рис. P15.3).

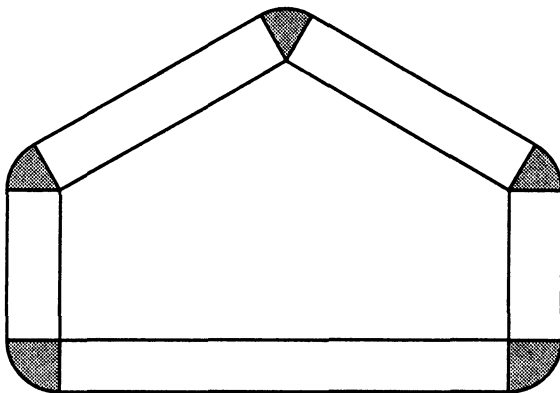


Рис. P15.3. Соотношение длины выпуклой оболочки и ответа задачи

Решение задачи 15 Е. Семечки

Тема: сокращение перебора.

Очевидно, оптимальность места и цены не зависит от количества всех покупателей на базаре. Поэтому можно рассчитывать доход для любого конкретного значе-

ния количества покупателей. Примем количество покупателей равным 200. Половина из них (100) идет с левого края ряда, половина — с правого. Таким образом, данные в условии проценты B_k теперь будут совпадать с количеством людей.

Обозначим $Pr(i)$ цену семечек на исходном i -м месте.

Сначала решим подзадачу — определим доход для фиксированных места L и цены P .

Рассмотрим случай, когда покупатель идет слева. Если слева от нашего места есть цена строго меньшая нашей, то есть $Pr(i) < P$ для какого-то $i = 1 \dots L$, то ни один из идущих слева наше место не выберет. Если справа от нашего места есть цена меньшая или равная нашей, то есть $Pr(i) \leq P$ для какого-то $i = L + 1 \dots N - 1$, то все покупатели, дошедшие до этого места, к нашему не вернуться. Обозначим ML — номер ближайшего справа места, где цена меньше или равна нашей. Если цены, меньшей или равной P , справа от нашего места нет, то установим $ML = N$. Обратите внимание на то, что все номера мест соответствуют положению, существовавшему до нашего выхода на рынок.

Итак, если слева от нашего места есть меньшая цена, то от покупателей, идущих слева, дохода не будет. Если же слева от нашего места нет меньшей цены, то доход от покупателей, идущих слева, составит $P(B_{L+1} + \dots + B_{ML})$.

Аналогично рассматривается случай, когда покупатель идет справа. Если справа от нашего места есть строго меньшая цена, то есть $Pr(i) < P$ для какого-то $i = L + 1 \dots N$, то все идущие справа покупатели для нас потеряны. Обозначим MR — номер ближайшего слева места, где цена меньше или равна нашей. Если цены, меньшей или равной P , слева от нашего места нет, то установим $MR = 1$. Опять все номера мест — до нашего выхода на рынок.

Итак, если справа от нашего места есть меньшая цена, то от покупателей, идущих справа, дохода не будет. Если же справа от нашего места нет меньшей цены, то доход от покупателей, идущих справа, составит $P(B_{N-L+1} + \dots + B_{N-MR+1})$. Обратите внимание на то, что для покупателей, идущих справа, B_1 соответствует не $Pr(1)$, а $Pr(N)$.

Рассмотрим, сколько действий нужно совершить, чтобы найти доход при фиксированных месте и цене. Нужно найти минимум цен слева и справа от нашего места — это можно сделать за N сравнений. Затем нужно сложить не более N разных B_k — это тоже не более N действий.

Будем считать, что для определения дохода при фиксированных цене и месте нужно совершить N «элементарных» действий.

Теперь определим, сколько всего «элементарных» действий нужно совершить, если перебирать все 9999 возможных цен и все $N - 1$ возможное положение. Для $N = 100$ количество «элементарных» действий составит чуть меньше 100 миллионов. Учитывая то, что «элементарное» действие заключается в нескольких индексированиях массивов, можно сделать вывод, что на указанном в предисловии компьютере полный перебор в требуемое время не уложится.

Сокращение перебора основано на следующей идее: не нужно рассматривать все 9999 возможных цен. Нужно рассматривать только цены, равные какой-то из исходных, а также цены на единицу меньше или больше какой-нибудь из исходных. Покажем, почему можно не рассматривать другие цены.

Пусть мы установили цену P , а на других местах нет ни цены P , ни цены $P + 1$. При этом оказывается, что если P была меньше какой-то исходной цены, то $P + 1$ тоже будет меньше нее, а если P была больше какой-то исходной цены, то $P + 1$ тоже будет больше. Из этого следует, что наше место с ценой $P + 1$ выберут точно те же покупатели, которые выбрали наше место с ценой P . То есть при повышении цены на 1 количество выбравших наше место покупателей не изменится. Если количество покупателей ненулевое, то наш доход возрастет.

Таким образом, показано, что достаточно рассмотреть не более $2N$ цен. При таком подходе для $N = 100$ количество «элементарных» действий составит около 2 миллионов, что вполне реально выполнить на указанном компьютере за указанное в условии время.

Решение задачи 15 F. Умножение многочленов

Тема: метод рекурсивного спуска.

В этой задаче проблемы две — ввод и вывод. Умножение многочленов, представленных в удобном виде (массивом коэффициентов), сложности не представляет.

Ввод лучше всего организовать в стиле рекурсивного спуска (подробности — в решении задачи 12 F). Правила РБНФ для многочлена таковы:

Многочлен = ["-"] Одночлен { ("+" | "-") Одночлен }.

Одночлен = число ["x" Степень] | "x" Степень.

Степень = ["^" число].

При выводе нужно учесть довольно много особых случаев:

- Слагаемые с коэффициентом 0 не пишутся, однако многочлен, являющийся нулем, нужно выводить.
- Когда первый коэффициент отрицательный, знак перед ним ставить нужно, когда положительный — нет.
- Коэффициенты 1 и -1 не пишутся, но только если они не относятся к свободному члену.
- Степень 1 не пишется, а если степень 0, то не пишется еще и x .

Предлагается выделить процедуру вывода одночлена:

type

integer = longint;

tPoly = array [0..20] of integer; {коэффициенты многочлена}

```
{
Вывод одночлена с положительным коэффициентом coef степени pow.
}
```

```
procedure showMono(coef, pow : integer);
```

```
begin
```

```
  if coef <= 0 then
```

```
    halt(1);
```

```
  if pow = 0 then
```

```
    write(coef)
```

```
  else begin
```

```
    if coef <> 1 then
```

```
      write(coef);
```

```
    write('x');
```

```
    if pow <> 1 then
```

```
      write('^', pow);
```

```
  end;
```

```
end;
```

```
{
Вывод многочлена
}
```

```
procedure show(p : tPoly);
```

```
var
```

```
  i : integer;
```

```
begin
```

```
  i := high(p);
```

```
  while (i > 0) and (p[i] = 0) do
```

```
    dec(i);
```

```
  if p[i] < 0 then begin
```

```
    write('-');
```

```
    showMono(-p[i], i);
```

```
  end
```

```
  else if p[i] > 0 then
```

```
    showMono(p[i], i)
```

```
  else
```

```
    write(0); {i = 0}
```

```
  for i := i - 1 downto 0 do begin
```

```
    if p[i] < 0 then begin
```

```
      write('-');
```

```
      showMono(-p[i], i);
```

```
    end
```

```
        else if p[i] > 0 then begin
            write('+');
            showMono(p[i], i);
        end;
    end;
    writeln;
end;
```


Приложение А.

Характеристики встроенных типов Турбо Паскаля

Целые типы

Тип	Размер, байт	Формула границ	Десятичное представление границ
shortint	1	$-2^7 \dots (2^7 - 1)$	-128...127
byte	1	$0 \dots (2^8 - 1)$	0...255
integer	2	$-2^{15} \dots (2^{15} - 1)$	-32 768...32 767
word	2	$0 \dots (2^{16} - 1)$	0...65 535
longint	4	$-2^{31} \dots (2^{31} - 1)$	-2 147 483 648...2 147 483 647
comp	8	$-2^{63} \dots (2^{63} - 1)$	-9 223 372 036 854 775 808...9 223 372 036 854 775 807

Тип comp формально является вещественным типом, в частности, при его использовании нужно писать { $\$N+$,E-}, а также для операндов этого типа не определены div и mod.

Вещественные типы

Тип	Размер, байт	Минимальное положительное число	Максимальное число	Точность, десятичных цифр
real	6	2.9e-39	1.7e38	11
single	4	1.5e-45	3.4e38	7
double	8	5.0e-324	1.7e308	15
extended	10	3.4e-4932	1.1e4932	19

Тип real не поддерживается аппаратно, поэтому скорость операций с ним на современных процессорах значительно ниже, чем для других типов.

Приложение Б.

Процедуры длинной арифметики

Ниже приводится простейший вариант процедур длинной арифметики, в котором

- все вычисления проводятся в десятичной системе счисления;
- не хранится длина числа.

Эти особенности уменьшают сложность написания процедур, но скорость их работы не будет максимально возможной. В подавляющем числе случаев такой подход себя оправдывает.

const

numlen = ...;

{

езде в циклах используются переменные типа integer.

если numlen > maxint, нужно использовать переменные типа word

}

type

number = **array** [1..numlen] of byte;

{

переменной a типа number представлено число $\sum a[i] \cdot 10^{(i-1)}$

}

{

Обнуление

}

procedure set0(var n : number);

var

i : integer;

begin

for i := 1 to numlen do

n[i] := 0;

end;

```
{
Занесение короткого числа
}
procedure setN(var n : number; short : integer);
var
    i : integer;
begin
    set0(n);
    i := 1;
    while short > 0 do begin
        n[i] := short mod 10;
        short := short div 10;
        inc(i);
    end;
end;
```

```
{
Получение длины числа
Если число представляет ноль, длина 1.
}
function len(var n : number) : integer;
var
    i : integer;
begin
    for i := numlen downto 1 do
        if n[i] <> 0 then begin
            len := i;
            exit;
        end;
    len := 1;
end;
```

```
{
Вывод числа и перевод строки
}
procedure show(var n : number);
var
    i : integer;
begin
    for i := len(n) downto 1 do
        write(n[i]);
    writeln;
end;
```

```
{  
Добавление короткого числа  
}
```

```
procedure addShort(var n : number; short : integer);  
var  
    i : integer;  
begin  
    i := 1;  
    while short > 0 do begin  
        short := short + n[i];  
        n[i] := short mod 10;  
        short := short div 10;  
        inc(i);  
    end;  
end;
```

```
{  
Умножение на короткое число  
}
```

```
procedure mulShort(var n : number; short : integer);  
var  
    i, carry : integer;  
begin  
    carry := 0;  
    for i := 1 to numlen do begin  
        carry := carry + n[i] * short;  
        n[i] := carry mod 10;  
        carry := carry div 10;  
    end;  
    if carry <> 0 then {диагностика переполнения}  
        halt(1);  
end;
```

```
{  
Деление на короткое число и получение остатка  
}
```

```
procedure divShort(var n : number; divisor : integer; var rem : integer);  
var  
    i : integer;  
begin  
    rem := 0;  
    for i := numlen downto 1 do begin  
        rem := rem * 10 + n[i];  
        n[i] := rem div divisor;
```

```

        rem := rem mod divisor;
    end;
end;

{
Добавление длинного числа
}
procedure add(var n1, n2 : number);
var
    i, carry : integer;
begin
    carry := 0;
    for i := 1 to numlen do begin
        carry := carry + n1[i] + n2[i];
        n1[i] := carry mod 10;
        carry := carry div 10;
    end;
    if carry <> 0 then {диагностика переполнения}
        halt(1);
    end;
end;

```

```

{
Умножение длинных чисел
}
procedure mul(n1, n2 : number; var n3 : number);
var
    i1, i2, i3, len1, len2, carry : integer;
begin
    set0(n3);
    len1 := len(n1);
    len2 := len(n2);
    for i1 := 1 to len1 do
        for i2 := 1 to len2 do begin
            i3 := i1 + i2 - 1;
            carry := n1[i1] * n2[i2];
            while carry > 0 do begin
                carry := carry + n3[i3];
                n3[i3] := carry mod 10;
                carry := carry div 10;
                inc(i3);
            end;
        end;
    end;
end;
end;

```

```
{
Сравнение чисел
если n1 < n2, выдает -1
если n1 = n2, выдает 0
если n1 > n2, выдает 1
}
function cmp(var n1, n2 : number) : integer;
var
    i : integer;
begin
    for i := numlen downto 1 do begin
        if n1[i] < n2[i] then begin
            cmp := -1;
            exit;
        end;
        if n1[i] > n2[i] then begin
            cmp := 1;
            exit;
        end;
    end;
    cmp := 0;
end;
```

Дополнительная информация о задачах

Тренировка 1

Задача А. Простые числа.

- Источник: [Брудно] — задача 80.1.1 (стр. 6).
- Модификация ФМ: задается нижняя граница, границы подобраны специально для описанного алгоритма.
- Тесты: ФМ.

Задача В. Выражение.

- Источник: задача известная, условие сформулировал ФМ.
- Тесты: ФМ.

Задача С. Возрастающая подпоследовательность.

- Источник: [Ленинград, 1987] — задача 14.
- Модификация ФМ: снято ограничение на сложность алгоритма (задача упрощена).
- Тесты: ФМ.

Задача D. Треугольник и точка.

- Источник: [Брудно] — задача 83.2 (стр. 9).
- Модификация ФМ: точки имеют целочисленные координаты.
- Тесты: ФМ.

Задача Е. Степень.

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача F. Покер.

- Источник: [Брудно] — задача 86.2 (стр. 12).
- Модификация ФМ: комбинациям даны названия (тест на внимательность), добавлен случай Straight.
- Тесты: ФМ.

Тренировка 2

Задача А. Простые числа (2).

- Источник: [Брудно] — задача 80.1.1 (стр. 6).
- Модификация ФМ: задается нижняя граница, границы подобраны специально для описанного алгоритма.
- Тесты: ФМ.

Задача В. Перестановки.

- Источник: [Брудно] — задача 80.1.2 (стр. 6).
- Модификация ФМ: переставляются символы, а не числа.
- Тесты: ФМ.

Задача С. Маршрут.

- Источник: [Сипин] — 1993 г. — 1 тур — задача 1 (стр. 7).
- Модификация ФМ: ограничения увеличены, что исключило перебор.
- Тесты: ФМ.

Задача D. Пересечение отрезков.

- Источник: [Бадин] — задача 11 (стр. 33) — Областная олимпиада 1990/91 уч. г.
- Модификация ФМ: требуется не чтобы отрезки пересекались, а чтобы у них была общая точка.
- Тесты: ФМ.

Задача Е. Длинная сумма.

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача F. Спираль.

- Источник: [Брудно] — задача 81.3 (стр. 8).
- Тесты: ФМ.

Тренировка 3

Задача А. Разложение на простые множители.

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача В. Перестановки (2).

- Автор: ФМ.
- Тесты: ФМ.

Задача С. Копилка.

- Источник: [CERC, 1999] — задача «Piggy-Bank».
- Модификация ФМ: нужно найти не только минимальную, но и максимальную сумму.
- Тесты: оригинальные.

Задача D. Открытка и конверт.

- Источник: [Бадин] — задача 10 (стр. 9) — Областная олимпиада 1990/91 уч. г. Теоретический тур.
- Тесты: получены ФМ из тестов [NEERC] — 2002 г. — задача В.

Задача E. Длинное произведение.

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача F. Змейка.

- Источник: [Абрамов] — задача 691, в (стр. 123), задача была на городской олимпиаде школьников 1996 [Вологда, школы].
- Тесты: ФМ.

Тренировка 4

Задача A. Совершенные числа.

- Источник: [Брудно] — задача 84.3 (стр. 10).
- Модификация ФМ: задается нижняя граница.
- Тесты: ФМ.

Задача B. Разложение на слагаемые.

- Источник: [Брудно] — задача 85.1 (стр. 11).
- Тесты: ФМ.

Задача C. Гангстеры.

- Источник: [NEERC] — 1998 г. — задача H.
- Тесты: оригинальные.

Задача D. Площадь многоугольника.

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача E. Деление длинного числа на короткое.

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача F. Скобки.

- Источник: [Ленинград, 1987] — Задача 12, б.
- Тесты: ФМ.

Тренировка 5

Задача А. Дружественные числа.

- Источник: [Абрамов] — задача 560 (стр. 95), задача была на городской олимпиаде школьников 1995 [Вологда, школы].
- Тесты: ФМ.

Задача В. Скобки (2).

- Источник: задача известная, сформулировал ФМ.
- Тесты: ФМ.

Задача С. Маршрут (2).

- Источник: [Дальний Восток] — 1999 г. — задача А.
- Тесты: ФМ.

Задача D. Выпуклая оболочка.

- Источник: [Сипин] — 1993 г. — 1 тур — задача 2 (стр. 8).
- Тесты: получены ФМ из тестов [NEERC] — 2001 г. — задача D.

Задача Е. Системы счисления.

- Источник: [Брудно] — задача 85.6 (стр. 12).
- Модификация ФМ: допускаются системы счисления с основанием большим 10.
- Тесты: ФМ.

Задача F. День рождения.

- Автор: ФМ.
- Тесты: ФМ.

Тренировка 6

Задача А. Закраска прямой.

- Источник: [Ленинград, 1987] — задача 13.
- Тесты: ФМ.

Задача В. Суммы.

- Источник: [SEERC, 1999] — задача F.
- Модификация ФМ: снижены ограничения (если предоставлялись только 16-разрядные компиляторы, задача упрощена).
- Тесты: ФМ.

Задача С. Игра «Даты».

- Источник: [Бадин] — задача 13 (стр. 9).
- Модификация ФМ: чуть-чуть изменены правила.
- Тесты: ФМ.

Задача D. Площадь прямоугольников.

- Источник: [Сипин] — 1992 г. — Теоретический тур — задача 4 (стр. 7).

- Тесты: ФМ.

Задача Е. Lines.

- Источник: сюжет известный, сформулировал ФМ.
- Тесты: ФМ.

Задача F. Покраска лабиринта.

- Источник: [Урал, 1999] — задача F.
- Автор: Владимир Николаевич Пинаев.
- Тесты: ФМ.

Тренировка 7

Задача А. Упорядоченные дроби.

- Источник: [Брудно] — задача 82.2 (стр. 8).
- Модификация ФМ: увеличены ограничения для использования быстрой сортировки.
- Тесты: ФМ.

Задача В. Сообщение.

- Источник: [ZVN] — в 2003/2004 учебном году задача F.
- Тесты: оригинальные.

Задача С. Игра умножения.

- Источник: [Waterloo, 22.09.2001] — задача E.
- Тесты: ФМ.

Задача D. Прямая и квадраты.

- Источник: [Дальний Восток] — 2001 г. — задача B.
- Тесты: оригинальные.

Задача E. Lines (2).

- Источник: сюжет известный, сформулировал ФМ.
- Тесты: ФМ.

Задача F. Удаление клеток.

- Источник: [Ленинград, 1986] — задача 9.
- Модификация ФМ: границы подобраны специально для Турбо Паскаля.
- Тесты: ФМ.

Тренировка 8

Задача А. Несоставляемое число.

- Источник: [Брудно] — задача 85.3 (стр. 11).
- Тесты: ФМ.

Задача В. SMS.

- Источник: [Москва, 2003] — задача I.
- Тесты: ФМ.

Задача С. Дерево игры.

- Источник: [Рыбинск] — 2003 г. — задача I.
- Автор: Владимир Николаевич Пинаев.
- Тесты: оригинальные.

Задача D. Дуга на сфере.

- Источник: ФМ выделил подзадачу реальных задач.
- Тесты: получены ФМ из тестов [Урал, 1999] — задача С.

Задача Е. Путь коня.

- Источник: [Бадин] — задача 45 (стр. 17) — городская олимпиада 1992/93 уч. г. Практический тур.
- Тесты: ФМ.

Задача F. Грядки.

- Источник: [Рыбинск, 1998] — задача С.
- Тесты: ФМ.

Тренировка 9

Задача А. Диалог компьютеров.

- Источник: [NEERC] — 1999 г. — задача D.
- Тесты: оригинальные.

Задача В. Бросание кубика.

- Источник: [Вологда, вузы] — 2002 г. — задача З.
- Автор: Александр Степанович Сипин.
- Тесты: ФМ.

Задача С. Игра с монетами.

- Источник: [Вологда, школы] — область-2004 — 1 тур — задача З.
- Тесты: оригинальные.

Задача D. Бассейн реки.

- Источник: [Рыбинск] — 2003 г. — задача Н.
- Автор: Владимир Николаевич Пинаев.
- Тесты: оригинальные.

Задача Е. Ближайшее число.

- Источник: [Дальний Восток] — 2003 г. — задача А.

- Автор: Александр Сергеевич Кленин.
- Тесты: оригинальные.

Задача F. Прямоугольное деление.

- Источник: [Дальний Восток] — 1999 г. — задача D.
- Тесты: оригинальные.

Тренировка 10

Задача A. Анти-QuickSort.

- Источник: [Летние сборы, 2000].
- Тесты: оригинальные.

Задача B. Строки Фибоначчи.

- Источник: [SEERC, 1999] — задача B.
- Тесты: ФМ.

Задача C. Игра в зачеркивание.

- Источник: [Сипин] — 1993 г. — 2 тур — задача 4 (стр. 9).
- Тесты: ФМ.

Задача D. Граница многоугольника.

- Источник: [Вологда, вузы] — отборочный тур ВГПУ 2001 г. — задача 2.
- Тесты: ФМ.

Задача E. Путь спелеолога.

- Источник: [Воронеж, 2003] — отборочный тур.
- Тесты: ФМ.

Задача F. Дырявая ткань.

- Источник: [Дальний Восток] — 2001 г. — задача F.
- Тесты: оригинальные+ФМ.

Тренировка 11

Задача A. Последовательность.

- Автор: ФМ.
- Тесты: ФМ.

Задача B. Провода.

- Источник: [NEERC] — 2001 г. — задача C.
- Модификация ФМ: убраны вещественные числа.
- Тесты: оригинальные.

Задача С. Палиндромы.

- Источник: [Вологда, школы] — область-2004 — 1 тур — задача 2.
- Тесты: оригинальные.

Задача D. Круговая площадь.

- Источник: [Дальний Восток] — 2000 г. — задача F.
- Тесты: оригинальные.

Задача E. Гомер Симпсон.

- Источник: [UVA, 28.02.2003] — задача C.
- Модификация ФМ: ограничения усилены.
- Тесты: ФМ.

Задача F. Дробная арифметика.

- Источник: [Рыбинск] — 2003 г. — задача A.
- Автор: Владимир Николаевич Пинаев.
- Тесты: оригинальные.

Тренировка 12

Задача A. Последовательность (2).

- Источник: [Бадин] — задача 15 (стр. 10) — областная олимпиада 1990/91 уч. г., теоретический тур.
- Тесты: ФМ.

Задача B. Гирлянда.

- Источник: [NEERC] — 2000 г. — задача G.
- Тесты: оригинальные.

Задача C. Головоломка умножения.

- Источник: [Дальний Восток] — 2001 г. — задача E.
- Тесты: оригинальные.

Задача D. Точки в многоугольнике.

- Источник: [Вологда, вузы] — отборочный тур ВГПУ 2001 г. — задача 2.
- Тесты: ФМ.

Задача E. Водопровод.

- Источник: [Дальний Восток] — 2003 г. — задача C.
- Автор: Александр Сергеевич Кленин.
- Тесты: оригинальные.

Задача F. Химические реакции.

- Источник: [NEERC] — 2001 г. — задача E.
- Тесты: оригинальные.

Тренировка 13

Задача А. Двойная решетка.

- Источник: [Дальний Восток] — 1999 г. — задача Е.
- Тесты: оригинальные.

Задача В. Последовательность Фибоначчи.

- Источник: [Рыбинск] — 2001 г. — задача А.
- Автор: Владимир Николаевич Пинаев.
- Тесты: ФМ.

Задача С. Скобки (3).

- Источник: [NEERC] — 2001 г. — задача В.
- Тесты: оригинальные.

Задача D. Центр тяжести.

- Источник: [CERC1999] — задача «Lifting the Stone»
- Тесты: ФМ.

Задача Е. Сумма произведений.

- Источник: [Урал, 2000] — английский тур — задача В.
- Тесты: оригинальные.

Задача F. Статическая сложность.

- Источник: [SWERC, 1997] — задача В.
- Тесты: оригинальные.

Тренировка 14

Задача А. Марковский цикл.

- Источник: [Дальний Восток] — 1999 г. — задача С.
- Тесты: оригинальные.

Задача В. Д-44.

- Источник: [Вологда, вузы] — 2000 г. — задача 2.
- Автор: Александр Степанович Сипин.
- Тесты: ФМ.

Задача С. Упаковка символов.

- Источник: [NEERC] — 2002 г. — задача F.
- Тесты: оригинальные.

Задача D. Пирамиды.

- Источник: [Северный, 2002] — задача Н.
- Тесты: оригинальные.

Задача Е. Поле для крикета.

- Источник: [NEERC] — 2002 г. — задача С.
- Тесты: оригинальные.

Задача F. Электронная таблица.

- Источник: [Дальний Восток] — 2000 г. — задача С.
- Тесты: оригинальные.

Тренировка 15

Задача А. Игра с калькулятором.

- Источник: [Рыбинск] — 1999 г. — задача Е.
- Автор: Сергей Геннадьевич Волченков.
- Тесты: ФМ.

Задача В. Площадь треугольника.

- Источник: [Рыбинск] — 1999 г. — задача А.
- Автор: Сергей Геннадьевич Волченков.
- Тесты: оригинальные.

Задача С. Формирование поезда.

- Источник: [Рыбинск] — 2003 г. — задача С.
- Автор: Михаил Юрьевич Копачев.
- Тесты: оригинальные.

Задача D. Стена

- Источник: [NEERC] — 2001 г. — задача D.
- Тесты: оригинальные.

Задача Е. Семечки.

- Источник: [Дальний Восток] — 2001 г. — задача С.
- Тесты: оригинальные+ФМ.

Задача F. Умножение многочленов.

- Источник: [Сипин] — 1989 г. — Второй тур — задача 1 (стр. 5).
- Тесты: ФМ.

Использованные сокращения

1. [Абрамов]

Абрамов С. А., Гнездилова Г. Г., Капустина Е. Н., Селюн М. Н. Задачи по программированию — М.: Наука. Гл. ред. физ.-мат. лит., 1988. — 224 с.

2. [Бадин]
Бадин Н. М., Волченков С. Г., Дашиц Н. Л., Корнилов П. А. Ярославские олимпиады по информатике. — Ярославль: Издательство Ярославского областного института повышения квалификации педагогических и руководящих работников образования, 1995.
3. [Брудно]
Брудно А. Л., Каплан Л. И. Московские олимпиады по программированию/ Под ред. акад. Б. Н. Наумова. — 2-е изд., доп. и перераб. — М.: Наука. Гл. ред. физ.-мат. лит., 1990. — 208 с.
4. [Вологда, вузы]
Вологодская межвузовская олимпиада по программированию.
<http://www.uni-vologda.ac.ru/olympiads/interuni/>
Сайт есть на компакт-диске.
5. [Вологда, школы]
Вологодские олимпиады школьников по информатике.
<http://www.uni-vologda.ac.ru/olympiads/school/informatics/>
Сайт есть на компакт-диске.
6. [Воронеж, 2003]
Всероссийская индивидуальная студенческая олимпиада — 2003 г., г. Воронеж
Страница олимпиады-2004: <http://www.main.vsu.ru/~pmmmo/SiteRoss2004/>
Задачи олимпиады — 2003 есть на компакт-диске.
7. [Грэхем]
Грэхем Р., Кнут Д., Паташник О. Конкретная математика. Основание информатики: пер. с англ. — М.: Мир, 1998. — 703 с.
8. [Дальний Восток]
ACM International Collegiate Programming Contest,
Northeastern European Regional Contest,
Far-Eastern subregional quarterfinal.
<http://imcs.dvgu.ru/acm/arch.html>
Материалы сайта есть на компакт-диске.
9. [Ленинград, 1986]
I Олимпиада школьников Ленинграда по информатике, 1986 г.
<http://neerc.ifmo.ru/school/1986-1993/ru-olymp-spb-1986.html>
Задачи олимпиады есть на компакт-диске.
10. [Ленинград, 1987]
II Олимпиада школьников Ленинграда по информатике, 1987 г.
<http://neerc.ifmo.ru/school/1986-1993/ru-olymp-spb-1987.html>
Задачи олимпиады есть на компакт-диске.

11. [Летние сборы, 2000]

Летние учебно-тренировочные сборы к Международной олимпиаде школьников по информатике — 2000

12. [Москва, 2003]

ACM International Collegiate Programming Contest,
Northeastern European Regional Contest,
Moscow subregional quarterfinal 2003.

<http://acm.msu.ru/2003/>

Задачи олимпиады есть на компакт-диске.

13. [Рыбинск]

ACM International Collegiate Programming Contest,
Northeastern European Regional Contest,
Central region of Russia

На официальном сайте <http://www.rgata.ru/p0.php> материалы прошлых лет отсутствуют.

Материалы олимпиады есть на компакт-диске. Часть из них не публиковалась, а была предоставлена Владимиром Николаевичем Пинаевым, членом жюри олимпиады.

14. [Рыбинск, 1998]

Пинаев В. Н. Четвертьфинальные соревнования студенческого командного первенства мира по программированию. Центральный регион России / РГАТА. — Рыбинск, 1999. — 30 с.

Книга в электронном виде есть на компакт-диске. Спасибо Владимиру Николаевичу Пинаеву.

15. [Северный, 2002]

ACM International Collegiate Programming Contest,
Northeastern European Regional Contest,
Northern Subregion 2002.

<http://neerc.ifmo.ru/subregions/northern.html>

На момент написания этих строк по указанному адресу уже не были доступны материалы за 2002 и более ранние годы.

Материалы олимпиады есть на компакт-диске.

16. [Сипин]

Сипин А. С., Дунаев А. И. Областные олимпиады по информатике / Методические материалы для студентов физико-математического факультета. — Вологда: Русь, 1994.

Книга в электронном виде есть на сайте вологодских олимпиад. Сайт есть на компакт-диске.

17. [Урал, 1999]
Чемпионат Урала 1999.
<http://contest.ur.ru/ural99/>
Материалы олимпиады есть на компакт-диске.
18. [Урал, 2000]
Чемпионат Урала 2000.
Официальный сайт <http://contest.psu.ru/> на момент написания этих строк не работал больше года.
Материалы олимпиады есть на компакт-диске.
19. [CERC, 1999]
ACM International Collegiate Programming Contest,
Central European Regional Contest, 1999.
<http://contest.felk.cvut.cz/99cerc/>
Материалы олимпиады есть на компакт-диске.
20. [NEERC]
ACM International Collegiate Programming Contest,
Northeastern European Regional Contest.
<http://neerc.ifmo.ru/>
Материалы олимпиад есть на компакт-диске.
21. [SEERC, 1999]
ACM International Collegiate Programming Contest,
Southeastern European Regional Contest 1999.
<http://www.acm.ro/index.php/ACM1999/Problems>
Задачи олимпиады есть на компакт-диске.
22. [SWERC, 1997]
ACM International Collegiate Programming Contest,
Southwestern European Regional Contest 1997.
<http://www.informatik.uni-ulm.de/acm/Regionals/1997/>
Материалы олимпиады есть на компакт-диске.
23. [UVA, 28.02.2003]
Valladolid Online Judge – Contest Hosting Service
Return of the AZTECS Contest.
<http://online-judge.uva.es/contest/data/0069/problemset/>
Задачи олимпиады есть на компакт-диске.
24. [Waterloo, 22.09.2001]
Waterloo local contest 22 September 2001.

<http://plg.uwaterloo.ca/~acm00/010922/>

Материалы олимпиады есть на компакт-диске.

25. [ZVN]

Телекоммуникационный проект «Задача в неделю».

Рассылка job.education.zvn на <http://subscribe.ru>

Сайт <http://zvn.uriit.ru/>, <http://attend.to/zvn>.

Материалы проекта за 2002–2003 и 2003–2004 учебные годы есть на компакт-диске.

Авторы задач и решений

- Евгений Белов — в марте 2004 г. студент 2 курса отделения прикладной математики ВГПУ.
- Сергей Геннадьевич Волченков — в 2004 г. доцент Ярославского государственного университета.
- Екатерина Грозина — в декабре 2003 г. студентка 3 курса отделения прикладной математики ВГПУ.
- Виктор Сергеевич Губа — в августе 2004 г. профессор ВГПУ.
- Эдуард Гудков — осенью 2001 г. студент 1 курса отделения прикладной математики ВГПУ, осенью 2003 г. — студент 3 курса.
- Алексей Ивченко — осенью 2003 г. ученик 11 класса Вологодского многопрофильного лицея.
- Александр Сергеевич Кленин — в 2004 г. преподаватель Института математики и компьютерных наук Дальневосточного государственного университета.
- Михаил Юрьевич Копачев — в 2004 г. преподаватель Рыбинской государственной авиационной технологической академии.
- Антон Малафеев — в январе 2004 г. ученик 9 класса Вологодского многопрофильного лицея.
- Александр Меркулов — весной 2002 г. студент 1 курса отделения прикладной математики ВГПУ, осенью 2002 г. — студент 2 курса.
- Алексей Моисеев — весной 2002 г. студент 1 курса отделения прикладной математики ВГПУ.
- Ярослав Музыкантов — в 2003/2004 уч. г. — студент 1 курса отделения прикладной математики ВГПУ.
- Владимир Николаевич Пинаев — в 2004 г. доцент Рыбинской государственной авиационной технологической академии.
- Александр Ржеуцкий — весной 2002 г. ученик 11 класса 32-й школы г. Вологды.
- Никита Рыбак — в 2003/2004 уч. г. — студент 2 курса отделения прикладной математики ВГПУ, преподаватель математики Центра дополнительного образования одаренных детей.

- Григорий Сизов — в январе 2004 г. ученик 10 класса 32-й школы г. Вологды.
- Александр Степанович Сипин — в 2004 г. доцент ВГПУ.
- Александр Сорокин — в феврале 2004 г. ученик 10 класса 32-й школы г. Вологды.
- Арсений Сорокин — весной 2001 г. студент 4 курса отделения прикладной математики ВГПУ.
- Сергей Шиловский — в марте 2004 г. ученик 9 класса Вологодского многопрофильного лицея.

Федор Владимирович Меньшиков

Олимпиадные задачи по программированию (+CD)

Главный редактор
Заведующий редакцией
Руководитель проекта
Научный редактор
Литературные редакторы
Художник
Корректоры
Верстка

*Е. Строганова
А. Кривцов
В. Шачин
К. Кноп
К. Кноп, Н. Рощина
Е. Дьяченко
Н. Лукина, И. Смирнова
Л. Харитонов*

Лицензия ИД № 05784 от 07.09.01.

Подписано в печать 04.08.05. Формат 70×100/16. Усл. п. л. 16,8. Тираж 3500 экз. Заказ № 2437.

ООО «Питер Принт». 194044, Санкт-Петербург, пр. Б. Сампсониевский, 29а.

Налоговая льгота — общероссийский классификатор продукции ОК 005-93, том 2; 953005 — литература учебная.

Отпечатано с готовых диапозитивов в ФГУП «Печатный двор» им. А. М. Горького

Федерального агентства по печати и массовым коммуникациям.

197110, Санкт-Петербург, Чкаловский пр., 15.

ОЛИМПИАДНЫЕ ЗАДАЧИ по программированию

Фёдор Меньшиков

Цель этой книги — познакомить читателей с некоторыми часто встречающимися типами задач, предлагаемых на олимпиадах по программированию. В разных источниках можно найти немало олимпиадных задач, но примеры решений публикуются далеко не всегда, а без них начинающему программисту-олимпиаднику подготовиться к реальным соревнованиям бывает тяжело. В этой книге опубликован детальный разбор задач, взятых из ранее проводившихся олимпиад. Кроме примеров решения, автор обращает внимание читателей на особенности языков программирования, которые часто упускают из виду, но которые очень важны для успешного решения задачи. К книге прилагается компакт-диск с набором тестовых данных и тестирующей системой, позволяющими проверить правильность решения каждой из 90 задач.

 ПИТЕР®

Заказ книг:

197198, Санкт-Петербург, а/я 619
тел.: (812) 103-73-74, postbook@piter.com

61093, Харьков-93, а/я 9130
тел.: (057) 712-27-05, piter@kharkov.piter.com

www.piter.com — вся информация о книгах и веб-магазин

ISBN 5-469-00765-0



9 785469 007654